

Project Vitriol - Integrated Tile-Based Map Editor and Runtime Engine

Table of Contents

Contents

Project Vitriol - Integrated Tile-Based Map Editor and Runtime Engine	1
Table of Contents	1
Project Vitriol – NEA Analysis	5
Problem Identification	5
Client and User Description	7
Key Client Objectives:	7
End User Profiles	8
User Needs Analysis	8
Usability Requirements	10
Research Into Existing Systems	10
RPG Maker (XP / MV / MZ)	10
Tiled Map Editor	11
Unity (2D Tilemap Workflow)	12
Lightweight Standalone Map Editors	13
Market Gap:	13
Problem Statement	13
Intended Solution	16
Core System Architecture	16
Data Management Strategy	17
Performance and Quality Targets	18
Architectural Quality Attributes	18
Functional Requirements	18
Technical Constraints	18
Non-Functional Requirements	20
Data Requirements and Modelling	21

Data Strategy	21
Domain Model Design	21
Layered Map Representation	21
Data Integrity Framework	22
Performance-Oriented Data Handling.....	22
Storage Efficiency Measures.....	22
Data Safety and Recovery.....	23
Corruption Prevention.....	23
Summary of Data Architecture	23
Project Objectives	23
Objective 1: Core System Infrastructure and Foundational Architecture.....	23
Objective 2: Map Data Model, Persistence, and Validation	25
Objective 3: Graphical Editor Interface System.....	27
Objective 4: Map Editor System	28
Objective 5: Runtime Playback System	28
Objective 6: Performance, Stability, and Quality Assurance	29
Objective 7: Testing and Verification	30
Objective 8: Documentation and Evaluation Evidence	31
Success Criteria	32
Functional Completeness Metrics	32
Performance Benchmark Compliance	32
User Experience Success Indicators	33
Workflow Efficiency Indicators	33
Reliability and Stability	34
Data Integrity	34
Long-Term Viability Indicators.....	34
Documentation and Evidence	34
Traceability Matrix	34
Project Vitriol – NEA Design	35
Design Overview and Objectives	35
Core Functional Objectives	36
Non-Functional Objectives	36

Data Design: ERD Progression.....	37
Conceptual ERD: Domain Entities	37
Logical ERD: Attributes and Keys	38
Physical Design: JSON Schema Mapping	39
Persistence View: JSON Files + Project Metadata	41
System Architecture: Project Structure and Layered Responsibilities	42
<i>High Level Architecture</i>	42
Project Folder Structure and Asset Conventions	43
Editor Workflow Architecture.....	44
Class Design: Engine, Editor, Runtime Separation.....	45
Class Overview: Shared DTOs and Services	45
Map Model and Layer + Event Composition.....	46
Editor Editing Model: Commands, Undo/Redo, Persistence	48
Runtime Classes: Load, Render, Input, Camera	49
User Interface Design: Iterative UI Designs	50
Editor UI Iterations	50
Runtime UI Iterations	53
Current UI Mapping.....	56
Current UI Screenshots.....	57
Behaviour and Control Flow: Sequence Diagrams.....	58
Map Creation: New Map vs Sample Map	58
Tileset Loading: Editor	59
Tile Painting + Undo/Redo + Autosave	60
Runtime Render Loop: Viewport-Only Rendering.....	62
Camera Deadzone Follow	62
Design Justification Summary	63
Traceability: Design Decisions	63
Architectural Trade-offs and Alternatives Considered	64
Diagram Downloads	65
Project Vitriol – NEA Implementation	65
Vitriol.Data.....	65
Vitriol.Editor	72

Vitriol.Engine.....	223
Vitriol.Runtime	225
Vitriol.Shared	266
Vitriol.Tests	272
Project Vitriol – NEA Testing	278
1 Testing Strategy Overview	278
2 Subsystem Testing	278
2.1 Map Creation and Validation	278
2.2 Tile Editing and Undo/Redo	282
2.3 Tileset Loading and Referencing	289
2.4 Event System Testing	296
2.5 Persistence and JSON Integrity.....	303
2.6 Runtime Rendering and Viewport Behaviour	311
2.7 Camera Deadzone.....	316
3 Boundary and Abnormal Data Testing	321
3.1 Map Dimension Boundary Testing	321
3.2 Tileset Boundary and Missing Asset Handling	324
3.3 Event Parameter and Command Validation	327
3.4 JSON Corruption and Recovery Handling.....	330
3.5 Runtime Movement Boundary Testing	331
4 Performance Testing	335
4.1 Large Map Rendering Performance.....	335
4.2 Rendering Stability Under Continuous Movement	338
4.3 Memory Usage During Extended Editing Session.....	340
4.4 Asset Loading Performance	342
5 Regression Testing	345
5.1 Map Creation Regression Checks.....	345
5.2 Event System Regression Checks.....	347
5.3 Persistence Regression Checks	349
5.4 Rendering Regression Checks.....	352
6 User Testing.....	355
6.1 Structured User Task Testing	355

6.2 Observed Error Patterns.....	359
7 Witness Statement	359
7.1 Witness Confirmation of Functionality	359
7.2 Summary of Demonstrated Features.....	359
8 Testing Summary and Coverage.....	360
8.1 Requirement Coverage Mapping	360
8.2 Non-Functional Requirement Validation.....	361
8.3 Final Conclusion	362
9 Playtest Video	362
Project Vitriol – NEA Evaluation.....	362

Project Vitriol – NEA Analysis

Problem Identification

Currently, the development of tile-based 2D game worlds faces a number of structural and architectural limitations which, while not always immediately visible, disproportionately affect beginner and intermediate creators. Although there are many map editors and game engines available, there is a fundamental mismatch between ease of use, architectural cleanliness, and runtime integration. The tools with the most flexibility often require significant technical understanding of engine internals, rendering pipelines, or file formats. Conversely, tools that are simple to use frequently lack scalability, performance stability, or safe persistence mechanisms.

This creates a development environment in which creative intent is often constrained not by imagination, but by the limitations of tooling design. In many cases, the learning curve is not steep because of conceptual difficulty in map design itself, but because of fragmented workflows, inefficient rendering models, or fragile data handling systems. As a result, technical familiarity becomes prioritised over creative flow, discouraging users who lack deep engine knowledge from progressing beyond small-scale experimentation.

Fundamental Limitations

- **Dependence on External Integration** – Many widely used tile map editors operate as isolated tools rather than complete systems. They allow the user to construct a visual map, but require manual integration into a separate runtime engine for playtesting. This separation forces creators to understand file exports,

coordinate systems, collision implementation, and rendering logic that are not inherently part of map design. Rather than the architecture serving the creator, the creator is required to adapt to the architecture. This introduces unnecessary complexity, particularly for beginners who simply wish to build and test a playable environment.

- **The Feel of Disjointed Workflow** – Managing map data often involves switching between multiple disconnected conceptual systems: visual tile editing, collision configuration, event logic, and runtime testing. In many environments, these systems are not tightly coupled, meaning that what is visually constructed does not always reflect what is functionally playable. For example, a tile may appear walkable in the editor but behave differently in the runtime due to separate collision handling. This fragmentation increases cognitive overhead, as the user must mentally synchronise multiple layers of logic. Over time, this discourages experimentation, because small changes require disproportionate effort to verify.
- **Performance Fragility at Scale** – A common architectural weakness in lightweight editors is naive rendering strategy. Rendering entire maps every frame, rather than calculating only the visible viewport region, leads to unnecessary computation. While this may be acceptable for small maps, performance degrades significantly as dimensions increase. Within UI-driven frameworks such as WPF, this problem becomes more pronounced due to the overhead of managing large numbers of visual elements. Without deliberate optimisation strategies such as viewport-only rendering or region recalculation, scalability becomes constrained and the editing experience deteriorates.
- **Persistence and Data Fragility** – Many beginner-oriented tools rely on loosely structured JSON or text-based storage formats without strict schema validation, version tracking, or defensive loading behaviour. This creates vulnerabilities when:
 - Files are moved between devices
 - Asset paths are changed
 - Map dimensions are modified
 - Format updates occur

In the absence of controlled autosave policies and validation layers, users risk overwriting valid data with incomplete states. The problem is not the use of JSON itself, but the lack of architectural discipline surrounding its structure and lifecycle.

- **Limited Structured Guidance** – Existing systems often expose their full feature set immediately, without distinguishing between foundational and advanced concepts. Beginners are presented with layers, file structures, rendering concerns, and collision systems simultaneously. Without a progressive workflow model, users are forced to learn through error, which shifts focus away from

design and toward troubleshooting. Over time, this erodes confidence and reduces creative momentum.

Supporting Evidence:

Across community discussions surrounding tile-based editors and small engine toolchains, recurring frustrations include runtime mismatch between editor and gameplay, map lag at larger scales, broken asset references when projects are transferred, and inconsistent undo behaviour across editing modes. These issues frequently appear in help threads and development blogs, indicating that the core limitation is not feature absence, but architectural misalignment.

The pattern that emerges is clear: when tools prioritise rapid feature expansion over structural clarity and integration, creators inherit instability and fragmentation. This demonstrates that the barrier to creativity is rarely a lack of imagination, but rather friction introduced by the design of the development environment itself.

Client and User Description

The main client for this project is the community of beginner-to-intermediate 2D game developers who wish to construct playable tile-based environments without needing to architect a complete engine pipeline themselves. These users typically lack deep familiarity with rendering systems, file format design, or runtime architecture, yet still expect professional-level stability and responsiveness from their tools.

The client's requirements stem from a prioritisation of accessibility and creative flow over low-level technical complexity. Rather than focusing on engine internals, these creators aim to build environments visually, test them immediately, and iterate rapidly without fear of instability or data corruption. Therefore, the system must balance architectural discipline internally while presenting a controlled and intuitive interface externally.

Key Client Objectives:

- **Elimination of Main Barriers** – The primary objective is to remove the friction that exists between map design and runtime playtesting. Users should not need to manually integrate exported files into a separate engine, configure collision logic through code, or interpret file schemas to see their work function. The editor and runtime must share a unified data model so that a created map is immediately playable. By abstracting file handling, rendering calculations, and collision indexing away from the user, the system enables immediate engagement with creative tasks rather than technical configuration.
- **Intuitive Workflow Design** – The interface should reflect the natural order in which maps are built: tile layout first, collision refinement second, event

placement third. Tools must be grouped by workflow relevance rather than technical categorisation. Editing modes (tile, collision, event) should be clearly separated to prevent conceptual overlap, while still maintaining visual cohesion within a single viewport. Immediate feedback is critical; painting a tile must update the screen instantly, collision overlays must clearly indicate blocked regions, and events must visibly occupy positions on the grid upon placement. The system's terminology and layout should align with how creators think about worlds, not how the software internally stores them.

- **Confidence Building Through Structural Safety** – Users must feel safe experimenting. This requires a robust undo/redo system that applies consistently across tile edits, collision changes, and event placement. Autosave behaviour must be carefully controlled: newly created maps require an initial manual save before autosave activation, preventing accidental file creation, while still protecting ongoing work from unexpected shutdown. Validation during file loading must prevent crashes caused by malformed JSON or missing tileset paths. Error messages should not simply report failure, but clarify cause and suggest resolution.
- **Progressive Mastery** – The system should allow beginners to engage with foundational features without being overwhelmed by advanced concerns. A user should be able to paint tiles without needing to understand collision systems, and test maps in the runtime without understanding rendering loops. As familiarity increases, users can begin exploring event systems, camera behaviour, and larger map scaling. This layered exposure reduces cognitive overload and encourages sustained development rather than abandonment due to complexity.

End User Profiles

Beginner Creator:

No formal experience with engine architecture. Requires clear editing modes, error prevention, predictable saving behaviour, and immediate visual confirmation of actions.

Intermediate Designer:

Some familiarity with development tools. Requires performance scalability for larger maps, deterministic runtime behaviour, efficient editing workflows (shortcuts and drag-based tools), and portable file structures.

User Needs Analysis

Beginner Creator Needs (Primary Users):

- **Need Guided Structural Flow:**
The user should be able to follow a clear progression:
Create Map -> Paint Tiles -> Define Collision -> Place Events -> Save -> Test in

Runtime.

The system must not require understanding of rendering algorithms or file hierarchies to complete this cycle.

- **Need for Error Prevention:**

The software must validate map dimensions before creation, verify tileset references before rendering, and handle corrupted files gracefully. Preventative validation reduces frustration and preserves confidence.

- **Need for Immediate Feedback:**

Every action must be visible instantly within the viewport. Collision overlays should toggle clearly, and event markers should display at exact grid positions, reinforcing alignment between data and visual state.

- **Need for Safe Experimentation:**

Undo/redo must be deterministic and reversible. Autosave must protect progress without creating file clutter or overwriting unsaved maps. The system must ensure that no editing action irreversibly damages the map structure.

- **Need for Progressive Complexity:**

The interface should allow users to focus on one conceptual system at a time (tiles, collision, events), preventing cognitive overload and reducing accidental cross-editing.

Intermediate Creator Needs (Secondary Users):

- **Need for Efficient Workflow:**

Keyboard shortcuts (e.g., undo/redo), drag-to-paint functionality, smooth zoom and pan controls, and responsive viewport rendering to reduce friction during large-scale edits.

- **Need for Creative Flexibility Within Structure:**

While the system abstracts engine internals, it must not overly restrict map structure. Event data should support extension through command lists, and map formats should remain adaptable via versioned JSON schemas.

- **Need for Data Integrity:**

Maps must include format version fields and consistent schema definitions. Relative tileset paths ensure portability between machines without broken references.

- **Need for Performance Stability:**

Maps exceeding typical beginner scale must remain responsive. Viewport-only rendering must prevent full-map redraw overhead, ensuring scalability as complexity increases.

- **Need for Runtime Consistency:**

Collision detection and camera behaviour must match editor expectations precisely. A deterministic runtime loop ensures that gameplay behaviour reflects map configuration without unexpected divergence.

Usability Requirements

The System Usability Scale (SUS) will be used to evaluate interface effectiveness, with a target score above 80% across user segments.

Core usability expectations include:

- Users must be able to create and test a playable map within a single session without external documentation.
- Editing modes must be visually distinguishable and logically separated.
- Undo/redo must function consistently across all editing actions.
- Save behaviour must clearly differentiate between first manual save and subsequent autosave operations.
- File portability between systems must not break tileset references or corrupt map data.

User feedback should indicate increased confidence over time, a clear understanding of editing modes, and the ability to recover from mistakes without external assistance.

Research Into Existing Systems

RPG Maker (XP / MV / MZ)

- **Architecture Analysis** – RPG Maker provides an integrated editor and runtime system, which superficially appears aligned with beginner-friendly development. However, its architecture is proprietary and tightly coupled, meaning map data, event systems, collision handling, and rendering behaviour are deeply embedded within the engine's internal structure. The developer does not directly control how tiles are batched, how collision grids are evaluated, or how map data is internally cached. This creates architectural opacity: while early development is simplified, extensibility becomes constrained. Modifications beyond predefined behaviours require scripting within the engine's embedded language (Ruby or JavaScript depending on version), introducing a dependency on internal APIs that were not designed for modular extension. Over time, this leads to version migration issues, where changes in engine internals affect previously stable projects.
- **Data and Persistence Model** – Map files in RPG Maker are stored in engine-specific formats that abstract away structural details. While convenient, this reduces transparency. Developers cannot easily validate schema, enforce version compatibility, or implement custom migration strategies. Event command systems are stored in structured but engine-bound representations, limiting portability outside the ecosystem. Additionally, data storage is distributed across map files, database files, and scripting layers, creating implicit dependencies between components.

- **User Experience Assessment** – Although marketed as accessible, the interface separates map editing, event configuration, database management, and playtesting into distinct conceptual modes. The event editor in particular introduces a non-spatial abstraction layer: event logic is constructed in command lists separate from the visual tile grid, requiring users to mentally associate spatial placement with conditional logic. This separation increases cognitive load. Furthermore, menu structures are often grouped by engine subsystem rather than creator workflow, meaning users must navigate technical categorisation rather than creative sequencing.
- **Technical Barriers** – Advanced customisation requires scripting knowledge and familiarity with engine execution order. Simple alterations to collision rules, rendering behaviour, or camera systems cannot be achieved purely through the visual interface. Error reporting, when scripting is involved, exposes raw engine traces rather than user-oriented guidance. This creates a learning divide between users who remain within predefined functionality and those who attempt deeper modification.
- **Performance Considerations** – Rendering optimisation is handled internally and abstracted from the user. While suitable for typical map sizes, users cannot explicitly enforce viewport-only rendering strategies or optimise draw regions manually. As a result, scalability is bounded by engine design decisions rather than deliberate architectural planning.

Tiled Map Editor

- **Architecture Analysis** – Tiled is an open-source, standalone tile map editor that supports multiple layers (tile layers, object layers, image layers) and exports to structured formats such as JSON and TMX. Architecturally, it is modular and extensible within its editing domain. However, it deliberately excludes runtime functionality. This design philosophy positions Tiled as a content creation tool rather than an integrated system.
- **Data Model Strength and Limitation** – Tiled's exported JSON is structured and relatively clean, but it is editor-centric rather than runtime-centric. Collision data is typically represented through object layers or tile properties, requiring interpretation by the consuming engine. While flexible, this shifts architectural responsibility onto the developer. There is no enforced schema discipline beyond what the editor produces; developers must implement their own validation, versioning, and migration strategies.
- **Workflow Fragmentation** – Because Tiled does not include a runtime, users must integrate exported maps into a separate engine manually. This introduces multiple friction points:
 - Coordinate translation
 - Tile index mapping

- Collision grid reconstruction
- Camera and viewport logic implementation

The visual design environment and the runtime execution environment remain separate, requiring iterative adjustment across two independent systems.

- **Technical Barriers** – Although map editing itself is straightforward, meaningful use in a playable project demands knowledge of rendering loops, JSON parsing, collision algorithms, and asset management. The barrier shifts from map design to integration engineering.
- **Performance Responsibility** – Tiled does not prescribe rendering optimisation strategies. If a developer implements naive full-map redraws in their runtime, performance degrades. Thus, scalability depends entirely on the architectural competence of the integration layer rather than the editor.

Unity (2D Tilemap Workflow)

- **Architecture Analysis** – Unity is a general-purpose game engine built around a component-based architecture. While it supports 2D tilemaps, these exist within a broader ecosystem designed primarily for 3D rendering. Tilemap components are attached to GameObjects, which exist within scene hierarchies governed by lifecycle callbacks (Update, FixedUpdate, etc.). This abstraction model introduces complexity disproportionate to the needs of lightweight 2D map editing.
- **System Overhead** – Unity’s architecture includes physics engines, lighting systems, audio subsystems, asset import pipelines, and serialisation frameworks that operate regardless of whether a project requires them. For simple tile-based map workflows, this introduces unnecessary computational and conceptual overhead. Memory usage and build sizes are typically significantly larger than specialised 2D solutions.
- **User Experience Assessment** – The editor interface distributes responsibilities across Scene view, Game view, Hierarchy, Inspector, and Project windows. While powerful, this layout demands understanding of multiple interacting subsystems. Tilemaps must be combined with collider components, camera scripts, and input systems manually. There is no enforced workflow ordering; beginners must determine architectural structure independently.
- **Technical Barriers** – Implementing deterministic collision systems, camera deadzone behaviour, event triggering, and viewport optimisation requires C# scripting. Knowledge of draw calls, batching, and garbage collection becomes relevant when scaling maps. This exceeds the intended scope for beginner creators seeking simple map-to-runtime iteration.
- **Versioning and Serialisation Risks** – Unity’s prefab and scene serialisation systems can change between versions, occasionally causing migration issues.

Large projects may encounter conflicts between tilemap colliders and physics layers, requiring architectural debugging rather than design focus.

Lightweight Standalone Map Editors

- **Tool Analysis** – Many lightweight editors prioritise rapid tile placement with minimal interface complexity. However, these tools often lack strict persistence models. Map files may not include version identifiers, schema validation, or enforced relative asset referencing. As a result, portability between systems becomes fragile.
- **Undo/Redo Limitations** – In some tools, undo functionality applies only to tile painting and not to object placement or property editing. This inconsistency undermines user confidence and discourages experimentation.
- **Integration and Schema Drift** – Without a shared runtime model, schema evolution can break older maps. Changes to tile indexing or layer definitions may invalidate existing files. Asset paths may be stored absolutely rather than relatively, causing reference failures when directories change.
- **Architectural Simplicity vs Structural Discipline** – While accessible, these editors frequently prioritise simplicity over robustness. They lack defensive loading, controlled autosave behaviour, and explicit version management — all of which are critical for long-term project stability.

Market Gap:

At present, no single lightweight solution simultaneously provides:

- Beginner-accessible visual editing
- Integrated runtime playtesting within the same architectural model
- Explicit viewport-only rendering in both editor and runtime
- Strict JSON schema discipline with version tracking
- Relative asset referencing for portability
- Command-pattern undo/redo consistency across all editing modes
- Deterministic runtime loop aligned with editor expectations

Existing systems either prioritise accessibility at the expense of architectural clarity, or prioritise extensibility at the expense of usability.

This reveals a clear gap for a system that balances disciplined software engineering principles with tightly integrated, creator-oriented workflow design.

Problem Statement

Currently, the ecosystem surrounding tile-based 2D map creation suffers from a fundamental mismatch between creator needs and tool architecture. While there are numerous editors and engines available, very few are designed with an intentional

balance between accessibility, structural discipline, and runtime integration. This mismatch creates a barrier that disproportionately affects beginner and intermediate developers – ironically, the very group most motivated to experiment with world-building and interactive design.

The core issue is not the absence of features. In fact, many existing tools are extremely feature-rich. The problem lies in how those features are architected and exposed. Many systems prioritise extensibility, abstraction, and technical flexibility over cohesive workflow and structural clarity. As a result, creators are often forced to adapt to the tool's internal logic rather than the tool adapting to their creative process.

Because of this imbalance, the following systemic issues emerge:

- **Experimentation is Discouraged Through Architectural Fragility** – In many workflows, errors manifest as broken file references, corrupted exports, inconsistent runtime behaviour, or unexplained rendering anomalies. When undo systems are incomplete or autosave behaviour is unpredictable, users develop a fear of making structural changes. The risk of accidentally breaking a map increases as complexity grows. This discourages experimentation, which is foundational to creative development. Instead of encouraging iteration, the tool becomes something to cautiously manage.
- **Fragmented Workflow Reinforces Cognitive Overhead** – Existing solutions frequently separate editing and runtime environments, or isolate logical systems from visual ones. Collision handling may not directly correspond to visual tile placement. Event systems may exist as abstract object lists separate from spatial context. This fragmentation forces users to maintain a mental model of how separate subsystems interact. As complexity increases, so does the cognitive burden required to maintain consistency between these disconnected representations.
- **Integration Responsibility is Shifted Onto the User** – When editors do not share a unified data model with their runtime environments, users must manually bridge the gap. This may involve interpreting JSON exports, rebuilding collision grids, implementing camera logic, or translating coordinate systems. Instead of the tool guaranteeing structural alignment, integration becomes a manual engineering task. This effectively raises the barrier to entry for users who lack formal engine development experience.
- **Scalability is Often an Afterthought** – Many lightweight editors function adequately at small scales but degrade as map size increases. Without viewport-based rendering or efficient region recalculation, performance suffers due to unnecessary full-map redraws. When performance issues emerge, users must debug rendering inefficiencies that stem from architectural design choices they never consciously made.

- **Learning is Reactionary Rather Than Structured** – Documentation for many systems focuses on solving isolated issues rather than explaining architectural reasoning. Users learn how to fix specific symptoms rather than understanding the underlying model. As a result, their knowledge becomes context-dependent and fragile. When faced with scenarios outside documentation examples, confidence collapses because the conceptual foundation was never established.

These issues demonstrate that the limitation is not creative capacity, nor feature availability, but rather the structural misalignment between user workflow and tool architecture.

Therefore, a new solution must be deliberately designed around creator-centric principles rather than engine-centric abstraction. Such a solution must provide:

- **Progressive Complexity** – The interface must allow users to engage with foundational tasks (tile painting and saving) before encountering more advanced systems (collision indexing, event logic, runtime behaviour). Advanced configuration should exist but not overwhelm initial workflow.
- **Integrated Data Management** – All map elements (tile data, collision data, event data) must exist within a unified schema shared between editor and runtime. Changes made in the editor must be directly consumable by the runtime without translation layers or manual interpretation.
- **Runtime-Editor Cohesion** – The runtime should reflect exactly what is constructed in the editor, eliminating discrepancies between visual representation and gameplay behaviour.
- **Error-Resistant Design** – The system must implement controlled autosave policies, command-pattern undo/redo, defensive JSON loading, and clear error messaging. Destructive actions should require confirmation, and persistence mechanisms must prioritise structural safety.
- **Scalable Rendering Architecture** – Both editor and runtime must implement viewport-only rendering strategies to ensure consistent performance as map dimensions increase.
- **Sustainable Modular Architecture** – The internal codebase must separate concerns between rendering, input handling, data persistence, and editing logic. This modular structure ensures that extending functionality does not destabilise existing behaviour.

In summary, the required solution is not simply “another map editor,” but a deliberately architected system that integrates disciplined software engineering with accessible creative workflow. Only by aligning structural integrity with usability can the barrier between creative intention and technical execution be meaningfully reduced.

Intended Solution

The intended solution provides a tightly integrated tile-based map editor and runtime environment built around architectural clarity, performance scalability, and creator-centric workflow design. Rather than overwhelming users with excessive abstraction or feature density, the system establishes sensible structural defaults while preserving extensibility through disciplined design.

The core philosophy is to separate internal architectural complexity from external user interaction. Rendering logic, collision indexing, persistence formatting, and runtime execution are handled internally through modular components, ensuring that technical mechanisms do not interrupt creative flow. At the same time, these systems are deliberately engineered to remain transparent, predictable, and scalable.

Core System Architecture

Editor Application

The editor is implemented using WPF, leveraging a structured MVVM-inspired separation between interface components and logic handling. The interface is centred around a single unified viewport, ensuring that all editing operations remain spatially contextualised.

The system is built around progressive workflow exposure rather than feature overload.

- **Mode-Based Editing Model** – The editor separates interaction into clearly defined modes: Tile Editing, Collision Editing, and Event Editing. This prevents cross-system interference and reduces cognitive overload. Each mode modifies a distinct layer within the map's data model while sharing a common rendering viewport.
- **Layered Map Structure** – The map is composed of independent layers:
 - Tile layer (visual data)
 - Collision layer (walkability grid)
 - Event layer (position-based triggers)

This separation ensures that visual representation does not become tightly coupled to logical behaviour, preventing accidental structural corruption during editing.

- **Unified Viewport Rendering** – All editing operations occur within a single rendering surface. The viewport dynamically calculates visible tile regions and renders only those within the camera bounds. This design preserves performance even as map dimensions increase.
- **Command-Based Editing System** – All editing actions are encapsulated as command objects supporting execution and reversal. This enables deterministic undo/redo functionality across tile painting, collision toggling, and event

placement. The command model ensures that state transitions are controlled and reversible.

- **Controlled Autosave Policy** – The system differentiates between:
 - Newly created maps (manual save required before autosave activation)
 - Existing maps (autosave enabled after initial save)
 - Sample maps (continuous autosave permitted)

This prevents file clutter and accidental overwriting while still protecting against data loss

- **Real-Time Validation** – Map dimensions, tile indices, and file paths are validated before persistence. Defensive loading prevents crashes caused by malformed or outdated map files.

Runtime Application

The runtime is deliberately lightweight and purpose-built for deterministic playback of maps created in the editor. It shares the same data model as the editor, eliminating format translation and integration overhead.

- **Deterministic Update Loop** – A fixed timestep game loop separates update logic from rendering frequency. This ensures consistent behaviour regardless of hardware variation.
- **Viewport-Only Rendering** – The runtime calculates visible tile bounds based on camera position and renders only necessary regions. This avoids full-map redraw inefficiencies and maintains stable frame rates even on larger maps.
- **Grid-Based Collision System** – Collision detection operates on a discrete grid aligned with the collision layer defined in the editor. Movement resolution ensures predictable traversal without clipping or inconsistent edge cases.
- **Camera Follow System** – The runtime includes a camera abstraction that defines the visible world region and smoothly follows the player within map bounds. This maintains visual stability while preserving spatial context.
- **Layered Rendering Order** – Rendering order is strictly defined to ensure correct visual hierarchy between background tiles, event markers, and player sprites.

Data Management Strategy

The system uses structured JSON files as its persistence format. Rather than relying on loosely defined schemas, the design enforces structural discipline.

- **Unified Data Model** – Both editor and runtime consume the same DTO structures, eliminating schema drift and translation inconsistencies.
- **Format Version Field** – Each map file includes a version identifier, allowing for controlled evolution of file structure without invalidating older maps.
- **Relative Asset Referencing** – Tileset paths are stored relative to project directories, ensuring portability between devices without broken references.

- **Layer Isolation Within JSON** – Tile, collision, and event data are stored in separate arrays within the same file, maintaining structural clarity and preventing cross-contamination of concerns.

Performance and Quality Targets

The solution is engineered to maintain consistent responsiveness on mid-range hardware.

- **Rendering Optimisation** – Viewport-bound tile rendering reduces unnecessary draw operations. This ensures editing and runtime playback remain responsive even for maps exceeding 100×100 tiles.
- **Memory Management Discipline** – Image assets are loaded once and reused rather than recreated per frame. Disposal routines ensure that unused resources do not accumulate.
- **Startup Efficiency** – Map loading is performed in structured stages, minimising blocking operations and ensuring startup times remain within acceptable limits for typical project sizes.
- **Frame Stability Target** – Runtime performance aims to maintain approximately 60 FPS on standard integrated graphics configurations.

Architectural Quality Attributes

The system is explicitly designed around the following quality principles:

- **Learnability** – Clear editing modes, visible overlays, and direct map-to-runtime integration reduce the time required to achieve a playable result.
- **Efficiency** – Drag-to-paint tools, keyboard shortcuts for undo/redo, and responsive viewport navigation reduce repetitive friction.
- **Error Tolerance** – Command-based undo/redo, defensive JSON loading, and controlled autosave policies protect against irreversible mistakes.
- **Consistency** – Shared data models between editor and runtime ensure predictable behaviour. Rendering order, collision evaluation, and camera logic behave deterministically.
- **Scalability** – Viewport-only rendering and structured layer separation prevent performance degradation as map complexity increases.

Functional Requirements

Technical Constraints

Hardware Compatibility Requirements

- **Target Specification** – The project must perform reliably on systems equipped with Intel UHD 620 integrated graphics, 8GB RAM, and mechanical hard drives. This ensures accessibility for users without dedicated GPUs or SSD storage.

- **Rendering Optimisation Strategy** – The system must implement viewport-only tile rendering in both editor and runtime applications. Only tiles within the visible camera region may be drawn per frame. This minimises unnecessary redraw operations and preserves frame stability as map dimensions increase.
- **Memory Management Discipline** – Image assets (tilesets) must be loaded once and reused rather than recreated per frame. Resource disposal must occur when maps are closed or replaced, preventing memory accumulation over extended editing sessions.

Software Dependency Management

- **.NET Framework Constraints** – The system must maintain compatibility with .NET 8+, while avoiding reliance on unstable preview features. This ensures long-term maintainability and stable runtime behaviour.
- **Third-Party Libraries** – External dependencies must be minimal and selected based on performance stability and compatibility with WPF. The architecture should avoid unnecessary abstraction layers that complicate debugging or migration.
- **Platform Isolation** – The system must remain Windows-compatible (Windows 11 target environment) without requiring platform-specific native integrations that complicate deployment.

Challenges Involving Resource Allocation

- **Solo Development Effort** – As a single-developer project, architectural modularity must be prioritised to reduce refactoring overhead. Shared DTO models between editor and runtime minimise duplication.
- **Structured Knowledge Management** – Architectural decisions must be documented to ensure clarity of rendering logic, persistence structure, and collision behaviour. This supports maintainability and extension.

ID	Capability	Priority
FR1	Create new maps with defined dimensions and initialise layered structure	High
FR2	Visually paint, erase, and modify tile data within a grid-based system	High
FR3	Collision Editing – Define walkable and blocked tiles via a dedicated collision layer	High
FR4	Event Placement – Add, edit, and remove position-based event triggers on the map	High
FR5	Map Persistence – Save and load maps using structured JSON with format versioning	Medium
FR6	Runtime Playback – Load maps into runtime and render with camera-based viewport logic	Medium

FR7	Player Movement & Collision – Handle deterministic grid-based collision detection	High
FR8	Editing History – Implement command-based Undo/Redo across tile, collision, and events	High
FR9	Autosave Control – Activate controlled autosave after initial manual save	Medium
FR10	Performance Stability – Maintain approximately 60 FPS on target specification hardware	High
FR11	Relative Asset Referencing – Store tileset paths relative to project directory	Medium
FR12	Layer Rendering Order – Enforce correct drawing hierarchy in editor and runtime	Medium

Non-Functional Requirements

Quality Attribute	Requirement	Measurable Target
Performance	Frame Rate Stability Map Loading Efficiency	60 FPS $\pm$ 5% variance during runtime on target hardware 2s- for 100x100 tile maps
Usability	Interface Intuitiveness Task Completion Rate	SUS Score 80%+ 85%+ successful completion of core workflow tasks
Maintainability	Code Structure Clarity Modular Architecture	Clear separation between Editor, Runtime, and Shared Models No circular dependencies between major components
Reliability	Application Stability Data Integrity	99%+ crash-free sessions during structured testing No data corruption during save/load cycles
Compatibility	Operating System Support	Full windows 11 compatibility (.NET 8+)
Scalability	Map Dimension Handling Rendering Efficiency	Stable performance for maps 100x100+ tiles Viewport-only rendering prevents full-map redraw overhead

Data Requirements and Modelling

Data Strategy

The system employs a structured, file-based data architecture designed to support both immediate usability and long-term project stability. All map data is stored using a clearly defined JSON schema shared between the editor and runtime applications. This unified model ensures consistency across editing and execution environments while maintaining portability and transparency.

The architecture prioritises:

- Clear separation between visual and logical data
- Structural validation during load and save operations
- Version-controlled schema evolution
- Direct interoperability between editor and runtime

By maintaining a disciplined but lightweight data structure, the system supports scalable map development without introducing unnecessary architectural overhead.

Domain Model Design

The central data structure is a Map DTO (Data Transfer Object), which directly represents the conceptual components of a tile-based world.

Each map file contains:

- Map Metadata (name, dimensions, format version)
- Tilesset Reference (stored as a relative file path)
- Tile Layer Data (grid of tile indices)
- Collision Layer Data (grid defining walkability)
- Event Layer Data (collection of position-based event objects)

This structure mirrors the mental model of map construction, reducing the distance between user interaction and internal representation.

Layered Map Representation

- **Tile Layer**
Stores integer indices referencing positions within the tileset. This layer is purely visual and does not encode gameplay logic.
- **Collision Layer**
Stored as a grid aligned with tile coordinates, defining whether each tile is

traversable. This separation ensures gameplay logic remains independent of visual appearance.

- **Event Layer**

Stored as a list of objects containing grid position and associated properties. Events are spatially anchored but logically independent from tile and collision data.

By isolating these concerns, the system prevents unintended cross-modification and preserves structural clarity.

Data Integrity Framework

Structural Validation

Validation is enforced at key lifecycle points:

- Tile and collision arrays must match declared map dimensions.
- Event positions must fall within valid bounds.
- Tile indices must correspond to valid tileset coordinates.
- Tileset paths must resolve correctly before rendering.

Invalid states are intercepted before runtime execution, preventing crashes and undefined behaviour.

Schema Versioning

Each map file includes a format version field. This enables controlled structural evolution while preserving compatibility with earlier map versions.

The schema is designed to be extensible, allowing additional properties to be appended without restructuring the entire model.

Performance-Oriented Data Handling

The data model directly supports rendering optimisation strategies:

- Grid-based storage enables efficient calculation of visible tile ranges.
- Layer separation allows selective rendering without recomputation of unrelated data.
- Tileset images are loaded once per session and reused, preventing repeated allocation.

This alignment between storage model and rendering logic ensures consistent performance as map dimensions increase.

Storage Efficiency Measures

- **Compact JSON Structure** – Tile and collision grids are stored in streamlined array formats, minimising file size without sacrificing readability.

- **Deterministic Serialisation** – Saving the same in-memory structure consistently produces predictable file output, simplifying debugging and reducing ambiguity.
- **Layer Isolation** – Separating tile, collision, and event data prevents corruption in one subsystem from cascading into others.

Data Safety and Recovery

Controlled Persistence

- Autosave activates only after an initial manual save for new maps, ensuring user awareness of file creation.
- Save operations overwrite files in a controlled manner to avoid partial writes.
- Malformed files are detected during load, and execution is prevented until resolved.

Corruption Prevention

- Dimension consistency checks prevent mismatched grid structures.
- Validation is applied before serialisation to ensure only valid states are written to disk.
- Defensive error handling prevents malformed data from propagating into runtime systems.

Summary of Data Architecture

The data model is intentionally structured, versioned, and layer-isolated. It directly mirrors the conceptual structure of tile-based world design while maintaining runtime compatibility and performance alignment.

Through disciplined schema design and validation, the system ensures:

- Portability
- Structural clarity
- Rendering efficiency
- Data integrity
- Scalability

Without unnecessary architectural complexity.

Project Objectives

Objective 1: Core System Infrastructure and Foundational Architecture

1.1 Project Structure and Solution Architecture

1.1.1 Solution Configuration built within Visual Studio

- 1.1.1.1 Setup and finalise a “Master” solution to contain all editor, runtime, and shared model projects
- 1.1.1.2 Establish a Runtime class library to isolate core playback systems (rendering, input, collision, camera)
- 1.1.1.3 Create an Editor application project to serve as the primary map creation interface
- 1.1.1.4 Design and implement a Shared DTO project to enforce consistency across editor and runtime

1.1.2 Project Reference Configuration

- 1.1.2.1 Configure the Editor project to reference Shared DTOs for all map structures and serialisation contracts
- 1.1.2.2 Configure the Runtime project to reference Shared DTOs to ensure direct consumption of editor outputs
- 1.1.2.3 Implement a namespace hierarchy that clearly separates concerns (e.g. Vitriol.Editor, Vitriol.Engine, Vitriol.Shared)
- 1.1.2.4 Validate project dependency direction (Shared referenced by both, no circular coupling between Editor and Runtime)

1.1.3 Validation for the main Build System

- 1.1.3.1 Successfully execute clean builds of the entire solution in Release and Debug configurations
- 1.1.3.2 Ensure compilation output is structured correctly and does not rely on missing local dependencies
- 1.1.3.3 Create repeatable run configurations for both Editor and Runtime for testing and demonstration
- 1.1.3.4 Validate that the runtime can launch independently and load maps created by the editor

1.2 Management of Dependencies and Package Configuration

1.2.1 NuGet and Dependency Discipline

- 1.2.1.1 Integrate JSON serialisation tooling appropriate for DTO persistence (stable and maintainable)
- 1.2.1.2 Ensure WPF rendering dependencies remain minimal to preserve performance and deployment stability

- 1.2.1.3 Ensure release builds include all required DLLs and execute on clean systems

1.3 Object-Oriented Architecture and Separation of Concerns

1.3.1 Design of Core Class Hierarchy

- 1.3.1.1 Implement core map structures as DTOs (MapData, Layer data, Event definitions)
- 1.3.1.2 Implement clear runtime abstractions (Camera, Player, Input handling, Render systems)
- 1.3.1.3 Separate map rendering logic from map storage logic to prevent structural coupling
- 1.3.1.4 Ensure event objects and collision data remain independent of tile rendering data

1.3.2 Interface Segregation and Modular Components

- 1.3.2.1 Define interfaces for update loops and renderable systems to standardise runtime execution
- 1.3.2.2 Encapsulate editor tools as modular behaviours (tile brush, collision brush, event placement tool)
- 1.3.2.3 Ensure shared validation utilities are reusable between editor and runtime
- 1.3.2.4 Document component boundaries for maintainability and future extension

Objective 2: Map Data Model, Persistence, and Validation

2.1 Structured JSON Schema Design

2.1.1 Map Data Format Definition

- 2.1.1.1 Define a stable JSON structure containing metadata, tileset reference, and layered map data
- 2.1.1.2 Define tile layer format using tile indices referencing the loaded tileset
- 2.1.1.3 Define collision layer format as a grid aligned to the tile grid
- 2.1.1.4 Define event layer format as a list of position-based event objects with typed properties

2.1.2 Schema Versioning

- 2.1.2.1 Add a format version field to map files for controlled schema evolution
- 2.1.2.2 Ensure version field is validated during load to prevent undefined behaviour

- 2.1.2.3 Establish rules for forward-compatible extension (new fields appended without breaking existing)
- 2.1.2.4 Maintain clear documentation of schema meaning to support maintenance and testing

2.2 Persistence Pipeline

2.2.1 Save and Load System

- 2.2.1.1 Implement map serialisation to JSON from the shared DTO model
- 2.2.1.2 Implement deserialization with defensive error handling and clear failure states
- 2.2.1.3 Ensure tile and collision arrays match declared map dimensions before saving
- 2.2.1.4 Implement relative tileset path persistence to preserve portability between machines

2.2.2 Data Integrity Validation

- 2.2.2.1 Validate tile indices against tileset bounds during runtime/editor render preparation
- 2.2.2.2 Validate event positions fall within map boundaries
- 2.2.2.3 Validate collision grid dimensions match tile grid dimensions
- 2.2.2.4 Prevent invalid sates from being written to disk by validating prior to serialisation

2.3 Autosave Behaviour and Persistence Safety

2.3.1 Autosave Activation Rules

- 2.3.1.1 Implement rule: new maps require at least one manual save before autosave is enabled
- 2.3.1.2 Implement rule: existing saved maps may autosave on subsequent edits
- 2.3.1.3 Implement rule: sample/testing maps may autosave continuously for rapid iteration
- 2.3.1.4 Ensure autosave never overwrites an unintended file path or produces uncontrolled file duplication

2.3.2 Failure Handling

- 2.3.2.1 Handle failed saves gracefully (e.g. file locked, invalid path) without corrupting in-memory state

- 2.3.2.2 Prevent runtime load attempts of known-invalid map files
- 2.3.2.3 Ensure error messages indicate cause and resolution direction
- 2.3.2.4 Validate that recovery from invalid data does not require manual file editing by the user

Objective 3: Graphical Editor Interface System

3.1 Main Editor Window Architecture

3.1.1 WPF UI Structure

- 3.1.1.1 Implement a main viewport-based editor layout centred around map rendering
- 3.1.1.2 Implement UI controls for loading tilesets, creating maps, and switching editing modes
- 3.1.1.3 Ensure window resizing and DPI behaviour remains stable and readable
- 3.1.1.4 Provide clear visual distinction between tile editing, collision editing, and event editing modes

3.1.2 Workspace Interaction Model

- 3.1.2.1 Implement selection logic appropriate for tools (tile selection from palette, event selection on map)
- 3.1.2.2 Implement status feedback to show active tool, coordinates, and current layer/mode
- 3.1.2.3 Ensure tool interactions do not conflict (e.g. collision edits not painting tiles)
- 3.1.2.4 Validate workflow through structured user task sequences (create -> edit -> save -> load)

3.2 Editor Tooling and Usability Features

3.2.1 Tileset Handling and Palette

- 3.2.1.1 Load tileset images from disk and slice into tile-sized regions for selection
- 3.2.1.2 Implement a palette panel that displays tiles and support selection of current brush tile
- 3.2.1.3 Ensure selected tile index maps correctly to tileset coordinated during painting
- 3.2.1.4 Validate tileset replacement does not silently corrupt map tile indices

3.2.2 Mode-Specific Tools

- 3.2.2.1 Implement tile painting tool supporting click-to-paint and drag-to-paint behaviour
- 3.2.2.2 Implement collision painting tool that toggles blocked/walkable states
- 3.2.2.3 Implement even placement tool that creates and removes events at grid coordinates
- 3.2.2.4 Ensure each tool updates the viewport immediately to maintain user trust and feedback

Objective 4: Map Editor System

4.1 Tile-Based Rendering Engine (Editor)

4.1.1 Map Visualisation System

- 4.1.1.1 Implement a WPF rendering surface for tile map drawing
- 4.1.1.2 Implement coordinate conversion between mouse position and tile grid indices
- 4.1.1.3 Implement camera/viewport panning and zoom controls appropriate for large maps
- 4.1.1.4 Implement viewport-only rendering (render only visible tiles rather than full map)

4.1.2 Layer Rendering and Overlays

- 4.1.2.1 Render tile layer as the base visual layer
- 4.1.2.2 Render collision overlay with clear visual distinction from tiles
- 4.1.2.3 Render event makers with identifiable icons/visuals at their grid positions
- 4.1.2.4 Ensure overlays are toggleable to reduce visual clutter during specific editing tasks

4.2 Editing History System

4.2.1 Command Pattern Implementation

- 4.2.1.1 Define command objects for reversible edits (tile placement, collision toggle, event add/remove)
- 4.2.1.2 Track command execution order to support deterministic undo/redo
- 4.2.1.3 Implement Ctrl+Z / Ctrl+Y keyboard shortcuts for editing history navigation
- 4.2.1.4 Ensure undo/redo updates both data model and viewport rendering immediately

4.2.2 History Safety and Correctness

- 4.2.2.1 Ensure undo never produces invalid dimension states
- 4.2.2.2 Ensure redo matches original edit behaviour exactly (no divergence)
- 4.2.2.3 Ensure history operates consistently across all three map layers
- 4.2.2.4 Test edge cases (rapid drag edits, repeated undos, mixed layer editing sequences)

Objective 5: Runtime Playback System

5.1 Architecture of Core Runtime Loop

5.1.1 Engine Initialisation

- 5.1.1.1 Implement a runtime entry point that initialises rendering, input, and map loading subsystems
- 5.1.1.2 Load map JSON and tileset resources using the shared DTO model
- 5.1.1.3 Validate map integrity before entering gameplay loop
- 5.1.1.4 Ensure runtime starts reliably with a selected map and does not require editor context

5.1.2 Deterministic Update Loop

- 5.1.2.1 Implement a fixed timestep loop for consistent movement and collision evaluation
- 5.1.2.2 Separate update frequency from rendering frequency to ensure stability
- 5.1.2.3 Ensure consistent movement speed across different hardware
- 5.1.2.4 Validate long-runtime stability (no drift, no progressive slowdown)

5.2 Rendering and Camera System

5.2.1 Viewport Rendering

- 5.2.1.1 Render only visible tiles within the camera bounds to maintain performance
- 5.2.1.2 Enforce layer rendering order (map tiles -> events/objects -> player sprite)
- 5.2.1.3 Ensure correct tile-to-screen coordinate mapping at different zoom levels
- 5.2.1.4 Validate rendering correctness at map edges and during camera movement

5.2.2 Camera Follow Behaviour

- 5.2.2.1 Implement camera positioning to follow player movement while respecting map bounds
- 5.2.2.2 Ensure camera movement remains smooth and does not jitter during collision contact
- 5.2.2.3 Maintain consistent visible area calculations for viewport rendering
- 5.2.2.4 Validate camera behaviour on small maps where viewport exceeds map size

5.3 Player Movement, Collision, and Event Interaction

5.3.1 Input Handling

- 5.3.1.1 Implement keyboard input tracking for movement directions
- 5.3.1.2 Prevent inconsistent movement behaviour during rapid key presses
- 5.3.1.3 Ensure input is processed within fixed timestep updates for determinism
- 5.3.1.4 Validate movement speed remains consistent regardless of framerate

5.3.2 Collision Detection

- 5.3.2.1 Implement grid-based collision checks against the collision layer
- 5.3.2.2 Prevent entering blocked tiles and prevent clipping across tile boundaries
- 5.3.2.3 Ensure collision checks align exactly to editor-defined collision cells
- 5.3.2.4 Validate edge-case interactions at map borders and corners

5.3.3 Event Trigger Behaviour

- 5.3.3.1 Trigger events based on player position interacting with event coordinates
- 5.3.3.2 Ensure event activation conditions remain consistent and do not double-trigger unintentionally
- 5.3.3.3 Provide a clear runtime response to event activation (e.g. debug output/UI indicator)
- 5.3.3.4 Validate event persistence (events saved in editor load identically in runtime)

Objective 6: Performance, Stability, and Quality Assurance

6.1 Performance Validation Targets

6.1.1 Editor Performance

- 6.1.1.1 Maintain responsive painting behaviour on maps $\geq 100 \times 100$ tiles

- 6.1.1.2 Ensure viewport rendering prevents lag spikes during camera movement
- 6.1.1.3 Validate no progressive slowdown during extended editing sections
- 6.1.1.4 Validate tileset loading does not lock the UI unnecessarily

6.1.2 Runtime Performance

- 6.1.2.1 Maintain approximately 60 FPS on target specification hardware
- 6.1.2.2 Ensure camera and collision updates remain stable under continuous movement
- 6.1.2.3 Validate map load times remain within acceptable limits for typical map sizes
- 6.1.2.4 Ensure performance does not degrade due to avoidable allocations or resource leaks

6.2 Reliability and Data Integrity

6.2.1 File Integrity

- 6.2.1.1 Ensure save/load round-trip does not alter map structure unexpectedly
- 6.2.1.2 Ensure invalid files fail safely and do not crash editor or runtime
- 6.2.1.3 Validate autosave does not corrupt map files during frequent edits
- 6.2.1.4 Ensure relative tileset paths preserve portability across devices

6.2.2 Stability

- 6.2.2.1 Validate that editor and runtime remain stable during repeated open/edit/save cycles
- 6.2.2.2 Validate that undo/redo cannot produce corrupted in-memory state
- 6.2.2.3 Validate that error handling paths are exercised and predictable
- 6.2.2.4 Maintain clear logs/messages for debugging during development and testing

Objective 7: Testing and Verification

7.1 Functional Test Planning

7.1.1 Core Workflow Test Cases

- 7.1.1.1 Create tests for map creation, tile painting, collision editing, event placement, and saving
- 7.1.1.2 Create tests for map loading, tileset loading, and schema validation
- 7.1.1.3 Create tests for undo/redo correctness across mixed edit sequences
- 7.1.1.4 Create tests for autosave rule enforcement (new map vs saved map vs sample map)

7.1.2 Runtime Verification

- 7.1.2.1 Create tests for runtime map load correctness matching editor output
- 7.1.2.2 Create tests for collision behaviour accuracy against collision layer definitions
- 7.1.2.3 Create tests for camera follow behaviour and edge-bound constraints
- 7.1.2.4 Create tests for event trigger activation conditions and persistence fidelity

7.2 Performance and Stability Testing

7.2.1 Performance Benchmarks

- 7.2.1.1 Measure runtime FPS stability on target specification hardware profile

- 7.2.1.2 Measure map load time for 100x100 and larger maps
- 7.2.1.3 Measure editor responsiveness during drag painting across large regions
- 7.2.1.4 Record memory behaviour across extended sessions to detect leaks

7.2.2 Failure and Recovery Testing

- 7.2.2.1 Test corrupted JSON input handling and ensure graceful failure
- 7.2.2.2 Test missing tileset reference handling
- 7.2.2.3 Test rapid undo/redo stress conditions
- 7.2.2.4 Test autosave interruption scenarios (e.g. sudden close) and ensure integrity remains preserved

Objective 8: Documentation and Evaluation Evidence

8.1 Technical Documentation

8.1.1 Documentation of Code

- 8.1.1.1 Implement structured inline documentation for public classes and key systems
- 8.1.1.2 Produce a clear explanation of architecture boundaries (Editor vs Runtime vs Shared)
- 8.1.1.3 Document design rationale for viewport-only rendering, layer separation, and command-based history
- 8.1.1.4 Maintain records of key implementation decisions and trade-offs to support evaluation

8.1.2 Documentation of the System

- 8.1.2.1 Create UML or component diagrams describing subsystem interactions
- 8.1.2.2 Document the JSON schema structure including layer definitions and version field
- 8.1.2.3 Provide diagrams o explanations of runtime loop and camera/viewport calculation
- 8.1.2.4 Document collision and event data representations with examples

8.2 Evaluation Evidence Compilation

8.2.1 Evidence Collection and Organisation

- 8.2.1.1 Capture annotated screenshots of editor features (tile, collision, event modes) as implementation evidence
- 8.2.1.2 Capture annotated screenshots of runtime playback to demonstrate editor-runtime cohesion
- 8.2.1.3 Store representative map JSON examples showing schema structure, versioning, and layer separation
- 8.2.1.4 Maintain a structured folder of test outputs, benchmark results, and failure/retest logs

8.2.2 Linking Evidence to Criteria

- 8.2.2.1 Map test cases directly to functional requirements IDs (FR1-FR12)

- 8.2.2.2 Map benchmarks (FPS/load times) directly to non-functional requirements targets
- 8.2.2.3 Record notable bug fixes as “issue -> fix -> retest” entries for evaluation credibility
- 8.2.2.4 Ensure each success criterion claim is backed by at least one concrete piece of evidence

8.3 Final Self-Evaluation Framework

- 8.3.1.1 Create an evaluation comparing success criteria against implemented outcomes
- 8.3.1.2 Record measurable benchmarks (load times, FPS stability, responsiveness on large maps)
- 8.3.1.3 Present scope boundaries as deliberate design constraints (clarity and performance focus)
- 8.3.1.4 Present testing evidence using structure test cases and retest outcomes

Success Criteria

Functional Completeness Metrics

Core Feature Implementation

- **Map Editing Completion** – Creation, editing, saving, and loading of maps must function with no structural data corruption. Tile, collision, and event layers must persist accurately across save/load cycles.
- **Editor-Runtime Cohesion** – Maps created in the editor must load into the runtime without requiring modification or manual conversion. Runtime behaviour (collision, event triggers, rendering order) must reflect editor configuration exactly.
- **Undo/Redo Reliability** – All reversible editing actions (tile paint, collision toggle, event placement/removal) must be fully reversible with deterministic behaviour and no divergence after repeated undo/redo sequences.
- **Autosave Rule Compliance** – Autosave must activate only under defined conditions (post-manual-save for new maps), and must never create unintended file duplication or overwrite unintended files.

Performance Benchmark Compliance

Runtime Performance Standards

- **Frame Rate Stability** – Runtime must maintain approximately 60 FPS ($\pm 5\%$ variance) on minimum target hardware (Intel UHD 620, 8GB RAM) during standard gameplay movement on maps $\geq 100 \times 100$ tiles.
- **Loading Time Targets** – Maps of size 100×100 must load within 2 seconds under standard test conditions.

- **Rendering Scalability** – Viewport-only rendering must prevent full-map redraw overhead, ensuring stable performance regardless of total map size beyond visible region.
- **Editor Responsiveness** – Drag-to-paint operations must remain visually responsive with no perceptible input lag on large maps.

User Experience Success Indicators

Learnability and Accessibility

- **First-Time Workflow Completion** – A user unfamiliar with the system must be able to:
Create a map -> Paint tiles -> Add collision -> Place an event -> Save -> Load in runtime
within a single session without external intervention.
- **Task Completion Rate** – At least 85% successful completion rate for core workflow tasks during structured testing sessions.
- **Error Recovery Success** – Users must be able to recover from common issues (e.g., invalid tileset path, accidental edits) using undo or error prompts without external debugging.

Workflow Efficiency Indicators

- **Reduced Cognitive Switching** – Editing of tiles, collision, and events must occur within a single unified viewport without switching to external tools.
- **SUS Target** – System Usability Scale score target $\geq 80\%$ across testing participants.
- **Confidence Growth Indicator** – Users should report increased confidence after completing a full create-edit-play cycle.

Technical Quality Measures

Code Quality and Maintainability

- **Architecture Separation Compliance** – Clear separation must exist between:
 - Editor logic
 - Runtime logic
 - Shared data models
 No circular dependencies between major components
- **Structured Namespace Organisation** – Namespaces must reflect architectural boundaries and maintain clarity of responsibility.
- **Consistent Coding Standards** – No critical compilation warnings in Release build configuration.

- **Documented Design Rationale** – Key architectural decisions (viewport rendering, layered data model, command pattern) must be documented for evaluation.

Reliability and Stability

Runtime Stability

- **Crash Frequency** – Fewer than 2 crashes per 100 hours of structured testing.
- **Deterministic Behaviour** – Movement speed, collision resolution, and event triggering must behave consistently across repeated runs on identical maps.

Data Integrity

- **Zero Data Corruption in Normal Usage** – Save -> Close -> Reload cycle must not alter tile, collision, or event data.
- **Validation Coverage** – Invalid JSON structures must fail gracefully without crashing the application.
- **Autosave Safety** – Autosave must not produce corrupted partial writes during frequent edits.

Long-Term Viability Indicators

Scalability

- **Map Size Handling** – Maps $\geq 100 \times 100$ tiles must remain performant in both editor and runtime environments.
- **Rendering Efficiency** – Performance must scale based on visible viewport rather than total map size.
- **Extensibility Foundation** – The presence of schema versioning and modular architecture must allow new layer types or map properties to be added without restructuring core systems.

Documentation and Evidence

- **Technical Documentation Completion** – Core architectural systems, JSON schema, runtime loop, and rendering logic must be clearly documented.
- **Testing Evidence Availability** – All major functional and performance claims must be supported by structured test cases and recorded results.
- **Evaluation Traceability** – Each functional and non-functional requirement must map to at least one objective and one test case.

Traceability Matrix

Project Focus Area	Supporting Objectives	Linked Functional Requirements
--------------------	-----------------------	--------------------------------

System Architecture	Obj 1.1, 1.3	FR5, FR8, FR11, FR12
Map Data & Persistence	Obj 2.1, 2.2, 2.3	FR1, FR5, FR9, FR11
Editor Interface	Obj 3.1, 3.2	FR1, FR2, FR3, FR4, FR8
Map Rendering Engine (Editor)	Obj 4.1	FR2, FR6, FR12
Editing History System	Obj 4.2	FR8
Runtime Engine	Obj 5.1, 5.2	FR6, FR7, FR10, FR12
Collision & Events	Obj 5.3	FR3, FR4, FR7
Performance Validation	Obj 6.1	FR10
Reliability & Integrity	Obj 6.2	FR5, FR8, FR9
Testing & Verification	Obj 7.1, 7.2	All FRs
Documentation & Evidence	Obj 8.1, 8.2	All FRs

Project Vitriol – NEA Design

Image Download Links are provided below each image in case of load failure

Design Overview and Objectives

This project consists of two user-facing applications which share a common data model: a WPF-based map editor used to create, modify, and persist tile-based maps as structured JSON files, and a WPF runtime application responsible for loading a selected map and rendering it within a scrolling viewport where the player can navigate and trigger events.

A central architectural objective was to maintain strict stability of the map data format while allowing the editor and runtime to function independently. To achieve this, shared Data Transfer Objects (DTOs) and serialisation logic were designed first, establishing a unified schema before implementing editor and runtime behaviours. The editor and runtime are therefore treated as separate operational layers that consume the same map model, rather than directly depending on one another.

An alternative approach would have been to combine editor and runtime functionality into a single monolithic application. However, this was rejected due to increased

coupling risks, reduced maintainability, and difficulty isolating rendering or logic faults. The layered separation adopted here ensures that issues in one subsystem do not propagate across the entire project.

Core Functional Objectives

- **Map Creation and Loading** – Allow users to create and load maps from a structured project directory (Vitriol.Projects).
- **Structured JSON Schema** – Map files must follow a predictable, versioned JSON schema to ensure forward compatibility and structural clarity.
- **Initial Map Generation** – New maps are initialised with a blank tile grid (default 32×32 tiles), with user-defined dimensions applied at creation.
- **Tileset Referencing** – Maps must reference tilesets through relative file paths. Tilesets can be loaded and unloaded in memory without invalidating existing tile indices, preserving map integrity.
- **Layer-Based Editing** – The editor must support:
 - Tile painting
 - Collision painting
 - Event placement
 - Undo and redo functionality (Ctrl+Z / Ctrl+Y)
- **Controlled Saving Behaviour** – The sample map supports continuous autosave. Newly created maps require at least one manual save before autosave is enabled. Manual save remains available at all times.
- **Runtime Playback Compatibility** – The runtime must load any valid map file and render only the visible region of the map. Viewport-based rendering must be implemented in both the editor and runtime to ensure consistent behaviour and performance.

Non-Functional Objectives

- **Performance** – Viewport-only rendering must be implemented to prevent unnecessary full-map redraw operations. Rendering must scale relative to visible area rather than total map size, ensuring responsiveness on minimum target hardware.
- **Reliability** – Validation and defensive checks must be applied when loading map files and tilesets. Invalid states must be detected before runtime execution.
- **Maintainability** – There must be clear architectural separation between:
 - Data model (Shared DTOs)
 - Editor logic
 - Runtime logic

This separation reduces coupling, supports scalability, and prevents cascading failures across subsystems.

- **Usability** – The interface must remain minimal and intuitive. Editing workflows should reflect common conventions used in established map editors, reducing the learning curve for new users.

Data Design: ERD Progression

The data model was designed in clearly defined layers:

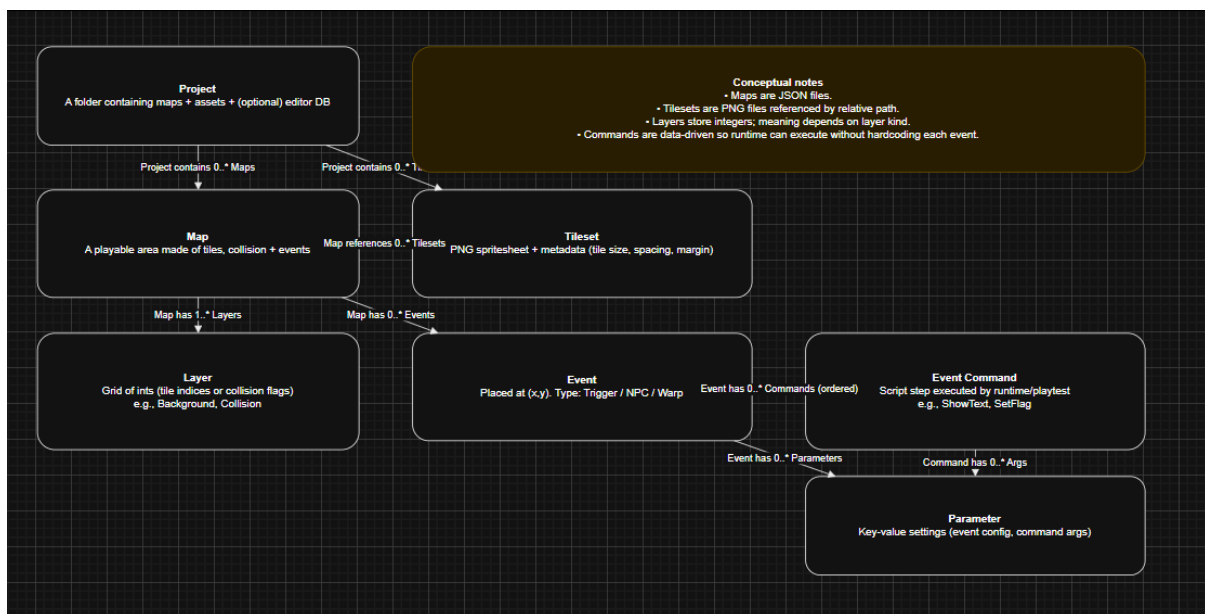
Conceptual -> Logical -> Physical (JSON) -> Persistence

This progression separates domain meaning from implementation detail. The conceptual model defines *what* exists in the system, the logical model defines *how* it is structured and constrained, the physical model defines *how it is encoded*, and the persistence view defines *how it is stored and accessed*.

Although map data is ultimately stored as JSON files, ERD-style modelling was still used during design to ensure structural clarity, scalability, and validation discipline. This layered modelling approach prevents implementation details from distorting domain reasoning.

Conceptual ERD: Domain Entities

Figure 01: Conceptual ERD – Core Entities and Relationships



[Design Section\Design Section Screenshots\image1.png](#)

At the conceptual level, the focus is purely on domain meaning. Storage format and technical encoding are intentionally excluded.

This level answers four essential domain questions:

- How large is a map and how is it identified?

- Which layers exist and what type of data does each store?
- Which tilesets provide the visual resources used by the map?
- Which events exist on the grid, and what behaviour occurs when they trigger?

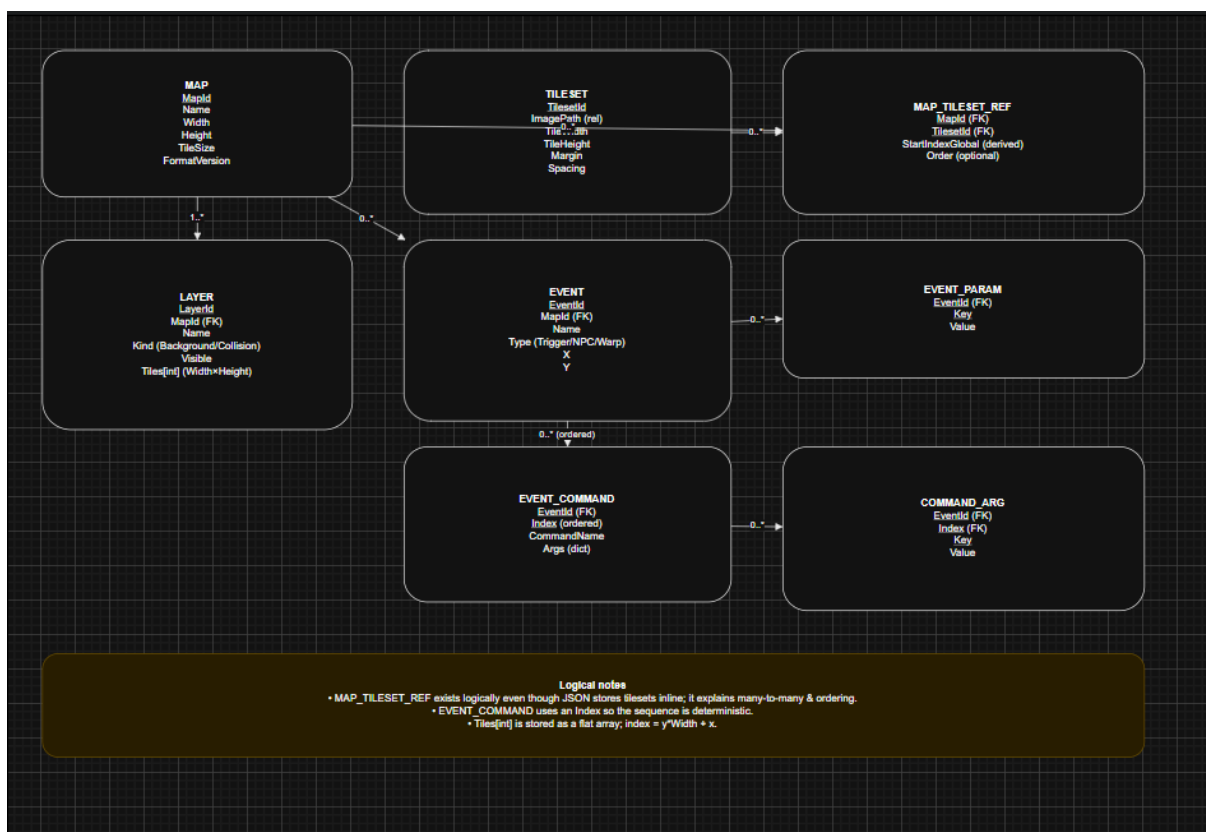
The primary conceptual entities are:

- **Map**
- **Layer**
- **Tileset**
- **Event**

By defining these entities independently of storage concerns, the design ensures that later implementation decisions (such as JSON structure) do not compromise conceptual clarity.

Logical ERD: Attributes and Keys

Figure 02: Logical ERD – Attributes, Keys, and Constraints



[Design Section\Design Section Screenshots\image2.png](#)

The logical stage introduces identifiers, attributes, and constraints aligned with runtime behaviour.

At this level:

- **Map** includes:
 - MapId (file-friendly identifier)
 - Name (display-friendly label)
 - Grid dimensions (Width, Height)
- **Layer** includes:
 - LayerId
 - Kind (e.g., background, collision)
 - Tile array sized Width × Height
- **TilesetRef** includes:
 - TilesetId
 - ImagePath
 - Layout metadata (TileSize, Margin, Spacing)
- **MapEvent** includes:
 - EventId (GUID)
 - Type (trigger, NPC, warp)
 - Position (X, Y)
 - Event-specific data

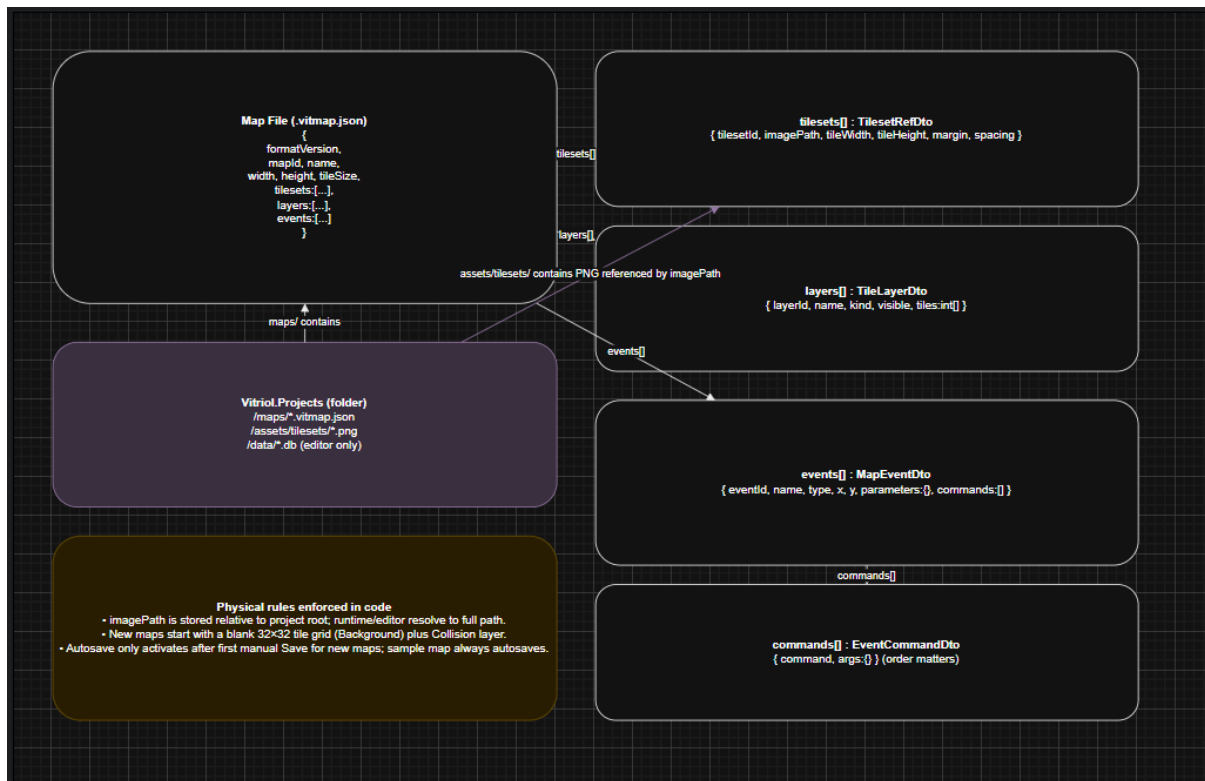
This logical definition introduces structural constraints that mirror runtime expectations. For example, tile arrays must align with declared map dimensions, and event positions must fall within map bounds.

Because the implementation uses C# and JSON, tile storage was designed using a **1D array rather than a 2D array**. This approach simplifies serialisation, reduces structural nesting in JSON, and enables faster index calculation using $\text{index} = y * \text{width} + x$. This decision improves both performance and structural clarity without affecting conceptual correctness.

An alternative 2D array representation was considered for readability; however, it was rejected due to increased serialisation complexity and reduced indexing efficiency.

Physical Design: JSON Schema Mapping

Figure 03: Physical Mapping – DTOs -> JSON File Structure



[Design Section\Design Section Screenshots\image3.png](#)

The physical model maps logical entities directly onto DTO records shared between the editor and runtime.

Each logical entity corresponds to a structured JSON object or array:

- Map metadata -> root JSON object
- Tile layer -> integer array
- Collision layer -> boolean or integer array
- Events -> list of event objects

Field names were kept consistent with DTO property names to minimise transformation logic and reduce deserialisation risk.

Over-normalisation was intentionally avoided. Since maps are loaded as complete documents rather than queried incrementally (as in relational systems), embedding related data within a single JSON file reduces dependency complexity and improves portability.

Key design decisions at this stage include:

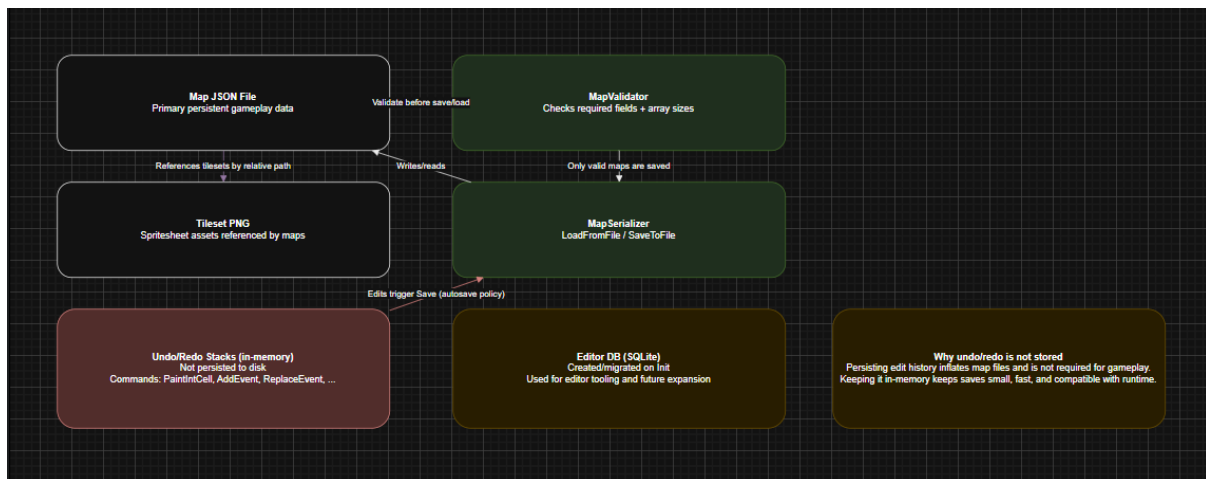
- **FormatVersion Field** – Included to support future schema evolution without breaking compatibility.

- **Events Stored as a List** – Events are sparse relative to tile grids. Storing them as a list avoids allocating large, mostly empty structures.
- **Tileset References Stored per Map** – Each map maintains its own tileset reference to ensure portability between projects, provided required assets are available.

These decisions prioritise portability, scalability, and runtime-editor cohesion.

Persistence View: JSON Files + Project Metadata

Figure 04: Persistence ERD – File Storage and DB Responsibilities



Design Section\Design Section Screenshots\image4.png

The persistence layer distinguishes between map storage and project-level metadata.

Two forms of persistence exist within the system:

- **Map Persistence** - Maps are stored as JSON files under: `Vitriol.Project/Maps/`. These files are written by the editor and read directly by the runtime.
- **Project Metadata Store** - The editor initialises lightweight project-level metadata to manage project configuration and structure. This is intentionally separate from individual map files.

This separation is deliberate.

Map files must remain:

- Easy to share
- Version-control friendly
- Self-contained
- Independent of editor internals

By isolating project-level concerns from map data, new editor-level features can be introduced without requiring structural changes to map files.

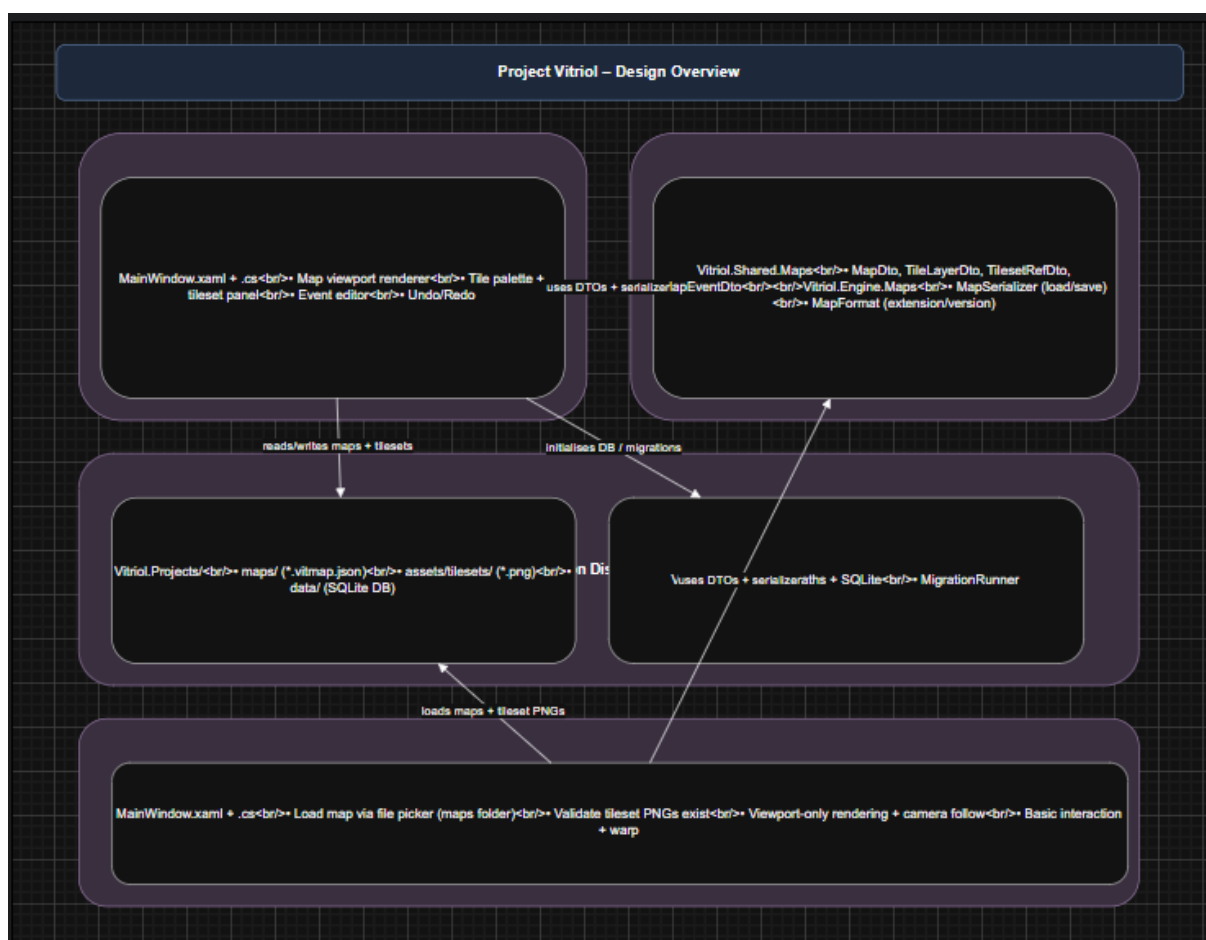
System Architecture: Project Structure and Layered Responsibilities

The system architecture is designed around strict separation of concerns. Shared map format definitions, DTOs, serialisation logic, and validation utilities are contained within shared assemblies. The editor and runtime applications depend on these shared components but maintain their own application-specific UI and behaviour layers.

This separation ensures consistency of data interpretation while preserving independence of execution environments.

High Level Architecture

Figure 05: Architecture Overview – Shared Libraries + Editor + Runtime



[Design Section\Design Section Screenshots\image5.png](#)

This diagram illustrates how both applications consume the same shared DTO definitions, serialisers, and validation logic.

Key architectural principles demonstrated here include:

- **Single Source of Truth** – There is one map schema definition.
- **Single Serialisation Pipeline** – Both applications use the same serialiser implementation.
- **Centralised Validation Logic** – Structural validation is not duplicated across applications.

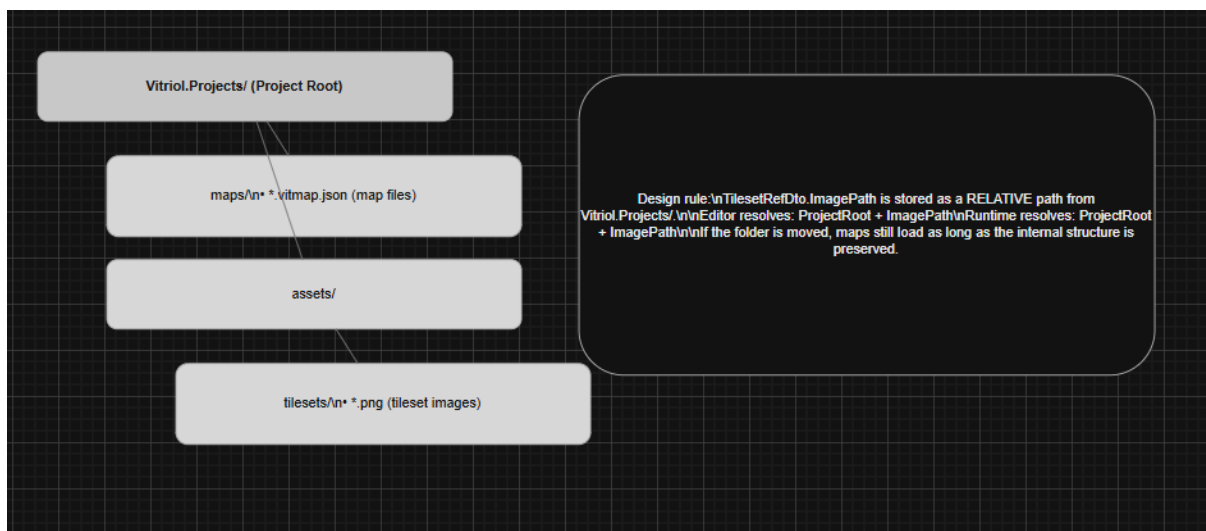
This eliminates divergence between editor and runtime behaviour. Consistency is enforced by shared architecture rather than informal development convention.

An alternative approach would have been to duplicate DTO definitions or allow the runtime to define its own interpretation of map data. This was rejected because it introduces schema drift risk and significantly increases maintenance complexity.

By enforcing dependency direction (Editor -> Shared, Runtime -> Shared), circular coupling is prevented. The runtime has no awareness of editor UI components, and the editor does not depend on runtime execution logic.

Project Folder Structure and Asset Conventions

Figure 06: Project Hierarchy – Vitriol.Projects Folder Layout



[Design Section\Design Section Screenshots\image6.png](#)

This diagram shows the standardised project directory layout:

- Vitriol.Projects/maps/ – Map JSON files
- Vitriol.Projects/assets/tilesets/ – Tileset PNG images

The folder structure is intentionally fixed to ensure both applications can deterministically compute file paths.

A key design rule is that tileset paths are stored as **relative paths from the project root**. This ensures:

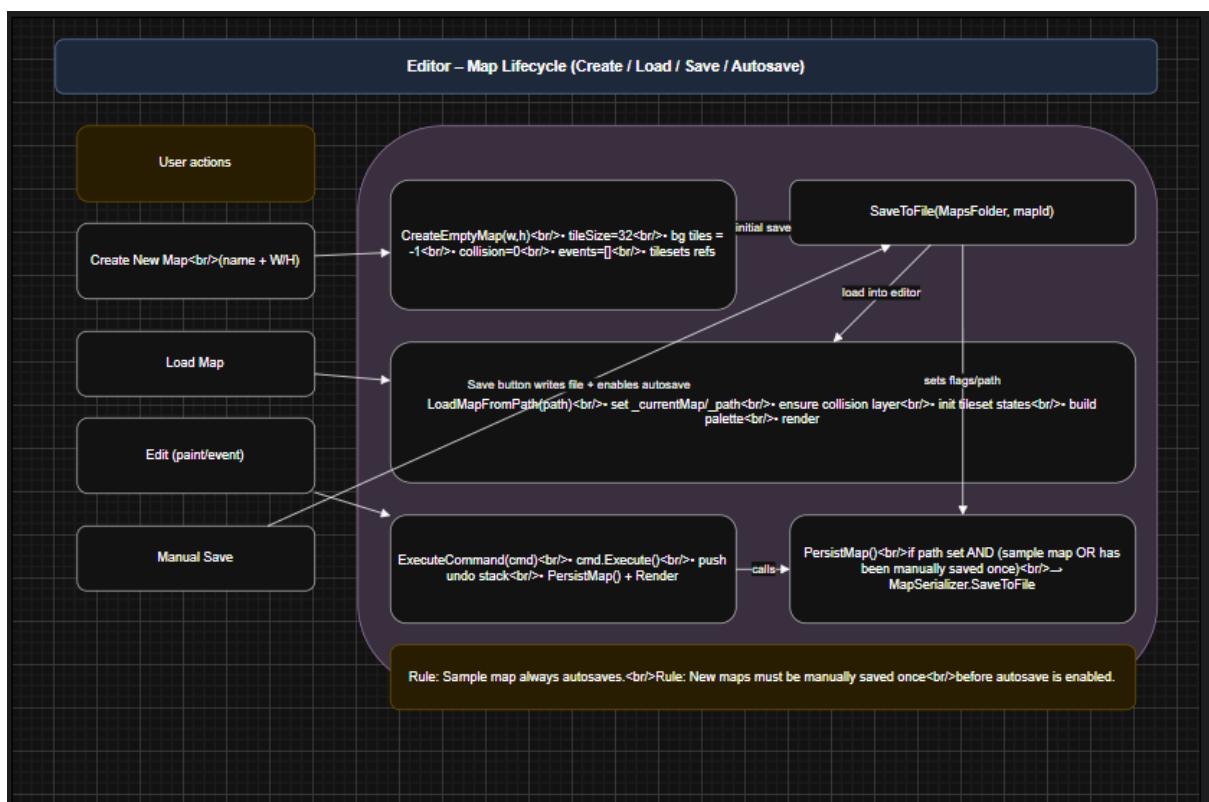
- Portability between machines
- Compatibility with version control systems
- Independence from absolute filesystem locations

As long as the internal folder structure remains consistent, maps remain valid when moved across devices.

Storing absolute paths was considered but rejected because it would tightly bind maps to a single machine's directory structure, significantly reducing portability and collaboration potential.

Editor Workflow Architecture

Figure 07: Editor Workflow – Create, Load, Save, Autosave Rules



[Design Section\Design Section Screenshots\image7.png](#)

This workflow diagram illustrates the lifecycle of a map within the editor:

- Creation
- Editing
- Manual save
- Autosave activation

The editor distinguishes between two operational contexts:

- **Sample Map** – Created for rapid experimentation and always autosaves. This supports immediate iteration during development and testing.
- **User-Created Maps** – Maps created via the “Create New Map” workflow must be manually saved at least once before autosave is enabled.

This design decision prevents:

- Accidental proliferation of unwanted files
- Unintended overwriting of unsaved work
- Loss of intentional filename control

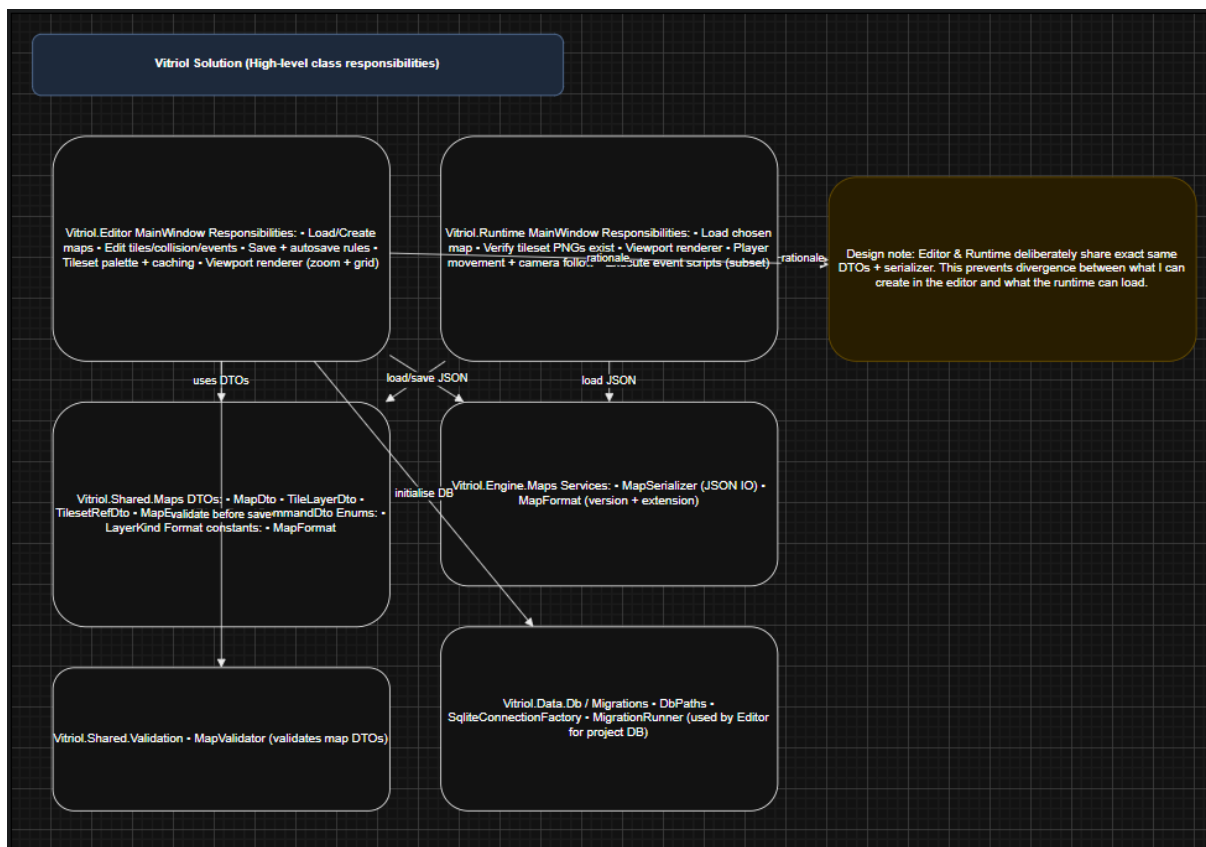
The requirement for an initial manual save ensures that each user map has a deliberate file identity before automated persistence begins.

This behaviour balances safety and efficiency: rapid testing remains frictionless via the sample map, while project structure remains controlled for user-defined maps.

Class Design: Engine, Editor, Runtime Separation

Class Overview: Shared DTOs and Services

Figure 08: Class Diagram – Shared Map Model, Serialisation, Validation



[Design Section\Design Section Screenshots\image8.png](#)

This diagram illustrates the shared map model, serialisation pipeline, and validation services consumed by both the editor and runtime applications.

A central design principle is that DTOs remain **passive data structures**. They contain no logic exclusive to either the editor or runtime. Their sole responsibility is representing structured data.

This constraint ensures:

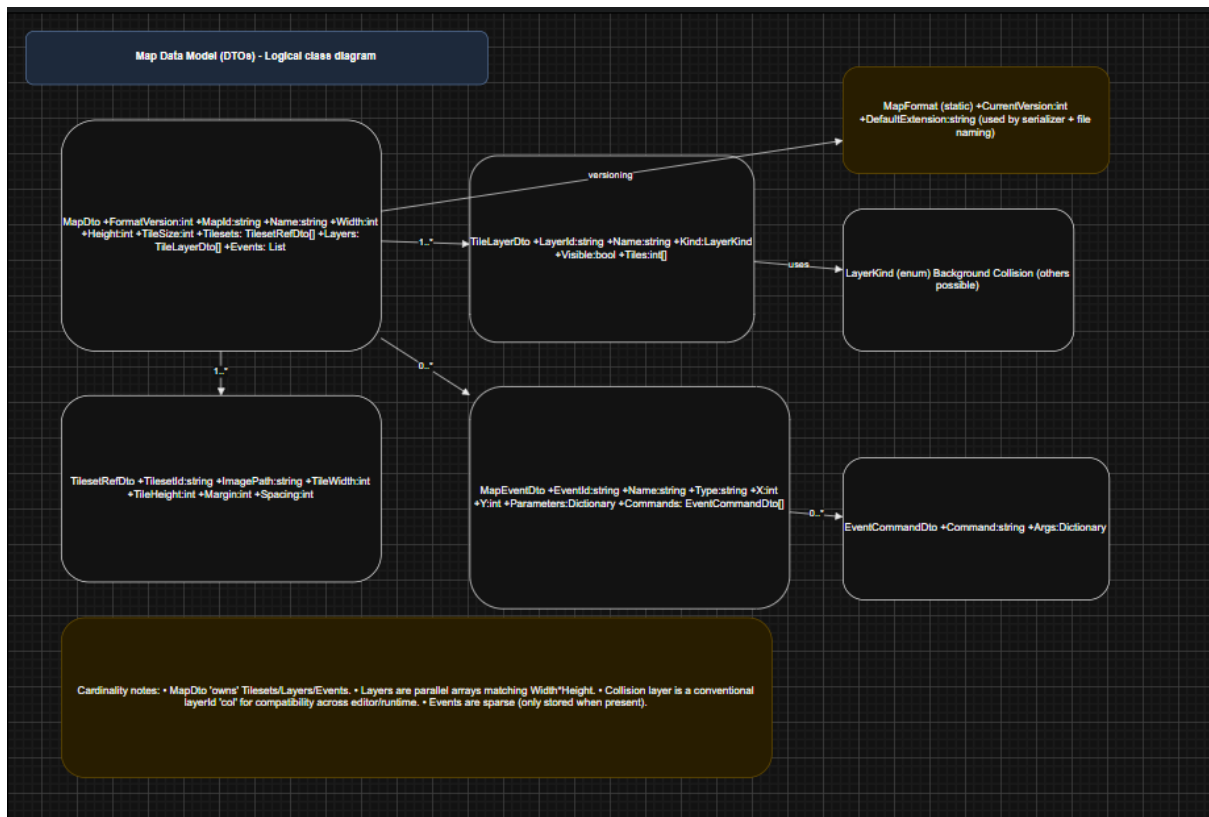
- Safe reuse across applications
- Predictable JSON serialisation behaviour
- Elimination of cross-layer logic leakage
- Reduced risk of circular dependencies

There is a single serialiser implementation and a single validation layer. Both applications consume these shared services rather than maintaining separate interpretations of the map format.

An alternative design would have embedded behaviour directly into DTO classes (for example, including rendering or validation logic within the model itself). This approach was rejected because it would tightly couple domain representation with application behaviour, reducing portability and increasing maintenance complexity.

Map Model and Layer + Event Composition

Figure 09: Class Diagram – MapDto, TileLayerDto, TilesetRefDto, MapEventDto



[Design Section\Design Section Screenshots\image9.png](#)

This diagram demonstrates how composition is prioritised within the map model.

The structure is intentionally hierarchical:

- MapDto contains:
 - Tile layers
 - Collision layer
 - Tileset reference
 - Event collection

Design details include:

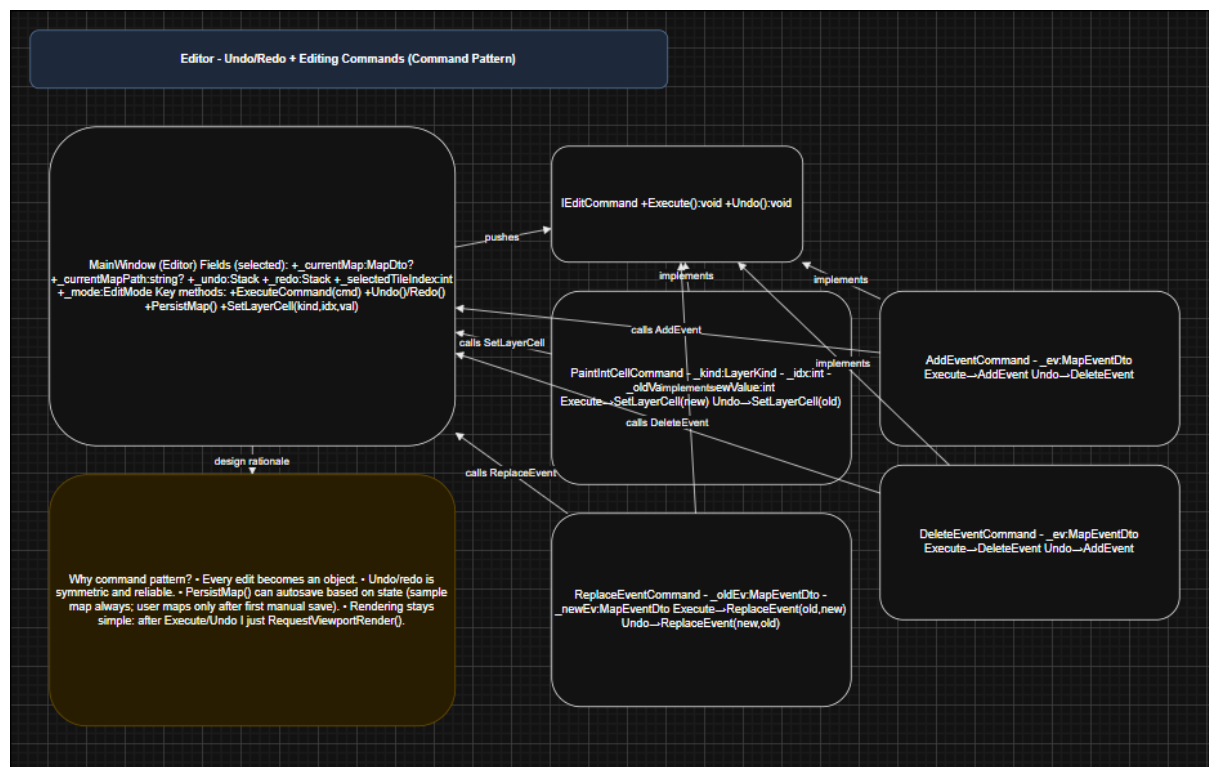
- **Layer Representation** – Layers are represented by TileLayerDto with a Kind property (e.g., visual or collision). This allows behaviour switching without introducing multiple subclass types, reducing class proliferation.
- **Event Extensibility** – Event scripts are represented as a list of EventCommandDto objects. Each command contains an argument dictionary, allowing new command types to be introduced without modifying the schema structure.

- **Null Normalisation on Load** – After deserialisation, the editor performs normalisation to ensure collections (events, commands, parameters) are never null. This simplifies UI logic by guaranteeing safe iteration.

Composition is preferred over inheritance throughout the model. This avoids unnecessary class hierarchies and keeps structural relationships explicit and predictable.

Editor Editing Model: Commands, Undo/Redo, Persistence

Figure 10: Class Diagram – Editor Commands and Undo/Redo Stack



[Design Section\Design Section Screenshots\image10.png](#)

This diagram shows the command-based editing system built around an `IEditCommand` interface, with concrete implementations such as:

- `PaintIntCellCommand`
- `AddEventCommand`
- `DeleteEventCommand`
- `ReplaceEventCommand`

Undo/Redo is implemented using the Command Pattern. This design was deliberately chosen because it:

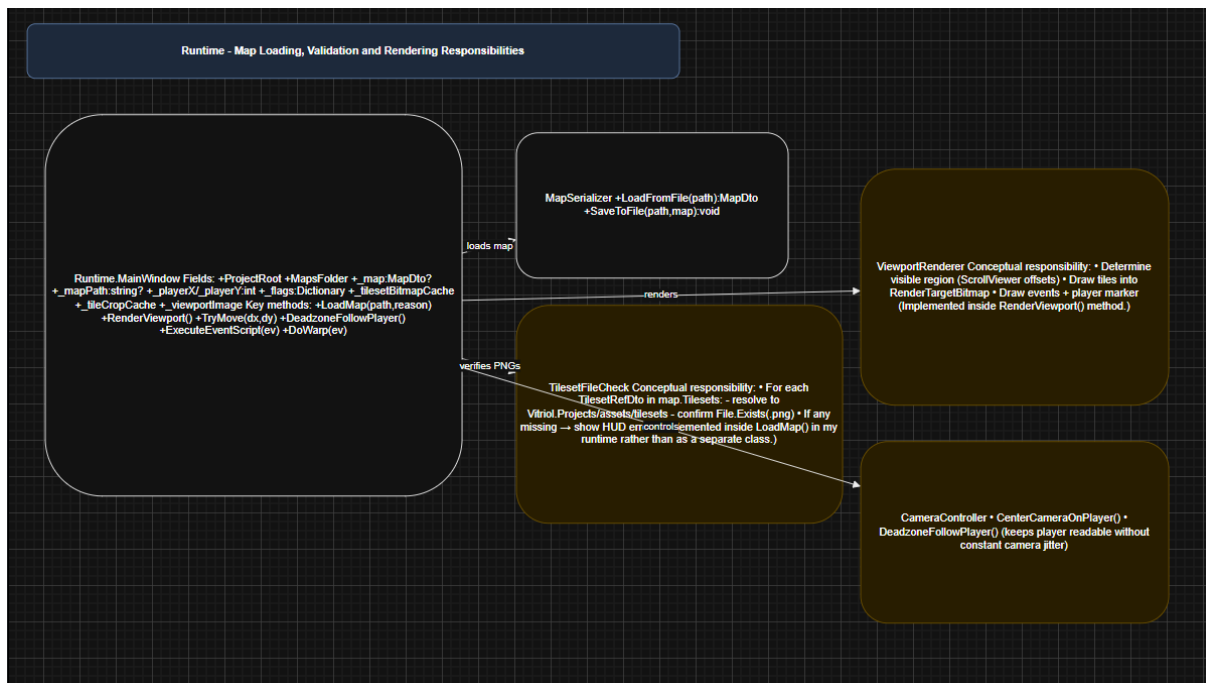
- Captures state changes in a reversible and explicit form
- Keeps UI code lightweight — the UI requests an action, and the command encapsulates execution logic
- Centralises modification behaviour, ensuring consistency across editing operations
- Simplifies autosave and persistence triggers by treating all edits uniformly

An alternative approach would have been to directly mutate the map model and manually track previous states. This was rejected because it complicates reversibility, increases coupling between UI and data logic, and risks inconsistent undo behaviour.

The command-based architecture ensures deterministic and maintainable editing state transitions.

Runtime Classes: Load, Render, Input, Camera

Figure 11: Class Diagram – Runtime Responsibilities (Loading, Rendering, Camera Follow)



[Design Section\Design Section Screenshots\image11.png](#)

This diagram illustrates separation within the runtime layer:

- Map loading logic
- Rendering subsystem

- Input handling
- Camera control

In the runtime architecture:

- **Map Loading** is isolated from rendering and input.
- **Rendering** consumes loaded data but does not modify it.
- **Input Handling** updates player state but does not directly manage rendering.
- **Camera Logic** calculates viewport bounds independently of input processing.

This separation prevents unnecessary asset reloads or cache rebuilding during gameplay. For example, player movement updates position state without triggering full asset reinitialisation.

An alternative tightly coupled runtime (where rendering and loading are intertwined) would increase the risk of performance instability and complicate debugging.

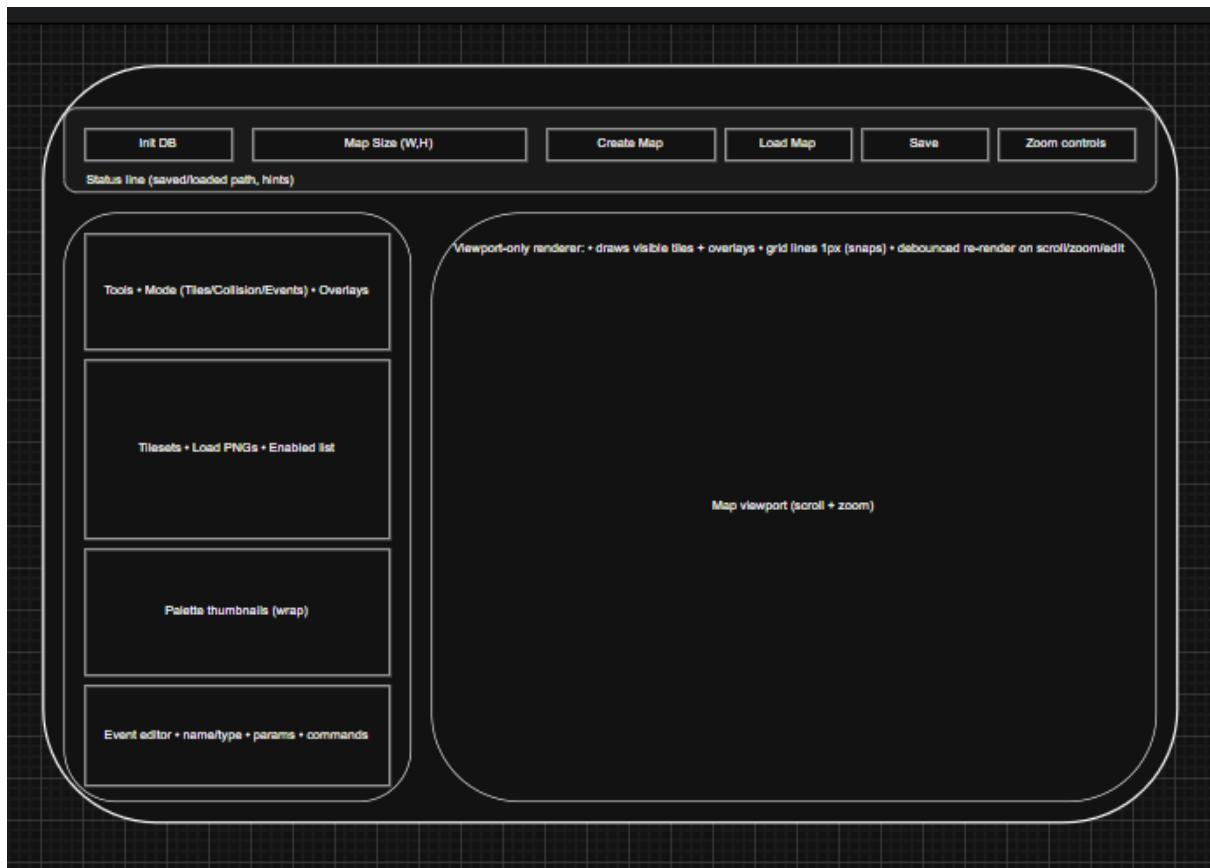
By separating these concerns, the runtime maintains:

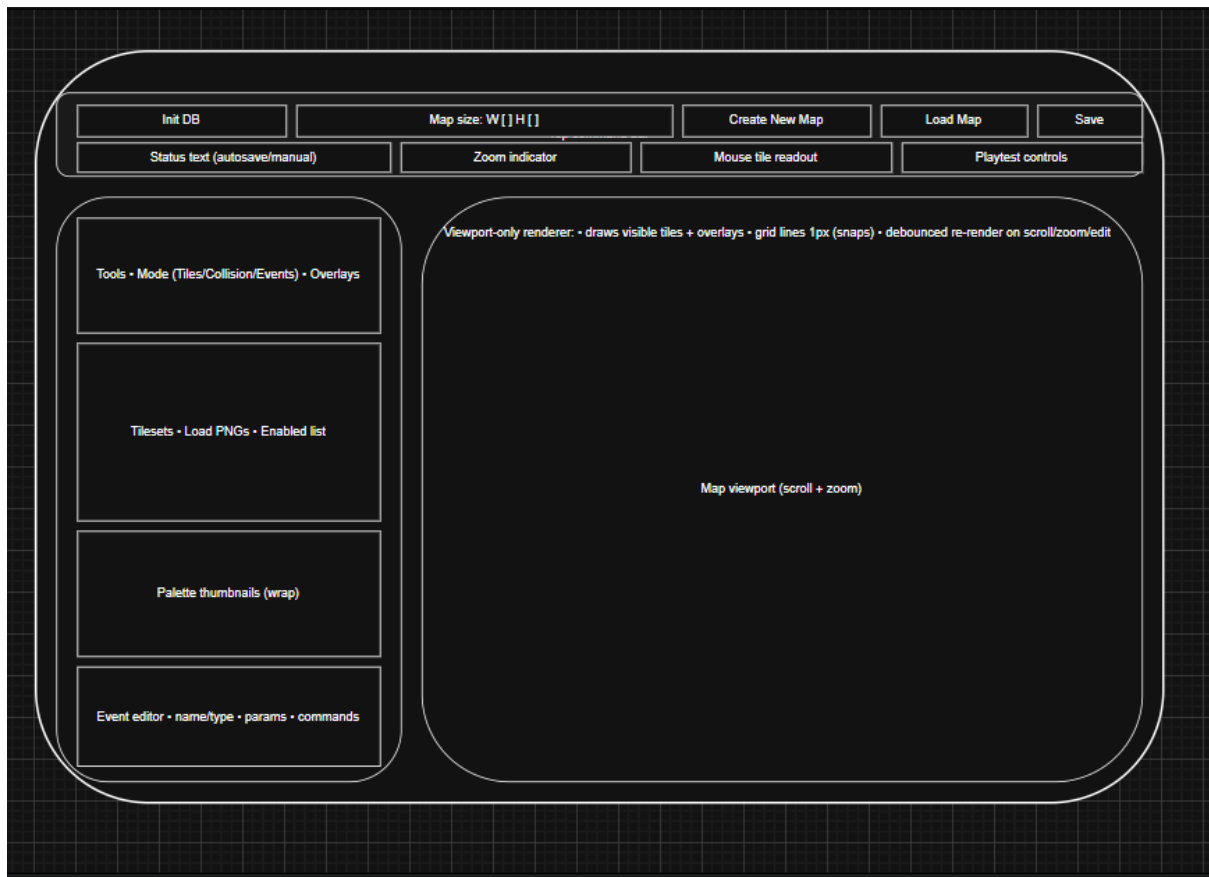
- Deterministic update behaviour
- Efficient viewport-only rendering
- Clear control flow ownership

User Interface Design: Iterative UI Designs

Editor UI Iterations

Figures 12, 13, 14: Editor UI Iteration Set – A -> C





[Design Section\Design Section Screenshots\image12.png](#)

[Design Section\Design Section Screenshots\image13.png](#)

[Design Section\Design Section Screenshots\image14.png](#)

These iterations demonstrate progressive refinement of the editor layout. Each version adjusted spatial hierarchy and control grouping to prioritise map editing efficiency.

The evolution moved toward a stable four-region layout:

- Top toolbar
- Left tools sidebar
- Central map viewport
- Expandable lower event editor panel

Main UI Decisions and Rationale

- **Viewport Priority** – A wide central viewport was prioritised to maximise spatial awareness during map editing. Map creation is inherently spatial; therefore, screen real estate was allocated primarily to the grid display rather than auxiliary controls.

- **Left-Aligned Tool Sidebar** – Editing tools were placed on the left to align with conventions used in established creative software (e.g., image editors and tile-based level editors). This reduces cognitive friction for users already familiar with such interfaces.
- **Top Toolbar Grouping** – The top toolbar groups actions into:
 - Project-level actions (create, load)
 - Map-level actions (save)
 - View-level actions (zoom controls)

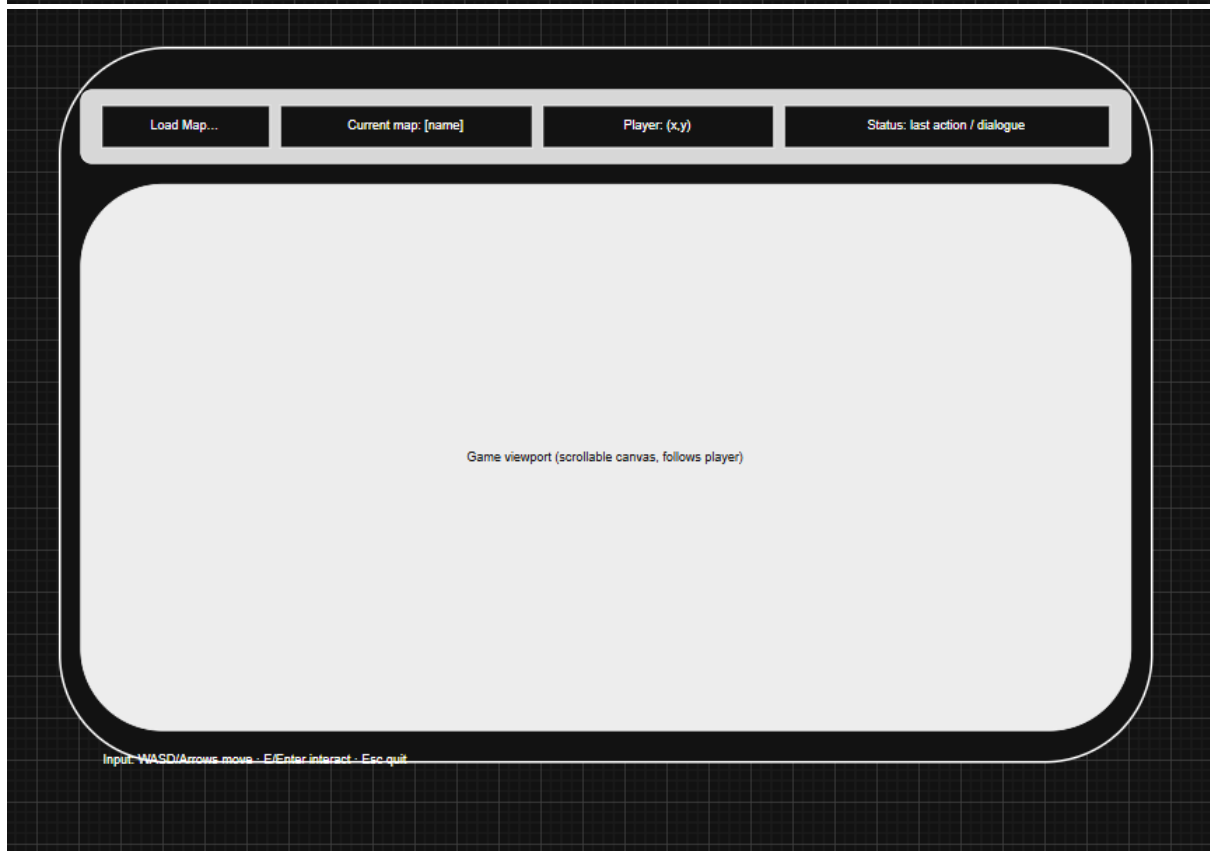
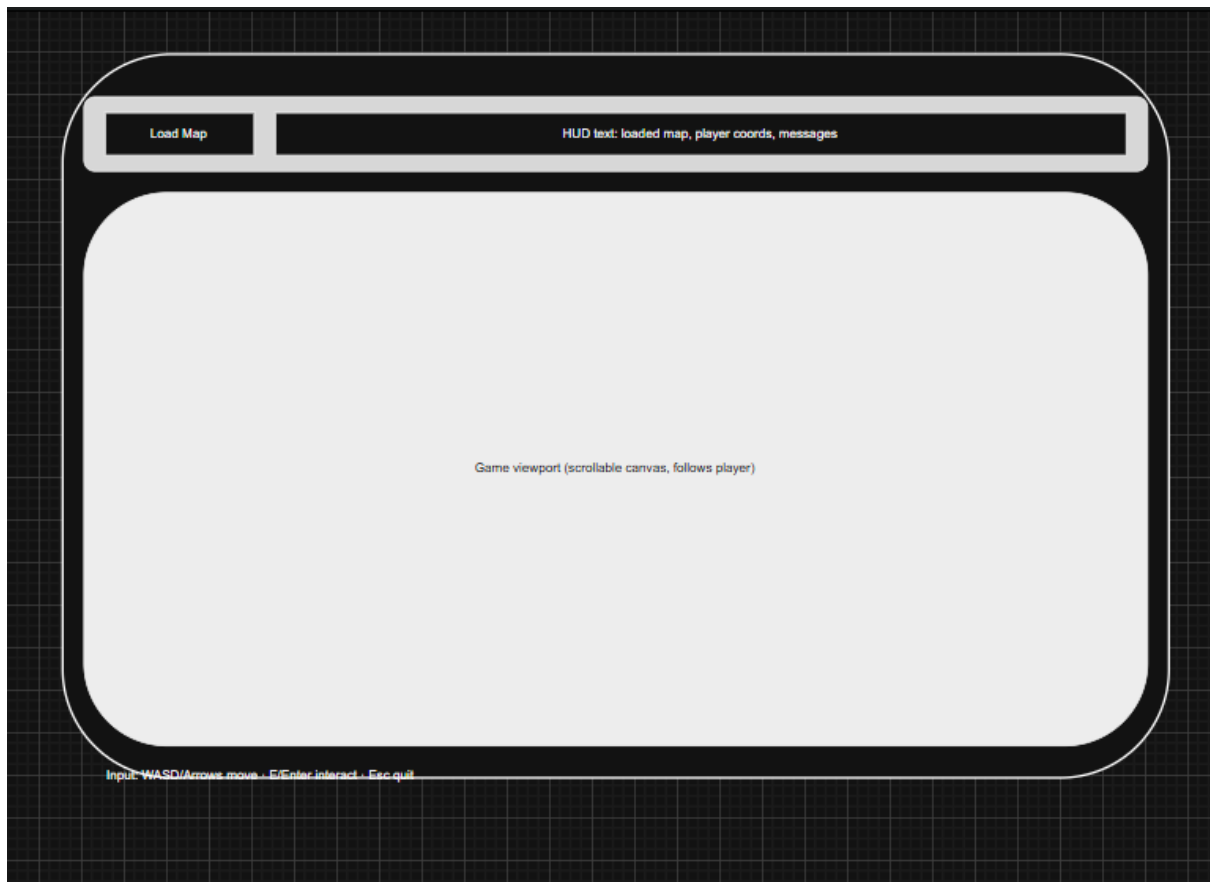
This grouping reflects task hierarchy rather than technical implementation, improving discoverability.

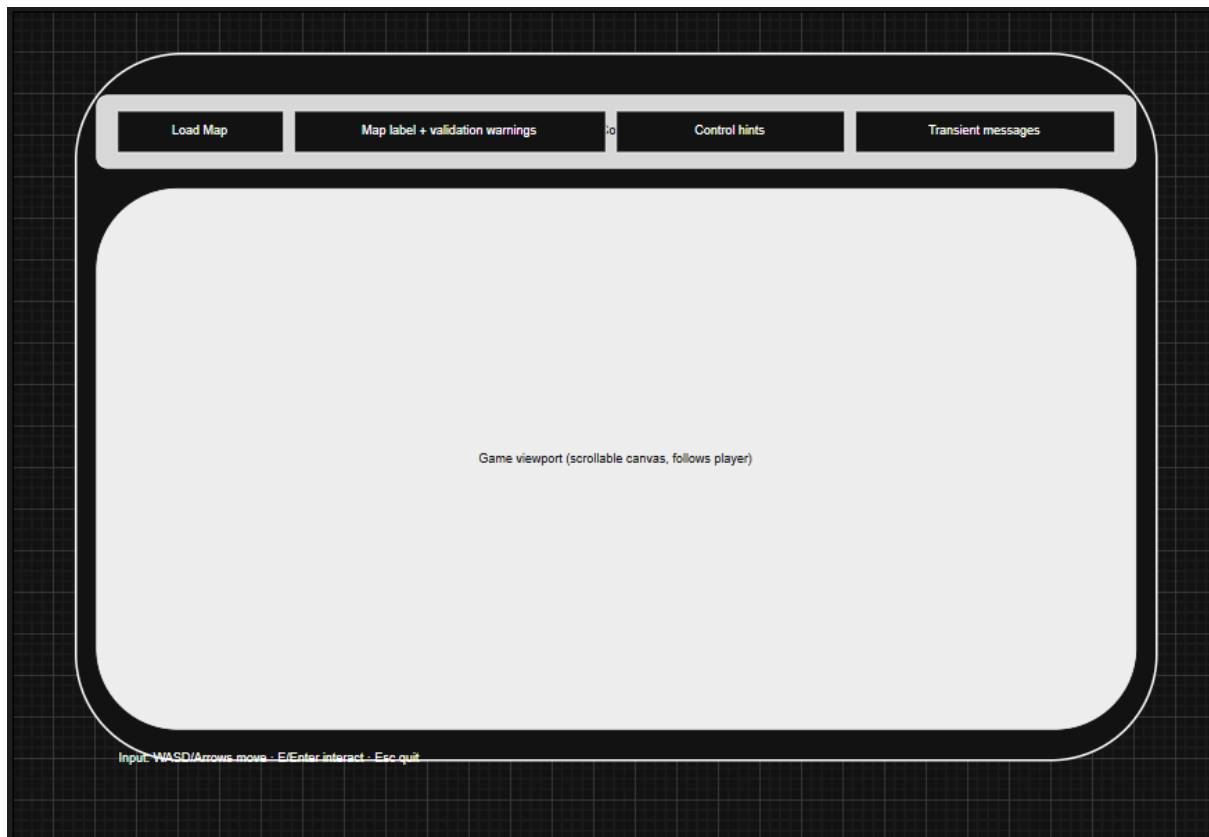
- **Event Editor Placement** – The event editor is positioned below the tile palette because event configuration is a secondary workflow compared to tile painting. Tile editing occurs continuously during map construction, whereas event editing is typically intermittent.

Earlier iterations experimented with alternative placements (including top docking and modal panels). These were rejected because they either reduced viewport area or disrupted editing flow.

Runtime UI Iterations

Figures 15, 16, 17: Runtime UI Iteration Set – A -> C





[Design Section\Design Section Screenshots\image15.png](#)

[Design Section\Design Section Screenshots\image16.png](#)

[Design Section\Design Section Screenshots\image17.png](#)

These iterations show the transition from an automatically loaded runtime model to an explicit, user-driven loading workflow.

Initially, the runtime loaded the most recently edited map automatically. While efficient for development testing, this approach limited user control.

The final design introduces:

- A **Load Map** button
- A minimal HUD displaying:
 - Current map
 - Player status
 - Control hints
- A central scrolling viewport

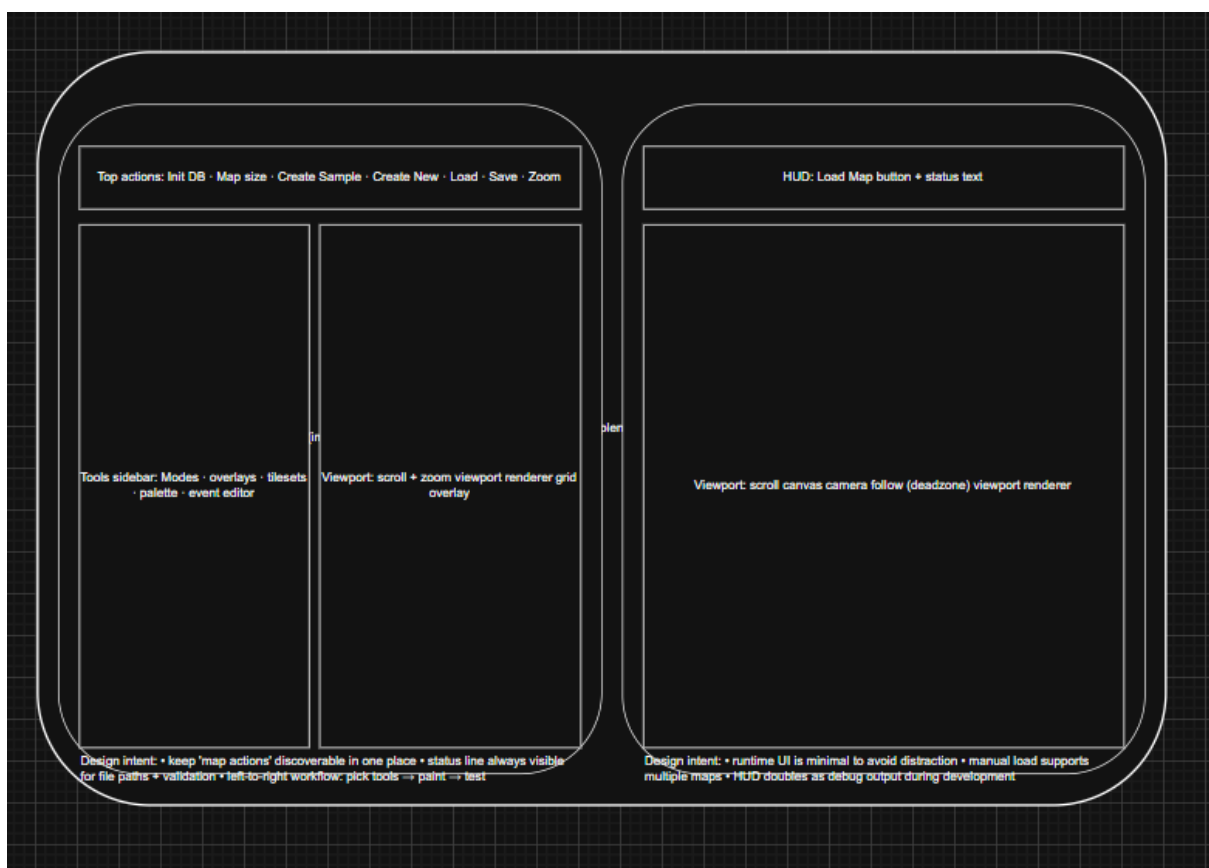
Runtime UI Design Principles

- **Intentional Minimalism** – The runtime interface avoids editor-style controls. It is focused exclusively on gameplay presentation.
- **Explicit User Choice** – The addition of the Load Map button allows the user to select which map to play at runtime. This increases flexibility and better reflects real-world deployment behaviour.
- **Separation from Editor Context** – The runtime does not assume any prior editing session state. It behaves independently of the editor application.

The progression from automatic loading to explicit loading demonstrates responsiveness to usability concerns and improved separation between development and runtime contexts.

Current UI Mapping

Figure 18: UI Control Mapping – XAML Controls -> Event Handlers



[Design Section\Design Section Screenshots\image18.png](#)

This diagram demonstrates traceability between design and implementation by mapping visual XAML controls to their corresponding event handlers and logic layers.

This ensures:

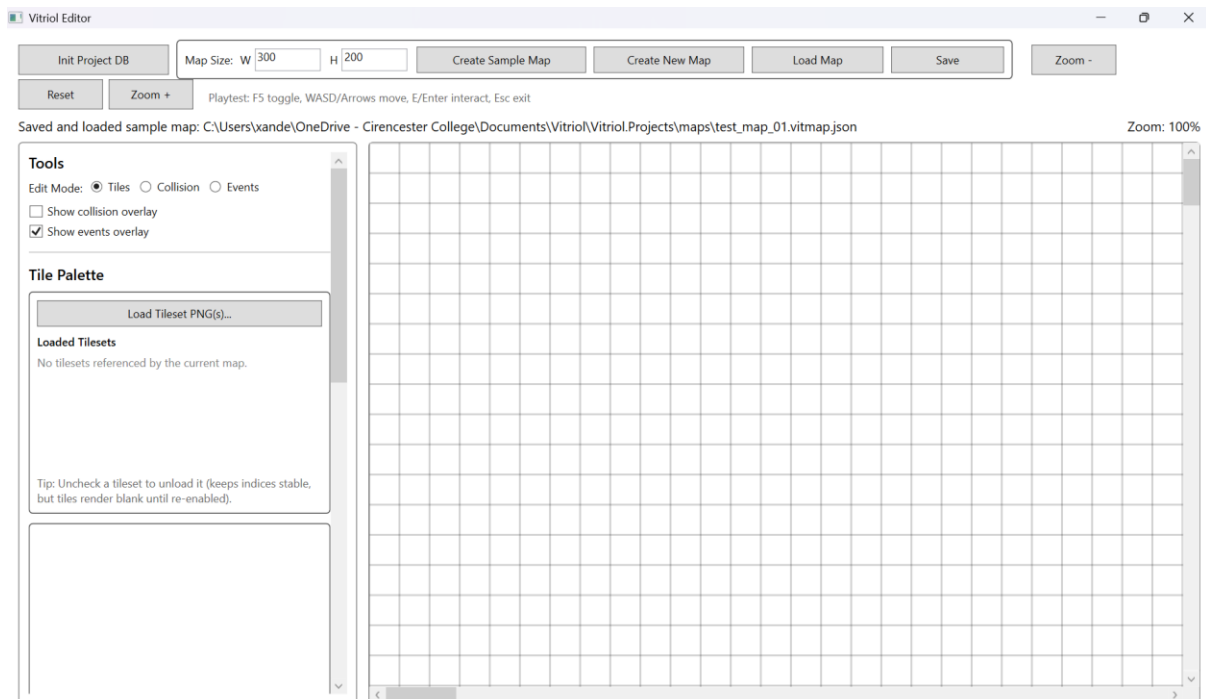
- Clear separation between presentation (XAML) and behaviour (code-behind or command handlers)

- Maintainable event routing
- Architectural transparency between UI and application logic

This mapping reinforces adherence to separation of concerns within the UI layer.

Current UI Screenshots

Figure X1: Current UI – WPF Editor

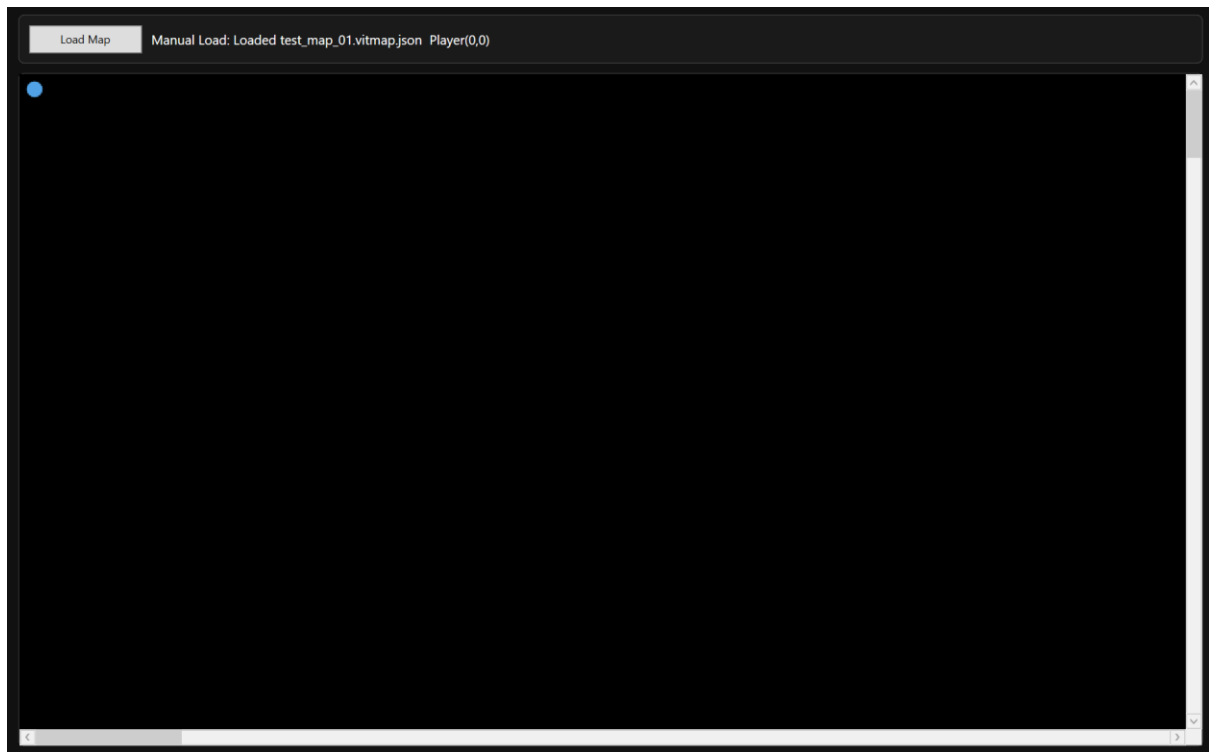


[Design Section\Design Section Screenshots\image19.png](#)

The final editor interface reflects the stabilised layout derived from iterative refinement:

- Wide viewport
- Persistent tools sidebar
- Structured top toolbar
- Status indicators

Figure X2: Current UI – WPF Runtime



[Design Section\Design Section Screenshots\image20.png](#)

The final runtime interface demonstrates:

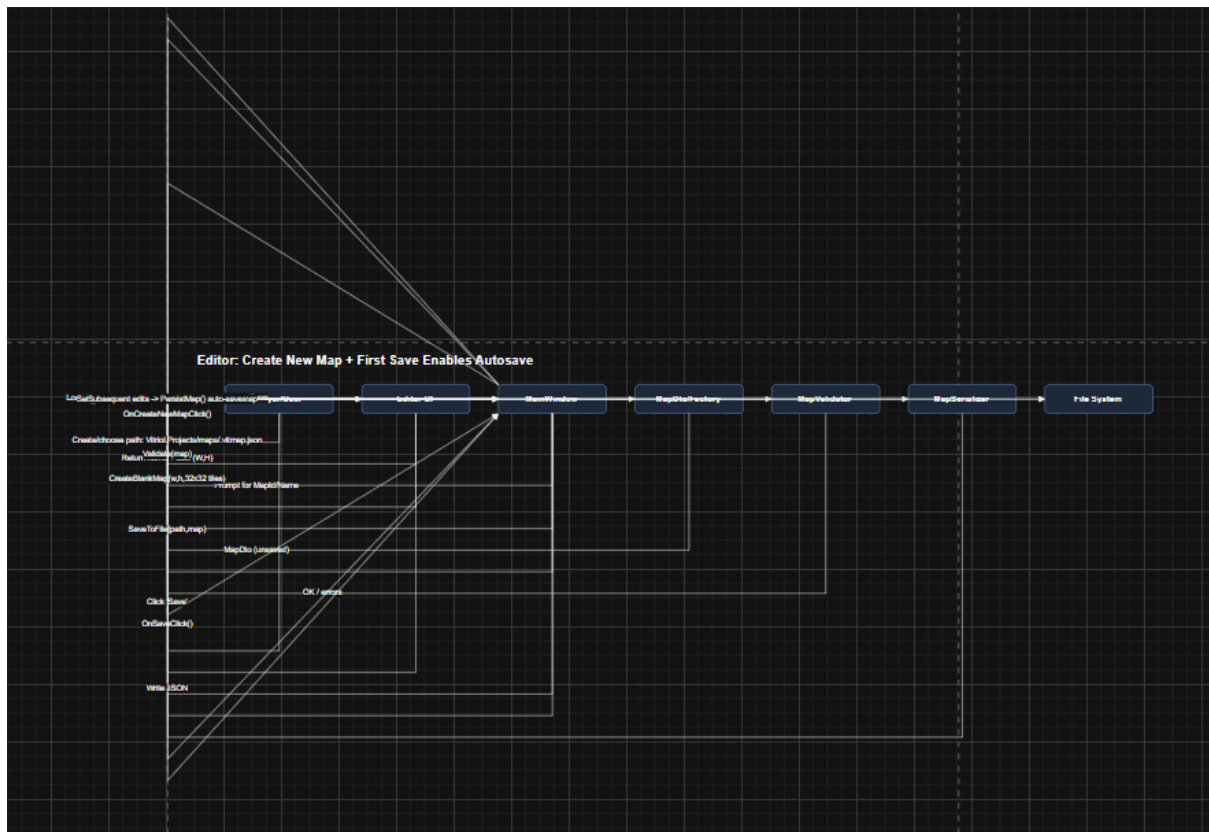
- Minimal HUD
- Explicit map loading
- Scrollable viewport with camera follow

The resulting interfaces prioritise clarity, task efficiency, and predictable behaviour, aligning directly with the usability objectives defined earlier in the design specification.

Behaviour and Control Flow: Sequence Diagrams

Map Creation: New Map vs Sample Map

Figure 19: Sequence – Map Creation and Initial Save State



[Design Section\Design Section Screenshots\image21.png](#)

This sequence diagram illustrates the lifecycle of a newly created map compared to the predefined sample map.

A key behavioural rule is enforced:

- Newly created maps must be manually saved at least once before autosave is enabled.
- The sample map always supports autosave.

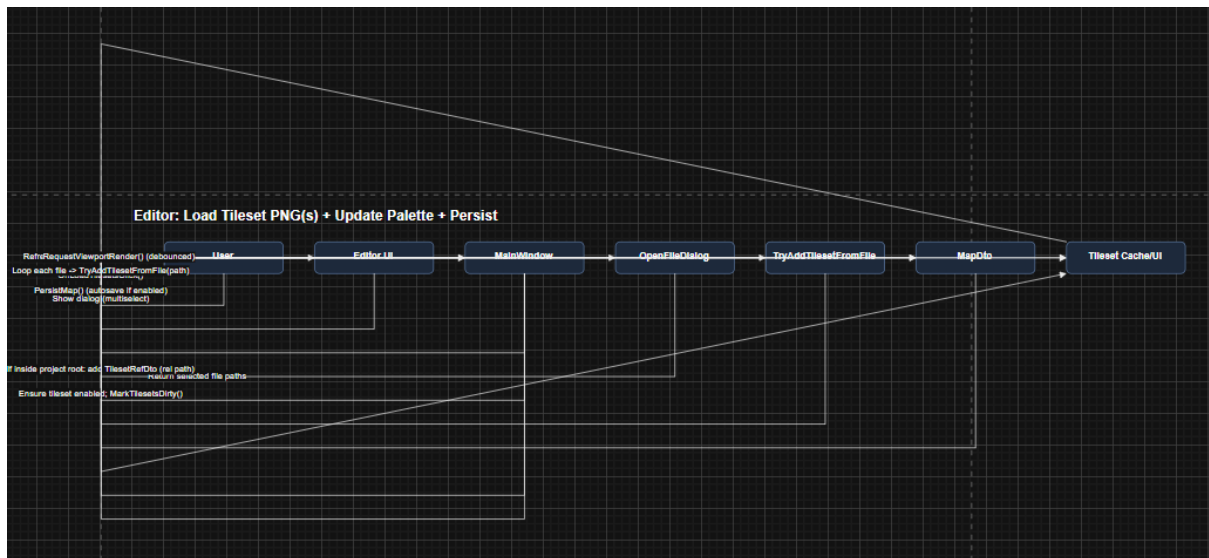
The primary risk during map creation is accidental persistence of invalid or incomplete data. The sequence therefore includes:

- Validation of map dimensions
- Construction of a safe, initialised tile grid
- Controlled activation of autosave only after a successful manual save

This ensures that file creation is deliberate and prevents unintended proliferation of incomplete map files.

Tileset Loading: Editor

Figure 20: Sequence – Tileset Selection, Validation, Referencing, Caching



[Design Section\Design Section Screenshots\image22.png](#)

This sequence diagram demonstrates the two-stage tileset loading model:

1. Reference registration (relative path storage)
2. Asset loading and caching

Tileset handling is intentionally separated into **reference** and **load** phases. This enables the editor to:

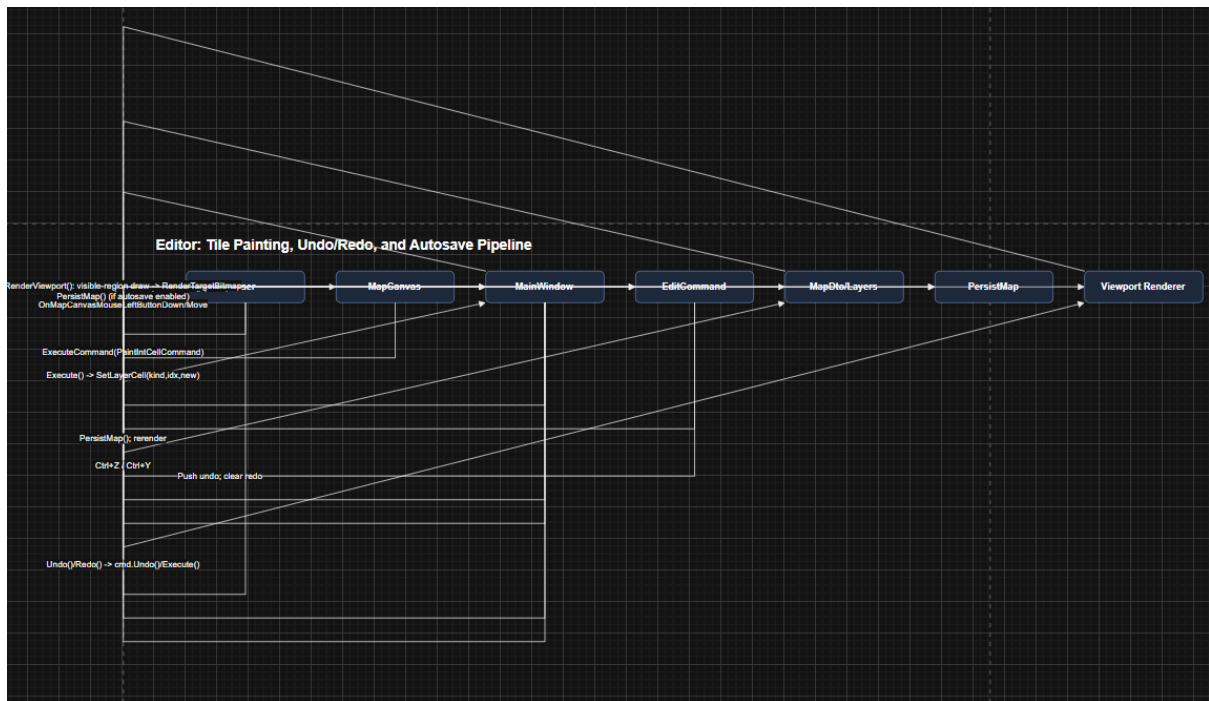
- Unload tilesets from memory to reduce resource usage
- Maintain stable global tile indices
- Prevent corruption when a tileset is toggled off

Stable indexing ensures that tile references stored within the map remain valid regardless of whether the tileset image is currently loaded into memory.

This design balances memory efficiency with structural consistency.

Tile Painting + Undo/Redo + Autosave

Figure 21: Sequence – Paint Action Routed Through Command + Persistence Rules



[Design Section\Design Section Screenshots\image23.png](#)

This sequence represents the convergence of multiple subsystems:

- Input handling
- Command stack
- Rendering updates
- Persistence rules

When a tile is painted:

1. Input is captured.
2. A command object is created.
3. The command mutates the data model.
4. The change is pushed onto the undo stack.
5. Rendering is refreshed.
6. Persistence rules determine whether autosave is triggered.

Explicitly modelling this flow ensured that:

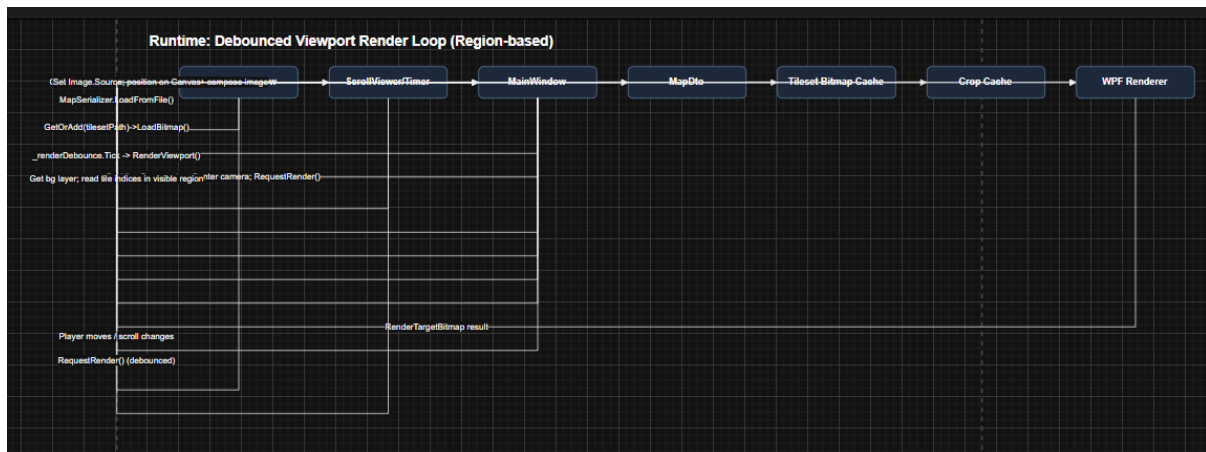
- The data model is updated exactly once per tile change.
- Undo/Redo replays actions through the same execution pathway, avoiding special-case logic.

- Autosave behaviour adheres strictly to the defined rules for sample versus user-created maps.

This structured approach prevents divergence between editing actions and persistence behaviour.

Runtime Render Loop: Viewport-Only Rendering

Figure 22: Sequence – Runtime Viewport Rendering and Cache Usage



[Design Section\Design Section Screenshots\image24.png](#)

This diagram illustrates region-based viewport rendering.

The runtime performs:

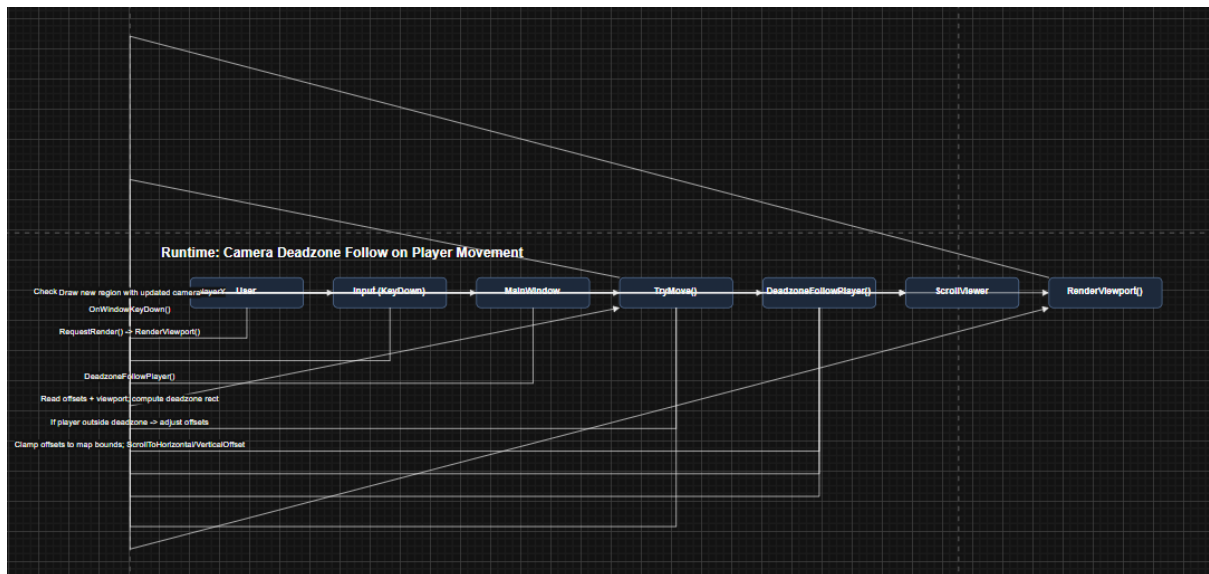
- Region calculation based on camera position
- Cropped tile selection
- Cached tile reuse where possible
- Single composited image output

Both the runtime and editor employ viewport-only rendering. This significantly improves performance for larger maps by limiting draw operations to visible tiles rather than the full grid.

Without region-based rendering, performance would degrade proportionally with map size, even when most tiles remain off-screen.

Camera Deadzone Follow

Figure 23: Sequence – Deadzone Follow Behaviour and Clamping



[Design Section\Design Section Screenshots\image25.png](#)

This diagram demonstrates the deadzone-based camera system.

The camera does not continuously track the player's exact position. Instead:

- A central deadzone region is defined within the viewport.
- The camera offset updates only when the player exits this region.
- Camera movement is clamped to map boundaries.

This prevents constant micro-adjustments and produces smoother perceived motion.

By shifting the camera only when necessary, the design reduces jitter and maintains visual stability, particularly during small player movements.

Design Justification Summary

Traceability: Design Decisions

The following architectural and behavioural decisions directly address the functional and non-functional requirements defined earlier. Each was selected deliberately to balance usability, maintainability, and performance.

- **Stable DTOs + Single Serialiser** -> Ensures both the editor and runtime interpret map files identically, eliminating format drift. By enforcing a single schema definition, long-term maintainability is improved and compatibility risks are reduced.
- **Flat Tile Arrays (1D Representation)** -> Chosen for faster indexing, simplified JSON serialisation, and predictable memory layout. This reduces overhead compared to nested 2D structures while maintaining conceptual clarity through coordinate conversion logic.

- **Command Pattern for Edits** -> Provides deterministic Undo/Redo behaviour across tile painting and event modification. This prevents inconsistent state restoration and centralises mutation logic, improving reliability and debugging clarity.
- **Viewport-Only Rendering** -> Ensures large maps remain performant by limiting draw operations to visible regions. This avoids excessive WPF element creation and prevents frame-rate degradation proportional to map size.
- **Tileset Reference vs Load State Separation** -> Maintains stable tile indexing even when tilesets are toggled in memory. This prevents structural corruption of maps and supports efficient memory management.
- **Explicit Save Rules (Manual Save Before Autosave)** -> Prevents accidental file creation and unintended project folder pollution, while preserving rapid iteration through the always-autosaving sample map. This balances safety and workflow efficiency.
- **Runtime Manual Load Button** -> Introduced to allow explicit map selection at runtime, improving usability, decoupling runtime behaviour from editor state, and increasing testing flexibility.

Each of these decisions reinforces separation of concerns and predictable behaviour across the system.

Architectural Trade-offs and Alternatives Considered

Several alternative approaches were evaluated during the design phase:

- Embedding logic inside DTOs was rejected in favour of passive models to prevent cross-layer coupling.
- Using full 2D tile arrays was considered but rejected due to increased serialisation complexity and reduced indexing efficiency.
- Automatic autosave for all new maps was considered but rejected because it risked accidental file proliferation and loss of intentional naming control.
- A fully dynamic runtime auto-loading model was replaced with explicit loading to improve user agency and deployment realism.

These trade-offs demonstrate that the final design is the result of deliberate decision-making rather than incremental accumulation.

Known Constraints

While the system meets its defined objectives, several constraints are acknowledged:

- **Grid Rendering Precision** – Grid lines may visually degrade during zoom transitions due to pixel snapping behaviour within WPF's rendering pipeline.
- **Tileset File Dependency** – Tilesets referenced by maps must exist as PNG files within
Vitriol.Projects/assets/tilesets/.
Missing assets will result in validation failure or incomplete rendering.
- **Flexible Event Command Schema** – The command dictionary structure supports extensibility but requires strict validation to prevent malformed argument structures or inconsistent parameter usage.

These constraints are acceptable within the scope of the project and do not compromise core functionality, but they define operational boundaries for safe usage.

Diagram Downloads

[Design Section\Design Section Diagrams](#)

Project Vitriol – NEA Implementation

Vitriol.Data

Vitriol.Data\Db

Vitriol.Data\Db\DbPaths.cs

```
namespace Vitriol.Data.Db;
```

```
public static class DbPaths
```

```
{
```

```
    public static string DefaultDbFile(string projectRoot)
```

```
        => Path.Combine(projectRoot, "data", "vitriol.db");
```

```
}
```

Vitriol.Data\Db\SqlConnectionFactory.cs

```
using Microsoft.Data.Sqlite;
```

```
namespace Vitriol.Data.Db;
```

```
public sealed class SqlConnectionFactory(string connectionString)
```

```

{
    public SqlConnection Create()
        => new(connectionString);

    public static SqlConnectionFactory ForFile(string dbFilePath)
    {
        Directory.CreateDirectory(Path.GetDirectoryName(dbFilePath)!);
        var cs = new SqlConnectionStringBuilder
        {
            DataSource = dbFilePath,
            Mode = SqliteOpenMode.ReadWriteCreate,
            Cache = SqliteCacheMode.Shared
        }.ToString();

        return new(cs);
    }
}

```

Vitriol.Data\Migrations

Vitriol.Data\Migrations\SqlConnectionFactory.cs

using Microsoft.Data.Sqlite;

namespace Vitriol.Data.Migrations;

public sealed class MigrationRunner

```

{
    public void EnsureMigrated(SqliteConnection conn)
    {
        // 1) migration history table
    }
}

```

```

using (var cmd = conn.CreateCommand())
{
    cmd.CommandText =
        """
        CREATE TABLE IF NOT EXISTS __migrations (
            id INTEGER PRIMARY KEY,
            name TEXT NOT NULL,
            applied_utc TEXT NOT NULL
        );
        """;
    cmd.ExecuteNonQuery();
}

// 2) Apply migrations in order
ApplyIfMissing(conn, 1, "001_init_species", """
CREATE TABLE IF NOT EXISTS pokemon_species (
    species_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL UNIQUE,
    primary_type TEXT NOT NULL,
    secondary_type TEXT NULL,
    base_hp INTEGER NOT NULL,
    base_atk INTEGER NOT NULL,
    base_def INTEGER NOT NULL,
    base_spa INTEGER NOT NULL,
    base_spd INTEGER NOT NULL,
    base_spe INTEGER NOT NULL
);
""");

```

```
}
```

```
private static void ApplyIfMissing(SqliteConnection conn, long id, string name, string sql)
```

```
{
```

```
    using var check = conn.CreateCommand();
```

```
    check.CommandText = "SELECT 1 FROM __migrations WHERE id = $id LIMIT 1;";
```

```
    check.Parameters.AddWithValue("$id", id);
```

```
    var exists = check.ExecuteScalar() is not null;
```

```
    if (exists) return;
```

```
    using var tx = conn.BeginTransaction();
```

```
    using (var cmd = conn.CreateCommand())
```

```
    {
```

```
        cmd.Transaction = tx;
```

```
        cmd.CommandText = sql;
```

```
        cmd.ExecuteNonQuery();
```

```
    }
```

```
    using (var insert = conn.CreateCommand())
```

```
    {
```

```
        insert.Transaction = tx;
```

```
        insert.CommandText =
```

```
            "INSERT INTO __migrations(id, name, applied_utc) VALUES($id, $name, $utc);";
```

```
        insert.Parameters.AddWithValue("$id", id);
```

```
        insert.Parameters.AddWithValue("$name", name);
```

```

        insert.Parameters.AddWithValue("$utc", DateTime.UtcNow.ToString("O"));

        insert.ExecuteNonQuery();
    }

```

```

        tx.Commit();
    }
}

```

Vitriol.Data.Repositories

Vitriol.Data.Repositories\PokemonSpeciesRepository.cs

```

using Microsoft.Data.Sqlite;

using Vitriol.Shared.Pokemon;

```

```

namespace Vitriol.Data.Repositories;

```

```

public sealed class PokemonSpeciesRepository

```

```

{
    public long Insert(SqliteConnection conn, PokemonSpeciesDto dto)
    {
        using var cmd = conn.CreateCommand();

        cmd.CommandText =
            """

            INSERT INTO pokemon_species

                (name, primary_type, secondary_type, base_hp, base_atk, base_def, base_spa,
base_spd, base_spe)

            VALUES

                ($name, $pt, $st, $hp, $atk, $def, $spa, $spd, $spe);

            SELECT last_insert_rowid();

            """,

```

```

cmd.Parameters.AddWithValue("$name", dto.Name);

cmd.Parameters.AddWithValue("$pt", dto.PrimaryType);

cmd.Parameters.AddWithValue("$st", (object?)dto.SecondaryType ??
DBNull.Value);

cmd.Parameters.AddWithValue("$hp", dto.BaseHP);

cmd.Parameters.AddWithValue("$atk", dto.BaseAttack);

cmd.Parameters.AddWithValue("$def", dto.BaseDefense);

cmd.Parameters.AddWithValue("$spa", dto.BaseSpAttack);

cmd.Parameters.AddWithValue("$spd", dto.BaseSpDefense);

cmd.Parameters.AddWithValue("$spe", dto.BaseSpeed);


return (long)cmd.ExecuteScalar()!;
}

public PokemonSpeciesDto? GetByName(SqliteConnection conn, string name)
{
    using var cmd = conn.CreateCommand();

    cmd.CommandText =
        """
        SELECT species_id, name, primary_type, secondary_type, base_hp, base_atk,
base_def, base_spa, base_spd, base_spe
        FROM pokemon_species
        WHERE name = $name
        LIMIT 1;
        """,
        cmd.Parameters.AddWithValue("$name", name);

    using var r = cmd.ExecuteReader();

    if (!r.Read()) return null;

```

```

return new PokemonSpeciesDto(
    SpeciesId: r.GetInt64(0),
    Name: r.GetString(1),
    PrimaryType: r.GetString(2),
    SecondaryType: r.IsDBNull(3) ? null : r.GetString(3),
    BaseHP: r.GetInt32(4),
    BaseAttack: r.GetInt32(5),
    BaseDefense: r.GetInt32(6),
    BaseSpAttack: r.GetInt32(7),
    BaseSpDefense: r.GetInt32(8),
    BaseSpeed: r.GetInt32(9)
);
}
}

```

Vitriol.Data\Class1.cs

```
namespace Vitriol.Data;
```

```
public class Class1
```

```
{
```

```
}
```

Vitriol.Data\Vitriol.Data.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
```

```
<ItemGroup>
```

```
<ProjectReference Include="..\Vitriol.Shared\Vitriol.Shared.csproj" />
```

```
</ItemGroup>
```

```
<ItemGroup>

  <PackageReference Include="Microsoft.Data.Sqlite" Version="10.0.1" />

</ItemGroup>
```

```
<PropertyGroup>

  <TargetFramework>net9.0</TargetFramework>

  <ImplicitUsings>enable</ImplicitUsings>

  <Nullable>enable</Nullable>

</PropertyGroup>
```

```
</Project>
```

Vitriol.Editor

Vitriol.Editor\App.xaml

```
<Application x:Class="Vitriol.Editor.App"

  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"

  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"

  ShutdownMode="OnMainWindowClose">

  <Application.Resources>

  </Application.Resources>

</Application>
```

Vitriol.Editor\App.xaml.cs

```
using System;

using System.Windows;

using System.Windows.Threading;


namespace Vitriol.Editor;
```



```

public partial class App : Application
{
    public App()
    {
        // Catch UI thread exceptions

        DispatcherUnhandledException += (_, e) =>
        {
            MessageBox.Show(e.Exception.ToString(), "Vitriol Editor crashed (UI thread)");

            e.Handled = true;

            Shutdown(-1);
        };

        // Catch non-UI exceptions

        AppDomain.CurrentDomain.UnhandledException += (_, e) =>
        {
            MessageBox.Show(e.ExceptionObject?.ToString() ?? "Unknown error",
                "Vitriol Editor crashed (AppDomain)");
        };
    }

    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);

        try
        {

```

```

        // Force-create and show MainWindow so StartupUri issues can't silently exit the
app
        var window = new MainWindow();

        MainWindow = window;

        window.Show();
    }

    catch (Exception ex)
    {
        MessageBox.Show(ex.ToString(), "Vitriol Editor failed during startup");

        Shutdown(-1);
    }
}
}

```

Vitriol.Editor\AssemblyInfo.cs

```
using System.Windows;
```

```
[assembly:ThemeInfo(
```

```
    ResourceDictionaryLocation.None,    //where theme specific resource
dictionaries are located
```

```
        //(used if a resource is not found in the page,
```

```
        // or application resource dictionaries)
```

```
    ResourceDictionaryLocation.SourceAssembly //where the generic resource
dictionary is located
```

```
        //(used if a resource is not found in the page,
```

```
        // app, or any theme specific resource dictionaries)
```

```
)]
```

Vitriol.Editor\MainWindow.xaml

```
<Window x:Class="Vitriol.Editor.MainWindow"
```

```
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
```

xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"

Title="Vitriol Editor"

Height="720"

Width="1280"

WindowState="Maximized"

WindowStartupLocation="CenterScreen">

<Grid Margin="12">

<Grid.RowDefinitions>

<RowDefinition Height="Auto"/>

<RowDefinition Height="Auto"/>

<RowDefinition Height="*/>

</Grid.RowDefinitions>

<WrapPanel Orientation="Horizontal" Margin="0,0,0,8"
VerticalAlignment="Center">

<Button Width="160" Height="32" Margin="0,0,8,0" Click="OnInitClick">

Init Project DB

</Button>

<Border BorderBrush="#444" BorderThickness="1" CornerRadius="4"
Padding="8,6" Margin="0,0,8,0">

<StackPanel Orientation="Horizontal">

<TextBlock Text="Map Size:" VerticalAlignment="Center" Margin="0,0,8,0"/>

<TextBlock Text="W" VerticalAlignment="Center" Margin="0,0,4,0"/>

<TextBox Name="MapWidthBox" Width="70" Height="24" Text="300"
Margin="0,0,10,0"/>

```
<TextBlock Text="H" VerticalAlignment="Center" Margin="0,0,4,0"/>

<TextBox Name="MapHeightBox" Width="70" Height="24" Text="200"
Margin="0,0,10,0"/>


<Button Width="180" Height="28" Click="OnCreateSampleMapClick">

    Create Sample Map

</Button>


<!-- NEW: Create New Map -->

<Button Width="160" Height="28" Margin="8,0,0,0"
Click="OnCreateNewMapClick">

    Create New Map

</Button>


<Button Width="140" Height="28" Margin="8,0,0,0"
Click="OnLoadMapClick">

    Load Map

</Button>


<!-- NEW: Save Map -->

<Button Width="120" Height="28" Margin="8,0,0,0" Click="OnSaveMapClick">

    Save

</Button>


</StackPanel>

</Border>


<Separator Width="12" Opacity="0"/>
```

```
<Button Width="90" Height="32" Margin="0,0,6,0"
Click="OnZoomOutClick">Zoom -</Button>
```

```
<Button Width="90" Height="32" Margin="0,0,6,0"
Click="OnZoomResetClick">Reset</Button>
```

```
<Button Width="90" Height="32" Click="OnZoomInClick">Zoom +</Button>
```

```
<TextBlock Margin="16,6,0,0"
```

```
    Foreground="#666"
```

```
    VerticalAlignment="Center"
```

```
    Text="Playtest: F5 toggle, WASD/Arrows move, E/Enter interact, Esc exit"/>
```

```
</WrapPanel>
```

```
<DockPanel Grid.Row="1" Margin="0,0,0,8">
```

```
    <TextBlock Name="StatusText"
```

```
        FontSize="14"
```

```
        DockPanel.Dock="Left"
```

```
        TextTrimming="CharacterEllipsis"/>
```

```
    <TextBlock Name="ZoomText"
```

```
        FontSize="14"
```

```
        DockPanel.Dock="Right"
```

```
        Margin="12,0,0,0"/>
```

```
    <TextBlock Name="MouseText"
```

```
        FontSize="14"
```

```
        DockPanel.Dock="Right"/>
```

```
</DockPanel>
```

```
<Grid Grid.Row="2">
```

```
<Grid.ColumnDefinitions>

    <ColumnDefinition Width="360"/>

    <ColumnDefinition Width="12"/>

    <ColumnDefinition Width="*" />

</Grid.ColumnDefinitions>
```

```
    <Border Grid.Column="0" BorderBrush="Gray" BorderThickness="1"
    CornerRadius="4" Padding="10">
```

```
        <ScrollViewer VerticalScrollBarVisibility="Auto"
            HorizontalScrollBarVisibility="Disabled"
            IsManipulationEnabled="True"
            ScrollViewer.PanningMode="VerticalOnly"
            ScrollViewer.CanContentScroll="False"
            Stylus.IsPressAndHoldEnabled="False"
            Stylus.IsFlicksEnabled="False">
```

```
            <StackPanel>
```

```
                <TextBlock FontSize="16" FontWeight="SemiBold" Text="Tools"
                Margin="0,0,0,8"/>
```

```
                <StackPanel Orientation="Horizontal" Margin="0,0,0,10">
                    <TextBlock Text="Edit Mode:" VerticalAlignment="Center"
                    Margin="0,0,8,0"/>
```

```
                    <RadioButton Name="ModeTiles"
                        Content="Tiles"
                        IsChecked="True"
                        Margin="0,0,10,0"
                        Checked="OnModeChanged"/>
```

```
<RadioButton Name="ModeCollision"
```

```
    Content="Collision"
```

```
    Margin="0,0,10,0"
```

```
    Checked="OnModeChanged"/>
```

```
<RadioButton Name="ModeEvents"
```

```
    Content="Events"
```

```
    Checked="OnModeChanged"/>
```

```
</StackPanel>
```

```
<CheckBox Name="ShowCollisionOverlay"
```

```
    Content="Show collision overlay"
```

```
    IsChecked="False"
```

```
    Checked="OnOverlayChanged"
```

```
    Unchecked="OnOverlayChanged"
```

```
    Margin="0,0,0,6"/>
```

```
<CheckBox Name="ShowEventsOverlay"
```

```
    Content="Show events overlay"
```

```
    IsChecked="True"
```

```
    Checked="OnOverlayChanged"
```

```
    Unchecked="OnOverlayChanged"
```

```
    Margin="0,0,0,14"/>
```

```
<Separator Margin="0,0,0,12"/>
```

```
<!-- Tile palette + tilesheet management -->
```

```
<TextBlock FontSize="16" FontWeight="SemiBold" Text="Tile Palette"
```

```
Margin="0,0,0,8"/>
```

```

<!-- Tileset controls (button + checkboxes live here) -->

<Border BorderBrush="#444" BorderThickness="1" CornerRadius="4"
Padding="8" Margin="0,0,0,10">

    <StackPanel>

        <Button Height="28"

            Margin="0,0,0,8"

            Click="OnLoadTilesetsClick">

                Load Tileset PNG(s)...

            </Button>

        <TextBlock FontWeight="SemiBold" Text="Loaded Tilesets"
Margin="0,0,0,6"/>

    <ScrollView VerticalScrollBarVisibility="Auto"

        HorizontalScrollBarVisibility="Disabled"

        Height="120"

        IsManipulationEnabled="True"

        ScrollView.PanningMode="VerticalOnly"

        ScrollView.CanContentScroll="False"

        Stylus.IsPressAndHoldEnabled="False"

        Stylus.IsFlicksEnabled="False">

        <StackPanel Name="TilesetListPanel"/>

    </ScrollView>

    <TextBlock Foreground="#666"

        Margin="0,8,0,0"

        TextWrapping="Wrap"

```


Text="Tip: Uncheck a tileset to unload it (keeps indices stable, but tiles render blank until re-enabled)."/>

</StackPanel>

</Border>

<!-- Palette thumbnails -->

<Border BorderBrush="#444" BorderThickness="1" CornerRadius="4" Padding="6" Margin="0,0,0,14">

<ScrollViewer VerticalScrollBarVisibility="Auto"

HorizontalScrollBarVisibility="Disabled"

Height="220"

IsManipulationEnabled="True"

ScrollViewer.PanningMode="VerticalOnly"

ScrollViewer.CanContentScroll="False"

Stylus.IsPressAndHoldEnabled="False"

Stylus.IsFlicksEnabled="False">

<WrapPanel Name="TilePalettePanel"/>

</ScrollViewer>

</Border>

<Separator Margin="0,0,0,12"/>

<TextBlock FontSize="16" FontWeight="SemiBold" Text="Event Editor" Margin="0,0,0,8"/>

<TextBlock Name="EventSelectedText" Foreground="#666" Margin="0,0,0,8"/>

<StackPanel Orientation="Horizontal" Margin="0,0,0,8">

<TextBlock Width="70" Text="Name:" VerticalAlignment="Center"/>

```
        <TextBox Name="EventNameBox" Width="240"
LostFocus="OnEventNameLostFocus"/>
```

```
    </StackPanel>
```

```
    <StackPanel Orientation="Horizontal" Margin="0,0,0,8">
```

```
        <TextBlock Width="70" Text="Type:" VerticalAlignment="Center"/>
```

```
        <ComboBox Name="EventTypeBox" Width="240"
SelectionChanged="OnEventTypeChanged"/>
```

```
    </StackPanel>
```

```
    <Button Name="DeleteEventButton"
```

```
        Height="28"
```

```
        Width="120"
```

```
        Margin="0,0,0,12"
```

```
        Click="OnDeleteEventClick">
```

```
        Delete
```

```
    </Button>
```

```
    <StackPanel Name="WarpFieldsPanel" Visibility="Collapsed"
Margin="0,0,0,12">
```

```
        <TextBlock FontWeight="SemiBold" Text="Warp Settings"
Margin="0,0,0,6"/>
```

```
        <StackPanel Orientation="Horizontal" Margin="0,0,0,6">
```

```
            <TextBlock Width="70" Text="destMap:" VerticalAlignment="Center"/>
```

```
            <ComboBox Name="WarpDestMapBox"
```

```
                Width="240"
```

```
                IsEditable="True"
```

```
                DropDownOpened="OnWarpDestMapDropDownOpened"
```

```
                LostFocus="OnWarpFieldsLostFocus"/>
```

```
</StackPanel>

<StackPanel Orientation="Horizontal" Margin="0,0,0,6">
    <TextBlock Width="70" Text="destX:" VerticalAlignment="Center"/>
    <TextBox Name="WarpDestXBox" Width="70"
LostFocus="OnWarpFieldsLostFocus"/>
    <TextBlock Width="30"/>
    <TextBlock Width="70" Text="destY:" VerticalAlignment="Center"/>
    <TextBox Name="WarpDestYBox" Width="70"
LostFocus="OnWarpFieldsLostFocus"/>
</StackPanel>

</StackPanel>
```

```
<Border BorderBrush="#444" BorderThickness="1" CornerRadius="4"
Padding="8" Margin="0,0,0,12">
```

```
<Grid>
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="*/>
    </Grid.RowDefinitions>
```

```
<TextBlock Grid.Row="0"
    FontSize="14"
    FontWeight="SemiBold"
    Text="Parameters"
    Margin="0,0,0,8"/>
```

```
<Grid Grid.Row="1">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
```

```
<RowDefinition Height="*/>
</Grid.RowDefinitions>
```

```
<Grid Grid.Row="0" Margin="0,0,0,8">
  <Grid.ColumnDefinitions>
    <ColumnDefinition Width="*/>
    <ColumnDefinition Width="Auto"/>
  </Grid.ColumnDefinitions>
```

```
<Grid Grid.Column="0" Margin="0,0,8,0">
  <Grid.ColumnDefinitions>
    <ColumnDefinition Width="Auto"/>
    <ColumnDefinition Width="*/>
    <ColumnDefinition Width="Auto"/>
    <ColumnDefinition Width="*/>
  </Grid.ColumnDefinitions>
```

```
<TextBlock Grid.Column="0" Text="Key:"
VerticalAlignment="Center" Margin="0,0,6,0"/>
```

```
<TextBox Grid.Column="1" Name="ParamKeyBox" Height="26"
MinWidth="90" Margin="0,0,12,0"/>
```

```
<TextBlock Grid.Column="2" Text="Value:"
VerticalAlignment="Center" Margin="0,0,6,0"/>
```

```
<TextBox Grid.Column="3" Name="ParamValueBox" Height="26"
MinWidth="90"/>
```

```
</Grid>
```

```
<StackPanel Grid.Column="1" Orientation="Vertical">
```

```
        <Button Width="76" Height="26" Click="OnAddParamClick"
Margin="0,0,0,6">Add</Button>
```

```
        <Button Width="76" Height="26"
Click="OnDeleteParamClick">Delete</Button>
```

```
    </StackPanel>
```

```
</Grid>
```

```
<ListBox Grid.Row="1"
```

```
    Name="ParamsList"
```

```
    SelectionChanged="OnParamSelectionChanged"
```

```
    MinHeight="110"
```

```
    IsManipulationEnabled="True"
```

```
    ScrollViewer.PanningMode="VerticalOnly"
```

```
    ScrollViewer.CanContentScroll="False"
```

```
    Stylus.IsPressAndHoldEnabled="False"
```

```
    Stylus.IsFlicksEnabled="False"/>
```

```
</Grid>
```

```
</Grid>
```

```
</Border>
```

```
<Separator Margin="0,0,0,12"/>
```

```
    <TextBlock FontSize="16" FontWeight="SemiBold" Text="Event Script"
Margin="0,0,0,8"/>
```

```
<ListBox Name="EventCommandList"
```

```
    Height="160"
```

```
    SelectionChanged="OnCommandSelectionChanged"
```

```
    Margin="0,0,0,8"
```

```
IsManipulationEnabled="True"

ScrollViewer.PanningMode="VerticalOnly"

ScrollViewer.CanContentScroll="False"

Stylus.IsPressAndHoldEnabled="False"

Stylus.IsFlicksEnabled="False"/>
```

```
<WrapPanel Margin="0,0,0,10">
```

```
    <Button Width="90" Height="28" Margin="0,0,6,6"
Click="OnAddShowTextClick">+ ShowText</Button>
```

```
    <Button Width="90" Height="28" Margin="0,0,6,6"
Click="OnAddSetFlagClick">+ SetFlag</Button>
```

```
    <Button Width="80" Height="28" Margin="0,0,6,6"
Click="OnDeleteCommandClick">Delete</Button>
```

```
    <Button Width="60" Height="28" Margin="0,0,6,6"
Click="OnMoveCommandUpClick">Up</Button>
```

```
    <Button Width="60" Height="28" Margin="0,0,6,6"
Click="OnMoveCommandDownClick">Down</Button>
```

```
</WrapPanel>
```

```
<Border BorderBrush="#444" BorderThickness="1" CornerRadius="4"
Padding="8">
```

```
    <StackPanel>
```

```
        <TextBlock Name="CommandEditorHeader"
```

```
            FontWeight="SemiBold"
```

```
            Text="No command selected"
```

```
            Margin="0,0,0,8"/>
```

```
    <StackPanel Name="ShowTextEditorPanel" Visibility="Collapsed">
```

```
        <TextBlock Text="Text:" Margin="0,0,0,4"/>
```

```

        <TextBox Name="CmdTextBox"
            Height="70"
            AcceptsReturn="True"
            TextWrapping="Wrap"
            LostFocus="OnCommandFieldsLostFocus"/>
    </StackPanel>

    <StackPanel Name="SetFlagEditorPanel" Visibility="Collapsed">
        <StackPanel Orientation="Horizontal" Margin="0,0,0,6">
            <TextBlock Width="70" Text="Flag:" VerticalAlignment="Center"/>
            <TextBox Name="CmdFlagNameBox" Width="240"
LostFocus="OnCommandFieldsLostFocus"/>
        </StackPanel>
        <StackPanel Orientation="Horizontal">
            <TextBlock Width="70" Text="Value:" VerticalAlignment="Center"/>
            <TextBox Name="CmdFlagValueBox" Width="240"
LostFocus="OnCommandFieldsLostFocus"/>
        </StackPanel>
    </StackPanel>

    </StackPanel>

    </Border>

    </StackPanel>

    </ScrollViewer>

    </Border>

    <Border Grid.Column="2" BorderBrush="Gray" BorderThickness="1"
CornerRadius="4">

```

```
<ScrollView Name="MapScroll"
    HorizontalScrollBarVisibility="Auto"
    VerticalScrollBarVisibility="Auto"
    ScrollChanged="OnScrollChanged"
    PreviewMouseWheel="OnScrollViewMouseWheel">
```

```
<Canvas Name="MapCanvas"
    Background="#111111"
    MouseMove="OnMapCanvasMouseMove"
    MouseLeave="OnMapCanvasMouseLeave"
    MouseLeftButtonDown="OnMapCanvasMouseLeftButtonDown"
    MouseRightButtonDown="OnMapCanvasMouseRightButtonDown"/>
```

```
</ScrollView>
```

```
</Border>
```

```
</Grid>
```

```
</Grid>
```

```
</Window>
```

Vitriol.Editor\MainWindow.xaml.cs

```
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
```



```

using System.Windows.Threading;

using Microsoft.Win32;

using Vitriol.Data.Db;

using Vitriol.Data.Migrations;

using Vitriol.Engine.Maps;

using Vitriol.Shared.Maps;

using Vitriol.Shared.Validation;

using IOPath = System.IO.Path;


namespace Vitriol.Editor;


public partial class MainWindow : Window
{
    // -----
    // Tileset caching + runtime compilation
    // -----

    private readonly ConcurrentDictionary<string, BitmapSource> _tilesetBitmapCache =
new(StringComparer.OrdinalIgnoreCase);

    private readonly ConcurrentDictionary<string, ConcurrentDictionary<int,
CroppedBitmap>> _tileCropCache = new(StringComparer.OrdinalIgnoreCase);


    // Stores computed metadata so global indices remain stable even if a tileset is
unloaded.

    private readonly ConcurrentDictionary<string, TilesetMeta> _tilesetMetaCache =
new(StringComparer.OrdinalIgnoreCase);


    // Tracks whether a tileset is currently enabled (checkbox state).

    private readonly ConcurrentDictionary<string, bool> _tilesetLoadedState =
new(StringComparer.OrdinalIgnoreCase);

```

```

private readonly object _tilesetCompileLock = new();
private List<TilesetRuntime> _compiledTilesets = new();
private bool _tilesetsDirty = true;

// -----
// Zoom state
// -----

private double _zoom = 1.0;
private const double ZoomMin = 0.25;
private const double ZoomMax = 6.0;
private const double ZoomStep = 1.15;

// -----
// Viewport renderer state
// -----

private MapDto? _currentMap;
private Image? _viewportImage;
private readonly DispatcherTimer _renderDebounce;

// -----
// Selection + undo/redo
// -----

private int _selectedTileIndex = 0;
private Border? _selectedPaletteBorder;

private readonly Stack<IEditCommand> _undo = new();
private readonly Stack<IEditCommand> _redo = new();

```

```

private string? _currentMapPath;

// -----
// Paint state
// -----
private bool _isPainting = false;
private int _lastPaintIdx = -1;

// -----
// Hover state (used for keyboard actions like Delete)
// -----
private int _hoverTileX = -1;
private int _hoverTileY = -1;

// -----
// Cursor tile selection (for arrow-key cell move)
// -----
private int _cursorTileX = -1;
private int _cursorTileY = -1;

// -----
// Event drag state (smooth overlay; no full redraw spam)
// -----
private bool _draggingEvent = false;
private string? _dragEventId = null;

// Mouse world position (MapCanvas coords)
private double _dragMouseWorldX = 0;

```

```

private double _dragMouseWorldY = 0;

// The snapped/clamped tile preview used for commit
private int _dragPreviewX = -1;
private int _dragPreviewY = -1;

// Overlay visuals (type-shaped ghost)
private System.Windows.Shapes.Shape? _dragGhostShape;
private System.Windows.Shapes.Shape? _dragGhostOutlineShape;

// Swap Hint Overlay (arrow-key swap feedback)
private System.Windows.Shapes.Rectangle? _swapHintFrom;
private System.Windows.Shapes.Rectangle? _swapHintTo;
private readonly DispatcherTimer _swapHintTimer = new();

// -----
// Drag autoscroll (while moving events)
// -----
private readonly DispatcherTimer _dragAutoScrollTimer = new();
private double _dragLastViewportX = 0;
private double _dragLastViewportY = 0;

private const double DragAutoScrollMargin = 48;
private const double DragAutoScrollSpeed = 26;
private const int DragAutoScrollTickMs = 16;

private void DeleteAtCurrentMode()
{

```

```

var map = _currentMap;

if (map is null) return;

// Needs a valid hovered tile
if (_hoverTileX < 0 || _hoverTileY < 0) return;
if (_hoverTileX >= map.Width || _hoverTileY >= map.Height) return;

int idx = _hoverTileY * map.Width + _hoverTileX;

// Don't allow destructive edits during playtest
if (_playtestActive)
{
    StatusText.Text = "Playtest is active (F5 to stop). Editing is disabled.";
    return;
}

if (_mode == EditMode.Tiles)
{
    var bg = GetBackgroundLayer();
    if (bg is null) return;

    int oldValue = bg.Tiles[idx];
    int newValue = -1; // erase tile

    if (oldValue == newValue) return;

    ExecuteCommand(new PaintIntCellCommand(this, LayerKind.Background, idx,
oldValue, newValue));
    return;
}

```

```
}
```

```
if (_mode == EditMode.Collision)
{
    var col = GetCollisionLayer();
    if (col is null) return;

    int oldValue = col.Tiles[idx];
    int newValue = 0; // clear collision

    if (oldValue == newValue) return;

    ExecuteCommand(new PaintIntCellCommand(this, col.Kind, idx, oldValue,
newValue));
    return;
}
```

```
if (_mode == EditMode.Events)
{
    var ev = GetEventAt(_hoverTileX, _hoverTileY) ?? GetSelectedEvent();
    if (ev is null) return;
```

```
ExecuteCommand(new DeleteEventCommand(this, ev));
```

```
if (_selectedEventId == ev.EventId)
{
    _selectedEventId = null;
    _selectedCommandIndex = -1;
    UpdateEventPanel(null);
```

```
RefreshCommandList(null);  
UpdateCommandEditor(null);  
}
```

```
RequestViewportRender();  
return;  
}  
}
```

```
// -----  
// Ctrl + D Duplicates Event  
// -----  
private MapEventDto CloneEventTo(MapEventDto src, int x, int y)  
{  
    var newParams = src.Parameters is null  
        ? new Dictionary<string, string>()  
        : new Dictionary<string, string>(src.Parameters);  
  
    var newCmds = (src.Commands ?? Array.Empty<EventCommandDto>())  
        .Select(c => new EventCommandDto(  
            (c.Command ?? "").Trim(),  
            c.Args is null ? new Dictionary<string, string>() : new Dictionary<string,  
string>(c.Args)))  
        .ToArray();  
  
    var name = string.IsNullOrEmpty(src.Name) ? "Event" : src.Name.Trim();  
  
    return new MapEventDto(  

```

```

    EventId: Guid.NewGuid().ToString("N"),
    Name: name + " Copy",
    Type: (src.Type ?? "Trigger").Trim(),
    X: x,
    Y: y,
    Parameters: newParams,
    Commands: newCmds
);
}

```

```

private bool TryFindFreeTileNear(int startX, int startY, out int outX, out int outY)
{
    outX = startX;
    outY = startY;

    var map = _currentMap;
    if (map is null) return false;

    // Spiral-ish search radius
    const int maxR = 25;

    bool IsFree(int x, int y) => GetEventAt(x, y) is null;

    if (startX >= 0 && startY >= 0 && startX < map.Width && startY < map.Height &&
        IsFree(startX, startY))
        return true;

    for (int r = 1; r <= maxR; r++)

```



```

{
    int minX = Math.Max(0, startX - r);
    int maxX = Math.Min(map.Width - 1, startX + r);
    int minY = Math.Max(0, startY - r);
    int maxY = Math.Min(map.Height - 1, startY + r);

    for (int x = minX; x <= maxX; x++)
    {
        if (IsFree(x, minY)) { outX = x; outY = minY; return true; }
        if (IsFree(x, maxY)) { outX = x; outY = maxY; return true; }
    }
    for (int y = minY; y <= maxY; y++)
    {
        if (IsFree(minX, y)) { outX = minX; outY = y; return true; }
        if (IsFree(maxX, y)) { outX = maxX; outY = y; return true; }
    }
}

return false;
}

```

```
// -----
```

```
// Editor modes
```

```
// -----
```

```
private enum EditMode { Tiles, Collision, Events }
```

```
private EditMode _mode = EditMode.Tiles;
```

```

private bool _showCollisionOverlay = false;

private bool _showEventsOverlay = true;


// -----
// Event selection
// -----

private string? _selectedEventId;


// -----
// Command selection (event script commands)
// -----

private int _selectedCommandIndex = -1;

private bool _updatingCommandUi = false;


// -----
// Playtest state
// -----

private bool _playtestActive = false;

private int _playerX = 0;

private int _playerY = 0;

private readonly Dictionary<string, string> _playtestFlags =
new(StringComparer.OrdinalIgnoreCase);


// -----
// Project folders
// -----

private string ProjectRoot => IOPath.Combine(Directory.GetCurrentDirectory(),
"Vitriol.Projects");

private string MapsFolder => IOPath.Combine(ProjectRoot, "maps");

```

```

private string PngsFolder => IOPath.Combine(ProjectRoot, "assets", "tilesets");

// Cached list of available map IDs (from the maps folder) for Warp destMap dropdown.
private List<string> _availableMapIds = new();

private static string? TryMapIdFromFileName(string fileName)
{
    // Supports both "*.vitmap.json" (default) and legacy "*.json"
    if (string.IsNullOrEmpty(fileName)) return null;

    var name = IOPath.GetFileName(fileName);

    if (name.EndsWith(MapFormat.DefaultExtension,
        StringComparison.OrdinalIgnoreCase))
        name = name[..^MapFormat.DefaultExtension.Length];
    else if (name.EndsWith(".json", StringComparison.OrdinalIgnoreCase))
        name = name[..^5];

    name = (name ?? "").Trim();
    return name.Length == 0 ? null : name;
}

private void RefreshAvailableMaps(bool preserveCurrentSelection = true)
{
    try
    {
        Directory.CreateDirectory(MapsFolder);
    }
}

```

```

var files = Directory.EnumerateFiles(MapsFolder, "*",
SearchOption.TopDirectoryOnly)

    .Where(f =>

        f.EndsWith(MapFormat.DefaultExtension,
StringComparison.OrdinalIgnoreCase) ||

        f.EndsWith(".json", StringComparison.OrdinalIgnoreCase))

    .ToList();

var ids = new HashSet<string>(StringComparer.OrdinalIgnoreCase);
foreach (var f in files)
{
    var id = TryMapIdFromFileName(f);
    if (id is null) continue;
    ids.Add(id);
}

var list = ids.OrderBy(x => x, StringComparer.OrdinalIgnoreCase).ToList();
_availableMapIds = list;

// If the Warp dest map UI exists (ComboBox in XAML), bind it.
if (WarpDestMapBox is ComboBox cb)
{
    var current = preserveCurrentSelection ? GetWarpDestMapUiValue() : null;

WarpDestMapBox.ItemsSource = null;
WarpDestMapBox.ItemsSource = _availableMapIds;

if (!string.IsNullOrEmpty(current))

```

```

{
    if (!_availableMapIds.Contains(current, StringComparer.OrdinalIgnoreCase))
    {
        _availableMapIds = _availableMapIds
            .Concat(new[] { current })
            .Distinct(StringComparer.OrdinalIgnoreCase)
            .OrderBy(x => x, StringComparer.OrdinalIgnoreCase)
            .ToList();

        WarpDestMapBox.ItemsSource = null;
        WarpDestMapBox.ItemsSource = _availableMapIds;
    }

    WarpDestMapBox.SelectedItem = _availableMapIds.FirstOrDefault(x =>
        x.Equals(current, StringComparer.OrdinalIgnoreCase));
}
else
{
    WarpDestMapBox.SelectedItem = null;
}

}
}
catch
{
    // Swallow folder IO errors; warp dropdown will just be empty.
    _availableMapIds = new();
}

```

```
}
```

```
private string GetWarpDestMapUiValue()
```

```
{
```

```
    // WarpDestMapBox is a ComboBox in XAML
```

```
    var sel = WarpDestMapBox.SelectedItem as string;
```

```
    if (!string.IsNullOrEmpty(sel)) return sel.Trim();
```

```
    // If editable, or if user typed
```

```
    return (WarpDestMapBox.Text ?? "").Trim();
```

```
}
```

```
private void SetWarpDestMapUiValue(string? value)
```

```
{
```

```
    value = (value ?? "").Trim();
```

```
    if (string.IsNullOrEmpty(value))
```

```
    {
```

```
        WarpDestMapBox.SelectedItem = null;
```

```
        WarpDestMapBox.Text = "";
```

```
        return;
```

```
    }
```

```
    // Ensure list contains the value so selection works.
```

```
    if (_availableMapIds.Count == 0 || !_availableMapIds.Contains(value,  
StringComparer.OrdinalIgnoreCase))
```

```
    {
```

```
        _availableMapIds = _availableMapIds
```

```
.Concat(new[] { value })  
.Distinct(StringComparer.OrdinalIgnoreCase)  
.OrderBy(x => x, StringComparer.OrdinalIgnoreCase)  
.ToList();
```

```
WarpDestMapBox.ItemsSource = null;  
WarpDestMapBox.ItemsSource = _availableMapIds;
```

```
}
```

```
WarpDestMapBox.SelectedItem = _availableMapIds.FirstOrDefault(x =>  
    x.Equals(value, StringComparison.OrdinalIgnoreCase));
```

```
// Keeps display sane even if not selectable
```

```
WarpDestMapBox.Text = value;
```

```
}
```

```
// XAML handler (ComboBox.DropDownOpened): refresh available maps right before  
showing the list.
```

```
private void OnWarpDestMapDropDownOpened(object sender, System.EventArgs e)
```

```
{
```

```
    RefreshAvailableMaps(preserveCurrentSelection: true);
```

```
}
```

```
// Manual save gating (new maps must be saved once before autosave)
```

```
private bool _manualSavePerformed = false;
```

```
private static bool IsSampleMapPath(string? path)
```

```
{
```

```

    if (string.IsNullOrEmpty(path)) return false;

    return path.IndexOf("test_map_01", StringComparison.OrdinalIgnoreCase) >= 0;
}

```

```

private bool IsSampleMapLoaded()
{
    if (_currentMap is null) return false;

    if (_currentMap.MapId.Equals("test_map_01",
StringComparison.OrdinalIgnoreCase)) return true;

    return IsSampleMapPath(_currentMapPath);
}

```

```

private static string MakeSafeMapId(string raw)
{

```

```

    raw = (raw ?? "").Trim();

    if (raw.Length == 0) return "new_map";

```

```

    var chars = raw.Select(ch =>
    {
        if (char.IsLetterOrDigit(ch)) return char.ToLowerInvariant(ch);

        if (ch == '_') return '_';

        if (ch == '-' || ch == '-') return '_';

        return '\0';
    }).Where(c => c != '\0').ToArray();

```

```

    var s = new string(chars);

    while (s.Contains("__", StringComparison.Ordinal)) s = s.Replace("__", "_");

    s = s.Trim('_');

```



```
if (s.Length == 0) s = "new_map";  
return s;  
}
```

```
public MainWindow()
```

```
{
```

```
    InitializeComponent();
```

```
    RefreshAvailableMaps(preserveCurrentSelection: false);
```

```
    StatusText.Text = "Editor started. Database not initialised.";
```

```
    MouseText.Text = "";
```

```
    EventTypeBox.ItemsSource = new[] { "Trigger", "NPC", "Warp" };
```

```
    EventTypeBox.SelectedIndex = 0;
```

```
    ApplyZoom();
```

```
    _renderDebounce = new DispatcherTimer { Interval =  
    TimeSpan.FromMilliseconds(30) };
```

```
    _renderDebounce.Tick += (_, _) =>
```

```
{
```

```
    _renderDebounce.Stop();
```

```
    RenderViewport();
```

```
};
```

```
    MapScroll.SizeChanged += (_, _) => RequestViewportRender();
```

```
PreviewKeyDown += OnMainWindowPreviewKeyDown;
```

```
MapCanvas.MouseLeftButtonUp += OnMapCanvasMouseLeftButtonUp;
```

```
MapCanvas.MouseRightButtonUp += (_, _) => StopPainting();
```

```
MapCanvas.MouseLeave += OnMapCanvasMouseLeave;
```

```
EnableTouchScrollForListBox(ParamsList);
```

```
EnableTouchScrollForListBox(EventCommandList);
```

```
UpdateEventPanel(null);
```

```
RefreshCommandList(null);
```

```
UpdateCommandEditor(null);
```

```
RefreshTilesetPanel();
```

```
    _dragAutoScrollTimer.Interval =  
TimeSpan.FromMilliseconds(DragAutoScrollTickMs);  
    _dragAutoScrollTimer.Tick += (_, _) => TickDragAutoScroll();  
}
```

```
// -----
```

```
// Tileset runtime structures
```

```
// -----
```

```
private sealed record TilesetMeta(  
    int PixelWidth,  
    int PixelHeight,
```

```
int TileWidth,  
int TileHeight,  
int Margin,  
int Spacing,  
int TilesAcross,  
int TilesDown,  
int TileCount  
);
```

```
private sealed class TilesetRuntime  
{  
    public required TilesetRefDto Ref { get; init; }  
    public required string FullPath { get; init; }  
    public required string Key { get; init; }  
    public required int StartIndexGlobal { get; init; }  
    public required int TileCount { get; init; }  
    public required int TilesAcross { get; init; }  
    public required int TileW { get; init; }  
    public required int TileH { get; init; }  
    public required int Margin { get; init; }  
    public required int Spacing { get; init; }  
    public required bool IsLoaded { get; init; }  
    public BitmapSource? Bitmap { get; init; }  
    public ConcurrentDictionary<int, CroppedBitmap>? CropCache { get; init; }  
}
```

```
private void MarkTilesetsDirty()  
{
```

```
lock (_tilesetCompileLock)
    _tilesetsDirty = true;
}
```

```
private static string NormalizeTilesetKey(string relativeOrAbsolutePath) =>
    (relativeOrAbsolutePath ?? "").Replace('\\', '/').Trim();
```

```
private bool IsTilesetLoaded(string tilesetKey)
{
    tilesetKey = NormalizeTilesetKey(tilesetKey);
    if (tilesetKey.Length == 0) return false;

    if (_tilesetLoadedState.TryGetValue(tilesetKey, out var loaded))
        return loaded;

    _tilesetLoadedState[tilesetKey] = true;

    return true;
}
```

```
private void SetTilesetLoaded(string tilesetKey, bool loaded, bool
clearCachesOnUnload = true)
{
    tilesetKey = NormalizeTilesetKey(tilesetKey);
    if (tilesetKey.Length == 0) return;

    _tilesetLoadedState[tilesetKey] = loaded;

    if (!loaded && clearCachesOnUnload)
```

```

{
    var full = ResolveTilesetFullPathFromKey(tilesetKey);
    if (full is not null)
    {
        _tilesetBitmapCache.TryRemove(full, out _);
        _tileCropCache.TryRemove(full, out _);
    }
}

MarkTilesetsDirty();
RefreshTilesetPanel();
BuildTilePalette();
RequestViewportRender();
}

private string? ResolveTilesetFullPathFromKey(string tilesetKey)
{
    if (tilesetKey.Length == 0) return null;

    if (IOPath.IsPathRooted(tilesetKey))
        return tilesetKey;

    return IOPath.Combine(ProjectRoot, tilesetKey.Replace('/', '\\'));
}

private void EnsureTilesetsCompiled()
{
    var map = _currentMap;

```

```

if (map is null)
{
    lock (_tilesetCompileLock)
    {
        _compiledTilesets = new List<TilesetRuntime>();
        _tilesetsDirty = false;
    }
    return;
}

lock (_tilesetCompileLock)
{
    if (!_tilesetsDirty) return;

    var compiled = new List<TilesetRuntime>();
    int runningStart = 0;

    var tilesets = map.Tilesets ?? Array.Empty<TilesetRefDto>();
    foreach (var ts in tilesets)
    {
        var key = NormalizeTilesetKey(ts.ImagePath ?? "");
        if (key.Length == 0) continue;

        bool loaded = IsTilesetLoaded(key);

        var fullPath = ResolveTilesetFullPathFromKey(key);
        if (fullPath is null) continue;
    }
}

```

```

BitmapSource? bmp = null;

if (loaded && File.Exists(fullPath))

    bmp = _tilesetBitmapCache.GetOrAdd(fullPath, LoadBitmap);

TilesetMeta meta;

if (bmp is not null)
{
    meta = ComputeMeta(ts, bmp);
    _tilesetMetaCache[fullPath] = meta;
}
else if (_tilesetMetaCache.TryGetValue(fullPath, out var metaCached))
{
    meta = metaCached;
}
else
{
    if (File.Exists(fullPath))
    {
        try
        {
            var temp = _tilesetBitmapCache.GetOrAdd(fullPath, LoadBitmap);
            meta = ComputeMeta(ts, temp);
            _tilesetMetaCache[fullPath] = meta;

            if (!loaded)
                _tilesetBitmapCache.TryRemove(fullPath, out _);
        }
    }
}

```

```

        catch
        {
            meta = new TilesetMeta(0, 0,
                Math.Max(1, ts.TileWidth),
                Math.Max(1, ts.TileHeight),
                Math.Max(0, ts.Margin),
                Math.Max(0, ts.Spacing),
                0, 0, 0);
        }
    }
    else
    {
        meta = new TilesetMeta(0, 0,
            Math.Max(1, ts.TileWidth),
            Math.Max(1, ts.TileHeight),
            Math.Max(0, ts.Margin),
            Math.Max(0, ts.Spacing),
            0, 0, 0);
    }
}

```

```

ConcurrentDictionary<int, CroppedBitmap>? cropCache = null;
if (loaded && bmp is not null && meta.TileCount > 0)
    cropCache = _tileCropCache.GetOrAdd(fullPath, _ => new
ConcurrentDictionary<int, CroppedBitmap>());

```

```

compiled.Add(new TilesetRuntime
{

```



```

        Ref = ts,
        FullPath = fullPath,
        Key = key,
        StartIndexGlobal = runningStart,
        TileCount = meta.TileCount,
        TilesAcross = meta.TilesAcross,
        TileW = meta.TileWidth,
        TileH = meta.TileHeight,
        Margin = meta.Margin,
        Spacing = meta.Spacing,
        IsLoaded = loaded && bmp is not null,
        Bitmap = bmp,
        CropCache = cropCache
    });

    runningStart += meta.TileCount;
}

_compiledTilesets = compiled;
_tilesetsDirty = false;
}
}

private static TilesetMeta ComputeMeta(TilesetRefDto ts, BitmapSource bmp)
{
    int tileW = Math.Max(1, ts.TileWidth);
    int tileH = Math.Max(1, ts.TileHeight);
    int margin = Math.Max(0, ts.Margin);

```

```

int spacing = Math.Max(0, ts.Spacing);

int stepX = tileW + spacing;
int stepY = tileH + spacing;

int tilesAcross = stepX > 0 ? (bmp.PixelWidth - margin) / stepX : 0;
int tilesDown = stepY > 0 ? (bmp.PixelHeight - margin) / stepY : 0;

tilesAcross = Math.Max(0, tilesAcross);
tilesDown = Math.Max(0, tilesDown);

int tileCount = tilesAcross * tilesDown;

return new TilesetMeta(
    PixelWidth: bmp.PixelWidth,
    PixelHeight: bmp.PixelHeight,
    TileWidth: tileW,
    TileHeight: tileH,
    Margin: margin,
    Spacing: spacing,
    TilesAcross: tilesAcross,
    TilesDown: tilesDown,
    TileCount: tileCount
);
}

private TilesetRuntime? FindRuntimeForGlobalTile(int globalTileIndex)
{

```

```

    EnsureTilesetsCompiled();

    if (globalTileIndex < 0) return null;

    foreach (var ts in _compiledTilesets)
    {
        if (globalTileIndex >= ts.StartIndexGlobal &&
            globalTileIndex < ts.StartIndexGlobal + ts.TileCount)
            return ts;
    }

    return null;
}

private static int GetLocalIndex(TilesetRuntime ts, int globalTileIndex) =>
    globalTileIndex - ts.StartIndexGlobal;

// -----
// Tileset UI panel (checkboxes)
// -----

private void RefreshTilesetPanel()
{
    if (TilesetListPanel is null) return;

    TilesetListPanel.Children.Clear();

    if (_currentMap is null)
    {

```

```
TilesetListPanel.Children.Add(new TextBlock { Text = "No map loaded.",  
Foreground = Brushes.Gray });
```

```
    return;  
}
```

```
var tilesets = _currentMap.Tilesets ?? Array.Empty<TilesetRefDto>();  
var visible = tilesets.Where(t => !string.IsNullOrEmpty(t.ImagePath)).ToList();
```

```
if (visible.Count == 0)  
{  
    TilesetListPanel.Children.Add(new TextBlock  
    {  
        Text = "No tilesets referenced by the current map.",  
        Foreground = Brushes.Gray  
    });  
    return;  
}
```

```
foreach (var ts in visible)  
{  
    var key = NormalizeTilesetKey(ts.ImagePath ?? "");  
    if (key.Length == 0) continue;  
  
    var row = new DockPanel { Margin = new Thickness(0, 0, 0, 6) };  
  
    var cb = new CheckBox  
    {  
        IsChecked = IsTilesetLoaded(key),
```

```

        VerticalAlignment = VerticalAlignment.Center,
        Margin = new Thickness(0, 0, 8, 0)
    };

    cb.Checked += (_, _) => SetTilesetLoaded(key, true);
    cb.Unchecked += (_, _) => SetTilesetLoaded(key, false);

    DockPanel.SetDock(cb, Dock.Left);
    row.Children.Add(cb);

    row.Children.Add(new TextBlock
    {
        Text = $"{ts.TilesetId} ({key})",
        TextWrapping = TextWrapping.Wrap
    });

    TilesetListPanel.Children.Add(row);
}
}

// -----
// Touch scrolling helper
// -----

private static void EnableTouchScrollForListBox(ListBox listBox)
{
    if (listBox == null) return;

    listBox.IsManipulationEnabled = true;

    listBox.PreviewTouchMove += (_, e) => { e.Handled = true; };
}

```

```

}

// -----
// Database initialization
// -----
private void OnInitClick(object sender, RoutedEventArgs e)
{
    Directory.CreateDirectory(IOPath.Combine(ProjectRoot, "data"));

    var dbFile = DbPaths.DefaultDbFile(ProjectRoot);
    var factory = SqliteConnectionFactory.ForFile(dbFile);

    using var conn = factory.Create();
    conn.Open();
    new MigrationRunner().EnsureMigrated(conn);

    StatusText.Text = $"Database ready: {dbFile}";
}

// -----
// Map creation/loading
// -----
private void OnCreateSampleMapClick(object sender, RoutedEventArgs e)
{
    if (!TryReadMapSize(out int w, out int h))
        return;

    Directory.CreateDirectory(MapsFolder);

```

```

var map = CreateSampleMap(w, h);

var errors = MapValidator.Validate(map);
if (errors.Count > 0)
{
    StatusText.Text = "Map validation failed: " +
        string.Join("; ", errors.Select(er => $"{er.Field}: {er.Message}"));
    return;
}

var path = IOPath.Combine(MapsFolder, map.MapId +
MapFormat.DefaultExtension);
MapSerializer.SaveToFile(path, map);

LoadMapFromPath(path, "Saved and loaded sample map");
}

private void OnLoadMapClick(object sender, RoutedEventArgs e)
{
    Directory.CreateDirectory(MapsFolder);

    var dlg = new OpenFileDialog
    {
        Title = "Open Vitriol Map",
        Filter = "Vitriol Map (*.vitmap.json)|*.vitmap.json|JSON (*.json)|*.json|All files (*.*)|*.*",
        InitialDirectory = MapsFolder,
        CheckFileExists = true,

```

```

        Multiselect = false
    };

    if (dlg.ShowDialog(this) != true)
        return;

    LoadMapFromPath(dlg.FileName, "Loaded map");
}

private void LoadMapFromPath(string path, string statusPrefix)
{
    if (!File.Exists(path))
    {
        StatusText.Text = $"Map file not found: {path}";
        return;
    }

    try
    {
        _currentMapPath = path;
        _currentMap = MapSerializer.LoadFromFile(path);
        _manualSavePerformed = !IsSampleMapPath(path);

        if (_currentMap.Events is null)
            _currentMap = _currentMap with { Events = Array.Empty<MapEventDto>() };

        var fixedEvents = _currentMap.Events
            .Select(ev =>

```



```

{
    var cmds = ev.Commands ?? Array.Empty<EventCommandDto>();
    var pars = ev.Parameters ?? new Dictionary<string, string>();
    if (ev.Commands == cmds && ev.Parameters == pars) return ev;
    return ev with { Commands = cmds, Parameters = pars };
})
.ToList();

_currentMap = _currentMap with { Events = fixedEvents };

_selectedEventId = null;
_selectedCommandIndex = -1;
UpdateEventPanel(null);
RefreshCommandList(null);
UpdateCommandEditor(null);

EnsureCollisionLayer();

_undo.Clear();
_redo.Clear();

var tilesets = _currentMap.Tilesets ?? Array.Empty<TilesetRefDto>();
foreach (var ts in tilesets)
{
    var key = NormalizeTilesetKey(ts.ImagePath ?? "");
    if (key.Length == 0) continue;
    _tilesetLoadedState.TryAdd(key, true);
}

```

```
MarkTilesetsDirty();
```

```
EnsureViewportImageExists();
```

```
RefreshTilesetPanel();
```

```
BuildTilePalette();
```

```
MapScroll.ScrollToHorizontalOffset(0);
```

```
MapScroll.ScrollToVerticalOffset(0);
```

```
RefreshAvailableMaps(preserveCurrentSelection: false);
```

```
StatusText.Text = $"{statusPrefix}: {path}";
```

```
RequestViewportRender();
```

```
}
```

```
catch (Exception ex)
```

```
{
```

```
    StatusText.Text = $"Failed to load map: {ex.Message}";
```

```
}
```

```
}
```

```
private bool TryReadMapSize(out int w, out int h)
```

```
{
```

```
    w = 0;
```

```
    h = 0;
```

```
    if (!int.TryParse(MapWidthBox.Text?.Trim(), out w) ||
```

```
        !int.TryParse(MapHeightBox.Text?.Trim(), out h))
```

```

{
    StatusText.Text = "Width/Height must be whole numbers.";
    return false;
}

if (w < 1 || h < 1)
{
    StatusText.Text = "Width/Height must be at least 1.";
    return false;
}

if (w > 2000 || h > 2000)
{
    StatusText.Text = "Width/Height too large (max 2000 each).";
    return false;
}

return true;
}

private static MapDto CreateNewBlankMap(string mapId, string name, int w, int h)
{
    const int tileSize = 32;

    var tiles = Enumerable.Repeat(-1, w * h).ToArray();
    var collision = new int[w * h];

    var map = new MapDto(

```

```

    FormatVersion: MapFormat.CurrentVersion,
    MapId: mapId,
    Name: name,
    Width: w,
    Height: h,
    TileSize: tileSize,
    Tilesets: new[]
    {
        new TilesetRefDto(
            TilesetId: "default_tileset",
            ImagePath: "",
            TileWidth: 32,
            TileHeight: 32,
            Margin: 0,
            Spacing: 0
        )
    },
    Layers: new[]
    {
        new TileLayerDto("bg", "Background", LayerKind.Background, true, tiles),
        new TileLayerDto("col", "Collision", LayerKind.Collision, true, collision)
    }
);

return map with { Events = Array.Empty<MapEventDto>() };
}

private static MapDto CreateSampleMap(int w, int h)

```

```

{
    const int tileSize = 32;

    var tiles = Enumerable.Repeat(-1, w * h).ToArray();
    var collision = new int[w * h];

    var map = new MapDto(
        FormatVersion: MapFormat.CurrentVersion,
        MapId: "test_map_01",
        Name: "Test Map 01",
        Width: w,
        Height: h,
        TileSize: tileSize,
        Tilesets: new[]
        {
            new TilesetRefDto(
                TilesetId: "default_tileset",
                ImagePath: "",
                TileWidth: 32,
                TileHeight: 32,
                Margin: 0,
                Spacing: 0
            )
        },
        Layers: new[]
        {
            new TileLayerDto("bg", "Background", LayerKind.Background, true, tiles),
            new TileLayerDto("col", "Collision", LayerKind.Collision, true, collision)
        }
    );
}

```

```

    }
);

return map with { Events = Array.Empty<MapEventDto>() };
}

// -----
// Collision layer compatibility
// -----

private void EnsureCollisionLayer()
{
    var map = _currentMap;
    if (map is null) return;

    bool hasCollision =
        map.Layers.Any(l =>
            l.Kind == LayerKind.Collision ||
            l.LayerId == "col" ||
            l.Name.Equals("Collision", StringComparison.OrdinalIgnoreCase));

    if (hasCollision) return;

    var col = new TileLayerDto(
        LayerId: "col",
        Name: "Collision",
        Kind: LayerKind.Collision,
        Visible: true,
        Tiles: new int[map.Width * map.Height]
    );
}

```

```
);
```

```
var newLayers = map.Layers.Concat(new[] { col }).ToArray();
```

```
_currentMap = map with { Layers = newLayers };
```

```
PersistMap();
```

```
}
```

```
private TileLayerDto? GetBackgroundLayer() =>
```

```
_currentMap?.Layers.FirstOrDefault(l => l.Kind == LayerKind.Background);
```

```
private TileLayerDto? GetCollisionLayer() =>
```

```
_currentMap?.Layers.FirstOrDefault(l => l.Kind == LayerKind.Collision || l.LayerId ==  
"col");
```

```
// -----
```

```
// Tool UI events
```

```
// -----
```

```
private void OnModeChanged(object sender, RoutedEventArgs e)
```

```
{
```

```
    if (!IsLoaded) return;
```

```
    if (ModeEvents.IsChecked == true) _mode = EditMode.Events;
```

```
    else if (ModeCollision.IsChecked == true) _mode = EditMode.Collision;
```

```
    else _mode = EditMode.Tiles;
```

```
    StatusText.Text = $"Mode: {_mode}";
```

```
}
```

```

private void OnOverlayChanged(object sender, RoutedEventArgs e)
{
    _showCollisionOverlay = ShowCollisionOverlay.IsChecked == true;
    _showEventsOverlay = ShowEventsOverlay.IsChecked == true;
    RequestViewportRender();
}

```

```

// -----

```

```

// Tileset loading

```

```

// -----

```

```

private void OnLoadTilesetsClick(object sender, RoutedEventArgs e)
{
    if (_currentMap is null)
    {
        StatusText.Text = "Load or create a map first.";
        return;
    }
}

```

```

Directory.CreateDirectory(PngsFolder);

```

```

var dlg = new OpenFileDialog
{
    Title = "Load Tileset PNG(s)",
    Filter = "PNG (*.png)|*.png|All files (*.*)|*.*",
    InitialDirectory = PngsFolder,
    CheckFileExists = true,
    Multiselect = true
}

```



```
};
```

```
if (dlg.ShowDialog(this) != true)
```

```
    return;
```

```
int added = 0;
```

```
int reenabled = 0;
```

```
foreach (var file in dlg.FileNames)
```

```
{
```

```
    if (!File.Exists(file)) continue;
```

```
    if (TryAddTilesetFromFile(file, out _, out var changed))
```

```
    {
```

```
        added++;
```

```
    }
```

```
    else
```

```
    {
```

```
        if (changed) reenabled++;
```

```
    }
```

```
}
```

```
if (added > 0 || reenabled > 0)
```

```
{
```

```
    MarkTilesetsDirty();
```

```
    RefreshTilesetPanel();
```

```
    BuildTilePalette();
```

```
    PersistMap();
```

```
RequestViewportRender();
```

```
    StatusText.Text = $"Tilesets updated. Added: {added}, Re-enabled: {reenabled}.";
}
else
{
    RefreshTilesetPanel();
    BuildTilePalette();
    RequestViewportRender();
}
}
```

```
private bool TryAddTilesetFromFile(string absoluteFile, out string statusMessage, out
bool stateChanged)
```

```
{
    statusMessage = "";
    stateChanged = false;

    if (_currentMap is null)
    {
        statusMessage = "No map is loaded.";
        return false;
    }
}
```

```
string rel;
```

```
try
```

```
{
    rel = IOPath.GetRelativePath(ProjectRoot, absoluteFile);
```

```

    }
    catch
    {
        statusMessage = $"Tileset path is invalid: {absoluteFile}";
        return false;
    }

    if (rel.StartsWith(".", StringComparison.Ordinal))
    {
        statusMessage = "Tileset must be inside the project folder (Vitriol.Projects).";
        return false;
    }

    rel = rel.Replace('\\', '/');
    var relKey = NormalizeTilesetKey(rel);

    var existingPaths = new HashSet<string>(
        (_currentMap.Tilesets ?? Array.Empty<TilesetRefDto>())
            .Select(t => NormalizeTilesetKey(t.ImagePath ?? "")),
        StringComparer.OrdinalIgnoreCase);

    if (existingPaths.Contains(relKey))
    {
        if (!IsTilesetLoaded(relKey))
        {
            _tilesetLoadedState[relKey] = true;
            stateChanged = true;
        }
    }

```

```

        statusMessage = $"Tileset already referenced: {rel}";
        return false;
    }

    var baseId = IOPath.GetFileNameWithoutExtension(absoluteFile);
    if (string.IsNullOrEmpty(baseId)) baseId = "tileset";

    var existingIds = new HashSet<string>(
        (_currentMap.Tilesets ?? Array.Empty<TilesetRefDto>()).Select(t => t.TilesetId),
        StringComparer.OrdinalIgnoreCase);

    string id = baseId;
    int n = 2;
    while (existingIds.Contains(id))
    {
        id = $"{baseId}_{n}";
        n++;
    }

    var newTs = new TilesetRefDto(
        TilesetId: id,
        ImagePath: rel,
        TileWidth: 32,
        TileHeight: 32,
        Margin: 0,
        Spacing: 0
    );

```

```

var list = (_currentMap.Tilesets ?? Array.Empty<TilesetRefDto>()).ToList();

list.Add(newTs);

_currentMap = _currentMap with { Tilesets = list.ToArray() };


_tilesetLoadedState[relKey] = true;


statusMessage = $"Tileset added: {id}";

stateChanged = true;

return true;
}


// -----
// Tile palette
// -----

private void BuildTilePalette()
{
    TilePalettePanel.Children.Clear();

    _selectedPaletteBorder = null;


var map = _currentMap;

if (map is null)

    return;


EnsureTilesetsCompiled();


bool any = false;

```

```

foreach (var ts in _compiledTilesets)
{
    if (!ts.IsLoaded || ts.Bitmap is null || ts.TileCount <= 0 || ts.CropCache is null)
        continue;

    any = true;

    for (int local = 0; local < ts.TileCount; local++)
    {
        int global = ts.StartIndexGlobal + local;

        var crop = ts.CropCache.GetOrAdd(local, tileIndex =>
        {
            int tx = ts.TilesAcross > 0 ? (tileIndex % ts.TilesAcross) : 0;
            int ty = ts.TilesAcross > 0 ? (tileIndex / ts.TilesAcross) : 0;

            int px = ts.Margin + tx * (ts.TileW + ts.Spacing);
            int py = ts.Margin + ty * (ts.TileH + ts.Spacing);

            if (px + ts.TileW > ts.Bitmap.PixelWidth || py + ts.TileH > ts.Bitmap.PixelHeight)
            {
                var dummy = new CroppedBitmap(ts.Bitmap, new Int32Rect(0, 0, 1, 1));
                dummy.Freeze();
                return dummy;
            }

            var cb = new CroppedBitmap(ts.Bitmap, new Int32Rect(px, py, ts.TileW,
ts.TileH));

```

```

        cb.Freeze();

        return cb;

    });

    var img = new Image
    {
        Width = 32,
        Height = 32,
        Source = crop,
        Stretch = Stretch.Fill,
        SnapsToDevicePixels = true
    };

    RenderOptions.SetBitmapScalingMode(img,
    BitmapScalingMode.NearestNeighbor);

    var border = new Border
    {
        BorderBrush = Brushes.Transparent,
        BorderThickness = new Thickness(2),
        Margin = new Thickness(4),
        Child = img,
        Tag = global
    };

    border.MouseLeftButtonDown += (_, _) => SelectPaletteTile(border);

    TilePalettePanel.Children.Add(border);

    if (global == _selectedTileIndex)

```

```

        SelectPaletteTile(border, updateStatus: false);
    }
}

if (!any)
    StatusText.Text = "No tilesets are currently enabled (palette is empty).";
}

private void SelectPaletteTile(Border border, bool updateStatus = true)
{
    if (_selectedPaletteBorder != null)
        _selectedPaletteBorder.BorderBrush = Brushes.Transparent;

    _selectedPaletteBorder = border;
    _selectedPaletteBorder.BorderBrush = Brushes.Gold;

    _selectedTileIndex = (int)border.Tag;

    if (updateStatus)
        StatusText.Text = $"Selected tile: {_selectedTileIndex}";
}

// -----
// Viewport rendering
// -----

private void EnsureViewportImageExists()
{
    if (_viewportImage != null) return;

```



```

        _viewportImage = new Image
    {
        Stretch = Stretch.None,
        SnapsToDevicePixels = true
    };

    RenderOptions.SetBitmapScalingMode(_viewportImage,
BitmapScalingMode.NearestNeighbor);

    MapCanvas.Children.Clear();

    MapCanvas.Children.Add(_viewportImage);

    // If overlays were created earlier, re-add them if they lost their parent.
    if (_dragGhostShape is not null && _dragGhostShape.Parent is null)
        MapCanvas.Children.Add(_dragGhostShape);
    if (_dragGhostOutlineShape is not null && _dragGhostOutlineShape.Parent is null)
        MapCanvas.Children.Add(_dragGhostOutlineShape);

}

private void RequestViewportRender()
{
    if (_currentMap is null) return;
    _renderDebounce.Stop();
    _renderDebounce.Start();
}

private void OnScrollChanged(object sender, ScrollChangedEventArgs e)

```

```
{  
    if (_currentMap is null) return;  
    RequestViewportRender();  
}
```

```
private void RenderViewport()
```

```
{  
    var map = _currentMap;  
    if (map is null) return;
```

```
    EnsureViewportImageExists();  
    EnsureTilesetsCompiled();
```

```
    int mapPxW = checked(map.Width * map.TileSize);  
    int mapPxH = checked(map.Height * map.TileSize);  
    MapCanvas.Width = mapPxW;  
    MapCanvas.Height = mapPxH;
```

```
    var bg = GetBackgroundLayer();  
    if (bg is null) return;
```

```
    var col = GetCollisionLayer();
```

```
    double viewLeftWorld = MapScroll.HorizontalOffset / _zoom;  
    double viewTopWorld = MapScroll.VerticalOffset / _zoom;  
    double viewWidthWorld = MapScroll.ViewportWidth / _zoom;  
    double viewHeightWorld = MapScroll.ViewportHeight / _zoom;
```

```
int pad = map.TileSize * 2;
```

```
int regionX = ClampToInt(viewLeftWorld) - pad;
```

```
int regionY = ClampToInt(viewTopWorld) - pad;
```

```
int regionW = ClampToInt(viewWidthWorld) + pad * 2;
```

```
int regionH = ClampToInt(viewHeightWorld) + pad * 2;
```

```
if (regionX < 0) regionX = 0;
```

```
if (regionY < 0) regionY = 0;
```

```
if (regionX + regionW > mapPxW) regionW = mapPxW - regionX;
```

```
if (regionY + regionH > mapPxH) regionH = mapPxH - regionY;
```

```
if (regionW <= 1 || regionH <= 1) return;
```

```
int startTileX = regionX / map.TileSize;
```

```
int startTileY = regionY / map.TileSize;
```

```
int endTileX = (regionX + regionW + map.TileSize - 1) / map.TileSize;
```

```
int endTileY = (regionY + regionH + map.TileSize - 1) / map.TileSize;
```

```
startTileX = Math.Max(0, startTileX);
```

```
startTileY = Math.Max(0, startTileY);
```

```
endTileX = Math.Min(map.Width, endTileX);
```

```
endTileY = Math.Min(map.Height, endTileY);
```

```
var visual = new DrawingVisual();
```

```
using (var dc = visual.RenderOpen())
```

```
{
```

```
    dc.DrawRectangle(Brushes.White, null, new Rect(0, 0, regionW, regionH));
```

```

for (int ty = startTileY; ty < endTileY; ty++)
for (int tx = startTileX; tx < endTileX; tx++)
{
    int idx = ty * map.Width + tx;

    int t = bg.Tiles[idx];

    if (t < 0) continue;

    var ts = FindRuntimeForGlobalTile(t);

    if (ts is null) continue;

    if (!ts.IsLoaded || ts.Bitmap is null || ts.CropCache is null || ts.TileCount <= 0)
        continue;

    int local = GetLocalIndex(ts, t);

    if (local < 0 || local >= ts.TileCount) continue;

    var crop = ts.CropCache.GetOrAdd(local, tileIndex =>
    {
        int cx = ts.TilesAcross > 0 ? (tileIndex % ts.TilesAcross) : 0;
        int cy = ts.TilesAcross > 0 ? (tileIndex / ts.TilesAcross) : 0;

        int px = ts.Margin + cx * (ts.TileW + ts.Spacing);
        int py = ts.Margin + cy * (ts.TileH + ts.Spacing);

        if (px + ts.TileW > ts.Bitmap.PixelWidth || py + ts.TileH > ts.Bitmap.PixelHeight)
        {
            var dummy = new CroppedBitmap(ts.Bitmap, new Int32Rect(0, 0, 1, 1));

```

```

        dummy.Freeze();

        return dummy;
    }

    var cb = new CroppedBitmap(ts.Bitmap, new Int32Rect(px, py, ts.TileW,
ts.TileH));

    cb.Freeze();

    return cb;
});

double drawX = tx * map.TileSize - regionX;
double drawY = ty * map.TileSize - regionY;

dc.DrawImage(crop, new Rect(drawX, drawY, map.TileSize, map.TileSize));
}

if (_showCollisionOverlay && col is not null && col.Tiles.Length == map.Width *
map.Height)
{
    var brush = new SolidColorBrush(Color.FromArgb(110, 255, 0, 0));
    brush.Freeze();

    for (int ty = startTileY; ty < endTileY; ty++)
    for (int tx = startTileX; tx < endTileX; tx++)
    {
        int idx = ty * map.Width + tx;

        if (idx < 0 || idx >= col.Tiles.Length) continue;

        if (col.Tiles[idx] != 0)

```

```

    {
        double drawX = tx * map.TileSize - regionX;
        double drawY = ty * map.TileSize - regionY;
        dc.DrawRectangle(brush, null, new Rect(drawX, drawY, map.TileSize,
map.TileSize));
    }
}
}

if (_showEventsOverlay && map.Events is not null && map.Events.Count > 0)
{
    var triggerFill = new SolidColorBrush(Color.FromArgb(220, 255, 215, 0));
    triggerFill.Freeze();

    var npcFill = new SolidColorBrush(Color.FromArgb(220, 80, 170, 255));
    npcFill.Freeze();

    var warpFill = new SolidColorBrush(Color.FromArgb(220, 190, 90, 255));
    warpFill.Freeze();

    var outlinePen = new Pen(new SolidColorBrush(Color.FromArgb(255, 10, 10,
10)), 1);
    outlinePen.Freeze();

    var selPen = new Pen(Brushes.White, 2);
    selPen.Freeze();

    foreach (var ev in map.Events)
    {

```

```

if (ev.X < startTileX || ev.X >= endTileX || ev.Y < startTileY || ev.Y >= endTileY)
    continue;

double tileLeft = ev.X * map.TileSize - regionX;
double tileTop = ev.Y * map.TileSize - regionY;

bool isSelected = !string.IsNullOrEmpty(_selectedEventId) && ev.EventId
== _selectedEventId;

var type = (ev.Type ?? "").Trim();

if (string.Equals(type, "NPC", StringComparison.OrdinalIgnoreCase))
{
    var center = new Point(tileLeft + map.TileSize / 2.0, tileTop + map.TileSize /
2.0);

    double r = map.TileSize * 0.22;

    dc.DrawEllipse(npcFill, outlinePen, center, r, r);

    if (isSelected)
        dc.DrawRectangle(null, selPen, new Rect(tileLeft + 1, tileTop + 1,
map.TileSize - 2, map.TileSize - 2));
}
else if (string.Equals(type, "Warp", StringComparison.OrdinalIgnoreCase))
{
    var cx = tileLeft + map.TileSize / 2.0;
    var cy = tileTop + map.TileSize / 2.0;
    double r = map.TileSize * 0.24;

    var geo = new StreamGeometry();

```

```

using (var g = geo.Open())
{
    g.BeginFigure(new Point(cx, cy - r), isFilled: true, isClosed: true);
    g.LineTo(new Point(cx + r, cy), isStroked: true, isSmoothJoin: false);
    g.LineTo(new Point(cx, cy + r), isStroked: true, isSmoothJoin: false);
    g.LineTo(new Point(cx - r, cy), isStroked: true, isSmoothJoin: false);
}
geo.Freeze();

dc.DrawGeometry(warpFill, outlinePen, geo);

if (isSelected)
    dc.DrawRectangle(null, selPen, new Rect(tileLeft + 1, tileTop + 1,
map.TileSize - 2, map.TileSize - 2));
}
else
{
    var rr = new Rect(
        tileLeft + map.TileSize * 0.28,
        tileTop + map.TileSize * 0.28,
        map.TileSize * 0.44,
        map.TileSize * 0.44);

    dc.DrawRectangle(triggerFill, outlinePen, rr);

    if (isSelected)
        dc.DrawRectangle(null, selPen, new Rect(tileLeft + 1, tileTop + 1,
map.TileSize - 2, map.TileSize - 2));
}

```



```
}  
}
```

```
if (_playtestActive)  
{  
    if (_playerX >= startTileX && _playerX < endTileX && _playerY >= startTileY &&  
        _playerY < endTileY)  
    {  
        double px = _playerX * map.TileSize - regionX;  
        double py = _playerY * map.TileSize - regionY;  
  
        var fill = new SolidColorBrush(Color.FromArgb(220, 90, 180, 255));  
        fill.Freeze();  
        var pen = new Pen(Brushes.Black, 1);  
        pen.Freeze();  
  
        dc.DrawEllipse(fill, pen,  
            new Point(px + map.TileSize / 2.0, py + map.TileSize / 2.0),  
            map.TileSize * 0.28,  
            map.TileSize * 0.28);  
    }  
}
```

```
var gridBrush = new SolidColorBrush(Color.FromArgb(90, 0, 0, 0));  
gridBrush.Freeze();
```

```
var gridPen = new Pen(gridBrush, 1);  
gridPen.Freeze();
```

```
RenderOptions.SetEdgeMode(visual, EdgeMode.Aliased);
```

```
var guidelines = new GuidelineSet();
```

```
for (int tx = startTileX; tx <= endTileX; tx++)
```

```
{
```

```
    double x = (tx * map.TileSize - regionX);
```

```
    guidelines.GuidelinesX.Add(x);
```

```
}
```

```
for (int ty = startTileY; ty <= endTileY; ty++)
```

```
{
```

```
    double y = (ty * map.TileSize - regionY);
```

```
    guidelines.GuidelinesY.Add(y);
```

```
}
```

```
dc.PushGuidelineSet(guidelines);
```

```
for (int tx = startTileX; tx <= endTileX; tx++)
```

```
{
```

```
    double x = (tx * map.TileSize - regionX);
```

```
    dc.DrawLine(gridPen, new Point(x, 0), new Point(x, regionH));
```

```
}
```

```
for (int ty = startTileY; ty <= endTileY; ty++)
```

```
{
```

```
    double y = (ty * map.TileSize - regionY);
```

```

        dc.DrawLine(gridPen, new Point(0, y), new Point(regionW, y));
    }

    dc.Pop();
}

var rtb = new RenderTargetBitmap(regionW, regionH, 96, 96, PixelFormats.Pbgra32);
rtb.Render(visual);
rtb.Freeze();

_viewportImage!.Source = rtb;
_viewportImage.Width = regionW;
_viewportImage.Height = regionH;

Canvas.SetLeft(_viewportImage, regionX);
Canvas.SetTop(_viewportImage, regionY);
}

private static int ClampToInt(double v)
{
    if (v <= int.MinValue) return int.MinValue;
    if (v >= int.MaxValue) return int.MaxValue;
    return (int)Math.Floor(v);
}

private static BitmapSource LoadBitmap(string path)
{
    var bmp = new BitmapImage();

```

```

    bmp.BeginInit();
    bmp.CacheOption = BitmapCacheOption.OnLoad;
    bmp.UriSource = new Uri(path, UriKind.Absolute);
    bmp.EndInit();
    bmp.Freeze();
    return bmp;
}

```

```

private void OnSaveMapClick(object sender, RoutedEventArgs e)
{
    if (_currentMap is null)
    {
        StatusText.Text = "Nothing to save (no map loaded).";
        return;
    }
}

```

```

Directory.CreateDirectory(MapsFolder);

```

```

if (string.IsNullOrEmpty(_currentMapPath))
{
    var dlg = new SaveFileDialog
    {
        Title = "Save Vitriol Map",
        Filter = "Vitriol Map (*.vitmap.json)|*.vitmap.json|JSON (*.json)|*.json|All files (*.*)|*.*",
        InitialDirectory = MapsFolder,
        FileName = (_currentMap.MapId ?? "new_map") + MapFormat.DefaultExtension,
        OverwritePrompt = true
    };
}

```

```

};

if (dlg.ShowDialog(this) != true)
    return;

    _currentMapPath = dlg.FileName;
}

try
{
    MapSerializer.SaveToFile(_currentMapPath, _currentMap);

    if (!IsSampleMapLoaded())
        _manualSavePerformed = true;

    StatusText.Text = $"Saved: {_currentMapPath}";
}
catch (Exception ex)
{
    StatusText.Text = $"Save failed: {ex.Message}";
}
}

// -----
// Mouse + painting + event clicks
// -----

private void OnMapCanvasMouseLeftButtonDown(object sender,
MouseButtonEventArgs e)

```

```

{
    if (_currentMap is null) return;

    if (_playtestActive)
    {
        StatusText.Text = "Playtest is active (F5 to stop). Editing is disabled.";
        return;
    }

    var p = e.GetPosition(MapCanvas);
    SetCursorFromPoint(p);

    if (_mode == EditMode.Events)
    {
        HandleEventLeftClick(p);
        return;
    }

    StartPainting();
    PaintAtMouse(p, isRightButton: false);
}

private void OnMapCanvasMouseRightButtonDown(object sender,
MouseButtonEventArgs e)
{
    if (_currentMap is null) return;

    if (_playtestActive)

```

```
{  
    StatusText.Text = "Playtest is active (F5 to stop). Editing is disabled.";  
    return;  
}
```

```
var p = e.GetPosition(MapCanvas);  
SetCursorFromPoint(p);
```

```
if (_mode == EditMode.Events)  
{  
    _selectedEventId = null;  
    _selectedCommandIndex = -1;  
    UpdateEventPanel(null);  
    RefreshCommandList(null);  
    UpdateCommandEditor(null);  
    RequestViewportRender();  
    return;  
}
```

```
StartPainting();  
PaintAtMouse(p, isRightButton: true);  
}
```

```
private void OnMapCanvasMouseLeftButtonUp(object sender,  
MouseButtonEventArgs e)  
{  
    StopPainting();  
    StopEventDrag(commit: true);
```

```
}
```

```
private void StartPainting()
```

```
{
```

```
    _isPainting = true;
```

```
    _lastPaintIdx = -1;
```

```
    MapCanvas.CaptureMouse();
```

```
}
```

```
private void StopPainting()
```

```
{
```

```
    _isPainting = false;
```

```
    _lastPaintIdx = -1;
```

```
    if (Mouse.Captured == MapCanvas)
```

```
        MapCanvas.ReleaseMouseCapture();
```

```
}
```

```
private void OnMapCanvasMouseMove(object sender, MouseEventArgs e)
```

```
{
```

```
    var map = _currentMap;
```

```
    if (map is null) return;
```

```
    var p = e.GetPosition(MapCanvas);
```

```
    int tileX = (int)Math.Floor(p.X / map.TileSize);
```

```
    int tileY = (int)Math.Floor(p.Y / map.TileSize);
```

```
    _hoverTileX = tileX;
```



```
_hoverTileY = tileY;
```

```
if (tileX < 0 || tileY < 0 || tileX >= map.Width || tileY >= map.Height)
```

```
{
```

```
    MouseText.Text = "";
```

```
    _hoverTileX = -1;
```

```
    _hoverTileY = -1;
```

```
    // If dragging, clamp so ghost doesn't disappear
```

```
    if (_draggingEvent)
```

```
    {
```

```
        _dragMouseWorldX = p.X;
```

```
        _dragMouseWorldY = p.Y;
```

```
        UpdateDragOverlays();
```

```
        UpdateDragViewportPoint(e.GetPosition(MapScroll));
```

```
    }
```

```
    return;
```

```
}
```

```
if (_mode == EditMode.Events && _draggingEvent)
```

```
{
```

```
    _dragMouseWorldX = p.X;
```

```
    _dragMouseWorldY = p.Y;
```

```
    UpdateDragOverlays();
```

```
    UpdateDragViewportPoint(e.GetPosition(MapScroll));
```

```
}
```

```

int idx = tileY * map.Width + tileX;

int colVal = GetCollisionLayer()?.Tiles.ElementAtOrDefault(idx) ?? 0;

var hoverEv = GetEventAt(tileX, tileY);
var hoverText = (hoverEv is null) ? "" : " | " + DescribeEventForHover(hoverEv);

MouseText.Text =
    $"Tile: ({tileX},{tileY}) Mode:{_mode} Col:{colVal}" +
    (_playtestActive ? $" Player:({_playerX},{_playerY})" : "") +
    hoverText;

if (_isPainting && _mode != EditMode.Events && !_playtestActive)
{
    bool right = e.RightButton == MouseButtonState.Pressed;
    bool left = e.LeftButton == MouseButtonState.Pressed;
    if (left || right)
        PaintAtMouse(p, isRightButton: right && !left);
}
}

private void SetCursorFromPoint(Point p)
{
    var map = _currentMap;
    if (map is null) return;

    int tileX = (int)Math.Floor(p.X / map.TileSize);
    int tileY = (int)Math.Floor(p.Y / map.TileSize);

```

```
if (tileX < 0 || tileY < 0 || tileX >= map.Width || tileY >= map.Height)
    return;

_cursorTileX = tileX;
_cursorTileY = tileY;
}
```

```
private void PaintAtMouse(Point p, bool isRightButton)
{
    var map = _currentMap;
    if (map is null) return;

    int tileX = (int)Math.Floor(p.X / map.TileSize);
    int tileY = (int)Math.Floor(p.Y / map.TileSize);
    if (tileX < 0 || tileY < 0 || tileX >= map.Width || tileY >= map.Height)
        return;

    int idx = tileY * map.Width + tileX;

    if (idx == _lastPaintIdx) return;
    _lastPaintIdx = idx;

    if (_mode == EditMode.Tiles)
    {
        var bg = GetBackgroundLayer();
        if (bg is null) return;
    }
}
```

```

        int oldValue = bg.Tiles[idx];

        int newValue = _selectedTileIndex;

        if (oldValue == newValue) return;

        ExecuteCommand(new PaintIntCellCommand(this, LayerKind.Background, idx,
oldValue, newValue));
    }

    else if (_mode == EditMode.Collision)
    {
        var col = GetCollisionLayer();

        if (col is null) return;

        int oldValue = col.Tiles[idx];

        int newValue = isRightButton ? 0 : 1;

        if (oldValue == newValue) return;

        ExecuteCommand(new PaintIntCellCommand(this, col.Kind, idx, oldValue,
newValue));
    }
}

private void OnCreateNewMapClick(object sender, RoutedEventArgs e)
{
    if (!TryReadMapSize(out int w, out int h))
        return;

    Directory.CreateDirectory(MapsFolder);

```

```

var dlg = new SaveFileDialog
{
    Title = "Create New Map",
    Filter = "Vitriol Map (*.vitmap.json)|*.vitmap.json|JSON (*.json)|*.json|All files (*.*)|*.*",
    InitialDirectory = MapsFolder,
    FileName = "new_map" + MapFormat.DefaultExtension,
    OverwritePrompt = false
};

if (dlg.ShowDialog(this) != true)
    return;

var chosenPath = dlg.FileName;

var baseName = IOPath.GetFileName(chosenPath);

if (baseName.EndsWith(".json", StringComparison.OrdinalIgnoreCase))
    baseName = baseName[..^5];

if (baseName.EndsWith(".vitmap", StringComparison.OrdinalIgnoreCase))
    baseName = baseName[..^7];

baseName = baseName.Trim();

if (baseName.Length == 0) baseName = "new_map";

var mapId = MakeSafeMapId(baseName);
var mapName = baseName;

```

```
var map = CreateNewBlankMap(mapId, mapName, w, h);
```

```
var errors = MapValidator.Validate(map);
```

```
if (errors.Count > 0)
```

```
{
```

```
    StatusText.Text = "Map validation failed: " +
```

```
        string.Join("; ", errors.Select(er => $"{er.Field}: {er.Message}"));
```

```
    return;
```

```
}
```

```
_currentMap = map;
```

```
_currentMapPath = chosenPath;
```

```
_manualSavePerformed = false;
```

```
if (_currentMap.Events is null)
```

```
    _currentMap = _currentMap with { Events = Array.Empty<MapEventDto>() };
```

```
_selectedEventId = null;
```

```
_selectedCommandIndex = -1;
```

```
UpdateEventPanel(null);
```

```
RefreshCommandList(null);
```

```
UpdateCommandEditor(null);
```

```
EnsureCollisionLayer();
```

```
_undo.Clear();
```

```
_redo.Clear();
```

```

var tilesets = _currentMap.Tilesets ?? Array.Empty<TilesetRefDto>();
foreach (var ts in tilesets)
{
    var key = NormalizeTilesetKey(ts.ImagePath ?? "");
    if (key.Length == 0) continue;
    _tilesetLoadedState.TryAdd(key, true);
}

MarkTilesetsDirty();

EnsureViewportImageExists();
RefreshTilesetPanel();
BuildTilePalette();

MapScroll.ScrollToHorizontalOffset(0);
MapScroll.ScrollToVerticalOffset(0);

RefreshAvailableMaps(preserveCurrentSelection: false);

    StatusText.Text = $"Created new map (not yet saved): {chosenPath} — press Save to
enable autosave.";

    RequestViewportRender();
}

// XAML handler: do not remove / rename
private void OnMapCanvasMouseLeave(object sender, MouseEventArgs e)
{

```

```

    MouseText.Text = "";

    _hoverTileX = -1;
    _hoverTileY = -1;

    StopPainting();
    StopEventDrag(commit: false);
}

// -----
// Playtest execution
// -----

private void TogglePlaytest()
{
    if (_currentMap is null)
    {
        StatusText.Text = "Load or create a map first.";
        return;
    }

    _playtestActive = !_playtestActive;

    if (_playtestActive)
    {
        _playtestFlags.Clear();

        _playerX = Math.Clamp(_playerX, 0, _currentMap.Width - 1);
        _playerY = Math.Clamp(_playerY, 0, _currentMap.Height - 1);

        StatusText.Text = "Playtest ON — F5 toggles, WASD/Arrows move, E/Enter interact,
Esc exit.";
    }
}

```



```

    }

    else

    {
        StatusText.Text = "Playtest OFF.";
    }

    RequestViewportRender();
}

private void ExitPlaytest()
{
    if (!_playtestActive) return;
    _playtestActive = false;
    StatusText.Text = "Playtest OFF.";
    RequestViewportRender();
}

private void TryMovePlayer(int dx, int dy)
{
    var map = _currentMap;
    if (map is null) return;
    if (!_playtestActive) return;

    int nx = _playerX + dx;
    int ny = _playerY + dy;

    if (nx < 0 || ny < 0 || nx >= map.Width || ny >= map.Height)
    {

```

```
    StatusText.Text = "Blocked: map boundary.";
    return;
}
```

```
var col = GetCollisionLayer();
if (col is not null && col.Tiles.Length == map.Width * map.Height)
{
    int idx = ny * map.Width + nx;
    if (idx >= 0 && idx < col.Tiles.Length && col.Tiles[idx] != 0)
    {
        StatusText.Text = "Blocked: collision tile.";
        return;
    }
}
```

```
_playerX = nx;
_playerY = ny;
```

```
RequestViewportRender();
```

```
var ev = GetEventAt(_playerX, _playerY);
if (ev is null) return;

if (string.Equals(ev.Type, "Warp", StringComparison.OrdinalIgnoreCase))
{
    DoWarp(ev);
    return;
}
```

```
        if (string.Equals(ev.Type, "Trigger", StringComparison.OrdinalIgnoreCase))
            ExecuteEventScript(ev);
    }
```

```
private void InteractAtPlayer()
{
    var ev = GetEventAt(_playerX, _playerY);
    if (ev is null)
    {
        StatusText.Text = "Interact: no event present.";
        return;
    }
}
```

```
        if (string.Equals(ev.Type, "NPC", StringComparison.OrdinalIgnoreCase) ||
            string.Equals(ev.Type, "Trigger", StringComparison.OrdinalIgnoreCase))
        {
            ExecuteEventScript(ev);
            return;
        }
    }
```

```
        StatusText.Text = $"Interact: {ev.Name} ({ev.Type})";
    }
```

```
private static string DescribeEventForHover(MapEventDto ev)
{
    var type = (ev.Type ?? "Trigger").Trim();
    var name = string.IsNullOrEmpty(ev.Name) ? "Event" : ev.Name;
}
```

```

if (type.Equals("Warp", StringComparison.OrdinalIgnoreCase))
{
    var p = ev.Parameters ?? new Dictionary<string, string>();
    p.TryGetValue("destMap", out var dm);
    p.TryGetValue("destX", out var dx);
    p.TryGetValue("destY", out var dy);

    dm = string.IsNullOrEmpty(dm) ? "?" : dm;
    dx = string.IsNullOrEmpty(dx) ? "?" : dx;
    dy = string.IsNullOrEmpty(dy) ? "?" : dy;

    return $"Event: {name} (Warp → {dm} @ {dx},{dy})";
}

return $"Event: {name} ({type})";
}

```

```

private MapEventDto? GetEventAt(int x, int y)
{
    var map = _currentMap;
    if (map is null) return null;
    if (map.Events is null || map.Events.Count == 0) return null;
    return map.Events.FirstOrDefault(e => e.X == x && e.Y == y);
}

```

```

private void ExecuteEventScript(MapEventDto ev)
{

```

```
var cmds = ev.Commands ?? Array.Empty<EventCommandDto>();  
if (cmds.Count == 0)  
{  
    StatusText.Text = $"Event '{ev.Name}' has no script commands.";   
    return;  
}
```

```
foreach (var cmd in cmds)  
{  
    switch ((cmd.Command ?? "").Trim())  
    {  
        case "ShowText":  
        {  
            cmd.Args.TryGetValue("text", out var text);  
            text ??= "";  
            if (text.Length == 0) text = "(empty text)";  
            MessageBox.Show(this, text, ev.Name);  
            break;  
        }  
  
        case "SetFlag":  
        {  
            cmd.Args.TryGetValue("flag", out var flag);  
            cmd.Args.TryGetValue("value", out var value);  
            flag = (flag ?? "").Trim();  
            value ??= "";  
  
            if (flag.Length == 0)
```

```

    {
        StatusText.Text = "SetFlag failed: flag is empty.";
        break;
    }

    _playtestFlags[flag] = value;
    StatusText.Text = $"Flag set: {flag} = '{value}'";
    break;
}

default:
    StatusText.Text = $"Unknown command: {cmd.Command}";
    break;
}
}
}

```

```

private void DoWarp(MapEventDto ev)
{
    var p = ev.Parameters ?? new Dictionary<string, string>();

    p.TryGetValue("destMap", out var destMap);
    p.TryGetValue("destX", out var destXStr);
    p.TryGetValue("destY", out var destYStr);

    destMap = (destMap ?? "").Trim();
    if (string.IsNullOrEmpty(destMap))
    {

```

```

        StatusText.Text = "Warp failed: destMap missing.";
        return;
    }

    int.TryParse(destXStr, out var destX);
    int.TryParse(destYStr, out var destY);

    var destPath = IOPath.Combine(MapsFolder, destMap +
MapFormat.DefaultExtension);

    if (!File.Exists(destPath))
    {
        StatusText.Text = $"Warp failed: map not found '{destMap}'";
        return;
    }

    LoadMapFromPath(destPath, $"Warped to {destMap}");

    if (_currentMap is null) return;

    _playerX = Math.Clamp(destX, 0, _currentMap.Width - 1);
    _playerY = Math.Clamp(destY, 0, _currentMap.Height - 1);

    StatusText.Text = $"Warped: {ev.Name} -> {destMap} ({_playerX},{_playerY})";
    RequestViewportRender();
}

// -----

```

```

// Events editor + dragging
// -----

private void HandleEventLeftClick(Point p)
{
    var map = _currentMap;
    if (map is null) return;

    int tileX = (int)Math.Floor(p.X / map.TileSize);
    int tileY = (int)Math.Floor(p.Y / map.TileSize);
    if (tileX < 0 || tileY < 0 || tileX >= map.Width || tileY >= map.Height)
        return;

    _cursorTileX = tileX;
    _cursorTileY = tileY;

    var existing = map.Events.FirstOrDefault(ev => ev.X == tileX && ev.Y == tileY);
    if (existing is not null)
    {
        _selectedEventId = existing.EventId;
        _selectedCommandIndex = -1;
        UpdateEventPanel(existing);
        RefreshCommandList(existing);
        UpdateCommandEditor(null);

        BeginEventDrag(existing);
        RequestViewportRender();
        return;
    }
}

```



```
var newEv = new MapEventDto(  
    EventId: Guid.NewGuid().ToString("N"),  
    Name: "New Event",  
    Type: "Trigger",  
    X: tileX,  
    Y: tileY,  
    Parameters: new Dictionary<string, string>(),  
    Commands: Array.Empty<EventCommandDto>()  
);
```

```
ExecuteCommand(new AddEventCommand(this, newEv));
```

```
_selectedEventId = newEv.EventId;
```

```
_selectedCommandIndex = -1;
```

```
UpdateEventPanel(newEv);
```

```
RefreshCommandList(newEv);
```

```
UpdateCommandEditor(null);
```

```
RequestViewportRender();
```

```
}
```

```
private void EnsureDragOverlays()
```

```
{
```

```
if (_dragGhostShape is not null && _dragGhostOutlineShape is not null) return;
```

```
// Filled ghost (shape changes by event type)
```

```
_dragGhostShape = new System.Windows.Shapes.Rectangle
```

```
{
    Width = 18,
    Height = 18,
    RadiusX = 4,
    RadiusY = 4,
    Fill = new SolidColorBrush(Color.FromArgb(170, 255, 215, 0)), // Trigger-ish default
    StrokeThickness = 0,
    IsHitTestVisible = false,
    Visibility = Visibility.Collapsed
};
```

```
// Outline ghost (shape changes too)
```

```
_dragGhostOutlineShape = new System.Windows.Shapes.Rectangle
```

```
{
    Width = 22,
    Height = 22,
    RadiusX = 5,
    RadiusY = 5,
    Stroke = new SolidColorBrush(Color.FromArgb(220, 0, 0, 0)),
    StrokeThickness = 2,
    Fill = Brushes.Transparent,
    IsHitTestVisible = false,
    Visibility = Visibility.Collapsed
};
```

```
MapCanvas.Children.Add(_dragGhostShape);
```

```
MapCanvas.Children.Add(_dragGhostOutlineShape);
```

```
}
```

```
private void ShowDragOverlays()
{
    EnsureDragOverlays();
    _dragGhostShape!.Visibility = Visibility.Visible;
    _dragGhostOutlineShape!.Visibility = Visibility.Visible;
}
```

```
private void HideDragOverlays()
{
    if (_dragGhostShape is not null) _dragGhostShape.Visibility = Visibility.Collapsed;
    if (_dragGhostOutlineShape is not null) _dragGhostOutlineShape.Visibility =
    Visibility.Collapsed;
}
```

```
private void EnsureSwapHintOverlays()
{
    if (_swapHintFrom is not null && _swapHintTo is not null) return;
```

```
_swapHintFrom = new System.Windows.Shapes.Rectangle
{
    Stroke = Brushes.Gold,
    StrokeThickness = 3,
```

```

        Fill = Brushes.Transparent,
        IsHitTestVisible = false,
        Visibility = Visibility.Collapsed
    };

    _swapHintTo = new System.Windows.Shapes.Rectangle
    {
        Stroke = Brushes.White,
        StrokeThickness = 3,
        Fill = Brushes.Transparent,
        IsHitTestVisible = false,
        Visibility = Visibility.Collapsed
    };

    EnsureViewportImageExists();
    MapCanvas.Children.Add(_swapHintFrom);
    MapCanvas.Children.Add(_swapHintTo);

    _swapHintTimer.Interval = TimeSpan.FromMilliseconds(140);
    _swapHintTimer.Tick += (_, _) =>
    {
        _swapHintTimer.Stop();
        if (_swapHintFrom is not null) _swapHintFrom.Visibility = Visibility.Collapsed;
        if (_swapHintTo is not null) _swapHintTo.Visibility = Visibility.Collapsed;
    };
}

```

```
private void ShowSwapHint(int fromX, int fromY, int toX, int toY)
{
    var map = _currentMap;
    if (map is null) return;

    EnsureSwapHintOverlays();

    double s = map.TileSize;

    _swapHintFrom!.Width = s;
    _swapHintFrom.Height = s;
    Canvas.SetLeft(_swapHintFrom, fromX * s);
    Canvas.SetTop(_swapHintFrom, fromY * s);
    _swapHintFrom.Visibility = Visibility.Visible;

    _swapHintTo!.Width = s;
    _swapHintTo.Height = s;
    Canvas.SetLeft(_swapHintTo, toX * s);
    Canvas.SetTop(_swapHintTo, toY * s);
    _swapHintTo.Visibility = Visibility.Visible;

    _swapHintTimer.Stop();
    _swapHintTimer.Start();
}

private void ConfigureDragGhostShapeForEvent(MapEventDto ev)
{

```

```
var type = (ev.Type ?? "Trigger").Trim();
```

```
Brush fill =
```

```
    string.Equals(type, "NPC", StringComparison.OrdinalIgnoreCase)
```

```
        ? new SolidColorBrush(Color.FromArgb(220, 80, 170, 255))
```

```
        : string.Equals(type, "Warp", StringComparison.OrdinalIgnoreCase)
```

```
            ? new SolidColorBrush(Color.FromArgb(220, 190, 90, 255))
```

```
            : new SolidColorBrush(Color.FromArgb(220, 255, 215, 0));
```

```
// Swap shape classes (Rectangle/Ellipse/Polygon) so the ghost matches type
```

```
if (string.Equals(type, "NPC", StringComparison.OrdinalIgnoreCase))
```

```
{
```

```
    EnsureOverlays<System.Windows.Shapes.Ellipse>(ref _dragGhostShape);
```

```
    EnsureOverlays<System.Windows.Shapes.Ellipse>(ref _dragGhostOutlineShape);
```

```
    _dragGhostShape!.Fill = fill;
```

```
    _dragGhostOutlineShape!.Fill = Brushes.Transparent;
```

```
    _dragGhostOutlineShape!.Stroke = new SolidColorBrush(Color.FromArgb(220, 0, 0, 0));
```

```
    _dragGhostOutlineShape!.StrokeThickness = 2;
```

```
}
```

```
else if (string.Equals(type, "Warp", StringComparison.OrdinalIgnoreCase))
```

```
{
```

```
    EnsureOverlays<System.Windows.Shapes.Polygon>(ref _dragGhostShape);
```

```
    EnsureOverlays<System.Windows.Shapes.Polygon>(ref _dragGhostOutlineShape);
```

```
((System.Windows.Shapes.Polygon)_dragGhostShape!).Fill = fill;
```

```

((System.Windows.Shapes.Polygon)_dragGhostShape!).StrokeThickness = 0;

var o = (System.Windows.Shapes.Polygon)_dragGhostOutlineShape!;
o.Fill = Brushes.Transparent;
o.Stroke = new SolidColorBrush(Color.FromArgb(220, 0, 0, 0));
o.StrokeThickness = 2;
}
else
{
    EnsureOverlays<System.Windows.Shapes.Rectangle>(ref _dragGhostShape);
    EnsureOverlays<System.Windows.Shapes.Rectangle>(ref
_dragGhostOutlineShape);

    var r = (System.Windows.Shapes.Rectangle)_dragGhostShape!;
    r.Fill = fill;
    r.StrokeThickness = 0;

    var ro = (System.Windows.Shapes.Rectangle)_dragGhostOutlineShape!;
    ro.Fill = Brushes.Transparent;
    ro.Stroke = new SolidColorBrush(Color.FromArgb(220, 0, 0, 0));
    ro.StrokeThickness = 2;
}
}

```

```

private void EnsureOverlays<T>(ref System.Windows.Shapes.Shape? shape) where T :
System.Windows.Shapes.Shape, new()
{
    if (shape is T) return;

```

```
// preserve current z-order by replacing in-place

int idx = -1;

if (shape is not null) idx = MapCanvas.Children.IndexOf(shape);

var newShape = new T
{
    IsHitTestVisible = false,
    Visibility = shape?.Visibility ?? Visibility.Collapsed
};

// carry over basic styling defaults

if (newShape is System.Windows.Shapes.Rectangle rr)
{
    rr.RadiusX = 4;
    rr.RadiusY = 4;
}

if (idx >= 0)
{
    MapCanvas.Children.RemoveAt(idx);
    MapCanvas.Children.Insert(idx, newShape);
}
else
{
    MapCanvas.Children.Add(newShape);
}

shape = newShape;
```



```
}
```

```
private void BeginEventDrag(MapEventDto ev)
```

```
{
```

```
    var map = _currentMap;
```

```
    if (map is null) return;
```

```
    _draggingEvent = true;
```

```
    _dragEventId = ev.EventId;
```

```
    var mouse = Mouse.GetPosition(MapCanvas);
```

```
    _dragMouseWorldX = mouse.X;
```

```
    _dragMouseWorldY = mouse.Y;
```

```
    _dragPreviewX = ev.X;
```

```
    _dragPreviewY = ev.Y;
```

```
    MapCanvas.CaptureMouse();
```

```
    ShowDragOverlays();
```

```
    ConfigureDragGhostShapeForEvent(ev);
```

```
    UpdateDragOverlays();
```

```
    StatusText.Text = $"Dragging event '{ev.Name}' — release to drop.";
```

```
}
```

```
private void UpdateDragOverlays()
```

```
{
```

```

var map = _currentMap;

if (map is null) return;

if (!_draggingEvent) return;

if (_dragGhostShape is null || _dragGhostOutlineShape is null) return;


var ev = GetSelectedEvent();

if (ev is not null)
    ConfigureDragGhostShapeForEvent(ev);


// Snap preview to tile but clamp to map bounds so it never disappears
int tx = (int)Math.Floor(_dragMouseWorldX / map.TileSize);
int ty = (int)Math.Floor(_dragMouseWorldY / map.TileSize);


tx = Math.Clamp(tx, 0, map.Width - 1);
ty = Math.Clamp(ty, 0, map.Height - 1);


_dragPreviewX = tx;
_dragPreviewY = ty;


// Ghost follows mouse, but if mouse is outside map, clamp ghost to tile center
double cx = _dragMouseWorldX;
double cy = _dragMouseWorldY;


if (_dragMouseWorldX < 0 || _dragMouseWorldY < 0 ||
    _dragMouseWorldX >= map.Width * map.TileSize ||
    _dragMouseWorldY >= map.Height * map.TileSize)
{
    cx = tx * map.TileSize + map.TileSize / 2.0;

```

```

        cy = ty * map.TileSize + map.TileSize / 2.0;
    }

    double inner = map.TileSize * 0.40;
    double outer = map.TileSize * 0.48;

    _dragGhostShape.Width = inner;
    _dragGhostShape.Height = inner;

    _dragGhostOutlineShape.Width = outer;
    _dragGhostOutlineShape.Height = outer;

    // Warp uses diamond points
    if (_dragGhostShape is System.Windows.Shapes.Polygon p1)
    {
        p1.Points = new PointCollection
        {
            new Point(inner/2, 0),
            new Point(inner, inner/2),
            new Point(inner/2, inner),
            new Point(0, inner/2)
        };
    }

    if (_dragGhostOutlineShape is System.Windows.Shapes.Polygon p2)
    {
        p2.Points = new PointCollection
        {
            new Point(outer/2, 0),

```

```

        new Point(outer, outer/2),
        new Point(outer/2, outer),
        new Point(0, outer/2)
    };
}

// Trigger uses rounded rect
if (_dragGhostShape is System.Windows.Shapes.Rectangle r1)
{
    r1.RadiusX = Math.Max(2, inner * 0.18);
    r1.RadiusY = Math.Max(2, inner * 0.18);
}
if (_dragGhostOutlineShape is System.Windows.Shapes.Rectangle r2)
{
    r2.RadiusX = Math.Max(2, outer * 0.18);
    r2.RadiusY = Math.Max(2, outer * 0.18);
}

Canvas.SetLeft(_dragGhostShape, cx - inner / 2.0);
Canvas.SetTop(_dragGhostShape, cy - inner / 2.0);

Canvas.SetLeft(_dragGhostOutlineShape, cx - outer / 2.0);
Canvas.SetTop(_dragGhostOutlineShape, cy - outer / 2.0);
}

private void StopEventDrag(bool commit)
{

```

```
if (!_draggingEvent)
    return;

try
{
    if (!commit)
        return;

    var ev = GetSelectedEvent();
    if (ev is null || _dragEventId is null || ev.EventId != _dragEventId)
        return;

    var map = _currentMap;
    if (map is null) return;

    int nx = _dragPreviewX;
    int ny = _dragPreviewY;

    nx = Math.Clamp(nx, 0, map.Width - 1);
    ny = Math.Clamp(ny, 0, map.Height - 1);

    if (nx == ev.X && ny == ev.Y)
        return;

    // Prevent stacking events by swapping if needed
    var other = GetEventAt(nx, ny);
    if (other is not null && other.EventId != ev.EventId)
    {
```

```
var updatedA = ev with { X = nx, Y = ny };  
var updatedB = other with { X = ev.X, Y = ev.Y };
```

```
ExecuteCommand(new SwapEventsCommand(this, ev, updatedA, other,  
updatedB));
```

```
    _selectedEventId = updatedA.EventId;  
    UpdateEventPanel(updatedA);  
    RefreshCommandList(updatedA);  
    UpdateCommandEditor(null);  
  
    StatusText.Text = $"Events swapped.";   
    return;  
}
```

```
var updated = ev with { X = nx, Y = ny };  
ExecuteCommand(new ReplaceEventCommand(this, ev, updated));  
_selectedEventId = updated.EventId;
```

```
UpdateEventPanel(updated);  
RefreshCommandList(updated);  
UpdateCommandEditor(null);
```

```
    StatusText.Text = $"Event moved to ({nx},{ny}).";  
}  
finally  
{  
    _draggingEvent = false;
```

```
_dragEventId = null;
_dragPreviewX = -1;
_dragPreviewY = -1;

HideDragOverlays();
_dragAutoScrollTimer.Stop();

if (Mouse.Captured == MapCanvas)
    MapCanvas.ReleaseMouseCapture();

RequestViewportRender();
}
}

private void UpdateDragViewportPoint(Point mouseViewport)
{
    _dragLastViewportX = mouseViewport.X;
    _dragLastViewportY = mouseViewport.Y;

    if (_draggingEvent)
    {
        if (!_dragAutoScrollTimer.IsEnabled)
            _dragAutoScrollTimer.Start();
    }
    else
    {
        _dragAutoScrollTimer.Stop();
    }
}
```

```
}
```

```
private void TickDragAutoScroll()
```

```
{
```

```
    if (!_draggingEvent) { _dragAutoScrollTimer.Stop(); return; }
```

```
    if (_currentMap is null) { _dragAutoScrollTimer.Stop(); return; }
```

```
    double vw = MapScroll.ViewportWidth;
```

```
    double vh = MapScroll.ViewportHeight;
```

```
    if (vw <= 1 || vh <= 1) return;
```

```
    double dx = 0;
```

```
    double dy = 0;
```

```
    if (_dragLastViewportX < DragAutoScrollMargin) dx = -DragAutoScrollSpeed;
```

```
    else if (_dragLastViewportX > vw - DragAutoScrollMargin) dx =  
+DragAutoScrollSpeed;
```

```
    if (_dragLastViewportY < DragAutoScrollMargin) dy = -DragAutoScrollSpeed;
```

```
    else if (_dragLastViewportY > vh - DragAutoScrollMargin) dy =  
+DragAutoScrollSpeed;
```

```
    if (dx == 0 && dy == 0) return;
```

```
    double newX = MapScroll.HorizontalOffset + dx;
```

```
    double newY = MapScroll.VerticalOffset + dy;
```

```
    if (newX < 0) newX = 0;
```

```
    if (newY < 0) newY = 0;
```



```

MapScroll.ScrollToHorizontalOffset(newX);
MapScroll.ScrollToVerticalOffset(newY);

var mouseWorld = Mouse.GetPosition(MapCanvas);
_dragMouseWorldX = mouseWorld.X;
_dragMouseWorldY = mouseWorld.Y;
UpdateDragOverlays();
}

private MapEventDto? GetSelectedEvent()
{
    var map = _currentMap;
    if (map is null) return null;
    if (string.IsNullOrEmpty(_selectedEventId)) return null;
    return map.Events.FirstOrDefault(ev => ev.EventId == _selectedEventId);
}

// -----
// Event panel + event list ops
// -----

private void UpdateEventPanel(MapEventDto? ev)
{
    if (ev is null)
    {
        EventSelectedText.Text = "No event selected";
        EventNameBox.Text = "";
        EventTypeBox.SelectedIndex = 0;
    }
}

```

```

    EventNameBox.IsEnabled = false;

    EventTypeBox.IsEnabled = false;

    DeleteEventButton.IsEnabled = false;


    WarpFieldsPanel.Visibility = Visibility.Collapsed;

    SetWarpDestMapUiValue("");

    WarpDestXBox.Text = "";

    WarpDestYBox.Text = "";

    WarpDestMapBox.IsEnabled = false;

    WarpDestXBox.IsEnabled = false;

    WarpDestYBox.IsEnabled = false;


    RefreshParamsList(null);

    return;
}


EventSelectedText.Text = $"Selected @ ({ev.X},{ev.Y}) id={ev.EventId}";

EventNameBox.Text = ev.Name;


var types = ((IEnumerable<string>)EventTypeBox.ItemsSource!).ToList();
var idx = types.IndexOf(ev.Type);
EventTypeBox.SelectedIndex = idx >= 0 ? idx : 0;


EventNameBox.IsEnabled = true;

EventTypeBox.IsEnabled = true;

DeleteEventButton.IsEnabled = true;

```

```

bool isWarp = string.Equals(ev.Type, "Warp", StringComparison.OrdinalIgnoreCase);
WarpFieldsPanel.Visibility = isWarp ? Visibility.Visible : Visibility.Collapsed;

WarpDestMapBox.IsEnabled = isWarp;
WarpDestXBox.IsEnabled = isWarp;
WarpDestYBox.IsEnabled = isWarp;

if (isWarp)
    RefreshAvailableMaps(preserveCurrentSelection: false);

if (isWarp)
{
    var p = ev.Parameters ?? new Dictionary<string, string>();
    SetWarpDestMapUiValue(p.TryGetValue("destMap", out var dm) ? dm : "");
    WarpDestXBox.Text = p.TryGetValue("destX", out var dx) ? dx : "0";
    WarpDestYBox.Text = p.TryGetValue("destY", out var dy) ? dy : "0";
}
else
{
    SetWarpDestMapUiValue("");
    WarpDestXBox.Text = "";
    WarpDestYBox.Text = "";
}

RefreshParamsList(ev);
}

private void AddEvent(MapEventDto ev)

```

```

{
    if (_currentMap is null) return;
    var list = _currentMap.Events.ToList();
    list.Add(ev);
    _currentMap = _currentMap with { Events = list };
}

```

```

private void DeleteEvent(MapEventDto ev)
{
    if (_currentMap is null) return;
    var list = _currentMap.Events.Where(x => x.EventId != ev.EventId).ToList();
    _currentMap = _currentMap with { Events = list };
}

```

```

private void ReplaceEvent(MapEventDto oldEv, MapEventDto newEv)
{
    if (_currentMap is null) return;
    var list = _currentMap.Events.ToList();
    var idx = list.FindIndex(x => x.EventId == oldEv.EventId);
    if (idx < 0) return;
    list[idx] = newEv;
    _currentMap = _currentMap with { Events = list };
}

```

```

private void OnDeleteEventClick(object sender, RoutedEventArgs e)
{
    var ev = GetSelectedEvent();
    if (ev is null) return;
}

```

```

ExecuteCommand(new DeleteEventCommand(this, ev));

_selectedEventId = null;
_selectedCommandIndex = -1;
UpdateEventPanel(null);
RefreshCommandList(null);
UpdateCommandEditor(null);

RequestViewportRender();
}

private void OnEventNameLostFocus(object sender, RoutedEventArgs e)
{
    var ev = GetSelectedEvent();
    if (ev is null) return;

    var newName = (EventNameBox.Text ?? "").Trim();
    if (string.IsNullOrEmpty(newName)) newName = "Event";
    if (newName == ev.Name) return;

    var updated = ev with { Name = newName };
    ExecuteCommand(new ReplaceEventCommand(this, ev, updated));
    UpdateEventPanel(updated);
    RefreshCommandList(updated);
}

private void OnEventTypeChanged(object sender, SelectionChangedEventArgs e)

```

```

{
    if (!IsLoaded) return;

    var ev = GetSelectedEvent();
    if (ev is null) return;

    var type = (EventTypeBox.SelectedItem as string) ?? "Trigger";
    if (type == ev.Type) return;

    var newParams = ev.Parameters is null
        ? new Dictionary<string, string>()
        : new Dictionary<string, string>(ev.Parameters);

    if (string.Equals(type, "Warp", StringComparison.OrdinalIgnoreCase))
    {
        if (!newParams.ContainsKey("destMap")) newParams["destMap"] =
"test_map_01";
        if (!newParams.ContainsKey("destX")) newParams["destX"] = "0";
        if (!newParams.ContainsKey("destY")) newParams["destY"] = "0";
    }

    var updated = ev with { Type = type, Parameters = newParams };
    ExecuteCommand(new ReplaceEventCommand(this, ev, updated));
    _selectedEventId = updated.EventId;

    UpdateEventPanel(updated);
    RefreshCommandList(updated);
    UpdateCommandEditor(null);

```

```
_selectedCommandIndex = -1;
```

```
RequestViewportRender();
```

```
}
```

```
private void OnWarpFieldsLostFocus(object sender, RoutedEventArgs e)
```

```
{
```

```
    var ev = GetSelectedEvent();
```

```
    if (ev is null) return;
```

```
    if (!string.Equals(ev.Type, "Warp", StringComparison.OrdinalIgnoreCase)) return;
```

```
    var destMap = GetWarpDestMapUiValue();
```

```
    if (string.IsNullOrEmpty(destMap))
```

```
    {
```

```
        StatusText.Text = "Warp destMap cannot be empty.";
```

```
        UpdateEventPanel(ev);
```

```
        return;
```

```
    }
```

```
    var destPathCheck = IOPath.Combine(MapsFolder, destMap +  
MapFormat.DefaultExtension);
```

```
    if (!File.Exists(destPathCheck))
```

```
    {
```

```
        StatusText.Text = $"Warp destMap '{destMap}' does not exist in the maps folder.";
```

```
        UpdateEventPanel(ev);
```

```
        return;
```

```
    }
```

```

        if (!int.TryParse((WarpDestXBox.Text ?? "").Trim(), out var destX) ||
            !int.TryParse((WarpDestYBox.Text ?? "").Trim(), out var destY))
        {
            StatusText.Text = "Warp destX/destY must be whole numbers.";
            UpdateEventPanel(ev);
            return;
        }

        var newParams = ev.Parameters is null
            ? new Dictionary<string, string>()
            : new Dictionary<string, string>(ev.Parameters);

        newParams["destMap"] = destMap;
        newParams["destX"] = destX.ToString();
        newParams["destY"] = destY.ToString();

        var updated = ev with { Parameters = newParams };
        ExecuteCommand(new ReplaceEventCommand(this, ev, updated));
        UpdateEventPanel(updated);
        RefreshCommandList(updated);
    }

    // -----
    // Parameters editor
    // -----

    private void RefreshParamsList(MapEventDto? ev)

```



```

{
    ParamsList.Items.Clear();

    ParamKeyBox.Text = "";

    ParamValueBox.Text = "";

    if (ev is null) return;

    var dict = ev.Parameters ?? new Dictionary<string, string>();
    foreach (var kv in dict.OrderBy(k => k.Key, StringComparer.OrdinalIgnoreCase))
        ParamsList.Items.Add($"{kv.Key} = {kv.Value}");
}

private void OnParamSelectionChanged(object sender, SelectionChangedEventArgs
e)
{
    var ev = GetSelectedEvent();

    if (ev is null) return;

    if (ParamsList.SelectedItem is not string row) return;

    var cut = row.IndexOf("=");

    if (cut <= 0) return;

    ParamKeyBox.Text = row[..cut].Trim();

    ParamValueBox.Text = row[(cut + 1)..].Trim();
}

private void OnAddParamClick(object sender, RoutedEventArgs e)

```

```

{
    var ev = GetSelectedEvent();
    if (ev is null) { StatusText.Text = "Select an event first."; return; }

    var key = (ParamKeyBox.Text ?? "").Trim();
    var val = ParamValueBox.Text ?? "";

    if (key.Length == 0)
    {
        StatusText.Text = "Parameter key cannot be empty.";
        return;
    }

    var dict = ev.Parameters is null
        ? new Dictionary<string, string>()
        : new Dictionary<string, string>(ev.Parameters);

    dict[key] = val;

    var updated = ev with { Parameters = dict };
    ExecuteCommand(new ReplaceEventCommand(this, ev, updated));

    _selectedEventId = updated.EventId;
    UpdateEventPanel(updated);
    RefreshCommandList(updated);
    UpdateCommandEditor(null);

    StatusText.Text = $"Param set: {key} = {val}";
}

```

```
}
```

```
private void OnDeleteParamClick(object sender, RoutedEventArgs e)
```

```
{
```

```
    var ev = GetSelectedEvent();
```

```
    if (ev is null) return;
```

```
    if (ParamsList.SelectedItem is not string row || row.Length == 0)
```

```
    {
```

```
        StatusText.Text = "Select a parameter row to delete.";
```

```
        return;
```

```
    }
```

```
    var cut = row.IndexOf("=");
```

```
    if (cut <= 0) return;
```

```
    var key = row[..cut].Trim();
```

```
    var dict = ev.Parameters is null
```

```
        ? new Dictionary<string, string>()
```

```
        : new Dictionary<string, string>(ev.Parameters);
```

```
    if (!dict.Remove(key))
```

```
    {
```

```
        StatusText.Text = "Parameter not found.";
```

```
        return;
```

```
    }
```

```

var updated = ev with { Parameters = dict };
ExecuteCommand(new ReplaceEventCommand(this, ev, updated));

_selectedEventId = updated.EventId;
UpdateEventPanel(updated);

StatusText.Text = $"Param removed: {key}";
}

// -----
// Command list + editor
// -----

private void RefreshCommandList(MapEventDto? ev)
{
    _updatingCommandUi = true;
    try
    {
        EventCommandList.Items.Clear();
        _selectedCommandIndex = -1;
        EventCommandList.SelectedIndex = -1;

        if (ev is null) return;

        var cmds = ev.Commands ?? Array.Empty<EventCommandDto>();
        for (int i = 0; i < cmds.Count; i++)
            EventCommandList.Items.Add(FormatCommandRow(i, cmds[i]));
    }
    finally

```

```

{
    _updatingCommandUi = false;
}
}

```

```

private static string FormatCommandRow(int i, EventCommandDto cmd)

```

```

{
    var c = (cmd.Command ?? "").Trim();

    if (c == "ShowText")
    {
        cmd.Args.TryGetValue("text", out var t);
        t ??= "";
        t = t.Replace("\r", " ").Replace("\n", " ");
        if (t.Length > 30) t = t[..30] + "...";
        return $"{i:00}: ShowText \"{t}\"";
    }

    if (c == "SetFlag")
    {
        cmd.Args.TryGetValue("flag", out var f);
        cmd.Args.TryGetValue("value", out var v);
        return $"{i:00}: SetFlag {f}={v}";
    }

    return $"{i:00}: {c}";
}

```

```

private void OnCommandSelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    if (_updatingCommandUi) return;

    var ev = GetSelectedEvent();
    if (ev is null)
    {
        UpdateCommandEditor(null);
        return;
    }

    _selectedCommandIndex = EventCommandList.SelectedIndex;
    if (_selectedCommandIndex < 0)
    {
        UpdateCommandEditor(null);
        return;
    }

    var cmds = ev.Commands ?? Array.Empty<EventCommandDto>();
    if (_selectedCommandIndex >= cmds.Count)
    {
        UpdateCommandEditor(null);
        return;
    }

    UpdateCommandEditor(cmds[_selectedCommandIndex]);
}

```

```

private void UpdateCommandEditor(EventCommandDto? cmd)
{
    _updatingCommandUi = true;

    try
    {
        if (cmd is null)
        {
            CommandEditorHeader.Text = "No command selected";
            ShowTextEditorPanel.Visibility = Visibility.Collapsed;
            SetFlagEditorPanel.Visibility = Visibility.Collapsed;
            return;
        }

        var c = (cmd.Command ?? "").Trim();
        CommandEditorHeader.Text = $"Editing: {c}";

        if (c == "ShowText")
        {
            ShowTextEditorPanel.Visibility = Visibility.Visible;
            SetFlagEditorPanel.Visibility = Visibility.Collapsed;

            cmd.Args.TryGetValue("text", out var t);
            CmdTextBox.Text = t ?? "";
            return;
        }

        if (c == "SetFlag")

```

```

{
    ShowTextEditorPanel.Visibility = Visibility.Collapsed;
    SetFlagEditorPanel.Visibility = Visibility.Visible;

    cmd.Args.TryGetValue("flag", out var f);
    cmd.Args.TryGetValue("value", out var v);
    CmdFlagNameBox.Text = f ?? "";
    CmdFlagValueBox.Text = v ?? "";
    return;
}

```

```

    ShowTextEditorPanel.Visibility = Visibility.Collapsed;
    SetFlagEditorPanel.Visibility = Visibility.Collapsed;
}
finally
{
    _updatingCommandUi = false;
}
}

```

```

private void OnAddShowTextClick(object sender, RoutedEventArgs e)
{
    var ev = GetSelectedEvent();
    if (ev is null) { StatusText.Text = "Select an event first."; return; }

    var cmd = new EventCommandDto("ShowText", new Dictionary<string, string> {
["text"] = "Hello!" });
    var updated = AppendCommand(ev, cmd);
}

```



```

ExecuteCommand(new ReplaceEventCommand(this, ev, updated));

RefreshCommandList(updated);

 UpdatingCommandUi = true;
try { EventCommandList.SelectedIndex = (updated.Commands?.Count ?? 1) - 1; }
finally { _updatingCommandUi = false; }

_selectedCommandIndex = EventCommandList.SelectedIndex;
UpdateCommandEditor(updated.Commands![_selectedCommandIndex]);
RequestViewportRender();
}

private void OnAddSetFlagClick(object sender, RoutedEventArgs e)
{
    var ev = GetSelectedEvent();
    if (ev is null) { StatusText.Text = "Select an event first."; return; }

    var cmd = new EventCommandDto("SetFlag", new Dictionary<string, string>
    {
        ["flag"] = "has_key",
        ["value"] = "true"
    });

    var updated = AppendCommand(ev, cmd);
    ExecuteCommand(new ReplaceEventCommand(this, ev, updated));

    RefreshCommandList(updated);

```

```
_updatingCommandUi = true;

try { EventCommandList.SelectedIndex = (updated.Commands?.Count ?? 1) - 1; }

finally { _updatingCommandUi = false; }
```

```
_selectedCommandIndex = EventCommandList.SelectedIndex;

UpdateCommandEditor(updated.Commands![_selectedCommandIndex]);

RequestViewportRender();

}
```

```
private static MapEventDto AppendCommand(MapEventDto ev, EventCommandDto
cmd)
```

```
{
    var list = (ev.Commands ?? Array.Empty<EventCommandDto>()).ToList();
    list.Add(cmd);
    return ev with { Commands = list };
}
```

```
private void OnDeleteCommandClick(object sender, RoutedEventArgs e)
```

```
{
    var ev = GetSelectedEvent();
    if (ev is null) return;
```

```
_selectedCommandIndex = EventCommandList.SelectedIndex;

if (_selectedCommandIndex < 0) { StatusText.Text = "Select a command to delete.";
return; }
```

```
var list = (ev.Commands ?? Array.Empty<EventCommandDto>()).ToList();

if (_selectedCommandIndex >= list.Count) return;
```

```
list.RemoveAt(_selectedCommandIndex);
```

```
var updated = ev with { Commands = list };
```

```
ExecuteCommand(new ReplaceEventCommand(this, ev, updated));
```

```
RefreshCommandList(updated);
```

```
UpdateCommandEditor(null);
```

```
_selectedCommandIndex = -1;
```

```
}
```

```
private void OnMoveCommandUpClick(object sender, RoutedEventArgs e) =>  
MoveCommand(-1);
```

```
private void OnMoveCommandDownClick(object sender, RoutedEventArgs e) =>  
MoveCommand(+1);
```

```
private void MoveCommand(int delta)
```

```
{
```

```
var ev = GetSelectedEvent();
```

```
if (ev is null) return;
```

```
var list = (ev.Commands ?? Array.Empty<EventCommandDto>()).ToList();
```

```
_selectedCommandIndex = EventCommandList.SelectedIndex;
```

```
if (_selectedCommandIndex < 0 || _selectedCommandIndex >= list.Count) return;
```

```
int j = _selectedCommandIndex + delta;
```

```
if (j < 0 || j >= list.Count) return;
```

```
(list[_selectedCommandIndex], list[j]) = (list[j], list[_selectedCommandIndex]);
```

```

var updated = ev with { Commands = list };
ExecuteCommand(new ReplaceEventCommand(this, ev, updated));

RefreshCommandList(updated);

 UpdatingCommandUi = true;
try { EventCommandList.SelectedIndex = j; }
finally { _updatingCommandUi = false; }

_selectedCommandIndex = j;
UpdateCommandEditor(updated.Commands![_selectedCommandIndex]);
}

private void OnCommandFieldsLostFocus(object sender, RoutedEventArgs e)
{
    if (_updatingCommandUi) return;

    var ev = GetSelectedEvent();
    if (ev is null) return;

    int idx = EventCommandList.SelectedIndex;
    if (idx < 0) return;

    var list = (ev.Commands ?? Array.Empty<EventCommandDto>()).ToList();
    if (list.Count == 0) return;
    if (idx >= list.Count) return;

```

```
var old = list[idx];

var c = (old.Command ?? "").Trim();

if (c == "ShowText")
{
    var text = CmdTextBox.Text ?? "";

    list[idx] = new EventCommandDto("ShowText",
        new Dictionary<string, string> { ["text"] = text });
}
else if (c == "SetFlag")
{
    var flag = (CmdFlagNameBox.Text ?? "").Trim();
    var value = CmdFlagValueBox.Text ?? "";

    if (flag.Length == 0)
    {
        StatusText.Text = "SetFlag: flag cannot be empty.";
        return;
    }

    list[idx] = new EventCommandDto("SetFlag",
        new Dictionary<string, string>
        {
            ["flag"] = flag,
            ["value"] = value
        });
}
else
```

```

{
    return;
}

var updatedEv = ev with { Commands = list };

 UpdatingCommandUi = true;
try
{
    ExecuteCommand(new ReplaceEventCommand(this, ev, updatedEv));

    RefreshCommandList(updatedEv);

    if (idx >= 0 && idx < (updatedEv.Commands?.Count ?? 0))
    {
        EventCommandList.SelectedIndex = idx;
        UpdateCommandEditor(updatedEv.Commands![idx]);
    }
    else
    {
        EventCommandList.SelectedIndex = -1;
        UpdateCommandEditor(null);
    }
}
finally
{
    UpdatingCommandUi = false;
}

```

```

        RequestViewportRender();
    }

    // -----
    // Undo/redo + persistence
    // -----

    private void ExecuteCommand(IEditCommand cmd)
    {
        cmd.Execute();
        _undo.Push(cmd);
        _redo.Clear();

        PersistMap();
        RequestViewportRender();
    }

    private void Undo()
    {
        if (_undo.Count == 0) return;
        var cmd = _undo.Pop();
        cmd.Undo();
        _redo.Push(cmd);

        PersistMap();
        RequestViewportRender();
    }

```

```

private void Redo()
{
    if (_redo.Count == 0) return;

    var cmd = _redo.Pop();

    cmd.Execute();

    _undo.Push(cmd);

    PersistMap();

    RequestViewportRender();
}

private void PersistMap()
{
    if (_currentMap is null || string.IsNullOrEmpty(_currentMapPath))
        return;

    if (!IsSampleMapLoaded() && !_manualSavePerformed)
    {
        StatusText.Text = $"Not saved yet — press Save to enable autosave.
(Undo:{_undo.Count} Redo:{_redo.Count})";

        return;
    }

    MapSerializer.SaveToFile(_currentMapPath, _currentMap);

    StatusText.Text = $"Saved: {_currentMapPath} (Undo:{_undo.Count}
Redo:{_redo.Count})";
}

// -----

```



```

// Undo/redo command implementations
// -----

private sealed class PaintIntCellCommand : IEditCommand
{
    private readonly MainWindow _w;
    private readonly LayerKind _kind;
    private readonly int _idx;
    private readonly int _oldValue;
    private readonly int _newValue;

    public PaintIntCellCommand(MainWindow w, LayerKind kind, int idx, int oldValue,
int newValue)
    {
        _w = w; _kind = kind; _idx = idx; _oldValue = oldValue; _newValue = newValue;
    }

    public void Execute() => _w.SetLayerCell(_kind, _idx, _newValue);
    public void Undo() => _w.SetLayerCell(_kind, _idx, _oldValue);
}

private sealed class AddEventCommand : IEditCommand
{
    private readonly MainWindow _w;
    private readonly MapEventDto _ev;
    public AddEventCommand(MainWindow w, MapEventDto ev) { _w = w; _ev = ev; }
    public void Execute() => _w.AddEvent(_ev);
    public void Undo() => _w.DeleteEvent(_ev);
}

```

```

private sealed class DeleteEventCommand : IEditCommand
{
    private readonly MainWindow _w;
    private readonly MapEventDto _ev;
    public DeleteEventCommand(MainWindow w, MapEventDto ev) { _w = w; _ev = ev; }
    public void Execute() => _w.DeleteEvent(_ev);
    public void Undo() => _w.AddEvent(_ev);
}

```

```

private sealed class ReplaceEventCommand : IEditCommand
{
    private readonly MainWindow _w;
    private readonly MapEventDto _oldEv;
    private readonly MapEventDto _newEv;
    public ReplaceEventCommand(MainWindow w, MapEventDto oldEv, MapEventDto
newEv) { _w = w; _oldEv = oldEv; _newEv = newEv; }
    public void Execute() => _w.ReplaceEvent(_oldEv, _newEv);
    public void Undo() => _w.ReplaceEvent(_newEv, _oldEv);
}

```

```

private sealed class SwapEventsCommand : IEditCommand
{
    private readonly MainWindow _w;
    private readonly MapEventDto _oldA;
    private readonly MapEventDto _newA;
    private readonly MapEventDto _oldB;
    private readonly MapEventDto _newB;

```

```
public SwapEventsCommand(MainWindow w, MapEventDto oldA, MapEventDto  
newA, MapEventDto oldB, MapEventDto newB)
```

```
{  
    _w = w;  
    _oldA = oldA; _newA = newA;  
    _oldB = oldB; _newB = newB;  
}
```

```
public void Execute()  
{  
    _w.ReplaceEvent(_oldA, _newA);  
    _w.ReplaceEvent(_oldB, _newB);  
}
```

```
public void Undo()  
{  
    _w.ReplaceEvent(_newA, _oldA);  
    _w.ReplaceEvent(_newB, _oldB);  
}  
}
```

```
private sealed class SwapCellContentsCommand : IEditCommand  
{  
    private readonly MainWindow _w;  
    private readonly int _ax, _ay;  
    private readonly int _bx, _by;
```

```
private readonly int _aBg, _bBg;

private readonly int _aCol, _bCol;


private readonly MapEventDto? _aEv;
private readonly MapEventDto? _bEv;


public SwapCellContentsCommand(MainWindow w, int ax, int ay, int bx, int by)
{
    _w = w;

    _ax = ax; _ay = ay;

    _bx = bx; _by = by;


    var map = _w._currentMap!;
    var bg = _w.GetBackgroundLayer!();
    var col = _w.GetCollisionLayer();


    int aldx = _ay * map.Width + _ax;
    int bldx = _by * map.Width + _bx;


    _aBg = bg.Tiles[aldx];
    _bBg = bg.Tiles[bldx];


    _aCol = (col is null) ? 0 : col.Tiles[aldx];
    _bCol = (col is null) ? 0 : col.Tiles[bldx];


    _aEv = _w.GetEventAt(_ax, _ay);
    _bEv = _w.GetEventAt(_bx, _by);
}
```

```
public void Execute() => _w.ApplySwapCellContents(_ax, _ay, _bx, _by);  
public void Undo() => _w.ApplySwapCellContents(_ax, _ay, _bx, _by);  
}
```

```
private void ApplySwapCellContents(int ax, int ay, int bx, int by)  
{  
    var map = _currentMap;  
    if (map is null) return;  
  
    int aldx = ay * map.Width + ax;  
    int bldx = by * map.Width + bx;  
  
    var bg = GetBackgroundLayer();  
    if (bg is null) return;  
  
    var col = GetCollisionLayer();  
  
    (bg.Tiles[aldx], bg.Tiles[bldx]) = (bg.Tiles[bldx], bg.Tiles[aldx]);  
  
    if (col is not null && col.Tiles.Length == map.Width * map.Height)  
        (col.Tiles[aldx], col.Tiles[bldx]) = (col.Tiles[bldx], col.Tiles[aldx]);  
  
    var aEv = GetEventAt(ax, ay);  
    var bEv = GetEventAt(bx, by);  
  
    if (aEv is not null && bEv is not null)  
    {
```

```

var newA = aEv with { X = bx, Y = by };
var newB = bEv with { X = ax, Y = ay };
ReplaceEvent(aEv, newA);
ReplaceEvent(bEv, newB);

if (_selectedEventId == aEv.EventId) _selectedEventId = newA.EventId;
else if (_selectedEventId == bEv.EventId) _selectedEventId = newB.EventId;
}
else if (aEv is not null)
{
var newA = aEv with { X = bx, Y = by };
ReplaceEvent(aEv, newA);
if (_selectedEventId == aEv.EventId) _selectedEventId = newA.EventId;
}
else if (bEv is not null)
{
var newB = bEv with { X = ax, Y = ay };
ReplaceEvent(bEv, newB);
if (_selectedEventId == bEv.EventId) _selectedEventId = newB.EventId;
}
}

```

```

private void SetLayerCell(LayerKind kind, int idx, int value)
{
var map = _currentMap;
if (map is null) return;

var layer = map.Layers.FirstOrDefault(l => l.Kind == kind);

```

```

    if (layer is null) return;

    if (idx < 0 || idx >= layer.Tiles.Length) return;

    layer.Tiles[idx] = value;
}

// -----
// Arrow-key: move cell contents (tile + collision + event) together
// -----

private void TryMoveSelectedCell(int dx, int dy)
{
    var map = _currentMap;

    if (map is null) return;

    if (_playtestActive)
        return;

    // Use cursor tile if set, else fallback to hover
    int x = _cursorTileX >= 0 ? _cursorTileX : _hoverTileX;
    int y = _cursorTileY >= 0 ? _cursorTileY : _hoverTileY;

    if (x < 0 || y < 0 || x >= map.Width || y >= map.Height)
        return;

    int nx = x + dx;
    int ny = y + dy;

    if (nx < 0 || ny < 0 || nx >= map.Width || ny >= map.Height)

```

```

        return;

        ExecuteCommand(new SwapCellContentsCommand(this, x, y, nx, ny));
        ShowSwapHint(x, y, nx, ny);

        _cursorTileX = nx;
        _cursorTileY = ny;

        var sel = GetSelectedEvent();
        UpdateEventPanel(sel);
        RefreshCommandList(sel);
        UpdateCommandEditor(null);
    }

    // -----
    // Keyboard handling
    // -----

    private void OnMainWindowPreviewKeyDown(object sender, KeyEventArgs e)
    {
        if ((Keyboard.Modifiers & ModifierKeys.Control) != 0)
        {
            if (e.Key == Key.Z) { Undo(); e.Handled = true; return; }
            if (e.Key == Key.Y) { Redo(); e.Handled = true; return; }
            if (e.Key == Key.D)
        {
            // Duplicate selected event (events mode only)
            if (_playtestActive) { e.Handled = true; return; }
            if (_mode != EditMode.Events) { e.Handled = true; return; }

```



```

var src = GetSelectedEvent();

if (src is null) { e.Handled = true; return; }


// Prefer cursor tile, else hover tile, else next to the event
int tx = _cursorTileX >= 0 ? _cursorTileX : (_hoverTileX >= 0 ? _hoverTileX : src.X);
int ty = _cursorTileY >= 0 ? _cursorTileY : (_hoverTileY >= 0 ? _hoverTileY : src.Y);


if (!TryFindFreeTileNear(tx, ty, out var fx, out var fy))
{
    StatusText.Text = "No free tile nearby to duplicate into.";
    e.Handled = true;
    return;
}


var copy = CloneEventTo(src, fx, fy);
ExecuteCommand(new AddEventCommand(this, copy));


_selectedEventId = copy.EventId;
UpdateEventPanel(copy);
RefreshCommandList(copy);
UpdateCommandEditor(null);
RequestViewportRender();


StatusText.Text = $"Duplicated event to ({fx},{fy}).";
e.Handled = true;
return;
}

```

```
}
```

```
// Arrow keys: if not playtesting, move the selected cell contents  
(tile+collision+event) together.
```

```
if (!_playtestActive && Keyboard.Modifiers == ModifierKeys.None)
```

```
{
```

```
    switch (e.Key)
```

```
    {
```

```
        case Key.Up: TryMoveSelectedCell(0, -1); e.Handled = true; return;
```

```
        case Key.Down: TryMoveSelectedCell(0, 1); e.Handled = true; return;
```

```
        case Key.Left: TryMoveSelectedCell(-1, 0); e.Handled = true; return;
```

```
        case Key.Right: TryMoveSelectedCell(1, 0); e.Handled = true; return;
```

```
    }
```

```
}
```

```
if (e.Key == Key.Delete)
```

```
{
```

```
    DeleteAtCurrentMode();
```

```
    e.Handled = true;
```

```
    return;
```

```
}
```

```
if (e.Key == Key.F5) { TogglePlaytest(); e.Handled = true; return; }
```

```
if (_playtestActive)
```

```
{
```

```
    if (e.Key == Key.Escape) { ExitPlaytest(); e.Handled = true; return; }
```

```

    if (e.Key == Key.E || e.Key == Key.Enter || e.Key == Key.Space)
    {
        InteractAtPlayer();

        e.Handled = true;

        return;
    }

    switch (e.Key)
    {
        case Key.Up:
        case Key.W: TryMovePlayer(0, -1); e.Handled = true; return;
        case Key.Down:
        case Key.S: TryMovePlayer(0, 1); e.Handled = true; return;
        case Key.Left:
        case Key.A: TryMovePlayer(-1, 0); e.Handled = true; return;
        case Key.Right:
        case Key.D: TryMovePlayer(1, 0); e.Handled = true; return;
    }
}

// -----
// Zoom
// -----

private void ApplyZoom()
{
    _zoom = Math.Clamp(_zoom, ZoomMin, ZoomMax);
}

```

```

MapCanvas.LayoutTransform = new ScaleTransform(_zoom, _zoom);
ZoomText.Text = $"Zoom: {(_zoom * 100):0}%";
}

private void ZoomAtViewportPoint(Point viewportPoint, double newZoom)
{
    newZoom = Math.Clamp(newZoom, ZoomMin, ZoomMax);
    if (Math.Abs(newZoom - _zoom) < 0.0001) return;

    double oldZoom = _zoom;

    double contentX = MapScroll.HorizontalOffset + viewportPoint.X;
    double contentY = MapScroll.VerticalOffset + viewportPoint.Y;

    _zoom = newZoom;
    ApplyZoom();

    double ratio = newZoom / oldZoom;

    MapScroll.Dispatcher.BeginInvoke(DispatcherPriority.Background, new Action(() =>
    {
        double newOffsetX = contentX * ratio - viewportPoint.X;
        double newOffsetY = contentY * ratio - viewportPoint.Y;

        if (newOffsetX < 0) newOffsetX = 0;
        if (newOffsetY < 0) newOffsetY = 0;

        MapScroll.ScrollToHorizontalOffset(newOffsetX);
    }
    ));
}

```

```
MapScroll.ScrollToVerticalOffset(newOffsetY);
```

```
RequestViewportRender();
```

```
}});
```

```
}
```

```
private void OnZoomInClick(object sender, RoutedEventArgs e)
```

```
{
```

```
    var viewportCenter = new Point(MapScroll.ViewportWidth / 2.0,  
MapScroll.ViewportHeight / 2.0);
```

```
    ZoomAtViewportPoint(viewportCenter, _zoom * ZoomStep);
```

```
}
```

```
private void OnZoomOutClick(object sender, RoutedEventArgs e)
```

```
{
```

```
    var viewportCenter = new Point(MapScroll.ViewportWidth / 2.0,  
MapScroll.ViewportHeight / 2.0);
```

```
    ZoomAtViewportPoint(viewportCenter, _zoom / ZoomStep);
```

```
}
```

```
private void OnZoomResetClick(object sender, RoutedEventArgs e)
```

```
{
```

```
    var viewportCenter = new Point(MapScroll.ViewportWidth / 2.0,  
MapScroll.ViewportHeight / 2.0);
```

```
    ZoomAtViewportPoint(viewportCenter, 1.0);
```

```
}
```

```
private void OnScrollViewerMouseWheel(object sender, MouseWheelEventArgs e)
```

```
{
```

```

        if ((Keyboard.Modifiers & ModifierKeys.Control) == 0) return;

        e.Handled = true;

        var viewportPos = e.GetPosition(MapScroll);

        double newZoom = _zoom;

        if (e.Delta > 0) newZoom *= ZoomStep;

        else newZoom /= ZoomStep;

        ZoomAtViewportPoint(viewportPos, newZoom);
    }

    // -----
    // Command interface
    // -----

    private interface IEditCommand
    {
        void Execute();

        void Undo();
    }
}

Vitriol.Editor\Vitriol.Editor.csproj
<Project Sdk="Microsoft.NET.Sdk">

<ItemGroup>

    <ProjectReference Include="..\Vitriol.Shared\Vitriol.Shared.csproj" />

    <ProjectReference Include="..\Vitriol.Data\Vitriol.Data.csproj" />

    <ProjectReference Include="..\Vitriol.Engine\Vitriol.Engine.csproj" />

```

```
</ItemGroup>
```

```
<PropertyGroup>
```

```
<OutputType>WinExe</OutputType>
```

```
<TargetFramework>net9.0-windows</TargetFramework>
```

```
<Nullable>enable</Nullable>
```

```
<ImplicitUsings>enable</ImplicitUsings>
```

```
<UseWPF>true</UseWPF>
```

```
</PropertyGroup>
```

```
</Project>
```

Vitriol.Engine

Vitriol.Engine\Maps

Vitriol.Engine\Maps\MapFormat.cs

```
namespace Vitriol.Engine.Maps;
```

```
public static class MapFormat
```

```
{
```

```
    public const int CurrentVersion = 1;
```

```
    public const string DefaultExtension = ".vitmap.json";
```

```
}
```

Vitriol.Engine\Maps\MapSerializer.cs

```
using System.Text.Json;
```

```
using Vitriol.Shared.Maps;
```

```
namespace Vitriol.Engine.Maps;
```

```
public static class MapSerializer
```

```

{
    private static readonly JsonSerializerOptions Options = new()
    {
        WriteIndented = true,
        PropertyNamingPolicy = JsonNamingPolicy.CamelCase
    };

    public static string ToJson(MapDto map)
        => JsonSerializer.Serialize(map, Options);

    public static MapDto FromJson(string json)
        => JsonSerializer.Deserialize<MapDto>(json, Options)
            ?? throw new InvalidOperationException("Failed to deserialize MapDto (null).");

    public static void SaveToFile(string path, MapDto map)
    {
        Directory.CreateDirectory(Path.GetDirectoryName(path)!);
        File.WriteAllText(path, ToJson(map));
    }

    public static MapDto LoadFromFile(string path)
        => FromJson(File.ReadAllText(path));
}

```

Vitriol.Engine\Class1.cs

namespace Vitriol.Engine;

public class Class1

{


```
}
```

Vitriol.Engine\Vitriol.Engine.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
```

```
<ItemGroup>
```

```
<ProjectReference Include="..\Vitriol.Shared\Vitriol.Shared.csproj" />
```

```
</ItemGroup>
```

```
<PropertyGroup>
```

```
<TargetFramework>net9.0</TargetFramework>
```

```
<ImplicitUsings>enable</ImplicitUsings>
```

```
<Nullable>enable</Nullable>
```

```
</PropertyGroup>
```

```
</Project>
```

Vitriol.Runtime

Vitriol.Runtime\App.xaml

```
<Application x:Class="Vitriol.Runtime.App"
```

```
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
```

```
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
```

```
    xmlns:local="clr-namespace:Vitriol.Runtime"
```

```
    StartupUri="MainWindow.xaml">
```

```
<Application.Resources>
```

```
</Application.Resources>
```

```
</Application>
```

Vitriol.Runtime\App.xaml.cs

```
using System.Configuration;
```

```
using System.Data;
```

```
using System.Windows;
```

```
namespace Vitriol.Runtime;
```

```
/// <summary>
```

```
/// Interaction logic for App.xaml
```

```
/// </summary>
```

```
public partial class App : Application
```

```
{
```

```
}
```

Vitriol.Runtime\AssemblyInfo.cs

```
using System.Windows;
```

```
[assembly:ThemeInfo(
```

```
    ResourceDictionaryLocation.None,    //where theme specific resource  
    dictionaries are located
```

```
        //(used if a resource is not found in the page,
```

```
        // or application resource dictionaries)
```

```
    ResourceDictionaryLocation.SourceAssembly //where the generic resource  
    dictionary is located
```

```
        //(used if a resource is not found in the page,
```

```
        // app, or any theme specific resource dictionaries)
```

```
)]
```

Vitriol.Runtime\MainWindow.xaml

```
<Window x:Class="Vitriol.Runtime.MainWindow"
```

```
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
```

xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"

Title="Vitriol Runtime"

Width="1100" Height="700"

Background="#111"

KeyDown="OnWindowKeyDown"

WindowStyle="None"

ResizeMode="NoResize"

WindowState="Maximized"

WindowStartupLocation="CenterScreen">

<Grid Margin="12">

<Grid.RowDefinitions>

<RowDefinition Height="Auto"/>

<RowDefinition Height="*/>

</Grid.RowDefinitions>

<!-- HUD -->

<Border Grid.Row="0"

BorderBrush="#444" BorderThickness="1"

CornerRadius="6"

Padding="10"

Margin="0,0,0,10"

Background="#1a1a1a">

<DockPanel LastChildFill="True">

<Button Width="120"

Height="30"

Margin="0,0,10,0"

```
Click="OnLoadMapClick"
DockPanel.Dock="Left"
Focusable="False"
IsTabStop="False"
IsDefault="False">
Load Map
</Button>
```

```
<TextBlock Name="HudText"
FontSize="14"
Foreground="White"
Text="Booting..."
TextTrimming="CharacterEllipsis"
VerticalAlignment="Center"/>
</DockPanel>
</Border>
```

```
<!-- Viewport -->
<Border Grid.Row="1"
BorderBrush="#444" BorderThickness="1"
CornerRadius="6"
Background="#000">
```

```
<ScrollViewer Name="ViewScroll"
HorizontalScrollBarVisibility="Auto"
VerticalScrollBarVisibility="Auto"
ScrollChanged="OnScrollChanged"
Focusable="True">
```

```

        <Canvas Name="ViewCanvas"
            Background="#000"
            Focusable="True"/>
    </ScrollViewer>

</Border>

</Grid>

</Window>
Vitriol.Runtime\MainWindow.xaml.cs
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Windows;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Controls;
using System.Windows.Threading;
using Microsoft.Win32;
using Vitriol.Engine.Maps;
using Vitriol.Shared.Maps;
using IOPath = System.IO.Path;
using System.Windows.Interop;

namespace Vitriol.Runtime;

```

```

public partial class MainWindow : Window
{
    // -----
    // Fullscreen lock
    // -----

    private bool _fullscreenApplying = false;

    private void ForceFullscreen()
    {
        if (_fullscreenApplying) return;
        _fullscreenApplying = true;

        try
        {
            if (WindowState != WindowStyle.None) WindowStyle = WindowStyle.None;
            if (ResizeMode != ResizeMode.NoResize) ResizeMode = ResizeMode.NoResize;
            if (WindowState != WindowState.Maximized) WindowState =
WindowState.Maximized;
        }
        finally
        {
            _fullscreenApplying = false;
        }
    }

    private void HookFullscreenGuards()
    {
        Loaded += (_, _) => ForceFullscreen();
    }
}

```

```
StateChanged += (_, _) => ForceFullscreen();  
SizeChanged += (_, _) => ForceFullscreen();  
Activated += (_, _) => ForceFullscreen();  
}
```

```
protected override void OnSourceInitialized(EventArgs e)
```

```
{  
    base.OnSourceInitialized(e);
```

```
    ForceFullscreen();
```

```
    var hwnd = new WindowInteropHelper(this).Handle;
```

```
    if (hwnd != IntPtr.Zero)
```

```
{  
    const int GWL_STYLE = -16;  
    const int WS_SYSMENU = 0x00080000;
```

```
    int style = NativeMethods.GetWindowLong(hwnd, GWL_STYLE);
```

```
    style &= ~WS_SYSMENU;
```

```
    NativeMethods.SetWindowLong(hwnd, GWL_STYLE, style);
```

```
}
```

```
}
```

```
private static class NativeMethods
```

```
{
```

```
    [System.Runtime.InteropServices.DllImport("user32.dll")]
```

```
    public static extern int GetWindowLong(IntPtr hWnd, int nIndex);
```

```

[System.Runtime.InteropServices.DllImport("user32.dll")]
public static extern int SetWindowLong(IntPtr hWnd, int nIndex, int dwNewLong);
}

// -----
// Project folders
// -----

private readonly string _projectRoot;
private string MapsFolder => IOPath.Combine(_projectRoot, "maps");
private string PngsFolder => IOPath.Combine(_projectRoot, "assets", "tilesets");

private MapDto? _map;
private string? _mapPath;

private const double DeadzoneFracX = 0.55;
private const double DeadzoneFracY = 0.28;

private int _playerX = 0;
private int _playerY = 0;

private readonly Dictionary<string, string> _flags =
new(StringComparer.OrdinalIgnoreCase);

// -----
// Tileset caches + compiled runtime layout (multi-tileset, global indices)
// -----

private readonly ConcurrentDictionary<string, BitmapSource> _tilesetBitmapCache =
new(StringComparer.OrdinalIgnoreCase);

private readonly ConcurrentDictionary<string, ConcurrentDictionary<int,
CroppedBitmap>> _tileCropCache = new(StringComparer.OrdinalIgnoreCase);

```



```
private readonly object _tilesetCompileLock = new();  
private List<TilesetRuntime> _compiledTilesets = new();  
private bool _tilesetsDirty = true;
```

```
private sealed record TilesetMeta(  
    int PixelWidth,  
    int PixelHeight,  
    int TileWidth,  
    int TileHeight,  
    int Margin,  
    int Spacing,  
    int TilesAcross,  
    int TilesDown,  
    int TileCount  
);
```

```
private sealed class TilesetRuntime  
{  
    public required TilesetRefDto Ref { get; init; }  
    public required string FullPath { get; init; }  
    public required int StartIndexGlobal { get; init; }  
    public required int TileCount { get; init; }  
    public required int TilesAcross { get; init; }  
    public required int TileW { get; init; }  
    public required int TileH { get; init; }  
    public required int Margin { get; init; }  
    public required int Spacing { get; init; }
```

```

    public required BitmapSource Bitmap { get; init; }

    public required ConcurrentDictionary<int, CroppedBitmap> CropCache { get; init; }
}

// -----
// Viewport-only renderer
// -----

private Image? _viewportImage;

private readonly DispatcherTimer _renderDebounce;

public MainWindow()
{
    InitializeComponent();

    _projectRoot = ResolveProjectRoot();

    HookFullscreenGuards();

    Loaded += (_, _) =>
    {
        ForceFullscreen();

        Boot();

        Dispatcher.BeginInvoke(DispatcherPriority.Input, new
Action(FocusGameSurface));
    };

    _renderDebounce = new DispatcherTimer { Interval =
TimeSpan.FromMilliseconds(16) };

    _renderDebounce.Tick += (_, _) =>

```

```
{  
    _renderDebounce.Stop();  
    RenderViewport();  
};  
}
```

```
private void FocusGameSurface()  
{  
    Keyboard.Focus(ViewCanvas);  
    ViewCanvas.Focus();  
}
```

```
// -----  
// Project root resolution  
// -----  
private static string ResolveProjectRoot()  
{  
    var start = AppContext.BaseDirectory;  
  
    var candidates = new[]  
    {  
        start,  
        Directory.GetCurrentDirectory()  
    }  
    .Where(p => !string.IsNullOrEmpty(p))  
    .Distinct(StringComparer.OrdinalIgnoreCase)  
    .ToArray();
```

```
const int maxUp = 8;
```

```
foreach (var baseDir in candidates)
```

```
{
```

```
    try
```

```
    {
```

```
        var dir = new DirectoryInfo(baseDir);
```

```
        for (int i = 0; i < maxUp && dir is not null; i++)
```

```
        {
```

```
            var probe = IOPath.Combine(dir.FullName, "Vitriol.Projects");
```

```
            if (Directory.Exists(probe))
```

```
                return probe;
```

```
            dir = dir.Parent;
```

```
        }
```

```
    }
```

```
    catch {}
```

```
}
```

```
return IOPath.Combine(Directory.GetCurrentDirectory(), "Vitriol.Projects");
```

```
}
```

```
private void Boot()
```

```
{
```

```
    HudText.Text = $"Runtime ready. Click 'Load Map'. (Project: {_projectRoot})";
```

```
}
```

```
// -----
```

```

// UI: Load Map button
// -----
private void OnLoadMapClick(object sender, RoutedEventArgs e)
{
    try
    {
        if (!Directory.Exists(MapsFolder))
        {
            HudText.Text = $"Maps folder not found: {MapsFolder}";
            FocusGameSurface();
            return;
        }

        var dlg = new OpenFileDialog
        {
            Title = "Open Vitriol Map",
            Filter = "Vitriol Map (*.vitmap.json)|*.vitmap.json|JSON (*.json)|*.json|All files (*.*)|*.*",
            InitialDirectory = MapsFolder,
            CheckFileExists = true,
            Multiselect = false
        };

        if (dlg.ShowDialog(this) != true)
        {
            FocusGameSurface();
            return;
        }
    }
}

```

```

        _playerX = 0;
        _playerY = 0;

        LoadMap(dlg.FileName, "Manual Load");

        Dispatcher.BeginInvoke(DispatcherPriority.Input, new
Action(FocusGameSurface));
    }
    catch (Exception ex)
    {
        HudText.Text = $"Load map dialog failed: {ex.Message}";
        FocusGameSurface();
    }
}

// -----
// Map loading
// -----

private void LoadMap(string path, string reason)
{
    try
    {
        _mapPath = path;
        _map = MapSerializer.LoadFromFile(path);

        if (_map.Events is null)
            _map = _map with { Events = Array.Empty<MapEventDto>() };
    }
}

```

```
EnsureCollisionLayer();
```

```
var errors = ValidateTilesetFilesExist(_map);
```

```
if (errors.Count > 0)
```

```
{
```

```
    _map = null;
```

```
    HudText.Text = "Map load blocked:\n" + string.Join("\n", errors);
```

```
    return;
```

```
}
```

```
MarkTilesetsDirty();
```

```
EnsureTilesetsCompiled();
```

```
_playerX = Math.Clamp(_playerX, 0, _map.Width - 1);
```

```
_playerY = Math.Clamp(_playerY, 0, _map.Height - 1);
```

```
EnsureViewportImageExists();
```

```
ViewCanvas.Width = checked(_map.Width * _map.TileSize);
```

```
ViewCanvas.Height = checked(_map.Height * _map.TileSize);
```

```
    HudText.Text = $"{reason}: Loaded {IOPath.GetFileName(path)}  
    Player({_playerX},{_playerY})";
```

```
Dispatcher.BeginInvoke(DispatcherPriority.Loaded, new Action(() =>
```

```
{
```

```
    CenterCameraOnPlayer();
```

```
    RequestRender();
```

```

        FocusGameSurface();
    }));
}
catch (Exception ex)
{
    HudText.Text = $"Load failed: {ex.Message}";
    FocusGameSurface();
}
}

```

```

private List<string> ValidateTilesetFilesExist(MapDto map)
{
    var issues = new List<string>();

    var tilesets = map.Tilesets ?? Array.Empty<TilesetRefDto>();
    foreach (var ts in tilesets)
    {
        var img = (ts.ImagePath ?? "").Trim();

        if (img.Length == 0)
            continue;

        var full = ResolveTilesetPathFromImagePath(img);
        if (full is null)
        {
            issues.Add($"Tileset '{ts.TilesetId}': ImagePath invalid: '{img}'");
            continue;
        }
    }
}

```



```

        if (!File.Exists(full))

            issues.Add($"Tileset '{ts.TilesetId}': missing PNG: '{full}'");

    }

    return issues;
}

private void EnsureCollisionLayer()
{
    if (_map is null) return;

    bool hasCollision =

        _map.Layers.Any(l => l.Kind == LayerKind.Collision || l.LayerId == "col");

    if (hasCollision) return;

    var col = new TileLayerDto(
        LayerId: "col",
        Name: "Collision",
        Kind: LayerKind.Collision,
        Visible: true,
        Tiles: new int[_map.Width * _map.Height]
    );

    _map = _map with { Layers = _map.Layers.Concat(new[] { col }).ToArray() };
}

```

```

private string? ResolveTilesetPathFromImagePath(string imagePath)
{
    if (string.IsNullOrEmpty(imagePath))
        return null;

    imagePath = imagePath.Trim();

    if (IOPath.IsPathRooted(imagePath))
        return imagePath;

    var rel = imagePath.Replace('/', '\\');
    return IOPath.Combine(_projectRoot, rel);
}

```

```

private TileLayerDto? GetBackgroundLayer() =>
    _map?.Layers.FirstOrDefault(l => l.Kind == LayerKind.Background);

```

```

private TileLayerDto? GetCollisionLayer() =>
    _map?.Layers.FirstOrDefault(l => l.Kind == LayerKind.Collision || l.LayerId == "col");

```

```

private string? ResolveTilesetFullPath(TilesetRefDto ts)
{
    var raw = (ts.ImagePath ?? "").Trim();

    if (raw.Length > 0 && IOPath.IsPathRooted(raw) && File.Exists(raw))
        return raw;

    if (raw.Length > 0)
    {

```

```

var fileName = IOPath.GetFileName(raw.Replace('/', '\\'));
if (!string.IsNullOrEmpty(fileName))
{
    var inTilesets = IOPath.Combine(PngsFolder, fileName);
    if (File.Exists(inTilesets))
        return inTilesets;

    if (IOPath.GetExtension(fileName).Length == 0)
    {
        var inTilesetsPng = IOPath.Combine(PngsFolder, fileName + ".png");
        if (File.Exists(inTilesetsPng))
            return inTilesetsPng;
    }
}

var fallbackName = (ts.TilesetId ?? "").Trim();
if (fallbackName.Length > 0)
{
    var p = IOPath.Combine(PngsFolder, fallbackName + ".png");
    if (File.Exists(p))
        return p;
}

return null;
}

private void MarkTilesetsDirty()

```

```
{  
    lock (_tilesetCompileLock)  
    {  
        _tilesetsDirty = true;  
    }  
}
```

```
private void EnsureTilesetsCompiled()
```

```
{  
    var map = _map;  
  
    lock (_tilesetCompileLock)  
    {  
        if (!_tilesetsDirty) return;  
  
        if (map is null)  
        {  
            _compiledTilesets = new List<TilesetRuntime>();  
            _tilesetsDirty = false;  
            return;  
        }  
  
        var compiled = new List<TilesetRuntime>();  
        int runningStart = 0;  
  
        var tilesets = map.Tilesets ?? Array.Empty<TilesetRefDto>();  
        foreach (var ts in tilesets)  
        {
```

```

var img = (ts.ImagePath ?? "").Trim();

if (img.Length == 0)
    continue;

var fullPath = ResolveTilesetPathFromImagePath(img);
if (fullPath is null || !File.Exists(fullPath))
    continue;

var bmp = _tilesetBitmapCache.GetOrAdd(fullPath, LoadBitmap);
var meta = ComputeMeta(ts, bmp);

var cropCache = _tileCropCache.GetOrAdd(fullPath, _ => new
ConcurrentDictionary<int, CroppedBitmap>());

compiled.Add(new TilesetRuntime
{
    Ref = ts,
    FullPath = fullPath,
    StartIndexGlobal = runningStart,
    TileCount = meta.TileCount,
    TilesAcross = meta.TilesAcross,
    TileW = meta.TileWidth,
    TileH = meta.TileHeight,
    Margin = meta.Margin,
    Spacing = meta.Spacing,
    Bitmap = bmp,
    CropCache = cropCache

```

```

    });

    runningStart += meta.TileCount;
}

_compiledTilesets = compiled;
_tilesetsDirty = false;
}
}

private static TilesetMeta ComputeMeta(TilesetRefDto ts, BitmapSource bmp)
{
    int tileW = Math.Max(1, ts.TileWidth);
    int tileH = Math.Max(1, ts.TileHeight);
    int margin = Math.Max(0, ts.Margin);
    int spacing = Math.Max(0, ts.Spacing);

    int stepX = tileW + spacing;
    int stepY = tileH + spacing;

    int tilesAcross = stepX > 0 ? (bmp.PixelWidth - margin) / stepX : 0;
    int tilesDown = stepY > 0 ? (bmp.PixelHeight - margin) / stepY : 0;

    tilesAcross = Math.Max(0, tilesAcross);
    tilesDown = Math.Max(0, tilesDown);

    int tileCount = tilesAcross * tilesDown;

```

```

return new TilesetMeta(
    PixelWidth: bmp.PixelWidth,
    PixelHeight: bmp.PixelHeight,
    TileWidth: tileW,
    TileHeight: tileH,
    Margin: margin,
    Spacing: spacing,
    TilesAcross: tilesAcross,
    TilesDown: tilesDown,
    TileCount: tileCount
);
}

private TilesetRuntime? FindRuntimeForGlobalTile(int globalTileIndex)
{
    EnsureTilesetsCompiled();
    if (globalTileIndex < 0) return null;

    foreach (var ts in _compiledTilesets)
    {
        if (globalTileIndex >= ts.StartIndexGlobal &&
            globalTileIndex < ts.StartIndexGlobal + ts.TileCount)
            return ts;
    }

    return null;
}

```

```

private static int GetLocalIndex(TilesetRuntime ts, int globalTileIndex) =>
    globalTileIndex - ts.StartIndexGlobal;

// -----
// Camera
// -----

private void CenterCameraOnPlayer()
{
    var map = _map;
    if (map is null) return;

    if (ViewScroll.ViewportWidth <= 0 || ViewScroll.ViewportHeight <= 0)
    {
        Dispatcher.BeginInvoke(DispatcherPriority.Loaded, new
        Action(CenterCameraOnPlayer));
        return;
    }

    double mapPxW = map.Width * map.TileSize;
    double mapPxH = map.Height * map.TileSize;

    double playerCenterX = _playerX * map.TileSize + map.TileSize / 2.0;
    double playerCenterY = _playerY * map.TileSize + map.TileSize / 2.0;

    double desiredX = playerCenterX - ViewScroll.ViewportWidth / 2.0;
    double desiredY = playerCenterY - ViewScroll.ViewportHeight / 2.0;

    double maxX = Math.Max(0, mapPxW - ViewScroll.ViewportWidth);

```



```

double maxY = Math.Max(0, mapPxH - ViewScroll.ViewportHeight);

desiredX = Math.Clamp(desiredX, 0, maxX);
desiredY = Math.Clamp(desiredY, 0, maxY);

ViewScroll.ScrollToHorizontalOffset(desiredX);
ViewScroll.ScrollToVerticalOffset(desiredY);
}

private void DeadzoneFollowPlayer()
{
    var map = _map;
    if (map is null) return;
    if (ViewScroll.ViewportWidth <= 0 || ViewScroll.ViewportHeight <= 0) return;

    double mapPxW = map.Width * map.TileSize;
    double mapPxH = map.Height * map.TileSize;

    double camX = ViewScroll.HorizontalOffset;
    double camY = ViewScroll.VerticalOffset;

    double vw = ViewScroll.ViewportWidth;
    double vh = ViewScroll.ViewportHeight;

    double dzW = vw * DeadzoneFracX;
    double dzH = vh * DeadzoneFracY;

    double dzLeft = camX + (vw - dzW) / 2.0;

```

```

double dzTop = camY + (vh - dzH) / 2.0;
double dzRight = dzLeft + dzW;
double dzBottom = dzTop + dzH;

double px = _playerX * map.TileSize + map.TileSize / 2.0;
double py = _playerY * map.TileSize + map.TileSize / 2.0;

if (px < dzLeft) camX -= (dzLeft - px);
else if (px > dzRight) camX += (px - dzRight);

if (py < dzTop) camY -= (dzTop - py);
else if (py > dzBottom) camY += (py - dzBottom);

double maxX = Math.Max(0, mapPxW - vw);
double maxY = Math.Max(0, mapPxH - vh);

camX = Math.Clamp(camX, 0, maxX);
camY = Math.Clamp(camY, 0, maxY);

ViewScroll.ScrollToHorizontalOffset(camX);
ViewScroll.ScrollToVerticalOffset(camY);
}

// -----
// Rendering
// -----

private void EnsureViewportImageExists()
{

```

```
if (_viewportImage != null) return;

_viewportImage = new Image
{
    Stretch = Stretch.None,
    SnapsToDevicePixels = true
};

RenderOptions.SetBitmapScalingMode(_viewportImage,
BitmapScalingMode.NearestNeighbor);
```

```
ViewCanvas.Children.Clear();
ViewCanvas.Children.Add(_viewportImage);
}
```

```
private void RequestRender()
{
    if (_map is null) return;
    _renderDebounce.Stop();
    _renderDebounce.Start();
}
```

```
private void OnScrollChanged(object sender, ScrollChangedEventArgs e)
{
    if (_map is null) return;
    RequestRender();
}
```

```
private void RenderViewport()
```

```
{  
    var map = _map;  
    if (map is null) return;  
  
    EnsureViewportImageExists();  
    EnsureTilesetsCompiled();  
  
    var bg = GetBackgroundLayer();  
    if (bg is null) { HudText.Text = "Map has no background layer."; return; }  
  
    int mapPxW = checked(map.Width * map.TileSize);  
    int mapPxH = checked(map.Height * map.TileSize);  
  
    double viewLeft = ViewScroll.HorizontalOffset;  
    double viewTop = ViewScroll.VerticalOffset;  
    double viewW = ViewScroll.ViewportWidth;  
    double viewH = ViewScroll.ViewportHeight;  
  
    int pad = map.TileSize * 2;  
    int regionX = ClampToInt(viewLeft) - pad;  
    int regionY = ClampToInt(viewTop) - pad;  
    int regionW = ClampToInt(viewW) + pad * 2;  
    int regionH = ClampToInt(viewH) + pad * 2;  
  
    if (regionX < 0) regionX = 0;  
    if (regionY < 0) regionY = 0;  
    if (regionX + regionW > mapPxW) regionW = mapPxW - regionX;  
    if (regionY + regionH > mapPxH) regionH = mapPxH - regionY;
```

```
if (regionW <= 1 || regionH <= 1) return;
```

```
int startTileX = regionX / map.TileSize;
```

```
int startTileY = regionY / map.TileSize;
```

```
int endTileX = (regionX + regionW + map.TileSize - 1) / map.TileSize;
```

```
int endTileY = (regionY + regionH + map.TileSize - 1) / map.TileSize;
```

```
startTileX = Math.Max(0, startTileX);
```

```
startTileY = Math.Max(0, startTileY);
```

```
endTileX = Math.Min(map.Width, endTileX);
```

```
endTileY = Math.Min(map.Height, endTileY);
```

```
var visual = new DrawingVisual();
```

```
using (var dc = visual.RenderOpen())
```

```
{
```

```
    dc.DrawRectangle(Brushes.Black, null, new Rect(0, 0, regionW, regionH));
```

```
    for (int y = startTileY; y < endTileY; y++)
```

```
        for (int x = startTileX; x < endTileX; x++)
```

```
        {
```

```
            int idx = y * map.Width + x;
```

```
            if ((uint)idx >= (uint)bg.Tiles.Length) continue;
```

```
            int t = bg.Tiles[idx];
```

```
            if (t < 0) continue;
```

```
            var ts = FindRuntimeForGlobalTile(t);
```

```

if (ts is null || ts.TileCount <= 0) continue;

int local = GetLocalIndex(ts, t);
if ((uint)local >= (uint)ts.TileCount) continue;

var crop = ts.CropCache.GetOrAdd(local, tileIndex =>
{
    int cx = ts.TilesAcross > 0 ? (tileIndex % ts.TilesAcross) : 0;
    int cy = ts.TilesAcross > 0 ? (tileIndex / ts.TilesAcross) : 0;

    int px = ts.Margin + cx * (ts.TileW + ts.Spacing);
    int py = ts.Margin + cy * (ts.TileH + ts.Spacing);

    if (px + ts.TileW > ts.Bitmap.PixelWidth || py + ts.TileH > ts.Bitmap.PixelHeight)
    {
        var dummy = new CroppedBitmap(ts.Bitmap, new Int32Rect(0, 0, 1, 1));
        dummy.Freeze();
        return dummy;
    }

    var cb = new CroppedBitmap(ts.Bitmap, new Int32Rect(px, py, ts.TileW,
ts.TileH));
    cb.Freeze();
    return cb;
});

double drawX = x * map.TileSize - regionX;
double drawY = y * map.TileSize - regionY;

```

```

        dc.DrawImage(crop, new Rect(drawX, drawY, map.TileSize, map.TileSize));
    }

    if (map.Events is not null && map.Events.Count > 0)
    {
        var triggerFill = new SolidColorBrush(Color.FromArgb(220, 255, 215, 0));
        triggerFill.Freeze();

        var npcFill = new SolidColorBrush(Color.FromArgb(220, 80, 170, 255));
        npcFill.Freeze();

        var warpFill = new SolidColorBrush(Color.FromArgb(220, 190, 90, 255));
        warpFill.Freeze();

        var outline = new Pen(Brushes.Black, 1); outline.Freeze();

        foreach (var ev in map.Events)
        {
            if (ev.X < startTileX || ev.X >= endTileX || ev.Y < startTileY || ev.Y >= endTileY)
                continue;

            double left = ev.X * map.TileSize - regionX;
            double top = ev.Y * map.TileSize - regionY;
            var type = (ev.Type ?? "").Trim();

            if (type.Equals("NPC", StringComparison.OrdinalIgnoreCase))
            {
                var c = new Point(left + map.TileSize / 2.0, top + map.TileSize / 2.0);
                dc.DrawEllipse(npcFill, outline, c, map.TileSize * 0.22, map.TileSize * 0.22);
            }
            else if (type.Equals("Warp", StringComparison.OrdinalIgnoreCase))

```

```

{
    var cx = left + map.TileSize / 2.0;
    var cy = top + map.TileSize / 2.0;
    double r = map.TileSize * 0.24;

    var geo = new StreamGeometry();
    using (var g = geo.Open())
    {
        g.BeginFigure(new Point(cx, cy - r), true, true);
        g.LineTo(new Point(cx + r, cy), true, false);
        g.LineTo(new Point(cx, cy + r), true, false);
        g.LineTo(new Point(cx - r, cy), true, false);
    }
    geo.Freeze();
    dc.DrawGeometry(warpFill, outline, geo);
}
else
{
    dc.DrawRectangle(triggerFill, outline,
        new Rect(left + map.TileSize * 0.28, top + map.TileSize * 0.28,
            map.TileSize * 0.44, map.TileSize * 0.44));
}
}

if (_playerX >= startTileX && _playerX < endTileX && _playerY >= startTileY &&
    _playerY < endTileY)
{

```



```

var fill = new SolidColorBrush(Color.FromArgb(230, 90, 180, 255)); fill.Freeze();

var outline = new Pen(Brushes.Black, 1); outline.Freeze();


double px = _playerX * map.TileSize - regionX;
double py = _playerY * map.TileSize - regionY;


dc.DrawEllipse(fill, outline,
    new Point(px + map.TileSize / 2.0, py + map.TileSize / 2.0),
    map.TileSize * 0.28,
    map.TileSize * 0.28);
}
}


var rtb = new RenderTargetBitmap(regionW, regionH, 96, 96, PixelFormats.Pbgra32);
rtb.Render(visual);
rtb.Freeze();


_viewportImage!.Source = rtb;
_viewportImage.Width = regionW;
_viewportImage.Height = regionH;


Canvas.SetLeft(_viewportImage, regionX);
Canvas.SetTop(_viewportImage, regionY);
}


private static int ClampToInt(double v)
{
    if (v <= int.MinValue) return int.MinValue;

```

```
    if (v >= int.MaxValue) return int.MaxValue;

    return (int)Math.Floor(v);
}
```

```
private static BitmapSource LoadBitmap(string path)
{
    var bmp = new BitmapImage();
    bmp.BeginInit();
    bmp.CacheOption = BitmapCacheOption.OnLoad;
    bmp.UriSource = new Uri(path, UriKind.Absolute);
    bmp.EndInit();
    bmp.Freeze();
    return bmp;
}
```

```
// -----
// Input + gameplay
// -----

private void OnWindowKeyDown(object sender, KeyEventArgs e)
{
    if (_map is null) return;

    if (e.Key == Key.Escape)
    {
        Close();
        return;
    }
}
```

```
if (e.Key == Key.E || e.Key == Key.Enter || e.Key == Key.Space)
{
    InteractAtPlayer();
    e.Handled = true;
    return;
}
```

```
switch (e.Key)
{
    case Key.Up:
    case Key.W: TryMove(0, -1); e.Handled = true; return;
    case Key.Down:
    case Key.S: TryMove(0, 1); e.Handled = true; return;
    case Key.Left:
    case Key.A: TryMove(-1, 0); e.Handled = true; return;
    case Key.Right:
    case Key.D: TryMove(1, 0); e.Handled = true; return;
}
}
```

```
private void TryMove(int dx, int dy)
{
    var map = _map;
    if (map is null) return;

    int nx = _playerX + dx;
    int ny = _playerY + dy;
```

```
if (nx < 0 || ny < 0 || nx >= map.Width || ny >= map.Height)
```

```
{
```

```
    HudText.Text = "Blocked: edge of map.";
```

```
    return;
```

```
}
```

```
var col = GetCollisionLayer();
```

```
if (col is not null && col.Tiles.Length == map.Width * map.Height)
```

```
{
```

```
    int idx = ny * map.Width + nx;
```

```
    if (idx >= 0 && idx < col.Tiles.Length && col.Tiles[idx] != 0)
```

```
    {
```

```
        HudText.Text = "Blocked: collision tile.";
```

```
        return;
```

```
    }
```

```
}
```

```
_playerX = nx;
```

```
_playerY = ny;
```

```
var ev = GetEventAt(_playerX, _playerY);
```

```
if (ev is not null)
```

```
{
```

```
    if (string.Equals(ev.Type, "Warp", StringComparison.OrdinalIgnoreCase))
```

```
        DoWarp(ev);
```

```
    else if (string.Equals(ev.Type, "Trigger", StringComparison.OrdinalIgnoreCase))
```

```
        ExecuteEventScript(ev);
```

```
}
```

```
HudText.Text = $"Player({_playerX},{_playerY})";
```

```
DeadzoneFollowPlayer();
```

```
RequestRender();
```

```
}
```

```
private void InteractAtPlayer()
```

```
{
```

```
    var ev = GetEventAt(_playerX, _playerY);
```

```
    if (ev is null)
```

```
    {
```

```
        HudText.Text = "Interact: no event here.";
```

```
        return;
```

```
    }
```

```
    if (string.Equals(ev.Type, "NPC", StringComparison.OrdinalIgnoreCase) ||
```

```
        string.Equals(ev.Type, "Trigger", StringComparison.OrdinalIgnoreCase))
```

```
    {
```

```
        ExecuteEventScript(ev);
```

```
        return;
```

```
    }
```

```
    HudText.Text = $"Interact: {ev.Name} ({ev.Type})";
```

```
}
```

```
private MapEventDto? GetEventAt(int x, int y)
```

```
{
```

```

var map = _map;

if (map?.Events is null || map.Events.Count == 0) return null;

return map.Events.FirstOrDefault(e => e.X == x && e.Y == y);
}

```

```

private void ExecuteEventScript(MapEventDto ev)
{
    var cmds = ev.Commands ?? Array.Empty<EventCommandDto>();

    if (cmds.Count == 0)
    {
        HudText.Text = $"Event '{ev.Name}' has no commands.";

        return;
    }

```

```

foreach (var cmd in cmds)
{
    switch ((cmd.Command ?? "").Trim())
    {
        case "ShowText":
        {
            cmd.Args.TryGetValue("text", out var text);

            text ??= "";

            if (text.Length == 0) text = "(empty text)";

            MessageBox.Show(this, text, ev.Name);

            break;
        }

```

```

        case "SetFlag":

```

```

{
    cmd.Args.TryGetValue("flag", out var flag);
    cmd.Args.TryGetValue("value", out var value);

    flag = (flag ?? "").Trim();
    value ??= "";

    if (flag.Length == 0)
    {
        HudText.Text = "SetFlag failed: flag is empty.";
        break;
    }

    _flags[flag] = value;
    HudText.Text = $"Flag set: {flag}='{value}'";
    break;
}

default:
    HudText.Text = $"Unknown command: {cmd.Command}";
    break;
}
}
}

```

```

private void DoWarp(MapEventDto ev)
{
    var map = _map;
    if (map is null) return;

```

```
var p = ev.Parameters ?? new Dictionary<string, string>();
```

```
p.TryGetValue("destMap", out var destMap);
```

```
p.TryGetValue("destX", out var destXStr);
```

```
p.TryGetValue("destY", out var destYStr);
```

```
destMap = (destMap ?? "").Trim();
```

```
if (string.IsNullOrEmpty(destMap))
```

```
{
```

```
    HudText.Text = "Warp failed: destMap missing.";
```

```
    return;
```

```
}
```

```
int.TryParse(destXStr, out var destX);
```

```
int.TryParse(destYStr, out var destY);
```

```
var destPath = IOPath.Combine(MapsFolder, destMap +  
MapFormat.DefaultExtension);
```

```
if (!File.Exists(destPath))
```

```
{
```

```
    HudText.Text = $"Warp failed: map not found '{destMap}'!";
```

```
    return;
```

```
}
```

```
LoadMap(destPath, $"Warp to {destMap}");
```



```

if (_map is null) return;

_playerX = Math.Clamp(destX, 0, _map.Width - 1);
_playerY = Math.Clamp(destY, 0, _map.Height - 1);

HudText.Text = $"Warped: {ev.Name} -> {destMap} ({_playerX},{_playerY})";

Dispatcher.BeginInvoke(DispatcherPriority.Background, new Action(() =>
{
    CenterCameraOnPlayer();
    RequestRender();
    FocusGameSurface();
}));
}
}

```

Vitriol.Runtime\Vitriol.Runtime.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
```

```
<ItemGroup>
```

```
<ProjectReference Include="..\Vitriol.Engine\Vitriol.Engine.csproj" />
```

```
<ProjectReference Include="..\Vitriol.Shared\Vitriol.Shared.csproj" />
```

```
</ItemGroup>
```

```
<PropertyGroup>
```

```
<OutputType>WinExe</OutputType>
```

```
<TargetFramework>net9.0-windows</TargetFramework>
```

```
<Nullable>enable</Nullable>
```

```
<ImplicitUsings>enable</ImplicitUsings>
```

```
<UseWPF>>true</UseWPF>
```

</PropertyGroup>

</Project>

Vitriol.Shared

Vitriol.Shared\Ids

Vitriol.Shared\Ids\EntityId.cs

namespace Vitriol.Shared.Ids;

```
public readonly record struct EntityId(long Value)
{
    public override string ToString() => Value.ToString();
}
```

Vitriol.Shared\Maps

Vitriol.Shared\Maps\EventCommandDto.cs

namespace Vitriol.Shared.Maps;

```
public sealed record EventCommandDto(
    string Command,
    Dictionary<string, string> Args
);
```

Vitriol.Shared\Maps\MapDto.cs

using System.Collections.Generic;

namespace Vitriol.Shared.Maps;

```
public sealed record MapDto(
    int FormatVersion,
    string MapId,
```

```

    string Name,
    int Width,
    int Height,
    int TileSize,
    IReadOnlyList<TilesetRefDto> Tilesets,
    IReadOnlyList<TileLayerDto> Layers
)
{
    public IReadOnlyList<MapEventDto> Events { get; init; }
        = System.Array.Empty<MapEventDto>();
}

```

[Vitriol.Shared\Maps\MapEventDto.cs](#)

```

namespace Vitriol.Shared.Maps;

public sealed record MapEventDto(
    string EventId,
    string Name,
    string Type,
    int X,
    int Y,
    Dictionary<string, string>? Parameters = null,
    IReadOnlyList<EventCommandDto>? Commands = null
);

```

[Vitriol.Shared\Maps\TileLayerDto.cs](#)

```

namespace Vitriol.Shared.Maps;

public sealed record TileLayerDto(
    string LayerId,

```

```
    string Name,  
    LayerKind Kind,  
    bool Visible,  
    int[] Tiles // length = Width*Height, tile index (or -1 for empty)  
);
```

```
public enum LayerKind  
{  
    Background = 0,  
    Collision = 1,  
    Objects = 2,  
    Overlay = 3  
}
```

[Vitriol.Shared\Maps\TilesetRefDto.cs](#)

```
namespace Vitriol.Shared.Maps;
```

```
public sealed record TilesetRefDto(  
    string TilesetId,  
    string ImagePath,    // relative to project root, e.g. "assets/tilesets/kanto.png"  
    int TileWidth,  
    int TileHeight,  
    int Margin,  
    int Spacing  
);
```

[Vitriol.Shared\Pokemon](#)

[Vitriol.Shared\Pokemon\PokemonSpeciesDto.cs](#)

```
namespace Vitriol.Shared.Pokemon;
```

```
public sealed record PokemonSpeciesDto(  

```

```
    long SpeciesId,  
    string Name,  
    string PrimaryType,  
    string? SecondaryType,  
    int BaseHP,  
    int BaseAttack,  
    int BaseDefense,  
    int BaseSpAttack,  
    int BaseSpDefense,  
    int BaseSpeed  
);
```

Vitriol.Shared.Validation

Vitriol.Shared.Validation.MapValidator.cs

```
using Vitriol.Shared.Maps;
```

```
namespace Vitriol.Shared.Validation;
```

```
public static class MapValidator
```

```
{
```

```
    public static IReadOnlyList<ValidationError> Validate(MapDto map)
```

```
    {
```

```
        var errors = new List<ValidationError>();
```

```
        if (map.FormatVersion <= 0)
```

```
            errors.Add(new("FormatVersion", "FormatVersion must be a positive integer."));
```

```
        if (string.IsNullOrEmpty(map.MapId))
```

```
            errors.Add(new("MapId", "MapId is required."));
```

```

if (map.Width <= 0 || map.Height <= 0)
    errors.Add(new("Size", "Width and Height must be positive.));

if (map.TileSize <= 0)
    errors.Add(new("TileSize", "TileSize must be positive.));

foreach (var layer in map.Layers)
{
    if (layer.Tiles is null)
    {
        errors.Add(new($"Layer:{layer.LayerId}", "Tiles array is missing.));
        continue;
    }

    var expected = map.Width * map.Height;

    if (layer.Tiles.Length != expected)
        errors.Add(new($"Layer:{layer.LayerId}", $"Tiles length must be Width*Height ({expected}).));
    }

    return errors;
}

```

[Vitriol.Shared.Validation.SpeciesValidator.cs](#)

using Vitriol.Shared.Pokemon;

namespace Vitriol.Shared.Validation;

```

public static class SpeciesValidator
{
    public static IReadOnlyList<ValidationError> Validate(PokemonSpeciesDto dto)
    {
        var errors = new List<ValidationError>();

        if (string.IsNullOrEmpty(dto.Name))
            errors.Add(new("Name", "Name is required."));

        if (string.IsNullOrEmpty(dto.PrimaryType))
            errors.Add(new("PrimaryType", "Primary type is required."));

        if (!string.IsNullOrEmpty(dto.SecondaryType) &&
            string.Equals(dto.PrimaryType, dto.SecondaryType,
                StringComparison.OrdinalIgnoreCase))
            errors.Add(new("SecondaryType", "Secondary type cannot be the same as primary
type."));

        int[] stats = [dto.BaseHP, dto.BaseAttack, dto.BaseDefense, dto.BaseSpAttack,
            dto.BaseSpDefense, dto.BaseSpeed];

        if (stats.Any(s => s < 1 || s > 255))
            errors.Add(new("BaseStats", "Each base stat must be between 1 and 255."));

        return errors;
    }
}

```

[Vitriol.Shared\Validation\ValidationError.cs](#)
namespace Vitriol.Shared.Validation;

```
public sealed record ValidationError(string Field, string Message);
```

Vitriol.Shared\Class1.cs

```
namespace Vitriol.Shared;
```

```
public class Class1
```

```
{
```

```
}
```

Vitriol.Shared\Vitriol.Shared.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
```

```
<PropertyGroup>
```

```
<TargetFramework>net9.0</TargetFramework>
```

```
<ImplicitUsings>enable</ImplicitUsings>
```

```
<Nullable>enable</Nullable>
```

```
</PropertyGroup>
```

```
</Project>
```

Vitriol.Tests

(do not mark this as part of the testing section)

Vitriol.Tests\DatabaseSmokeTests.cs

```
using Microsoft.Data.Sqlite;
```

```
using NUnit.Framework;
```

```
using Vitriol.Data.Migrations;
```

```
using Vitriol.Data.Repositories;
```

```
using Vitriol.Shared.Pokemon;
```

```
using Vitriol.Shared.Validation;
```



```
namespace Vitriol.Tests;
```

```
public class DatabaseSmokeTests
```

```
{
```

```
    [Test]
```

```
    public void Migration_creates_species_table_and_allows_insert_and_query()
```

```
    {
```

```
        using var conn = new SqlConnection("Data Source=:memory:");
```

```
        conn.Open();
```

```
        new MigrationRunner().EnsureMigrated(conn);
```

```
        var repo = new PokemonSpeciesRepository();
```

```
        var bulbasaur = new PokemonSpeciesDto(
```

```
            SpeciesId: 0,
```

```
            Name: "Bulbasaur",
```

```
            PrimaryType: "Grass",
```

```
            SecondaryType: "Poison",
```

```
            BaseHP: 45,
```

```
            BaseAttack: 49,
```

```
            BaseDefense: 49,
```

```
            BaseSpAttack: 65,
```

```
            BaseSpDefense: 65,
```

```
            BaseSpeed: 45
```

```
        );
```

```
        var errors = SpeciesValidator.Validate(bulbasaur);
```

```
        Assert.That(errors, Is.Empty);
```

```

var id = repo.Insert(conn, bulbasaur);
Assert.That(id, Is.GreaterThan(0));

var loaded = repo.GetByName(conn, "Bulbasaur");
Assert.That(loaded, Is.Not.Null);
Assert.That(loaded!.PrimaryType, Is.EqualTo("Grass"));
Assert.That(loaded.SecondaryType, Is.EqualTo("Poison"));
}

[Test]
public void Validator_rejects_duplicate_types()
{
    var dto = new PokemonSpeciesDto(
        SpeciesId: 0,
        Name: "Testmon",
        PrimaryType: "Fire",
        SecondaryType: "Fire",
        BaseHP: 50, BaseAttack: 50, BaseDefense: 50, BaseSpAttack: 50,
        BaseSpDefense: 50, BaseSpeed: 50
    );

    var errors = SpeciesValidator.Validate(dto);
    Assert.That(errors.Any(e => e.Field == "SecondaryType"), Is.True);
}
}

Vitriol.Tests\MapFormatTests.cs
using NUnit.Framework;
using Vitriol.Engine.Maps;

```

```
using Vitriol.Shared.Maps;
using Vitriol.Shared.Validation;

namespace Vitriol.Tests;

public class MapFormatTests
{
    [Test]
    public void Map_json_round_trip_preserves_dimensions_and_tiles()
    {
        var w = 8;
        var h = 6;

        var tiles = new int[w * h];
        for (int i = 0; i < tiles.Length; i++) tiles[i] = i % 3;

        var map = new MapDto(
            FormatVersion: MapFormat.CurrentVersion,
            MapId: "round_trip",
            Name: "Round Trip",
            Width: w,
            Height: h,
            TileSize: 32,
            Tilesets: new List<TilesetRefDto>(),
            Layers: new List<TileLayerDto>
            {
                new("bg", "Background", LayerKind.Background, true, tiles)
            }
        );
    }
}
```

```
);
```

```
var errors = MapValidator.Validate(map);
```

```
Assert.That(errors, Is.Empty);
```

```
var json = MapSerializer.ToJson(map);
```

```
var loaded = MapSerializer.FromJson(json);
```

```
Assert.That(loaded.Width, Is.EqualTo(map.Width));
```

```
Assert.That(loaded.Height, Is.EqualTo(map.Height));
```

```
Assert.That(loaded.Layers.Count, Is.EqualTo(1));
```

```
Assert.That(loaded.Layers[0].Tiles, Is.EqualTo(map.Layers[0].Tiles));
```

```
}
```

```
}
```

Vitriol.Tests\UnitTest1.csj

```
namespace Vitriol.Tests;
```

```
public class Tests
```

```
{
```

```
    [SetUp]
```

```
    public void Setup()
```

```
    {
```

```
    }
```

```
    [Test]
```

```
    public void Test1()
```

```
    {
```

```
        Assert.Pass();
```

```
}  
}
```

Vitriol.Tests\Vitriol.Tests.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
```

```
<PropertyGroup>
```

```
<TargetFramework>net9.0</TargetFramework>
```

```
<LangVersion>latest</LangVersion>
```

```
<ImplicitUsings>enable</ImplicitUsings>
```

```
<Nullable>enable</Nullable>
```

```
<IsPackable>false</IsPackable>
```

```
</PropertyGroup>
```

```
<ItemGroup>
```

```
<PackageReference Include="coverlet.collector" Version="6.0.2" />
```

```
<PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
```

```
<PackageReference Include="NUnit" Version="4.2.2" />
```

```
<PackageReference Include="NUnit.Analyzers" Version="4.4.0" />
```

```
<PackageReference Include="NUnit3TestAdapter" Version="4.6.0" />
```

```
</ItemGroup>
```

```
<ItemGroup>
```

```
<Using Include="NUnit.Framework" />
```

```
</ItemGroup>
```

```
<ItemGroup>
```

```
<ProjectReference Include="..\Vitriol.Shared\Vitriol.Shared.csproj" />
```

```
<ProjectReference Include="..\Vitriol.Data\Vitriol.Data.csproj" />
```

```
<ProjectReference Include="..\Vitriol.Engine\Vitriol.Engine.csproj" />

</ItemGroup>

</Project>
```

Project Vitriol – NEA Testing

Image Download Links are provided below each image in case of load failure

1 Testing Strategy Overview

Testing was conducted to verify that the implemented system satisfies the functional and non-functional objectives defined in the Analysis section. The purpose of this stage is not simply to demonstrate that the program runs, but to show that it behaves correctly under normal, boundary, and abnormal conditions.

Testing was structured into:

- **Internal Logic Verification** – confirming that core calculations and control logic (indexing, command execution, viewport calculations, camera behaviour, serialisation) operate deterministically.
- **Functional Subsystem Testing** – validating that observable behaviours (map creation, editing, persistence, runtime rendering) align with specified requirements.
- **Boundary and Abnormal Data Testing** – evaluating validation rules and system resilience when exposed to extreme or invalid input.
- **Performance Observation** – assessing rendering behaviour and responsiveness under larger map conditions.

Where issues were identified, corrective modifications were applied and the affected subsystem was retested to confirm stability. Representative samples are presented below to demonstrate that the system fulfils its intended purpose.

2 Subsystem Testing

2.1 Map Creation and Validation

This subsystem verifies that map initialisation follows the defined structural rules and that invalid configurations are rejected prior to persistence. It directly supports objectives relating to data integrity, autosave behaviour, and safe project management.

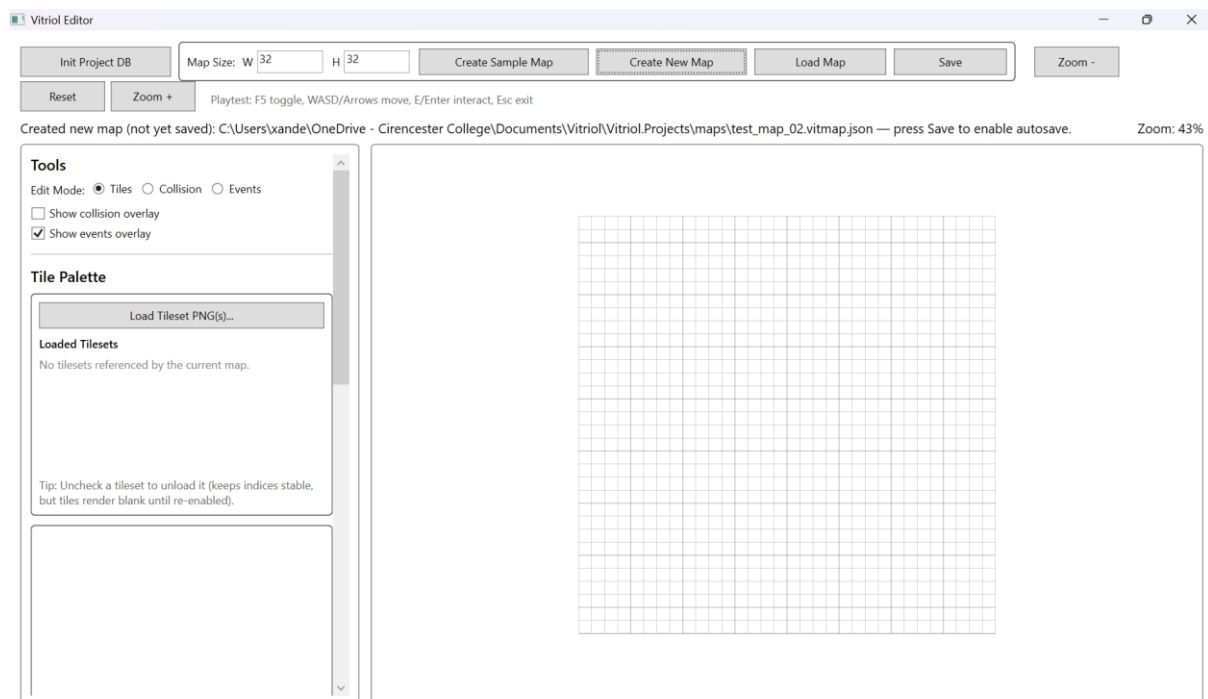
Test Table 1 – Map Creation & Validation

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
---------	-----------------	-----------	------------------	----------------	---------------

MC1	Verify new map creates valid empty grid	Width = 32 Height = 32	1024-tile grid initialised with default values	Pass	Figures T1
MC2	Validate manual save required before autosave	Create new map -> modify tile without saving	Autosave disabled until first manual save	Pass	Figures T2.1, T2.2
MC3	Boundary test – minimum size	Width = 1 Height = 1	Valid 1x1 grid created without error	Pass	Figures T3
MC4	Boundary test – large size	Width = 300 Height = 300	Map created successfully. Viewport rendering prevents performance degradation	Pass	Figures T4
MC5	Abnormal input – negative size	Width = -10	Map creation rejected with validation error	Pass	Figures T5
MC6	Abnormal input – zero dimension	Width = 0	Validation prevents creation	Pass	Figures T6

Test Table 1 – Screenshot Evidence

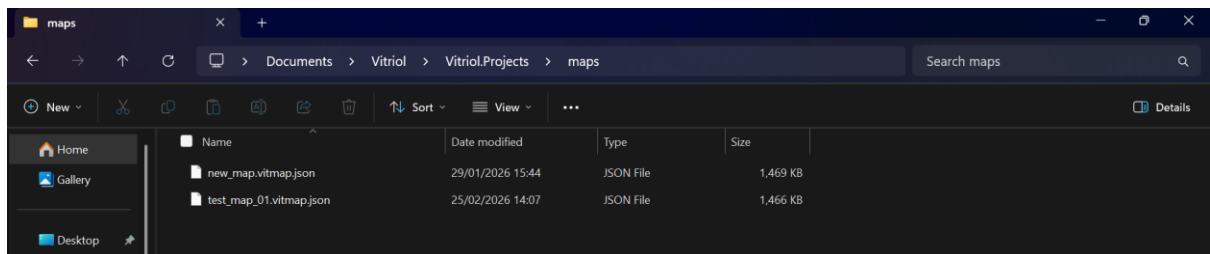
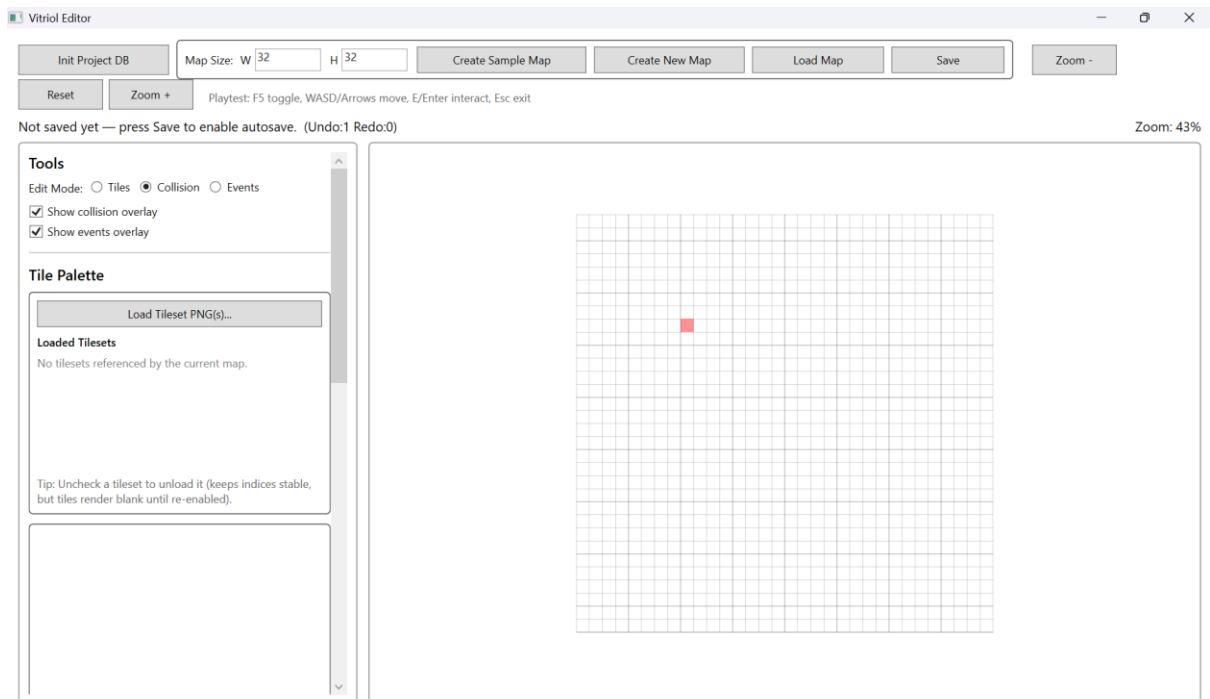
Figure T1 - Standard Map Creation (32×32 Grid)



[Testing Section\Testing Section Images\Figure T1.png](#)

Demonstrates successful creation of a default-sized map with correctly initialised tile grid.

Figure T2.1, T2.2 – Autosave Disabled Prior to First Manual Save

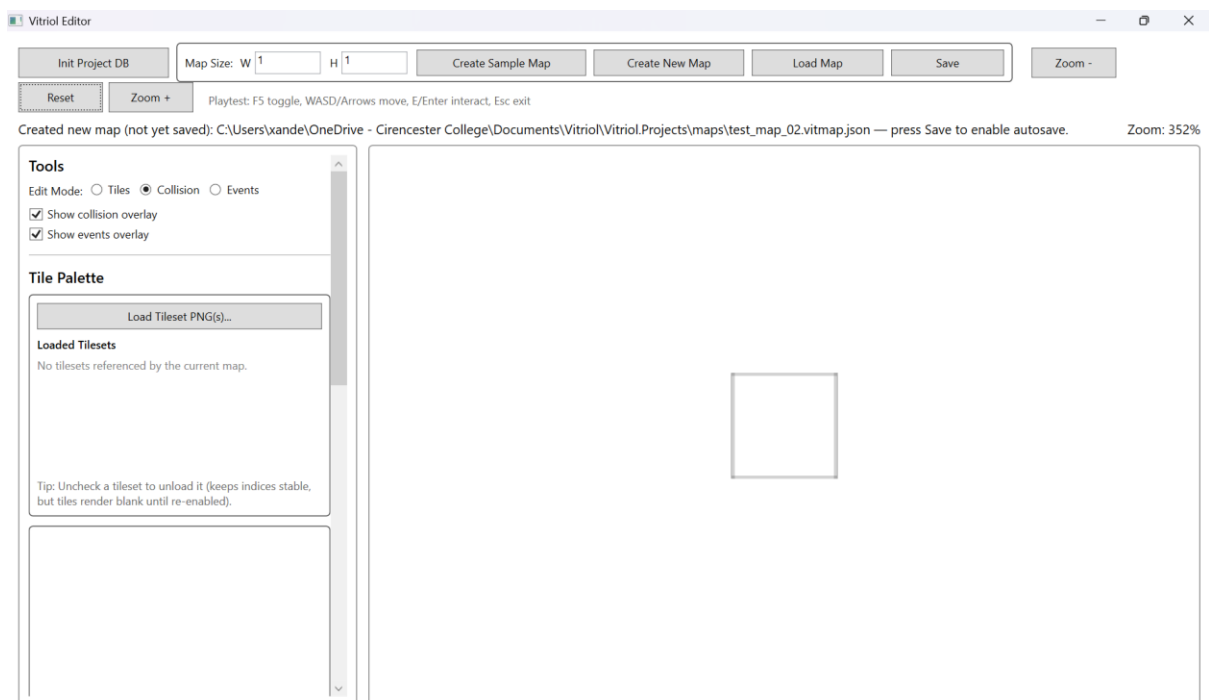


[Testing Section\Testing Section Images\Figure T2.1.png](#)

[Testing Section\Testing Section Images\Figure T2.2.png](#)

Demonstrates that modifying a newly created map does not trigger autosave before an explicit manual save is performed.

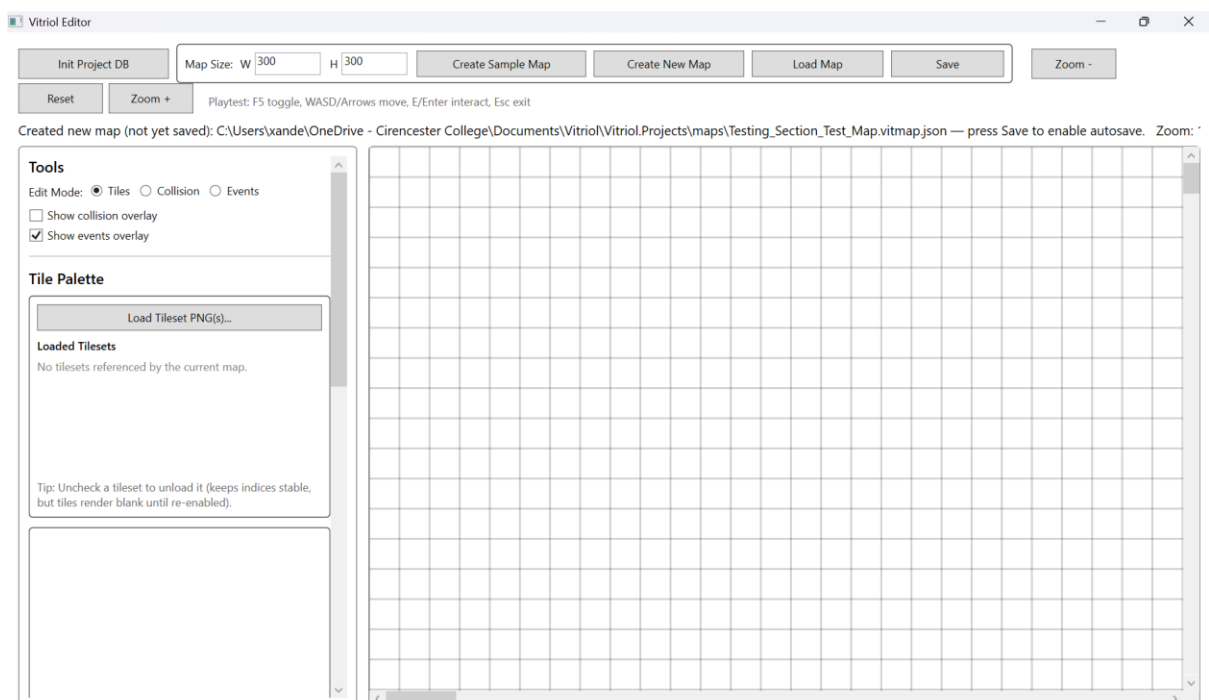
Figure T3 – Minimum Map Dimension Boundary Test (1×1 Grid)



[Testing Section\Testing Section Images\Figure T3.png](#)

Demonstrates correct handling of the smallest valid map configuration without runtime instability.

Figure T4 – Large Map Creation (300×300) with Viewport Rendering Active

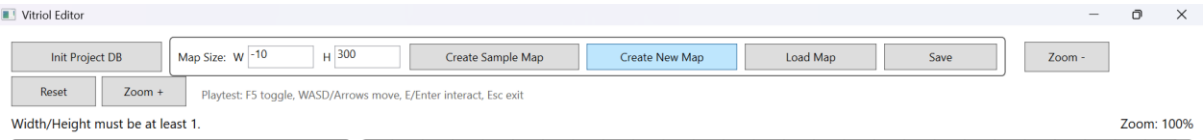


[Testing Section\Testing Section Images\Figure T4.png](#)

Demonstrates stable creation and navigation of a large map using region-based viewport rendering.

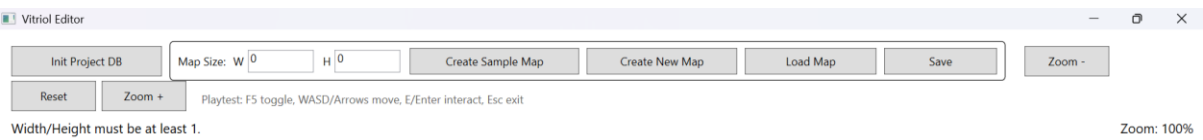
Figure T5 – Negative Dimension Validation Rejection

Demonstrates system rejection of invalid negative dimension input prior to grid allocation.



[Testing Section\Testing Section Images\Figure T5.png](#)

Figure T6 – Zero Dimension Validation Handling



[Testing Section\Testing Section Images\Figure T6.png](#)

Demonstrates prevention of zero-sized map creation through input validation logic.

2.2 Tile Editing and Undo/Redo

This subsystem verifies that editing operations correctly modify the underlying data model and that the Undo/Redo mechanism reliably restores previous states without introducing inconsistencies.

The command-based editing model was tested to ensure:

- Each paint operation produces exactly one reversible command
- Undo restores the previous tile state
- Redo reapplies the change deterministically
- New edits correctly clear the redo stack
- Autosave behaviour does not interfere with command execution

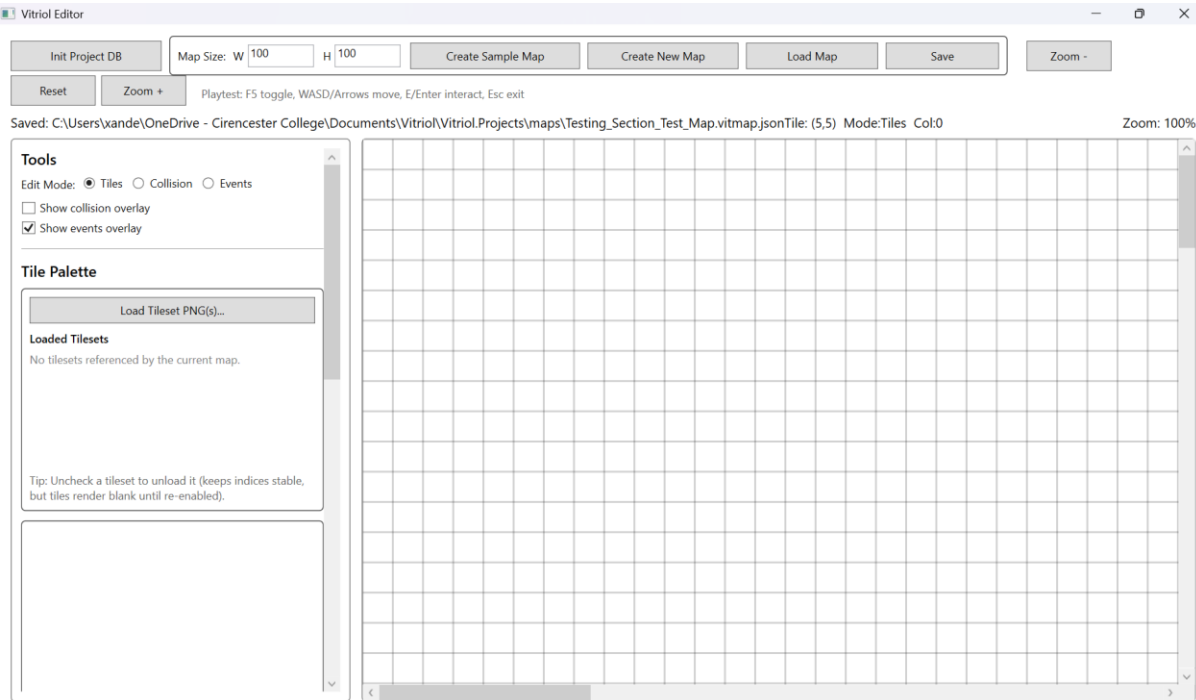
Test Table 2 – Tile Editing & Undo/Redo

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
UR1	Verify tile paint modifies correct grid index	Paint tile at (5,5)	Tile at (5,5) updated only	Pass	Figures T7.1, T7.2
UR2	Verify Undo restores previous state	Paint tile -> Undo	Tile returns to original value	Pass	Figures T8.1, T8.2
UR3	Verify Redo reapplies change	Undo -> Redo	Tile updated again	Pass	Figures T9.1, T9.2

UR4	Verify redo stack clears after new action	Paint -> Undo -> Paint new tile	Redo unavailable	Pass	Figures T10.1, T10.2
UR5	Multi-step undo stability	Paint 3 tiles -> Undo 3 times	Grid returns to original state	Pass	Figures T11.1, T11.2, T11.3, T11.4

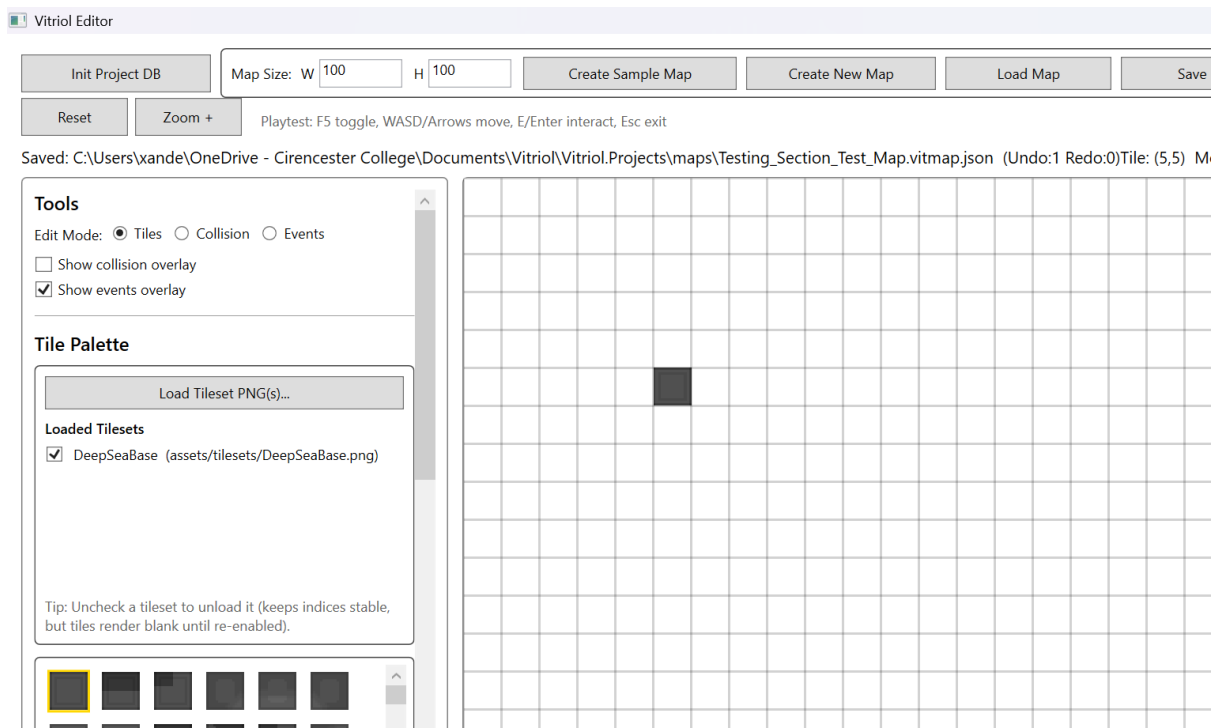
Test Table 2 – Screenshot Evidence

Figure T7.1 – Tile Grid Before Paint Operation at Coordinate (5,5)



Testing Section\Testing Section Images\Figure T7.1.png

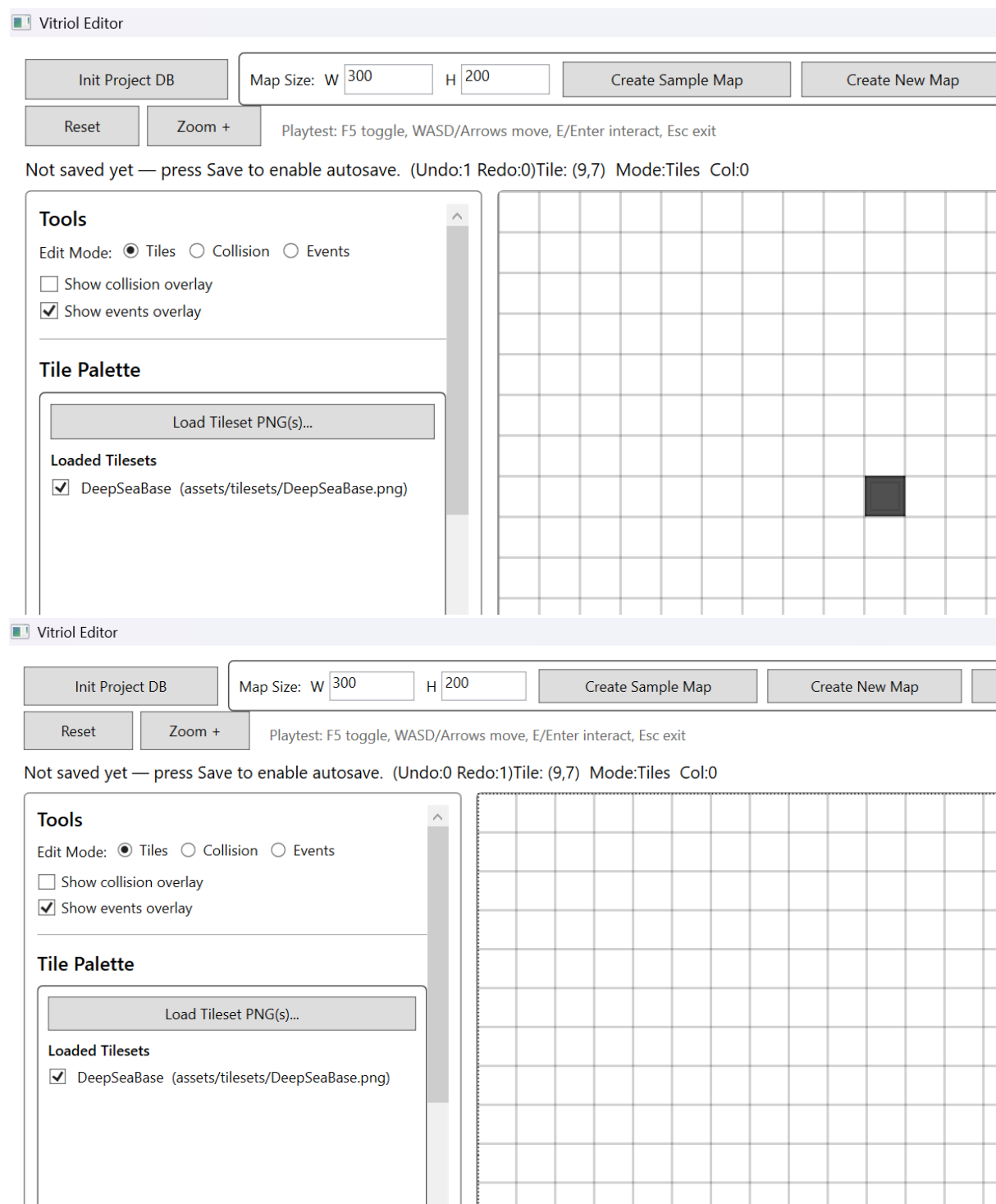
Figure T7.2 – Tile Grid After Paint Operation at Coordinate (5,5)



[Testing Section\Testing Section Images\Figure T7.2.png](#)

Demonstrates that the paint command updates only the intended grid coordinate, confirming correct 2D-to-1D index conversion logic and isolated state mutations.

Figure T8.1 – Tile State Immediately After Paint operation



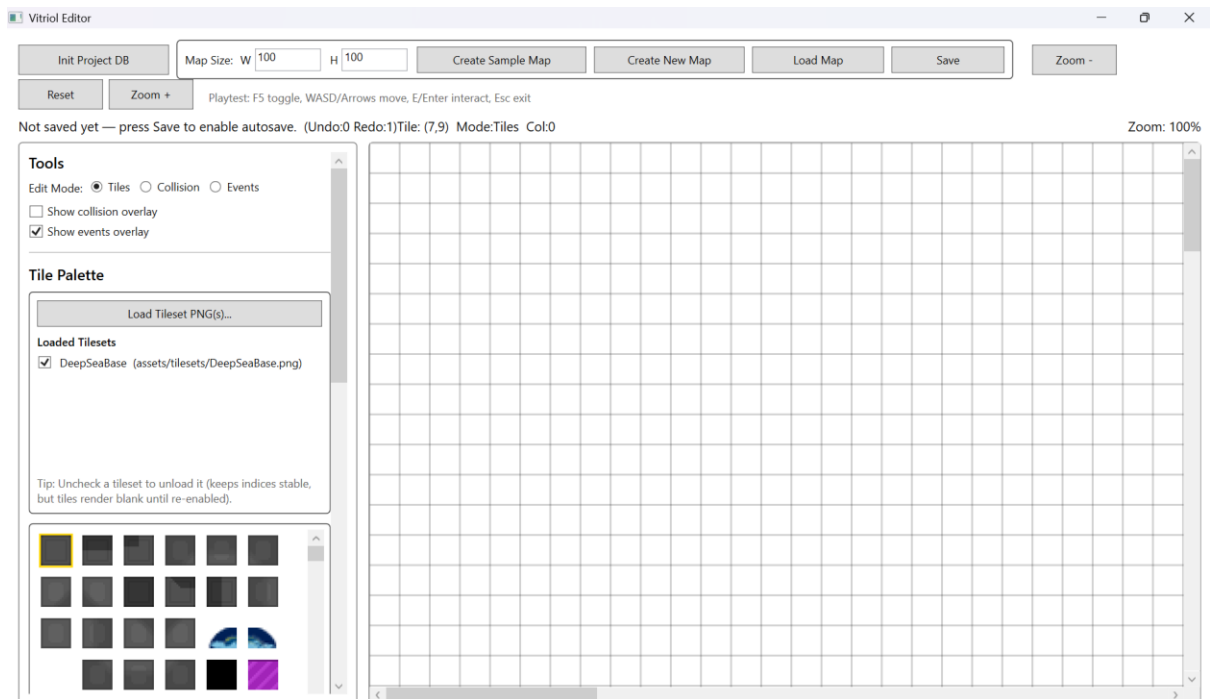
[Testing Section\Testing Section Images\Figure T8.1.png](#)

[Testing Section\Testing Section Images\Figure T8.2.png](#)

Figure T8.2 Tile State After Undo Invocation (Ctrl+Z)

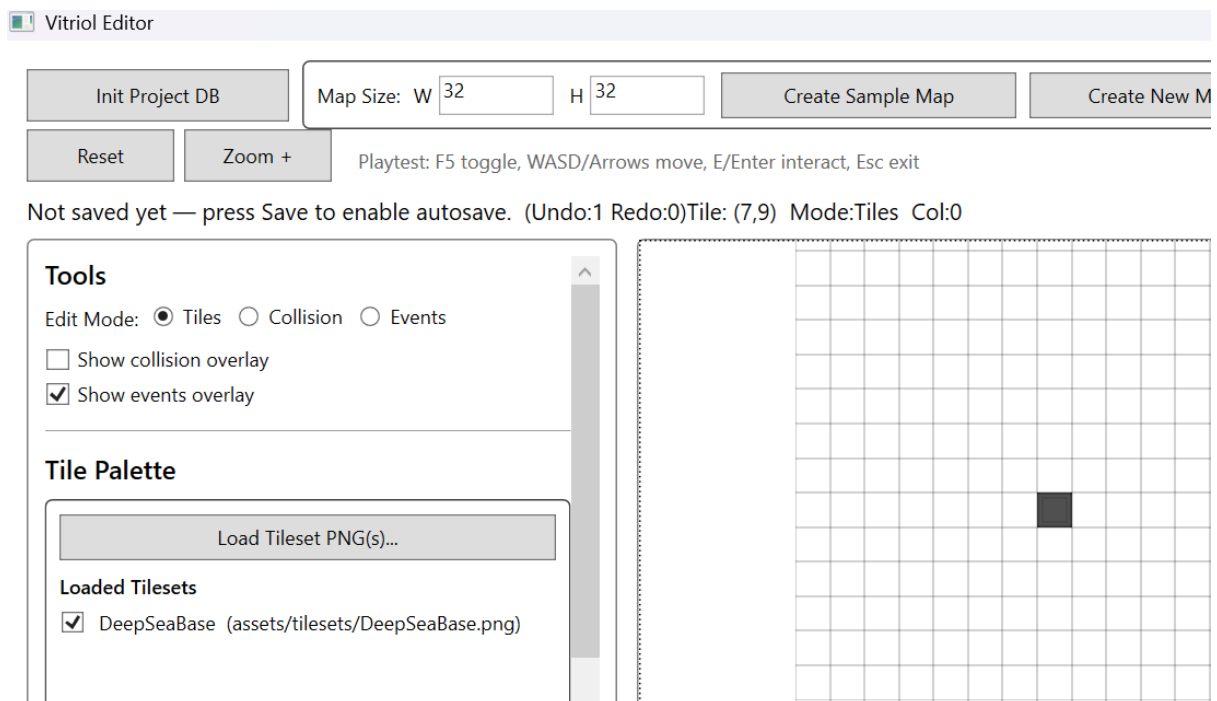
Demonstrates that Undo restores the previous tile value without affecting unrelated grid cells, validating reversible command execution

Figure T9.1 – Tile State after Undo Operation



[Testing Section\Testing Section Images\Figure T9.1.png](#)

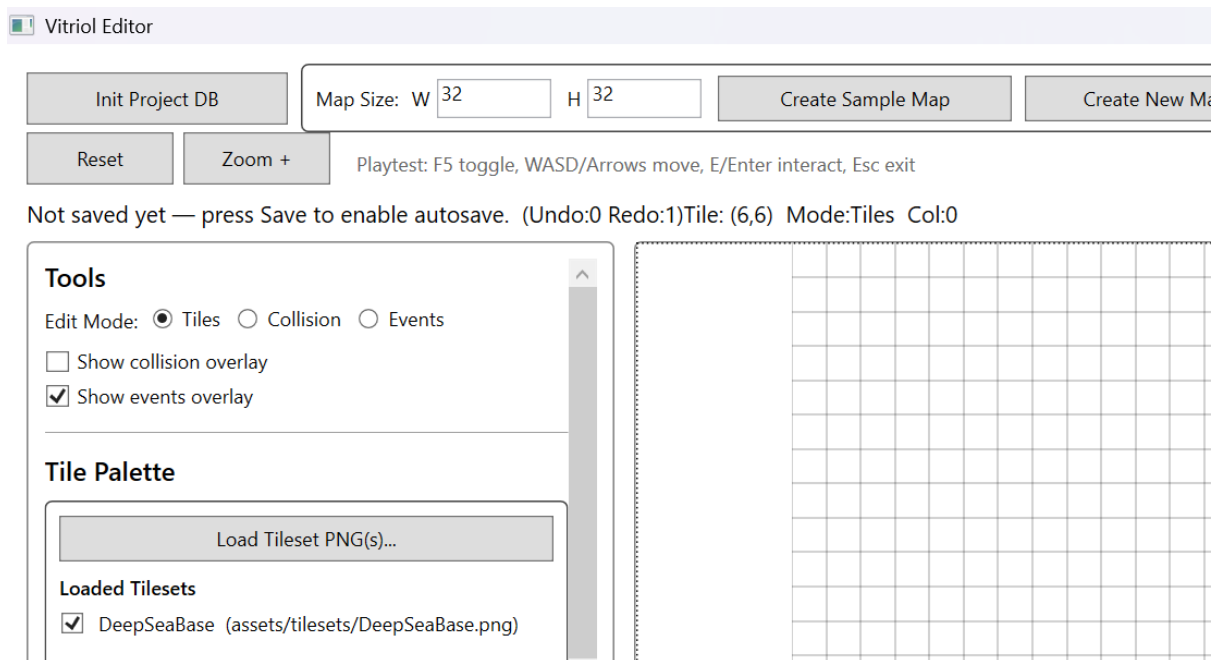
Figure T9.2 – Tile State after Redo Invocation (Ctrl+Y)



[Testing Section\Testing Section Images\Figure T9.2.png](#)

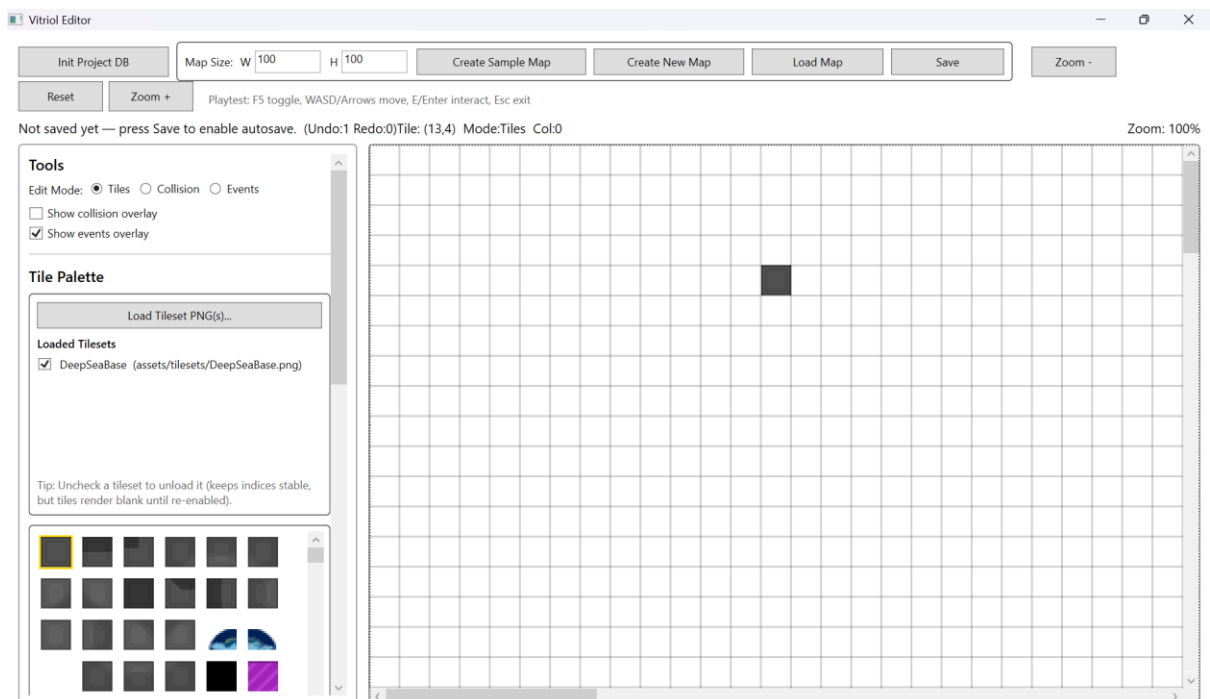
Demonstrates deterministic reapplication of a previously undone command, confirming symmetrical execution behaviour

Figure T10.1 – Tile State After Undo Operation



[Testing Section\Testing Section Images\Figure T10.1.png](#)

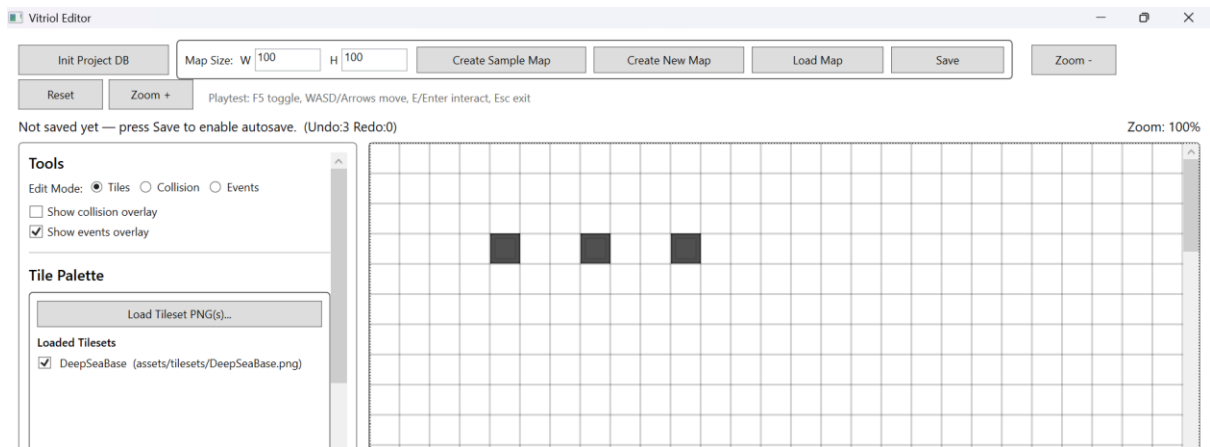
Figure 10.2 – Tile State After New Paint Action (Redo Attempt Blocked)



[Testing Section\Testing Section Images\Figure T10.2.png](#)

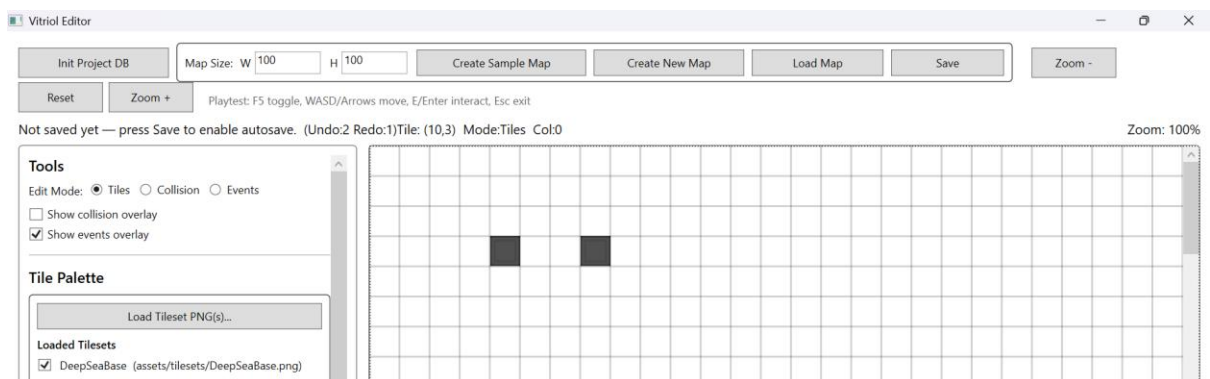
Demonstrates that performing a new edit after Undo correctly clear the redo stack, preventing invalid state replay

Figure T11.1 Grid After 3 Paint Operations



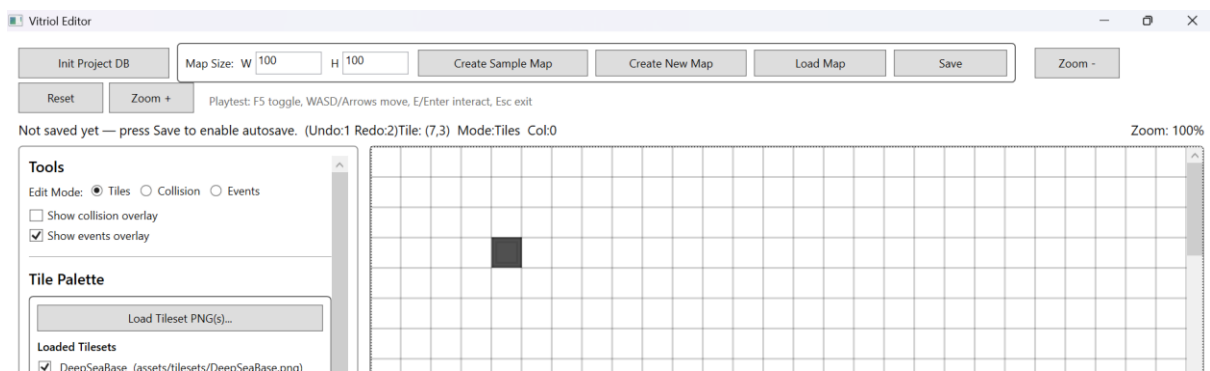
[Testing Section\Testing Section Images\Figure T11.1.png](#)

Figure T11.2 – Grid After 1st Undo Operation



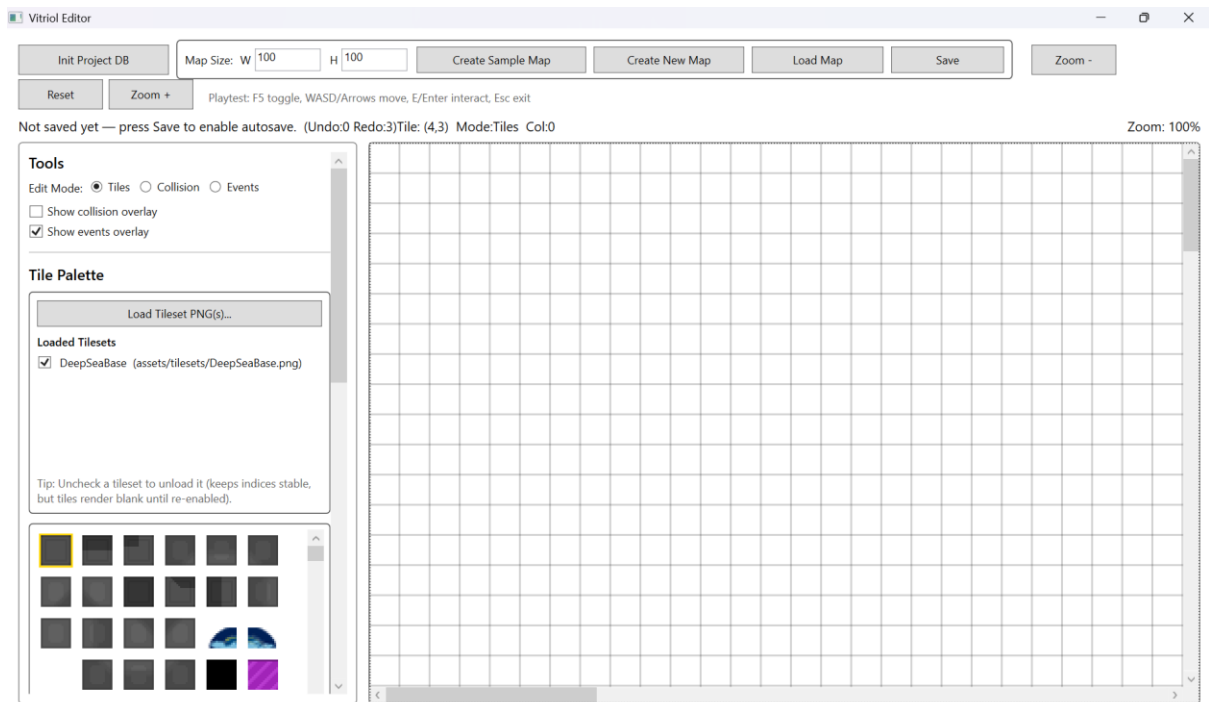
[Testing Section\Testing Section Images\Figure T11.2.png](#)

Figure T11.3 – Grid After 2nd Undo Operation



[Testing Section\Testing Section Images\Figure T11.3.png](#)

Figure T11.4 – Grid after 3th Undo Operation



[Testing Section\Testing Section Images\Figure T11.4.png](#)

Demonstrates correct last-in-first-out stack behaviour, restoring the grid to its original state after sequential command reversals

2.3 Tileset Loading and Referencing

This subsystem verifies that tilesets are referenced and loaded in a controlled and stable manner, preserving tile indices while allowing memory-efficient toggling of loaded assets.

Testing focused on:

- Correct storage of relative paths
- Successful loading and rendering of tileset images
- Stable tile indexing when toggling tilesets
- Correct behaviour when referenced tileset files are missing

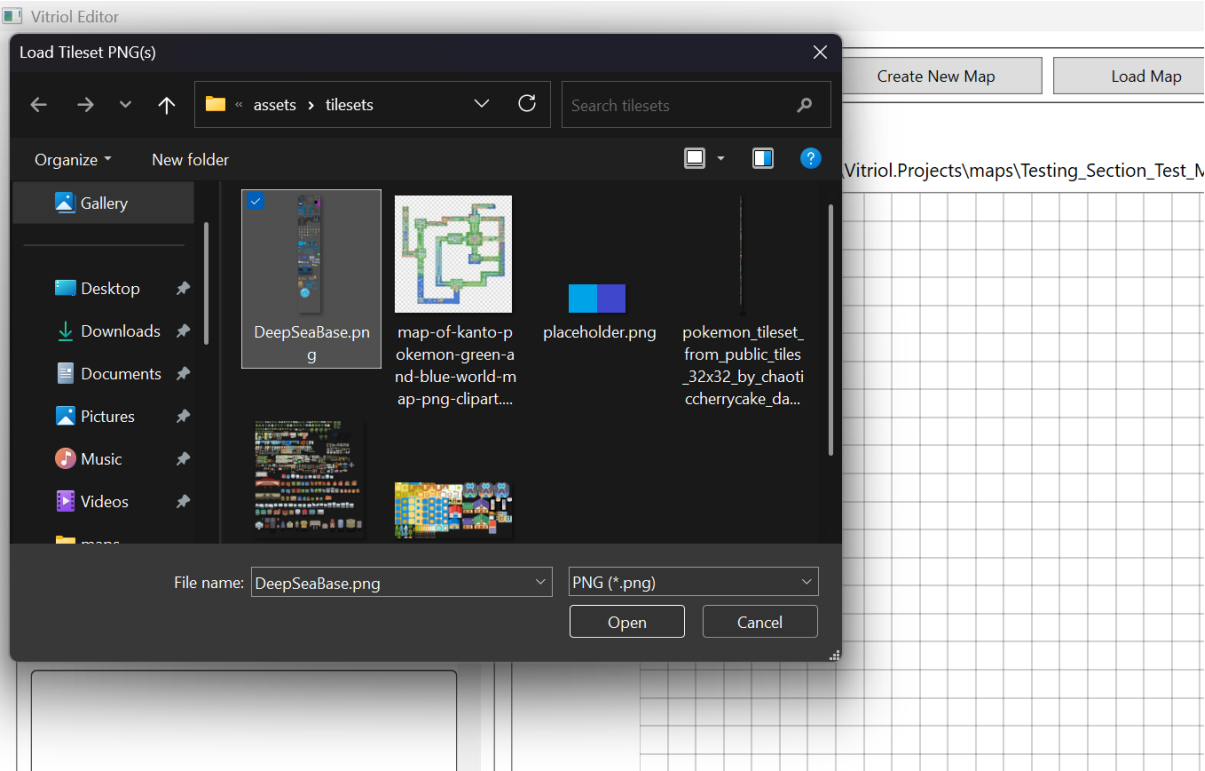
Test Table 3 – Tileset Loading & Referencing

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
TS1	Verify tileset reference stored as relative path	Select tileset from project folder	JSON stores relative path from project root	Pass	Figures T12.1, T12.2
TS2	Verify tileset loads and renders correctly	Load tileset and paint tile	Tiles render correctly in viewport	Pass	Figures T13.1, T13.2

TS3	Verify toggling tileset does not corrupt map	Load -> Unload -> Reload	Tile indices preserved	Pass	Figures T14.1, T14.2, T14.3
TS4	Abnormal test – missing tileset file	Rename/remove PNG file	Validation error shown or load prevented	Pass	Figures T15.1, T15.2

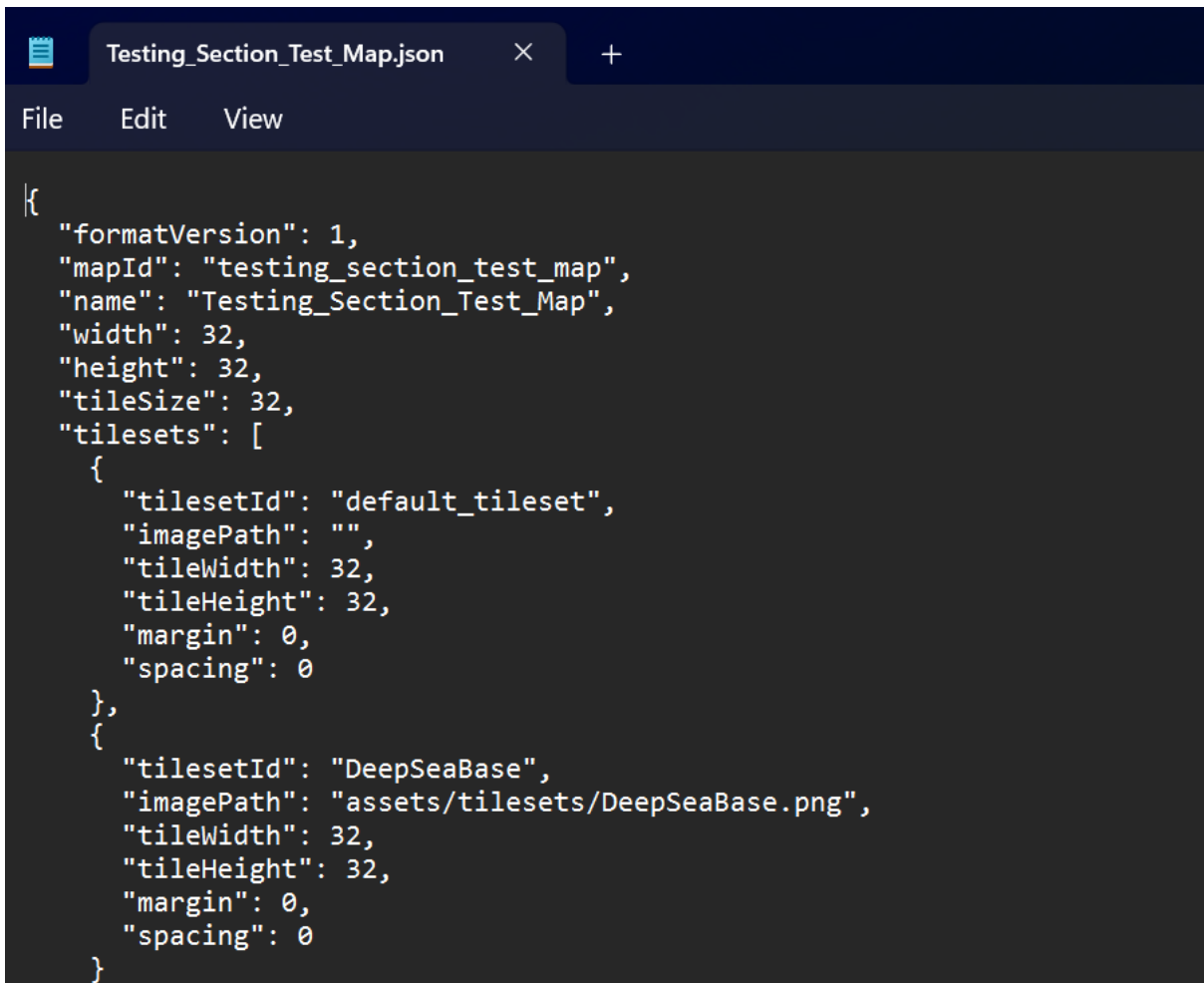
Test Table 3 – Screenshot Evidence

Figure T12.1 – Tileset Selected in Editor



Testing Section\Testing Section Images\Figure T12.1.png

Figure T12.2 – JSON File Showing Relative Tileset Path

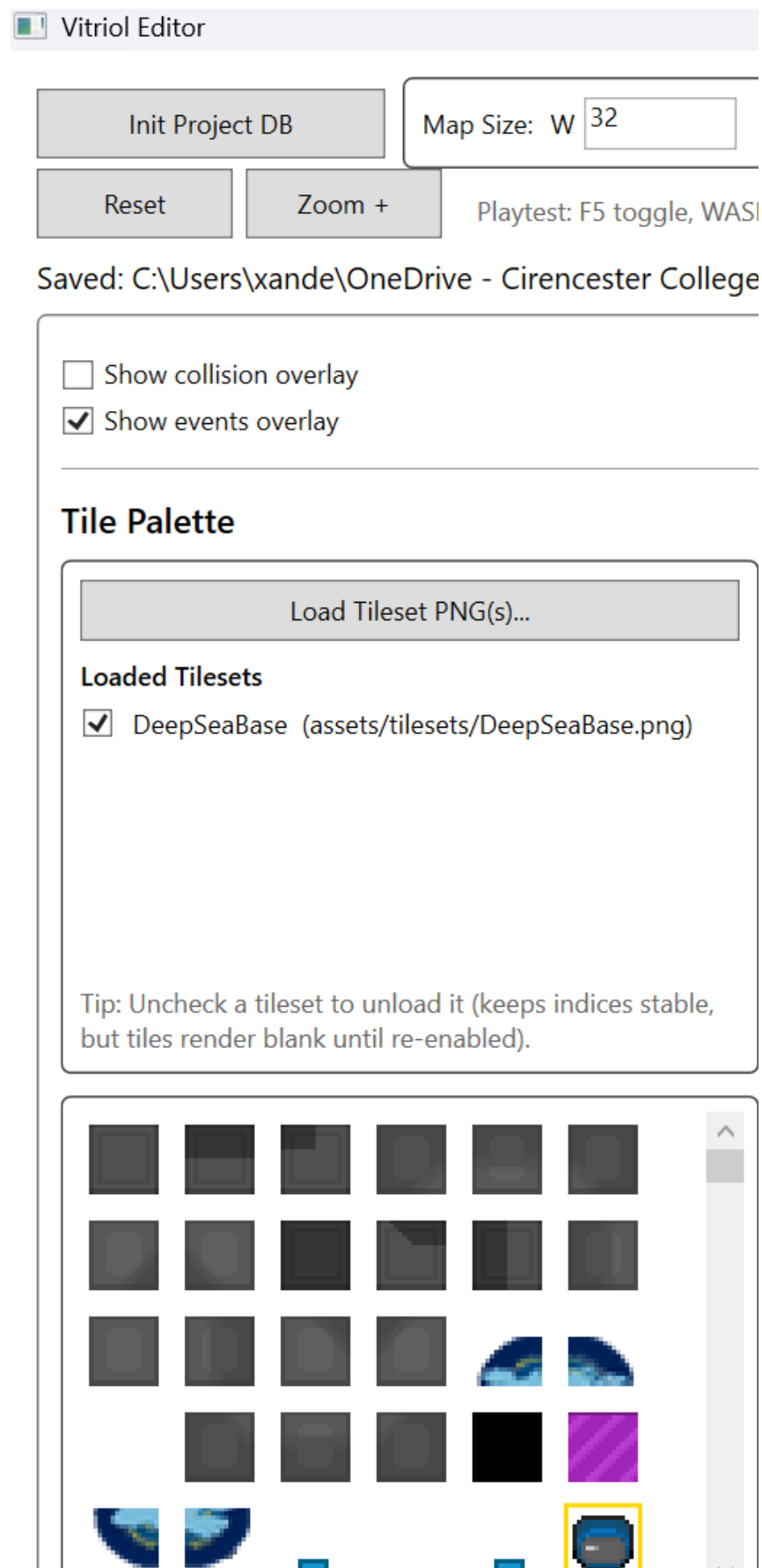


```
{
  "formatVersion": 1,
  "mapId": "testing_section_test_map",
  "name": "Testing_Section_Test_Map",
  "width": 32,
  "height": 32,
  "tileSize": 32,
  "tilesets": [
    {
      "tilesetId": "default_tileset",
      "imagePath": "",
      "tileWidth": 32,
      "tileHeight": 32,
      "margin": 0,
      "spacing": 0
    },
    {
      "tilesetId": "DeepSeaBase",
      "imagePath": "assets/tilesets/DeepSeaBase.png",
      "tileWidth": 32,
      "tileHeight": 32,
      "margin": 0,
      "spacing": 0
    }
  ]
}
```

[Testing Section\Testing Section Images\Figure T12.2.png](#)

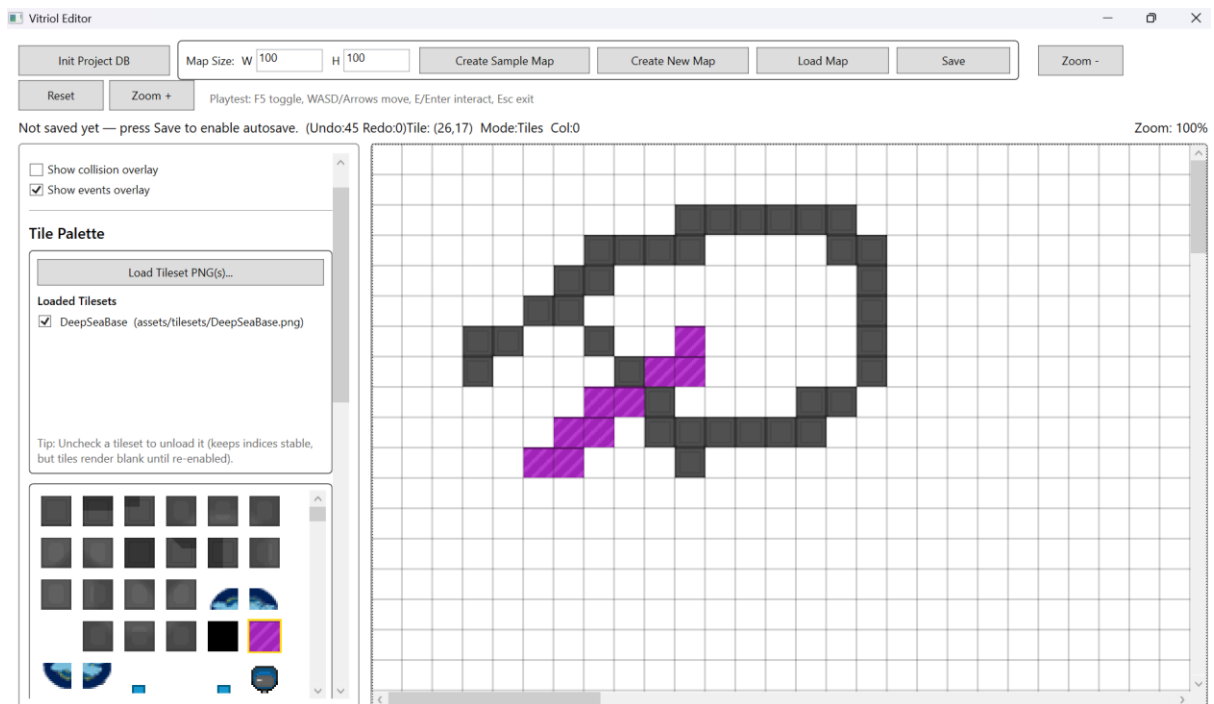
Demonstrates that tileset references are stored using project-relative paths, ensuring portability and version-control compatibility

Figure T13.1 – Tileset Loaded in Editor Palette



[Testing Section\Testing Section Images\Figure T13.1.png](#)

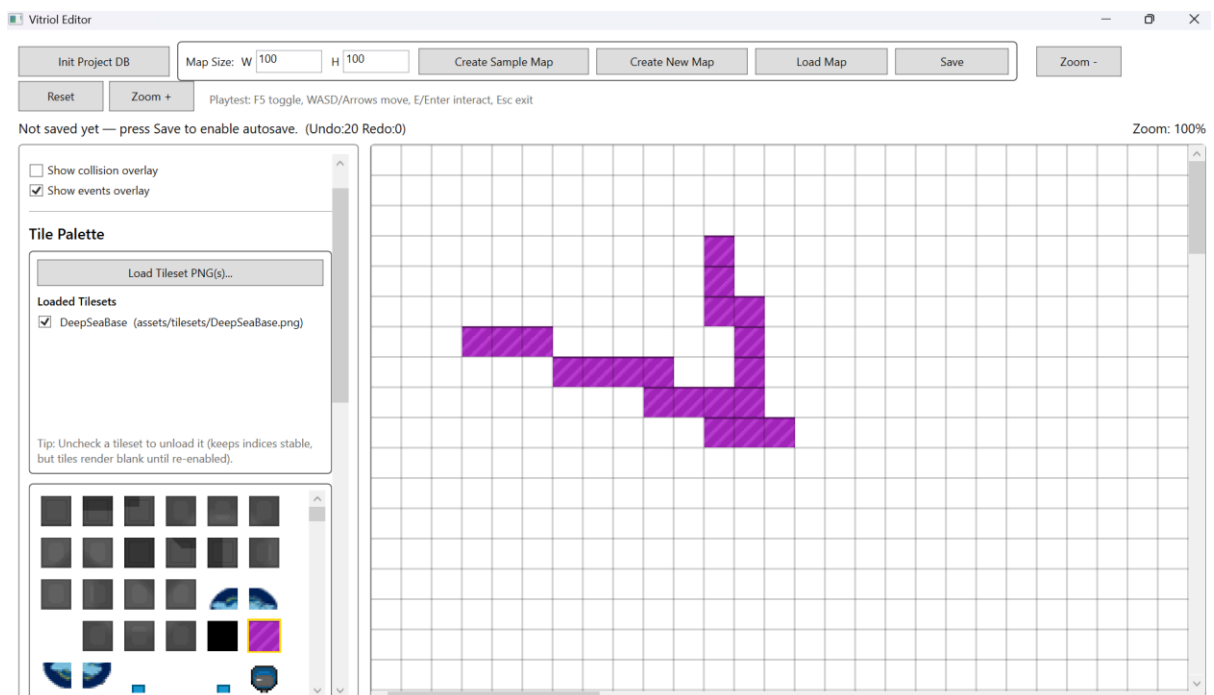
Figure T13.2 – Painted Tiles Rendering Correctly in Viewport



[Testing Section\Testing Section Images\Figure T13.2.png](#)

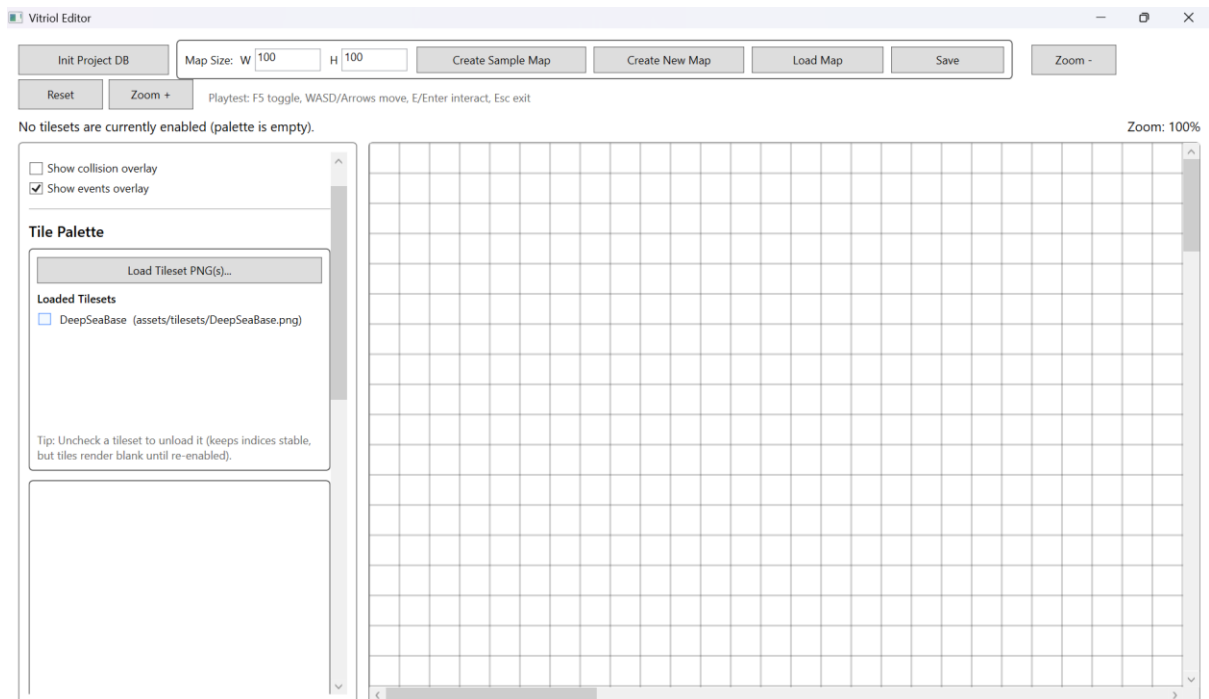
Demonstrates correct tileset loading and rendering within the editor viewport

Figure T14.1 – Painted Map Before Tileset Toggle



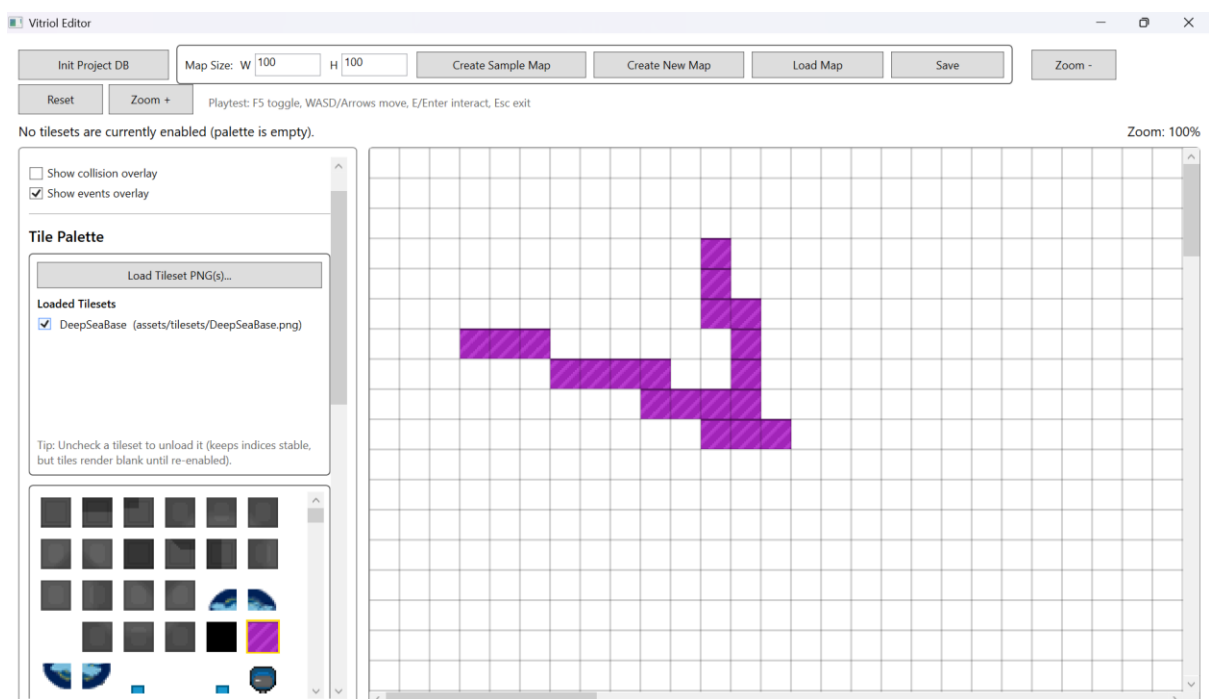
[Testing Section\Testing Section Images\Figure T14.1.png](#)

Figure T14.2 – Map After Tileset Toggle



[Testing Section\Testing Section Images\Figure T14.2.png](#)

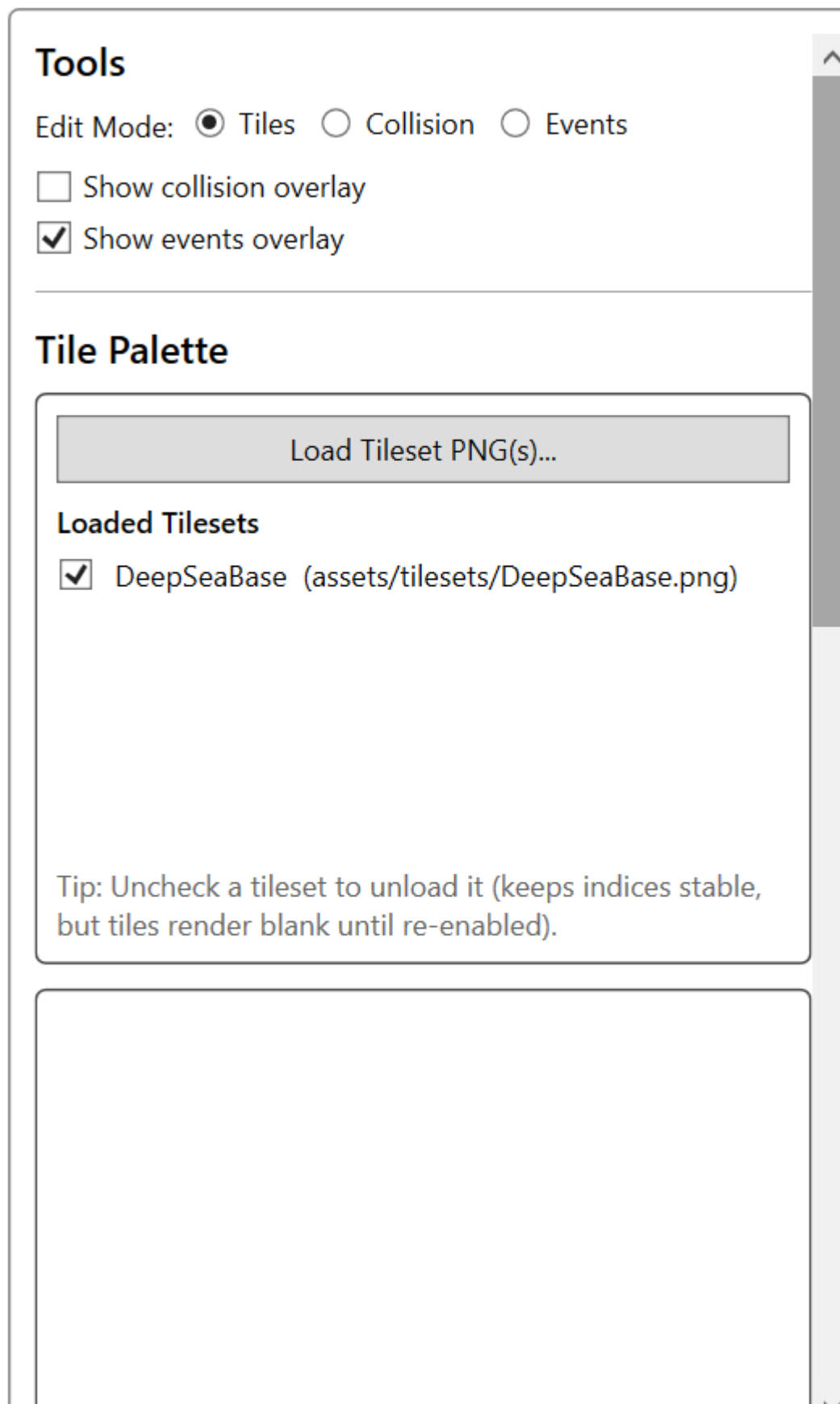
Figure T14.3 – Map After Tileset Reload (Indices Preserved)



[Testing Section\Testing Section Images\Figure T14.3.png](#)

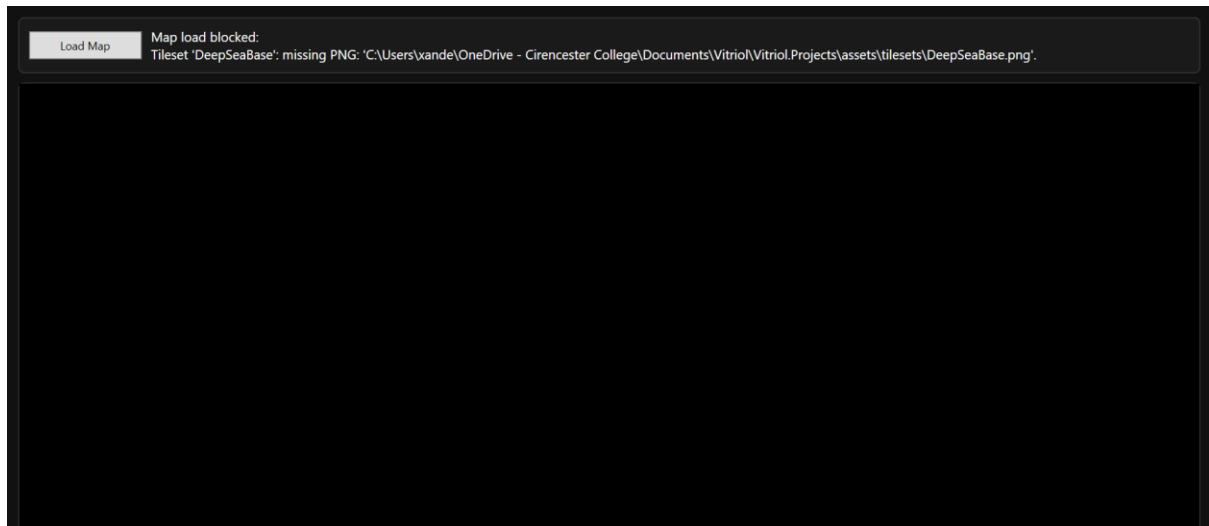
Demonstrates that tile indices remain stable when tilesets are unloaded and reloaded, preventing structural corruption

Figure T15.1 – Loaded PNG File Renamed



[Testing Section\Testing Section Images\Figure T15.1.png](#)

Figure T15.2 – Validation/Error Output for “Missing” Asset



[Testing Section\Testing Section Images\Figure T15.2.png](#)

Demonstrates controlled handling of missing asset references without runtime crash

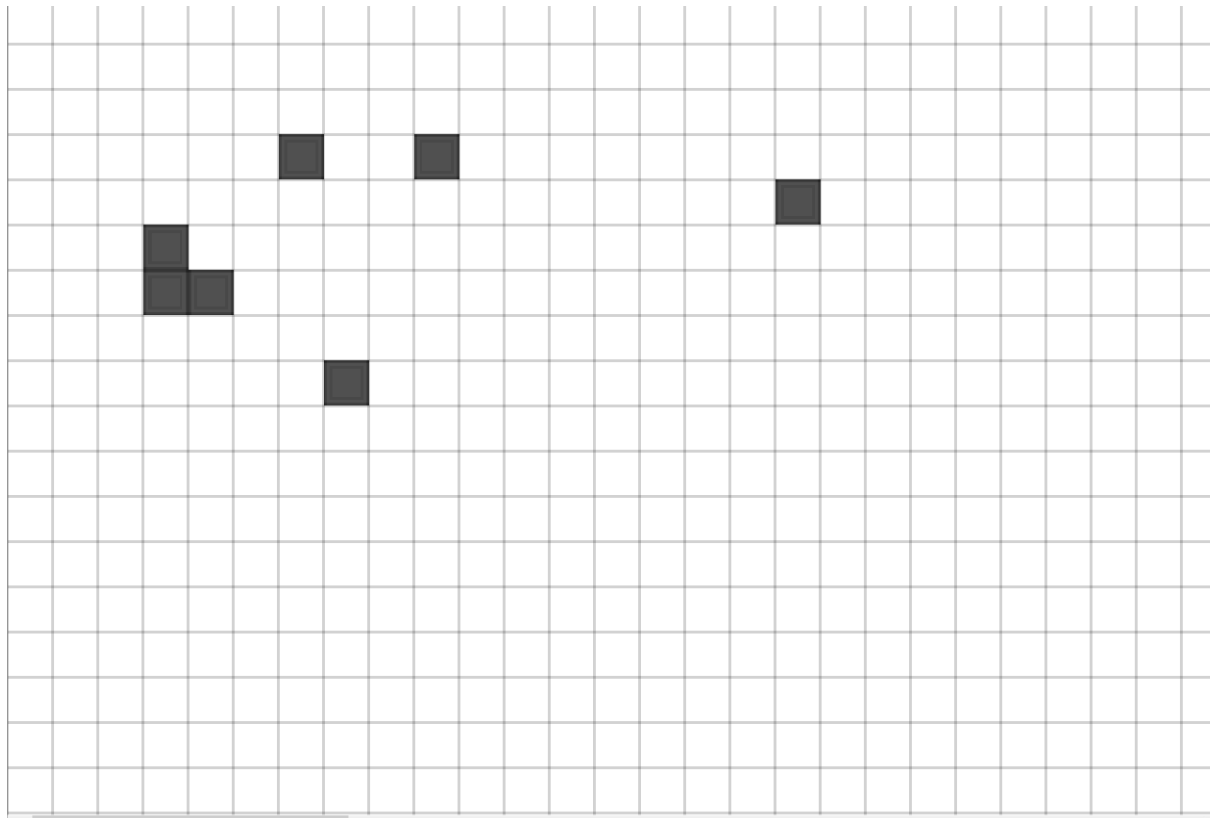
2.4 Event System Testing

Test Table 4 – Event System Testing

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
EV1	Verify event placement on map grid	Place NPC event at (10,10)	Event appears at correct coordinate	Pass	Figures T16.1, T16.2
EV2	Verify event serialisation integrity	Save map after event creation	Event stored correctly in JSON	Pass	Figures T17.1, T17.2
EV3	Verify event reload stability	Close editor -> reopen map	Event reappears at correct location	Pass	Figures T18.1, T18.2
EV4	Verify runtime event trigger execution	Trigger event in runtime	Event executes associated behaviour	Pass	Figures T19.1, T19.2
EV5	Abnormal test – invalid event parameters	Remove required argument manually in JSON	Validation prevents crash/handles safely	Pass	Figures T20.1, T20.2

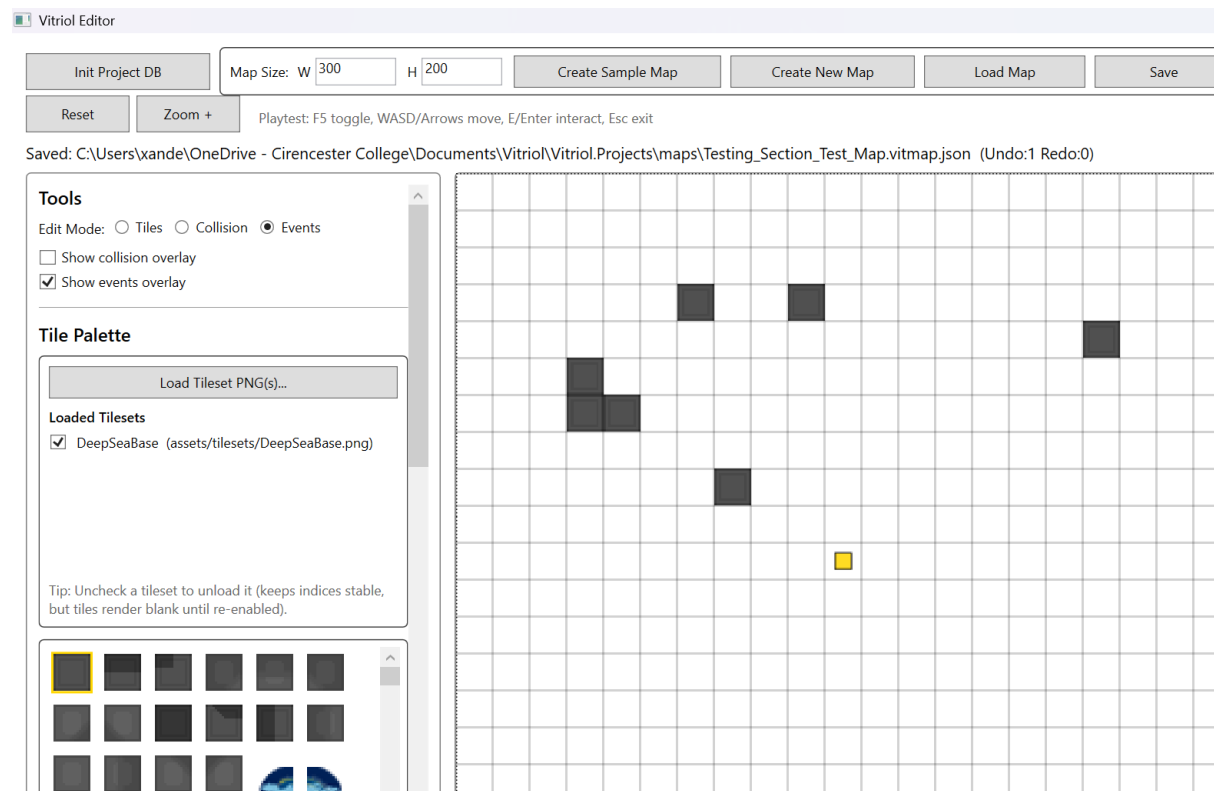
Test Table 4 – Screenshot Evidence

Figure T16.1 – Map Grid Before Event Placement



[Testing Section\Testing Section Images\Figure T16.1.png](#)

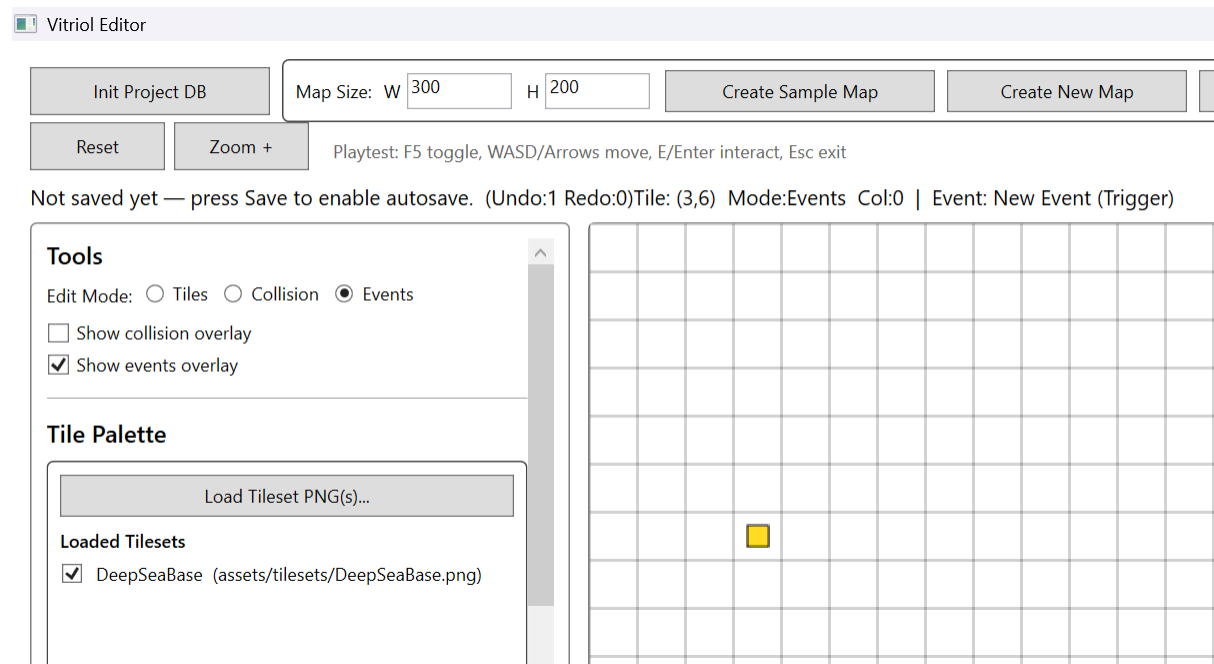
Figure T16.2 – Event Placed At Coordinate (10,10)



[Testing Section\Testing Section Images\Figure T16.2.png](#)

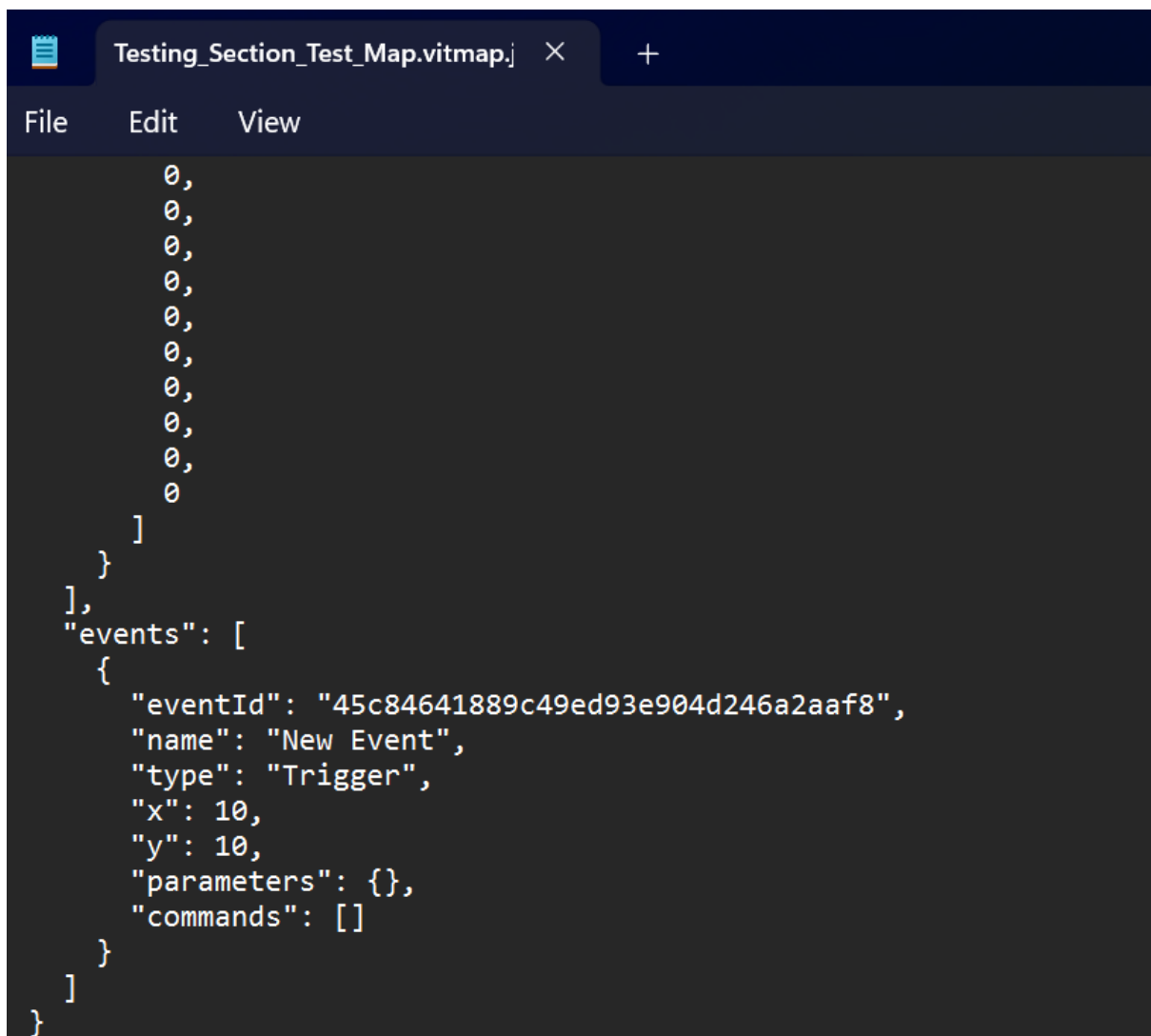
Demonstrates correct mapping between grid coordinate and event object placement

Figure T17.1 – Event Visible in Editor Prior to Save



[Testing Section\Testing Section Images\Figure T17.1.png](#)

Figure T17.2 – JSON Representation of Saved Event Data

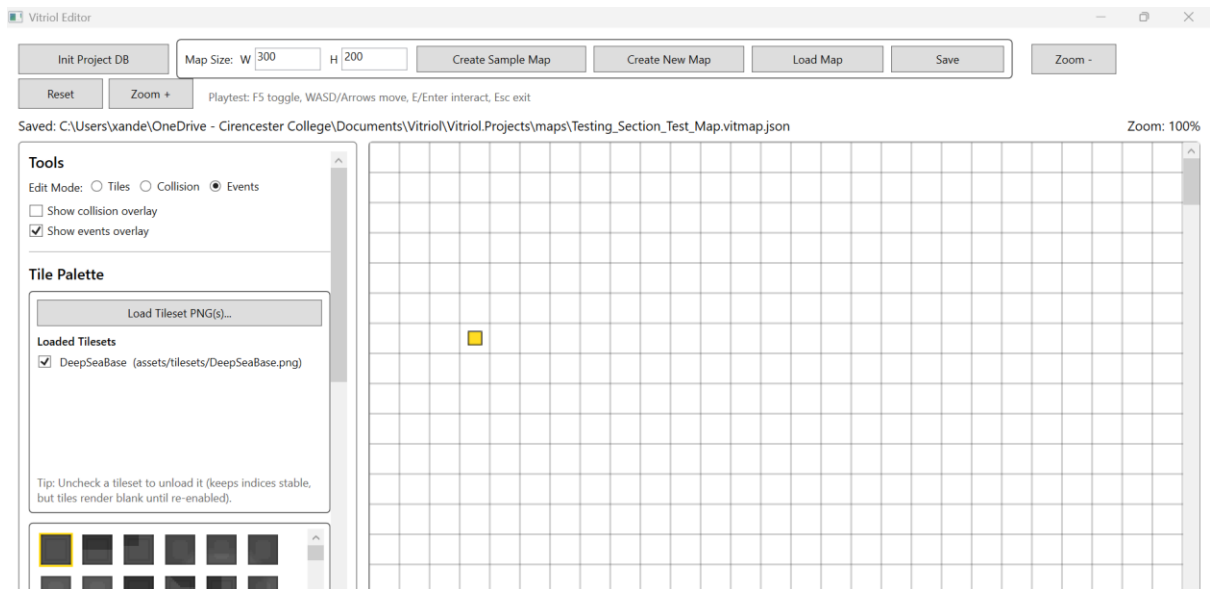
A screenshot of a code editor window with a dark theme. The title bar shows a file named 'Testing_Section_Test_Map.vitmap.j' with a close button and a plus sign for additional tabs. The menu bar includes 'File', 'Edit', and 'View'. The editor displays a JSON object. The first part is an array of ten zeros. This is followed by a closing bracket, a closing brace, a comma, and an 'events' array. The 'events' array contains a single object with the following properties: 'eventId' (a long hexadecimal string), 'name' ('New Event'), 'type' ('Trigger'), 'x' (10), 'y' (10), 'parameters' (an empty object), and 'commands' (an empty array).

```
0,
0,
0,
0,
0,
0,
0,
0,
0,
0
]
}
],
"events": [
{
  "eventId": "45c84641889c49ed93e904d246a2aaf8",
  "name": "New Event",
  "type": "Trigger",
  "x": 10,
  "y": 10,
  "parameters": {},
  "commands": []
}
]
```

[Testing Section\Testing Section Images\Figure T17.2.png](#)

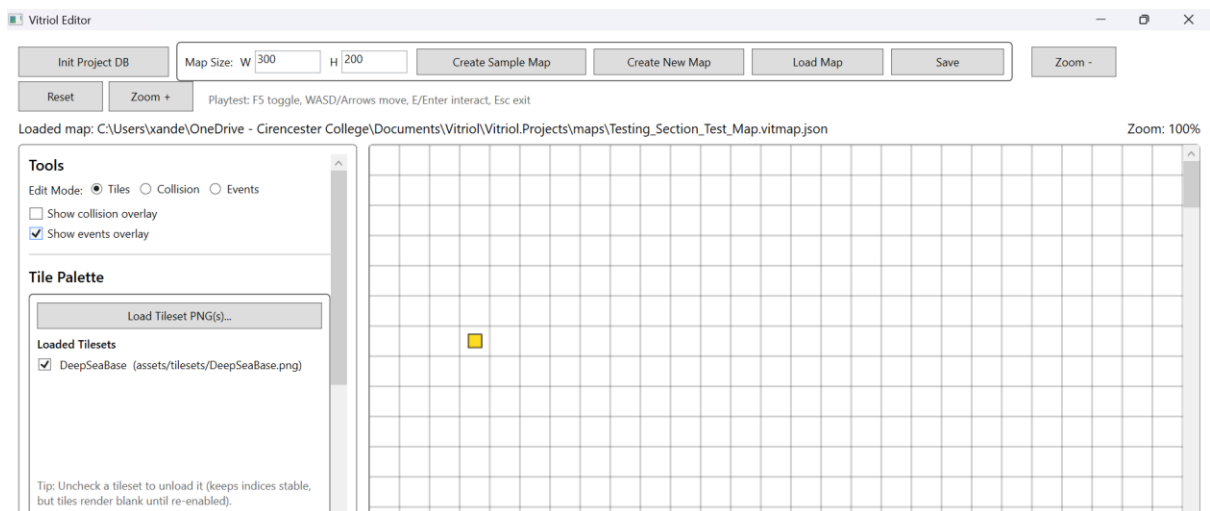
Demonstrates accurate serialisation of event metadata including ID, type, position, and command structure

Figure T18.1 – Event Visible Before Editor Restart



[Testing Section\Testing Section Images\Figure T18.1.png](#)

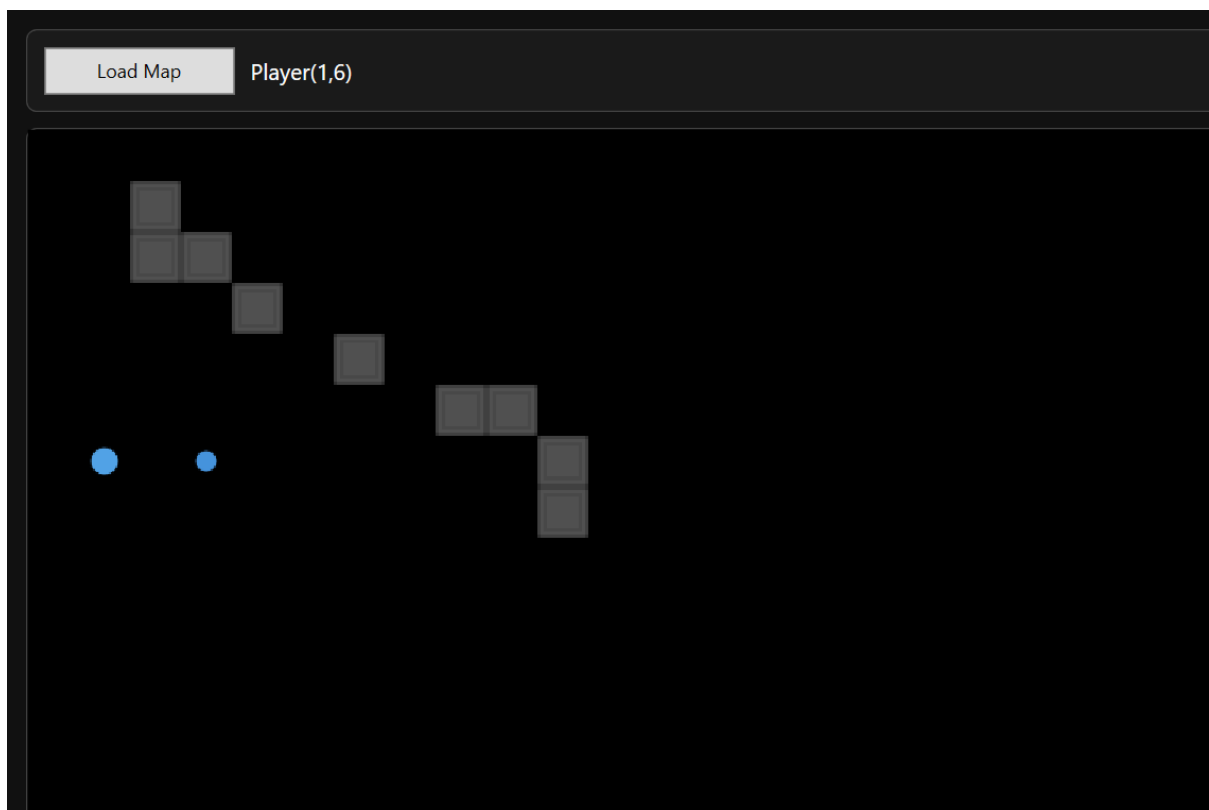
Figure T18.2 – Event Correctly Restored After Reload



[Testing Section\Testing Section Images\Figure T18.2.png](#)

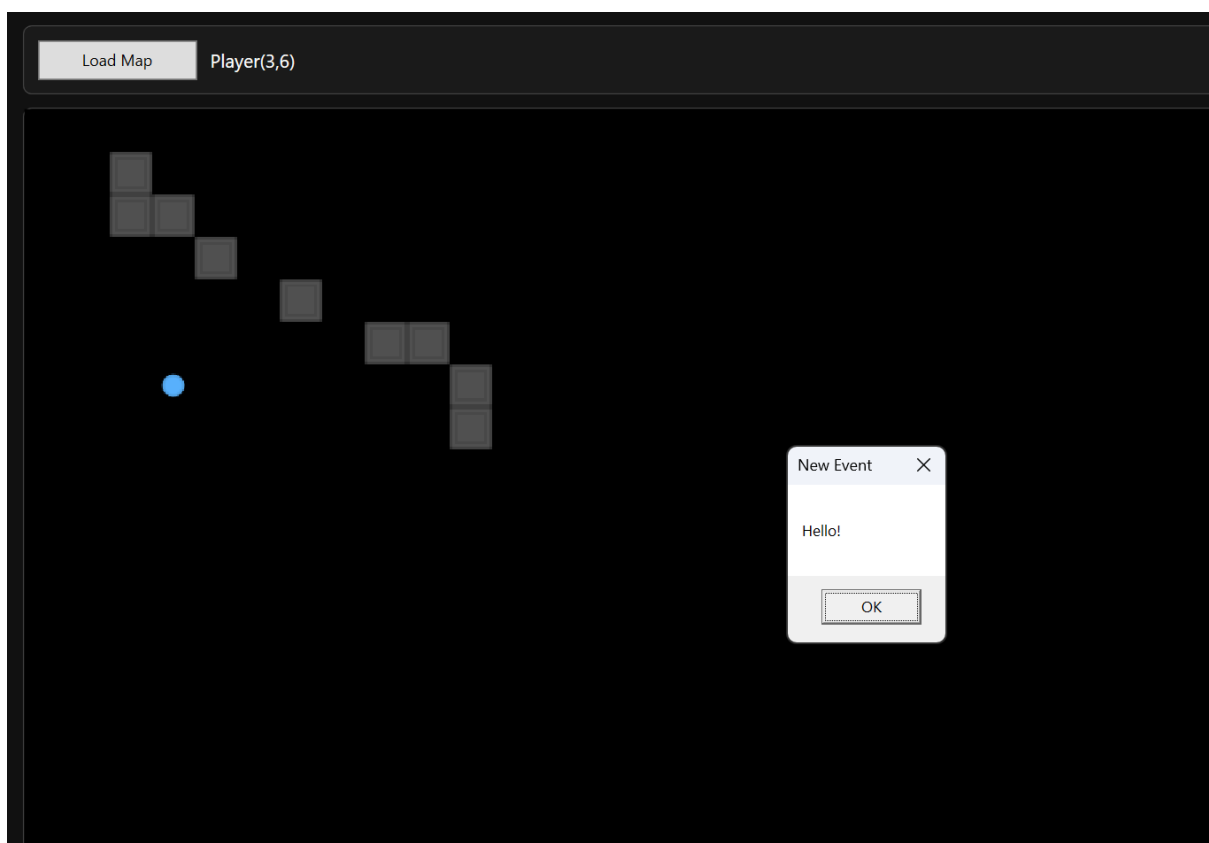
Demonstrates persistence integrity across sessions without data loss

Figure T19.1 – Player Approaching Event Trigger Tile



Testing Section\Testing Section Images\Figure T19.1.png

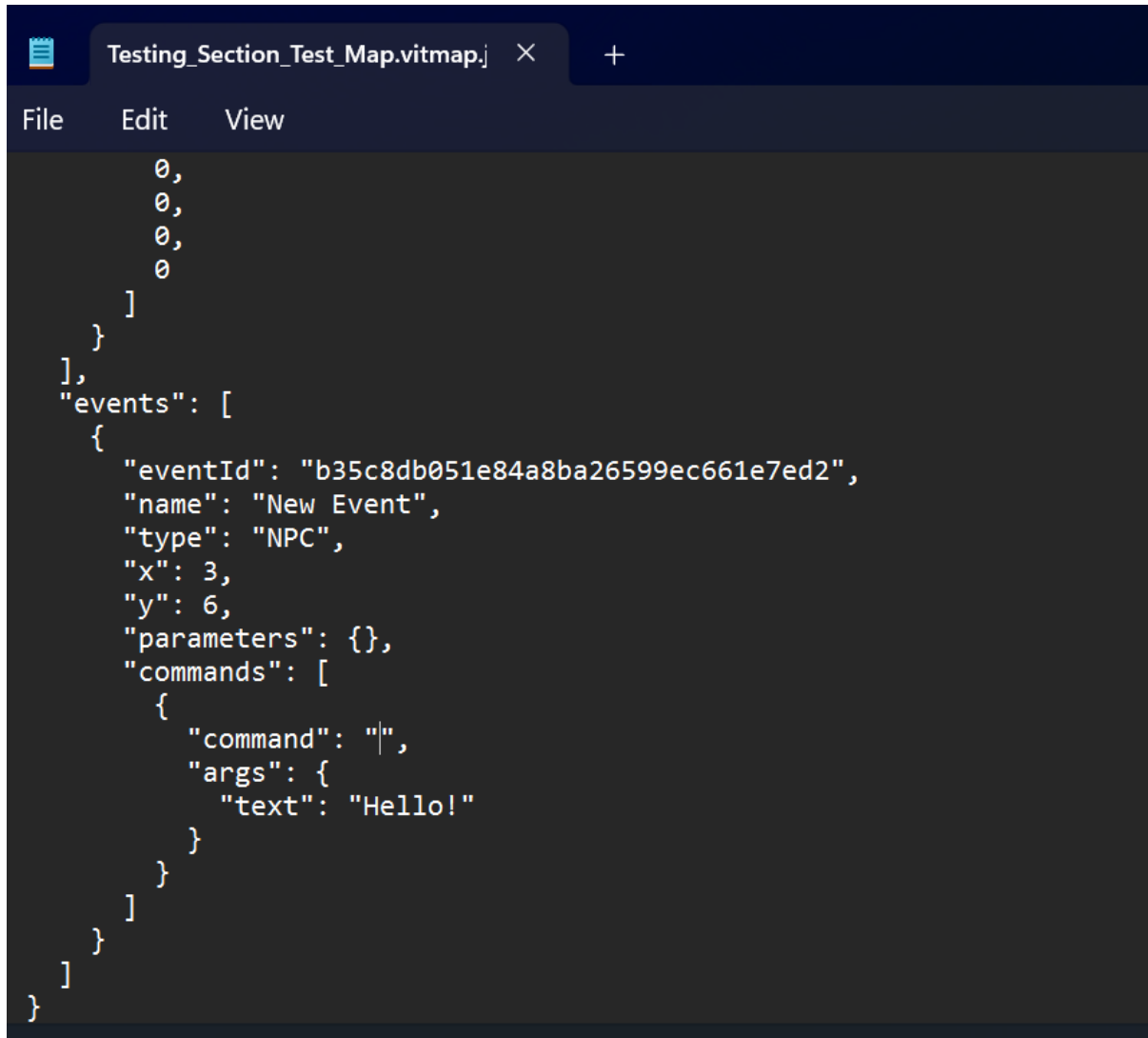
Figure T19.2 – Event Behaviour Executed in Runtime (Dialogue/Effect Displayed)



[Testing Section\Testing Section Images\Figure T19.2.png](#)

Demonstrates correct linkage between editor-defined event data and runtime execution logic

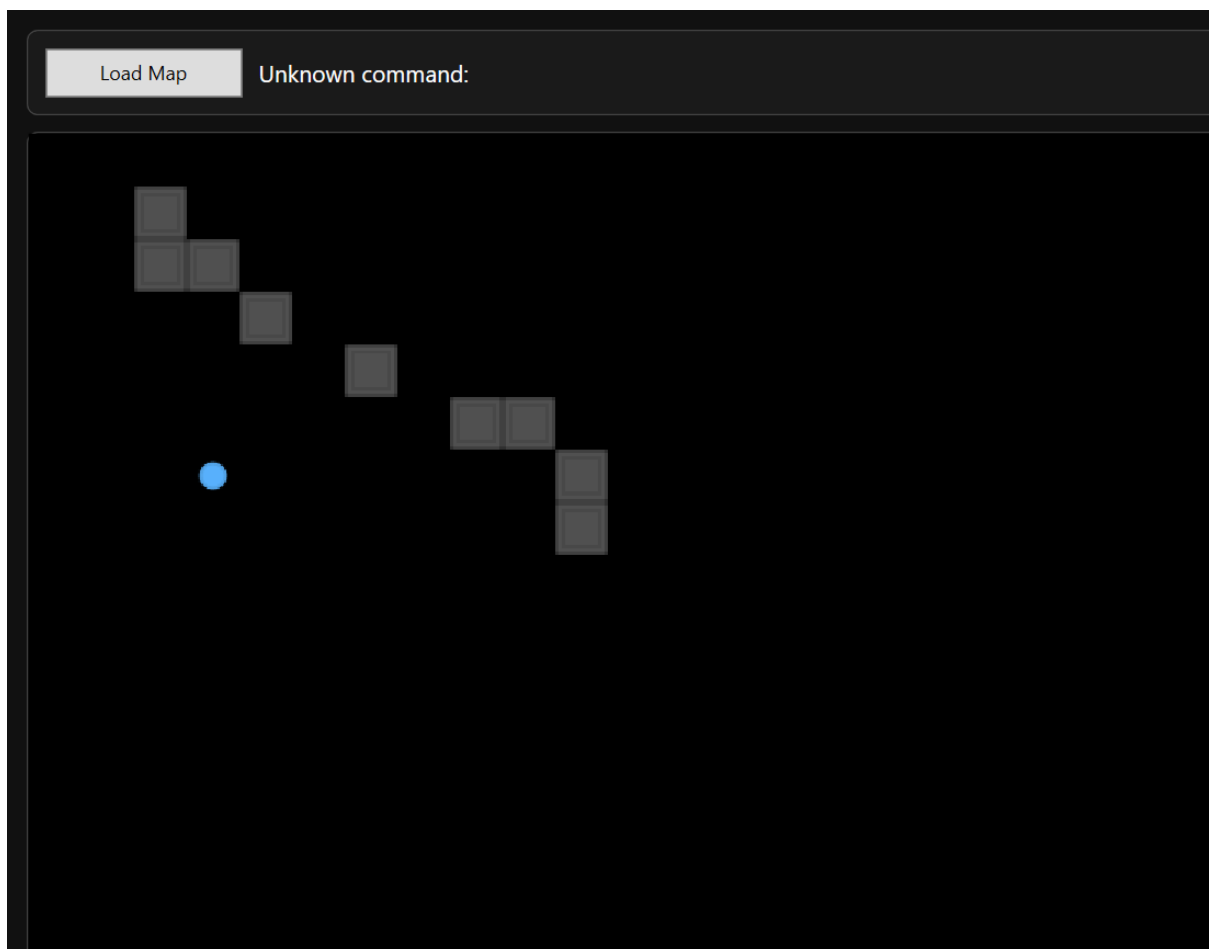
Figure T20.1 Modified JSON with Missing Event Parameter

A screenshot of a code editor window with a dark theme. The title bar shows the file name 'Testing_Section_Test_Map.vitmap.j' and a plus sign for additional tabs. The menu bar includes 'File', 'Edit', and 'View'. The code is a JSON object with the following structure:

```
{
  0,
  0,
  0,
  0
]
},
"events": [
  {
    "eventId": "b35c8db051e84a8ba26599ec661e7ed2",
    "name": "New Event",
    "type": "NPC",
    "x": 3,
    "y": 6,
    "parameters": {},
    "commands": [
      {
        "command": "|",
        "args": {
          "text": "Hello!"
        }
      }
    ]
  }
]
}
```

[Testing Section\Testing Section Images\Figure T20.1.png](#)

Figure T20.2 – Validation Message or Safe Handling During Load



[Testing Section\Testing Section Images\Figure T20.2.png](#)

Demonstrates defensive validation preventing malformed event data from causing runtime instability

2.5 Persistence and JSON Integrity

The persistence subsystem is responsible for ensuring that map data is reliably stored as structured JSON files and can be reloaded without corruption or structural drift. Because the project deliberately uses JSON files instead of a relational database, testing focused on validating:

- Correct file creation and naming
- Autosave behaviour rules
- JSON schema stability
- Round-trip integrity (save -> close -> reload)
- Cross-application compatibility (Editor -> Runtime)

The following tests validate that:

- Manual save produces a valid .vitmap.json file

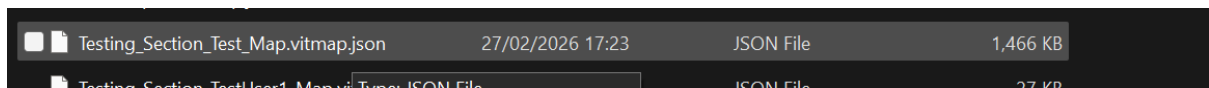
- Autosave follows intended behavioural rules
- JSON structure remains consistent with the defined schema
- Map state survives editor restart
- Runtime can load editor-produced files without format mismatch

Test Table 5 – Persistence & JSON Integrity

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
PJ1	Verify manual save creates valid JSON file	Create new user map -> paint tiles -> Save As	JSON file created in maps folder, correct structure present	Pass	Figures T21.1, T21.2
PJ2	Verify autosave rules (sample vs user map)	Edit sample map and new user map	Sample autosaves automatically, new map requires first manual save	Pass	Figures T22.1, T22.2 T22.3
PJ3	Verify JSON schema structure	Open saved JSON in Notepad	File contains required root fields (FormatVersion, Width, Height, Layers, Tilesets, Events)	Pass	Figures T23.1, T23.2, T23.3
PJ4	Verify round-trip persistence	Save -> close editor -> reopen -> reload map	Tiles, events, and layout restored correctly	Pass	Figures T24.1, T24.2
PJ5	Verify runtime compatibility	Load same map in runtime	Runtime loads successfully and renders identical layout	Pass	Figures T25.1, T25.2

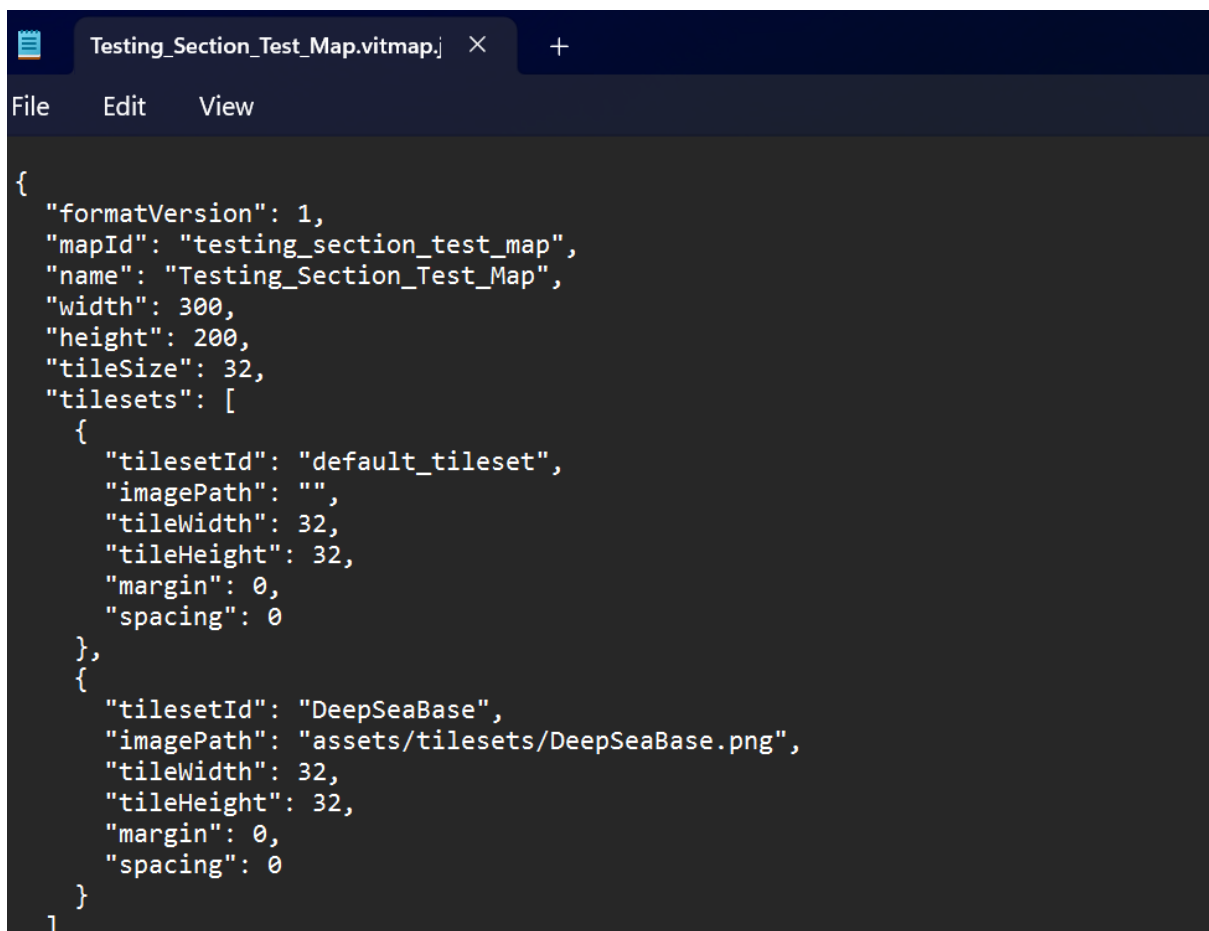
Test Table 5 – Screenshot Evidence

Figure T21.1 – Maps Folder Showing Newly Created .vitmap.json File



[Testing_Section\Testing_Section Images\Figure T21.1.png](#)

Figure T21.2 – JSON File Open in Notepad Showing Root Structure

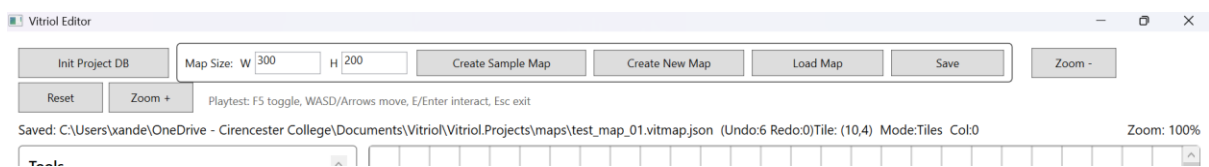


```
{
  "formatVersion": 1,
  "mapId": "testing_section_test_map",
  "name": "Testing_Section_Test_Map",
  "width": 300,
  "height": 200,
  "tileSize": 32,
  "tilesets": [
    {
      "tilesetId": "default_tileset",
      "imagePath": "",
      "tileWidth": 32,
      "tileHeight": 32,
      "margin": 0,
      "spacing": 0
    },
    {
      "tilesetId": "DeepSeaBase",
      "imagePath": "assets/tilesets/DeepSeaBase.png",
      "tileWidth": 32,
      "tileHeight": 32,
      "margin": 0,
      "spacing": 0
    }
  ]
}
```

[Testing Section\Testing Section Images\Figure T21.2.png](#)

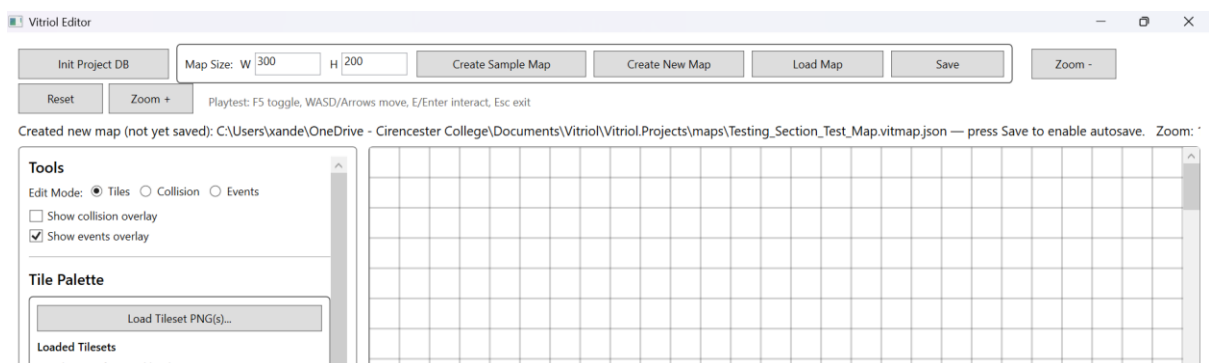
Demonstrates correct file creation and schema structure after manual save

Figure T22.1 – Sample Map Modified Timestamp Updating Automatically



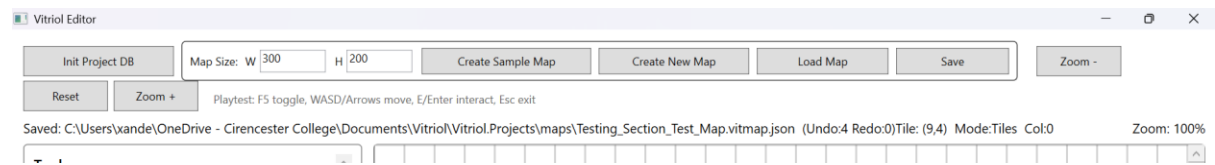
[Testing Section\Testing Section Images\Figure T22.1.png](#)

Figure T22.2 – User Map Not Saved Before First Manual Save



[Testing Section\Testing Section Images\Figure T22.2.png](#)

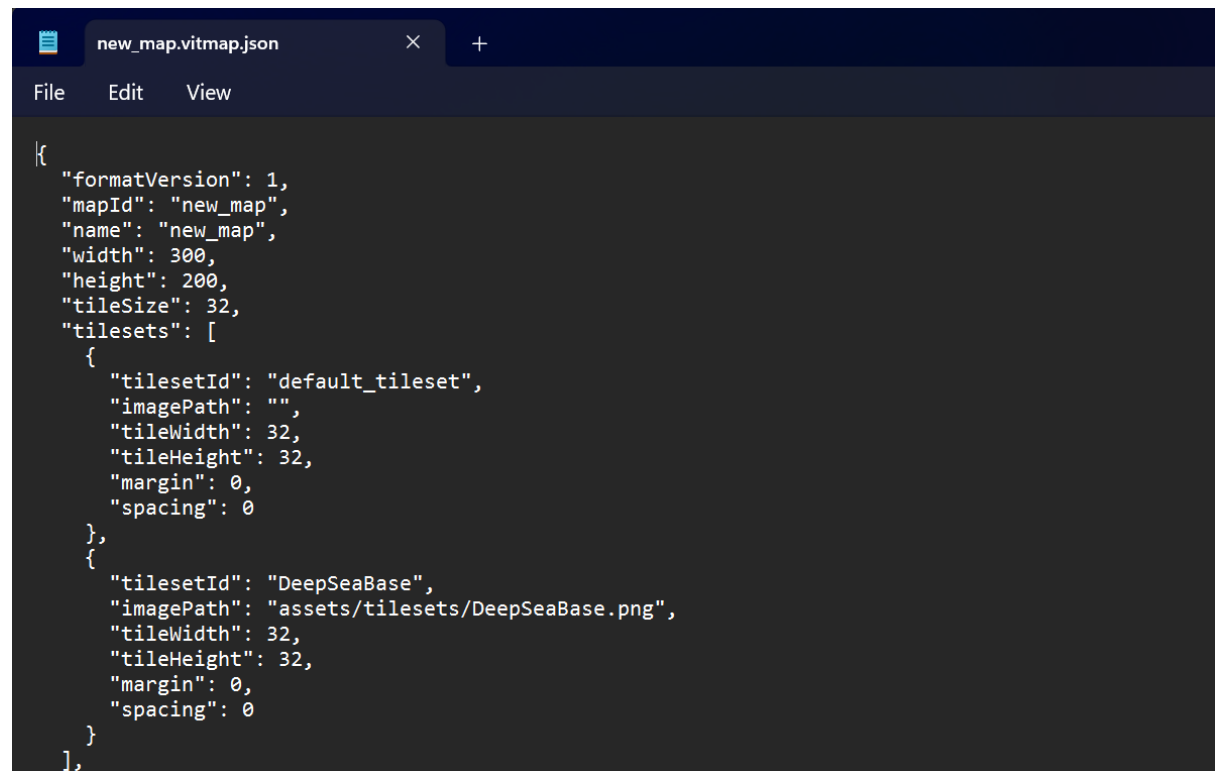
Figure T22.3 – User Map Autosave Active After Initial Manual Save



[Testing Section\Testing Section Images\Figure T22.3.png](#)

Demonstrates correct enforcement of autosave safety logic

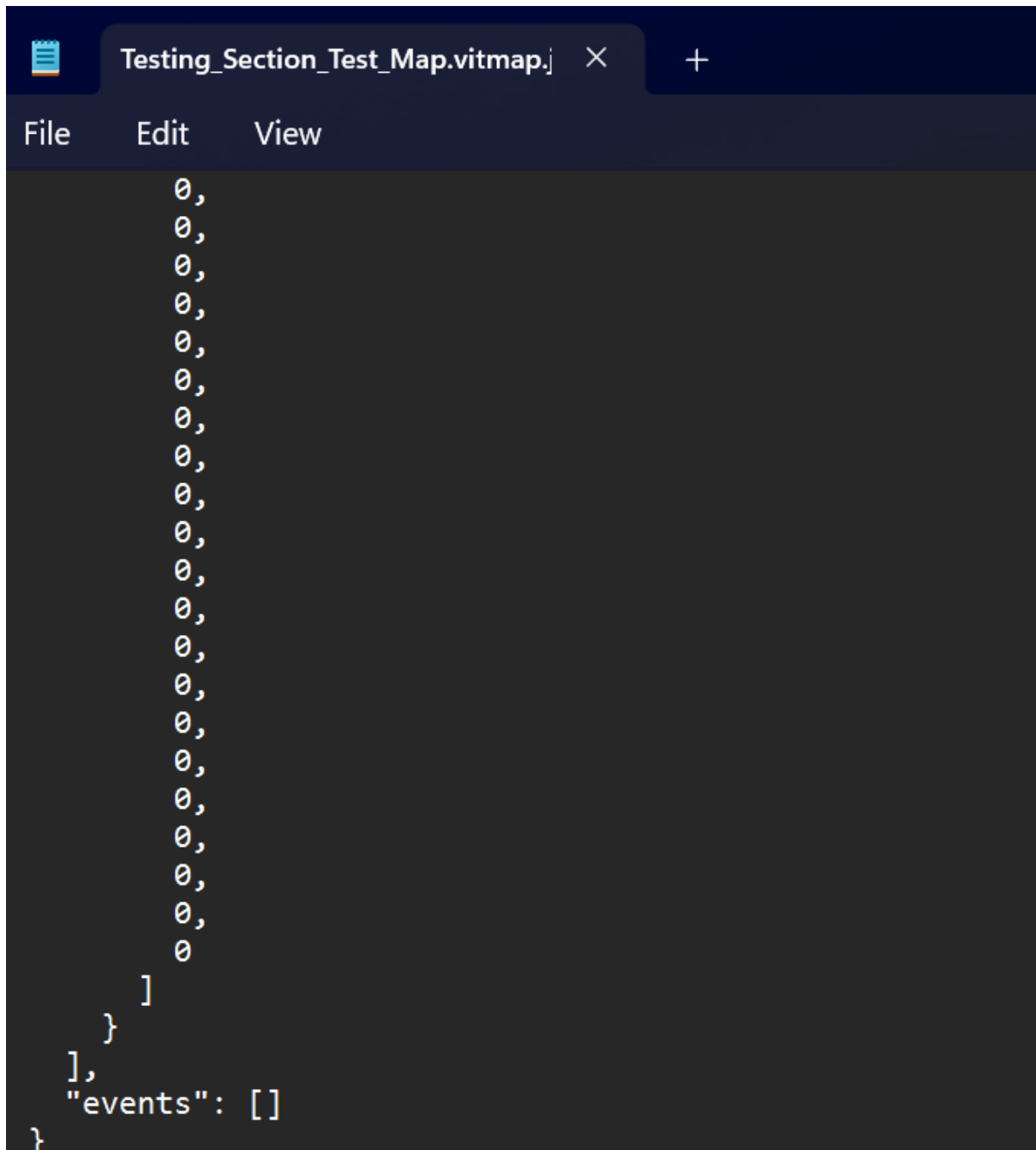
Figure T23.1 – JSON Root Fields Visible



[Testing Section\Testing Section Images\Figure T23.1.png](#)

Figure T23.2, T23.3 – Layers and Events Structure Visible

[illegible]

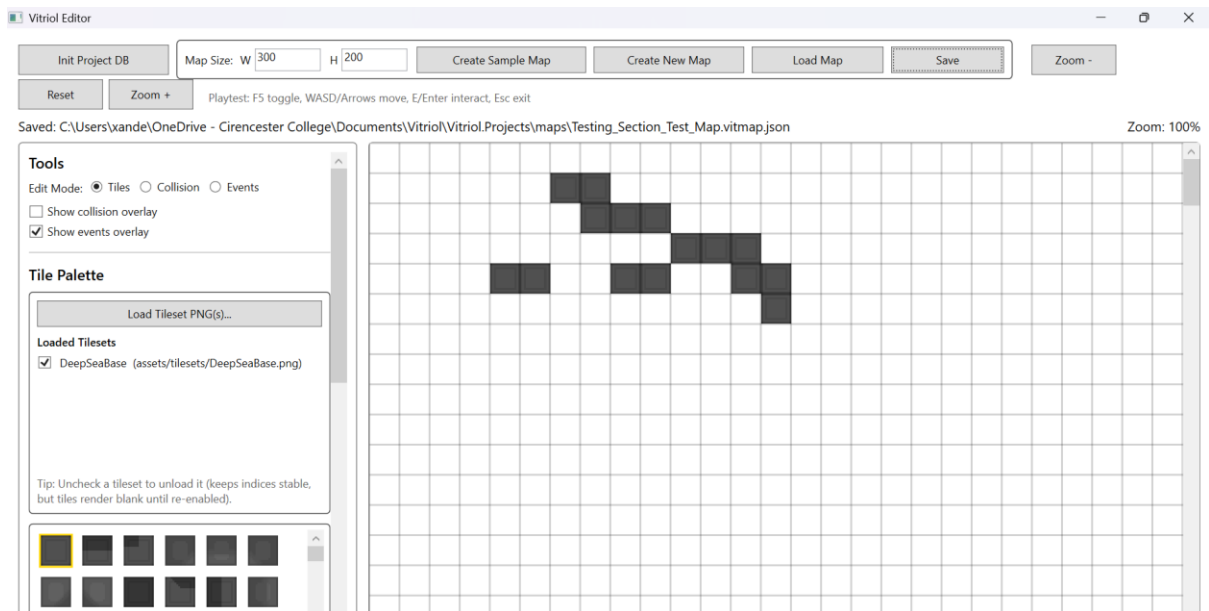


Testing Section\Testing Section Images\Figure T23.2.png

Testing Section\Testing Section Images\Figure T23.3.png

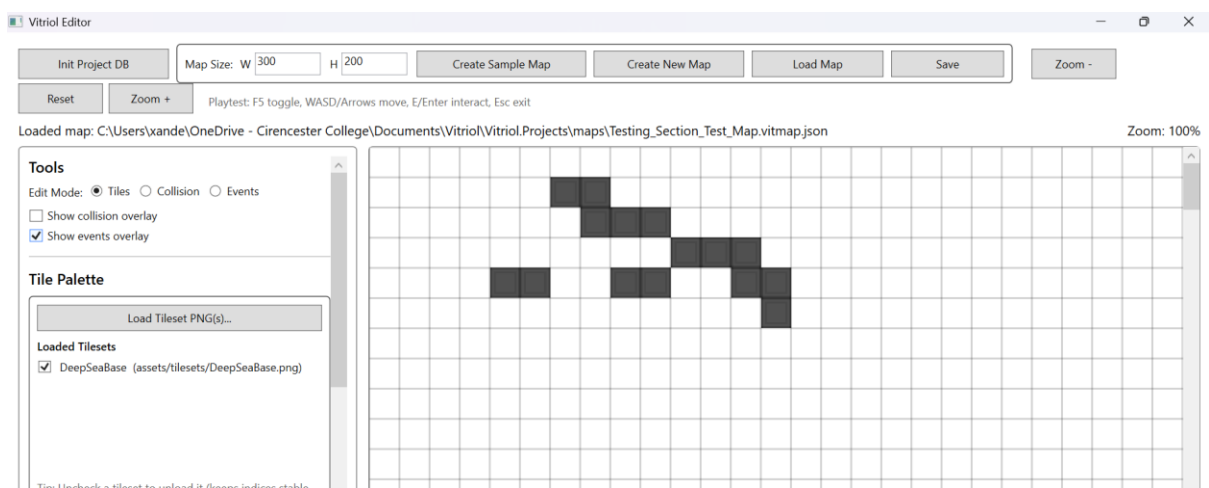
Demonstrates Structural Consistency with Defined Schema

Figure T24.1 – Map State Before Closing Editor



[Testing Section\Testing Section Images\Figure T24.1.png](#)

Figure T24.2 – Map State Restored After Reload



[Testing Section\Testing Section Images\Figure T24.2.png](#)

Demonstrates no data loss across sessions

Figure T25.1 – Runtime Loading Map File



[Testing Section\Testing Section Images\Figure T25.1.png](#)

Figure T25.2 – Runtime Rendering Saved Map Layout



[Testing Section\Testing Section Images\Figure T25.2.png](#)

Demonstrates editor/runtime format alignment and absence of schema drift

2.6 Runtime Rendering and Viewport Behaviour

The runtime rendering subsystem is responsible for displaying the map efficiently while maintaining responsive gameplay. Because Vitriol uses viewport-only rendering, the runtime should never attempt to draw the entire map every frame. Instead, it renders only the visible region (plus padding), producing a single composited output image that updates as the camera moves.

Testing focused on validating:

- Viewport-only rendering correctness (only visible region drawn)
- Stable rendering while scrolling (no blank gaps / tearing)
- Correct tile alignment as player moves across tile boundaries
- Render update logic triggered by movement and camera shifts
- Behaviour on larger maps where full rendering would be expensive

The following tests validate that:

- The runtime renders a cropped region rather than the entire map
- The viewport updates correctly when the player moves
- Large maps remain usable without frame collapse
- Visual output stays aligned to the map grid during camera movement

Test Table 6 – Runtime Rendering & Viewport

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
RV1	Verify viewport-only rendering (cropped region)	Load a large map and observe only current region visible	Only visible area is drawn, map can scroll without rendering full map	Pass	Figures T26.1, T26.2
RV2	Verify render updates on player movement	Move player across several tiles	Viewport redraws correctly, no lag/tearing	Pass	Figures T27.1, T27.2
RV3	Verify render stability during scroll/camera change	Move until camera shifts multiple times	Camera shifts smoothly, no blank regions or misalignment	Pass	Figures T28.1, T28.2
RV4	Boundary test – map edge rendering	Move player to far edge of map	Viewport clamps correctly, no drawing outside bounds	Pass	Figures T29.1, T29.2

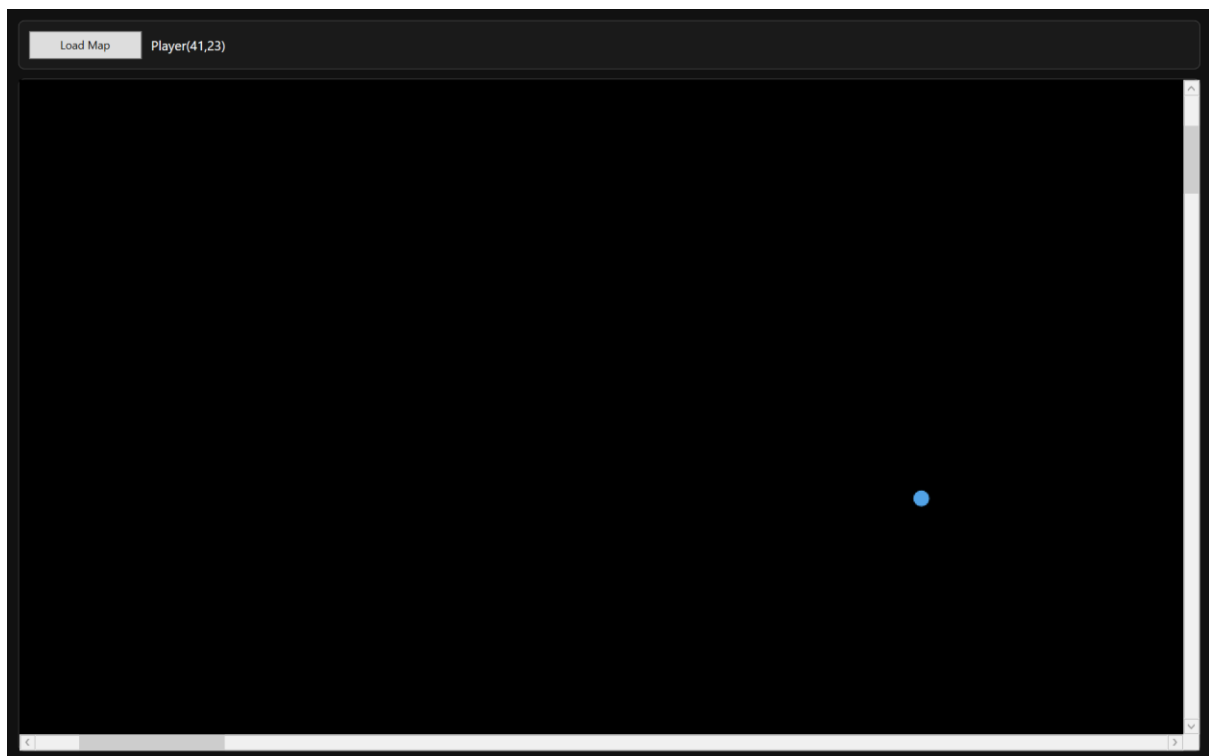
Test Table 6 – Screenshot Evidence

Figure T26.1 – Runtime Loaded Large Map (Initial Viewport Region)



[Testing Section\Testing Section Images\Figure T26.1.png](#)

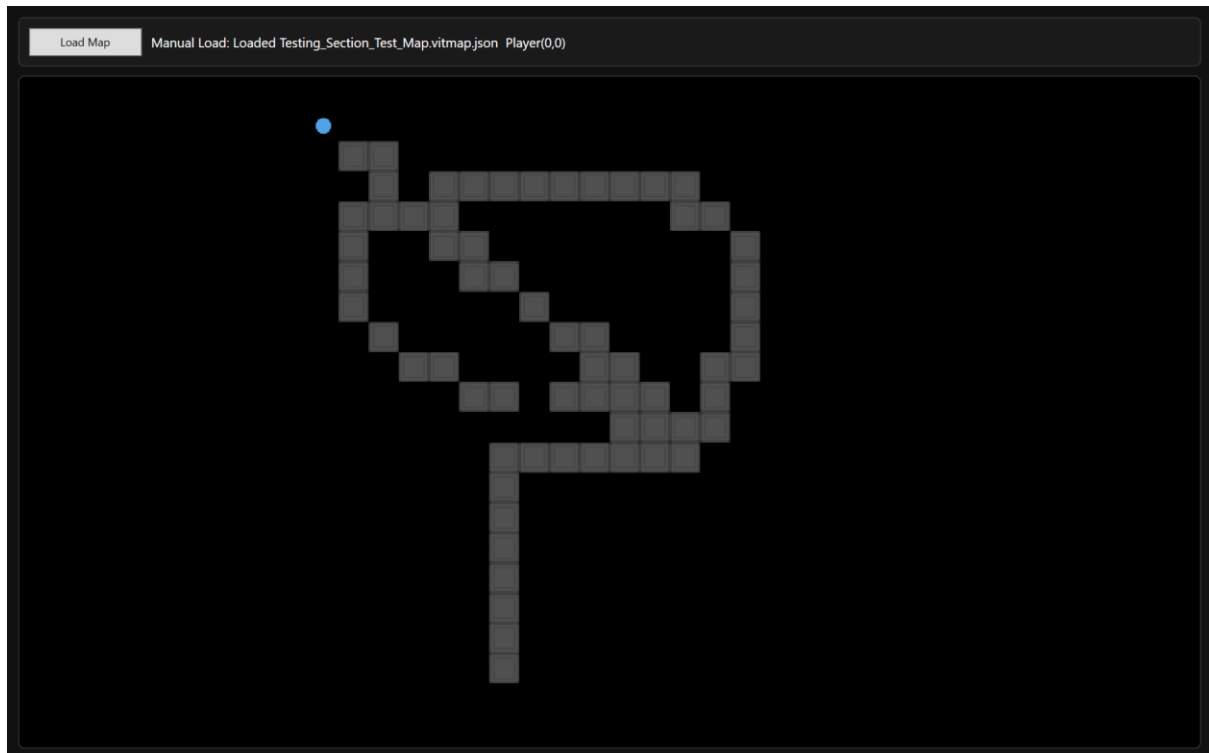
Figure T26.2 – Same Map After Moving Player (Viewport Region Changed)



[Testing Section\Testing Section Images\Figure T26.2.png](#)

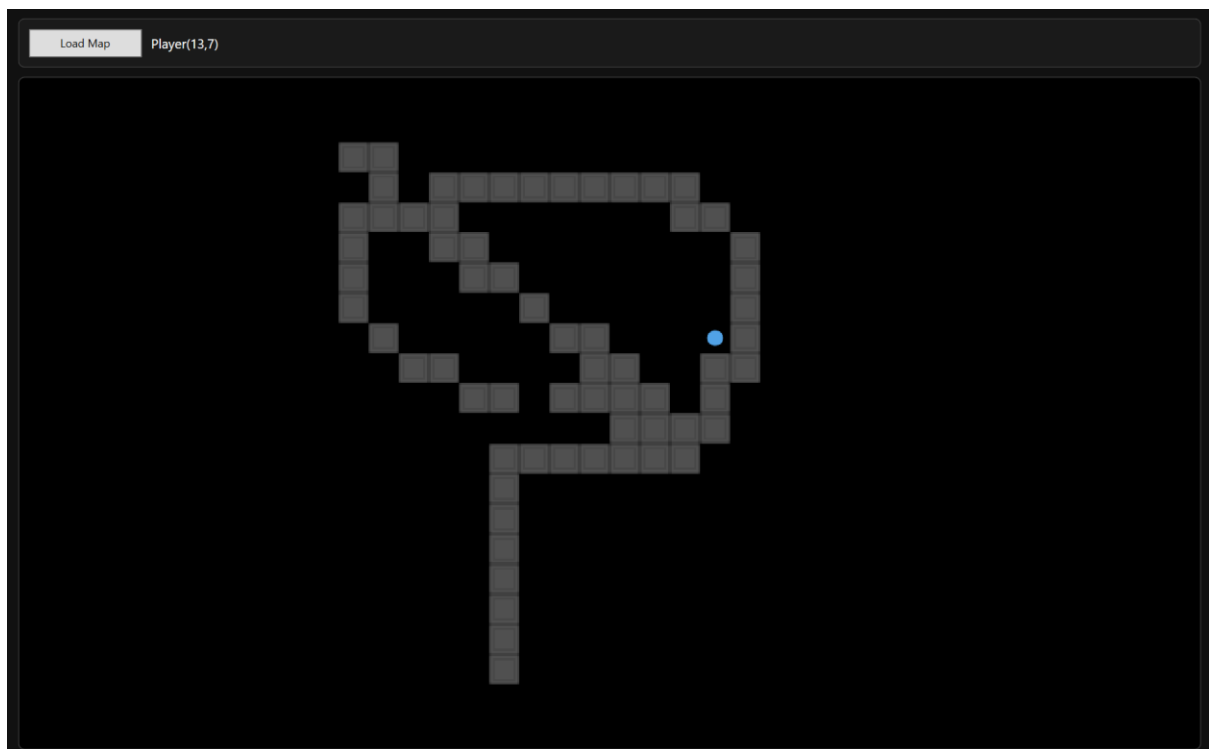
Demonstrates viewport-only rendering where the visible region updates rather than full-map rendering

Figure T27.1 – Player Position Before Movement



[Testing Section\Testing Section Images\Figure T27.1.png](#)

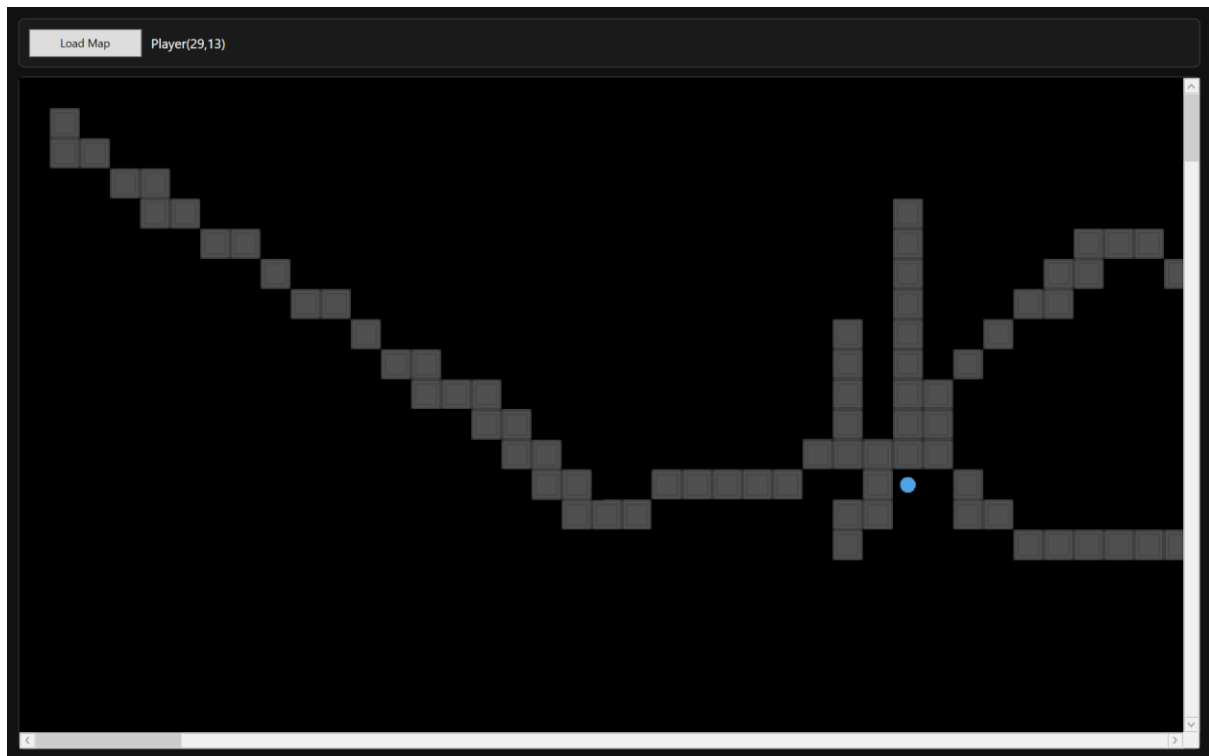
Figure T27.2 – Player Position After Movement (Viewport Updated Correctly)



[Testing Section\Testing Section Images\Figure T27.2.png](#)

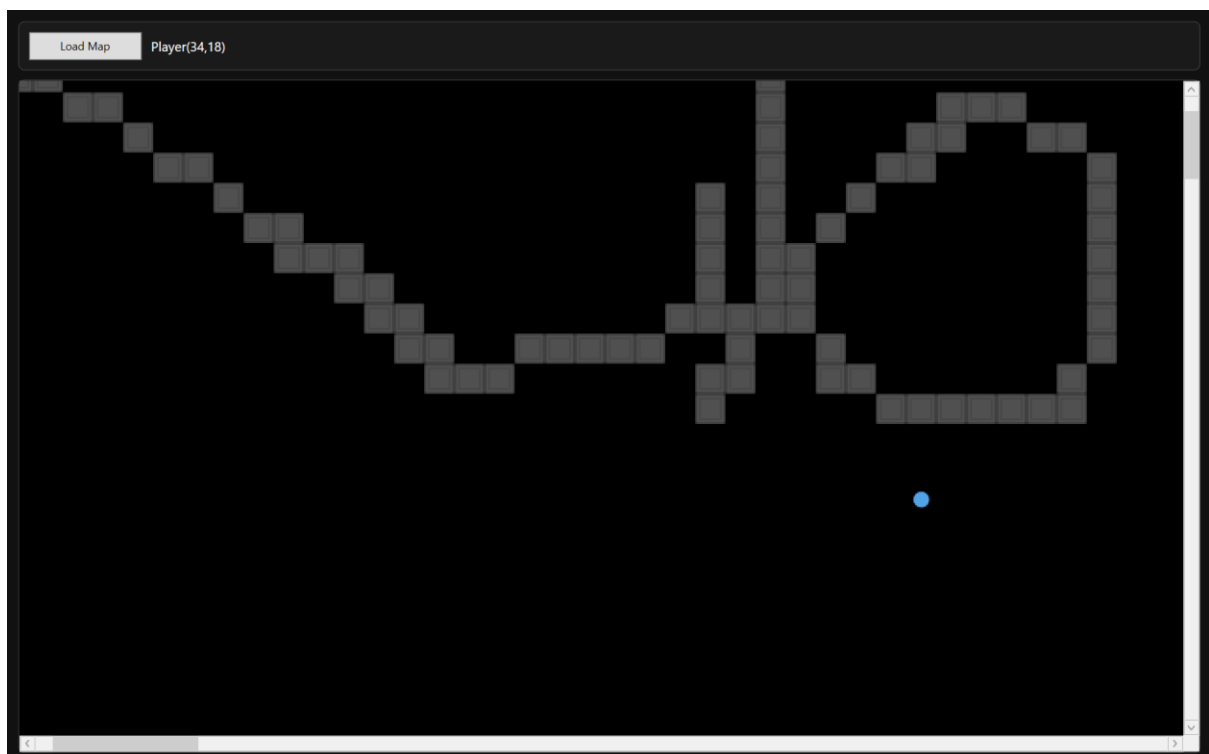
Demonstrates render refresh correctness per movement step

Figure T28.1 – Camera Before Deadzone Exit (Player Near Edge of Deadzone)



[Testing Section\Testing Section Images\Figure T28.1.png](#)

Figure T28.2 – Camera After Deadzone Exit (Viewport Shifted, Still Correct)



[Testing Section\Testing Section Images\Figure T28.2.png](#)

Demonstrates stable redraw during camera-driven viewport relocation

Figure T29.1 – Player at Map Edge (Viewport Clamped)



[Testing Section\Testing Section Images\Figure T29.1.png](#)

Figure T29.2 – Opposite Edge (Viewport Still Clamped Correctly)



[Testing Section\Testing Section Images\Figure T29.2.png](#)

Demonstrates correct clamping and safe region calculation at boundaries

2.7 Camera Deadzone

The camera subsystem uses a deadzone-based follow system rather than centering on the player every frame. This design prevents constant micro-adjustments and produces smoother visual motion. The camera only shifts when the player exits a defined deadzone region within the viewport.

Testing focused on validating:

- Deadzone boundaries behave as defined
- Camera does not move unnecessarily
- Camera shifts only when player crosses threshold
- Camera clamping prevents scrolling outside map bounds
- Player remains visible at all times

The following tests validate that:

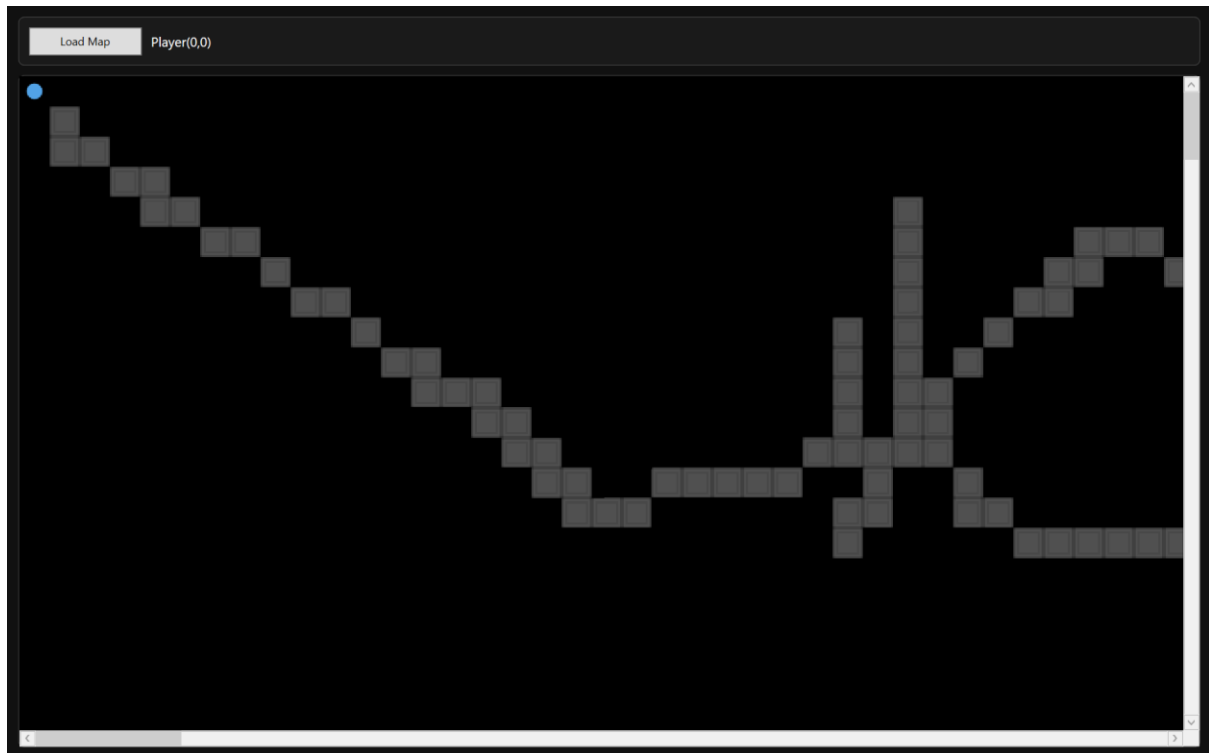
- The player can move within the deadzone without camera movement
- Exiting the deadzone triggers controlled camera shift
- Camera does not overshoot or jitter
- Camera remains clamped at map edges

Test Table 7 – Camera Deadzone

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
CD1	Verify no camera movement inside deadzone	Move player small distance within viewport centre	Camera remains static	Pass	Figures T30.1, T30.2
CD2	Verify camera shift on deadzone exit	Move player until crossing deadzone boundary	Camera shifts smoothly to re-centre region	Pass	Figures T31.1, T31.2
CD3	Verify no jitter during continuous movement	Hold movement key to repeatedly cross deadzone	Camera updates smoothly without flicker	Pass	Figures T32
CD4	Verify camera clamping at map boundaries	Move player to extreme edges of map	Camera does not scroll beyond map bounds	Pass	Figures T33.1, T33.2

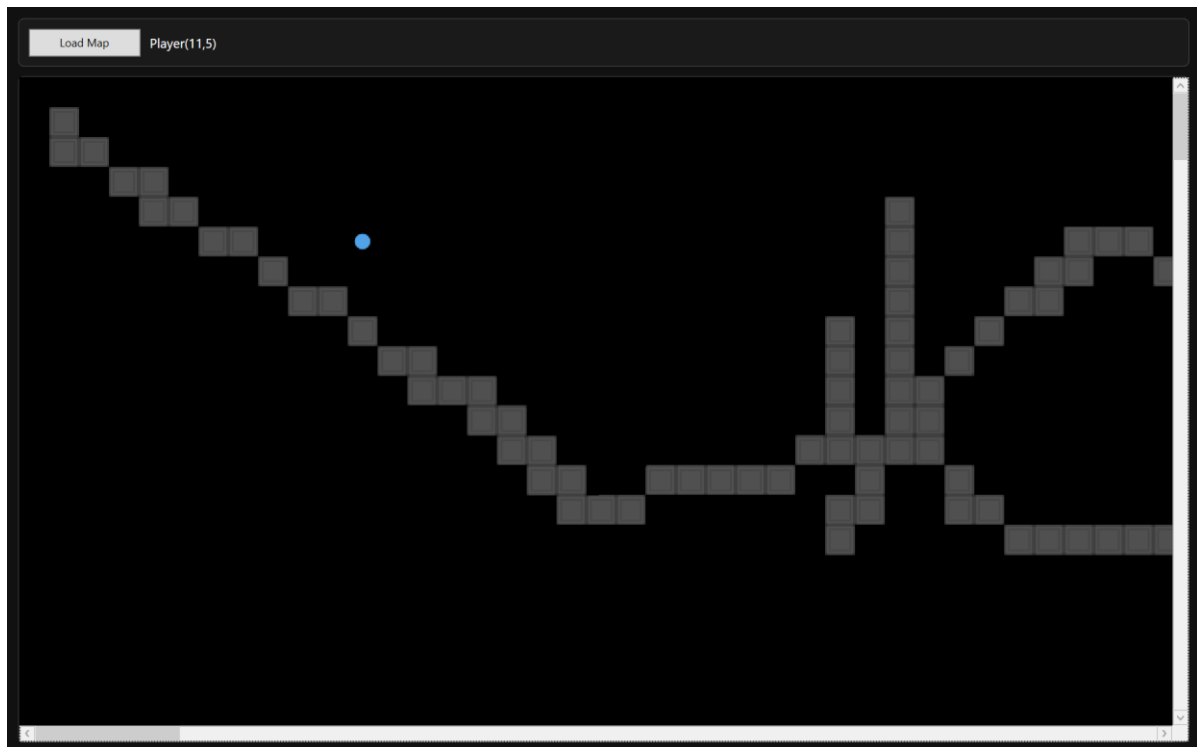
Test Table 7 – Screenshot Evidence

Figure T30.1 – Player Inside Deadzone (Initial Position)



[Testing Section\Testing Section Images\Figure T30.1.png](#)

Figure T30.2 – Player Moved Slightly (Camera Unchanged)



[Testing Section\Testing Section Images\Figure T30.2.png](#)

Demonstrates that camera does not respond to minor player movement within deadzone

Figure T31.1 – Player at Deadzone Boundary (Before Camera Shift)



[Testing Section\Testing Section Images\Figure T31.1.png](#)

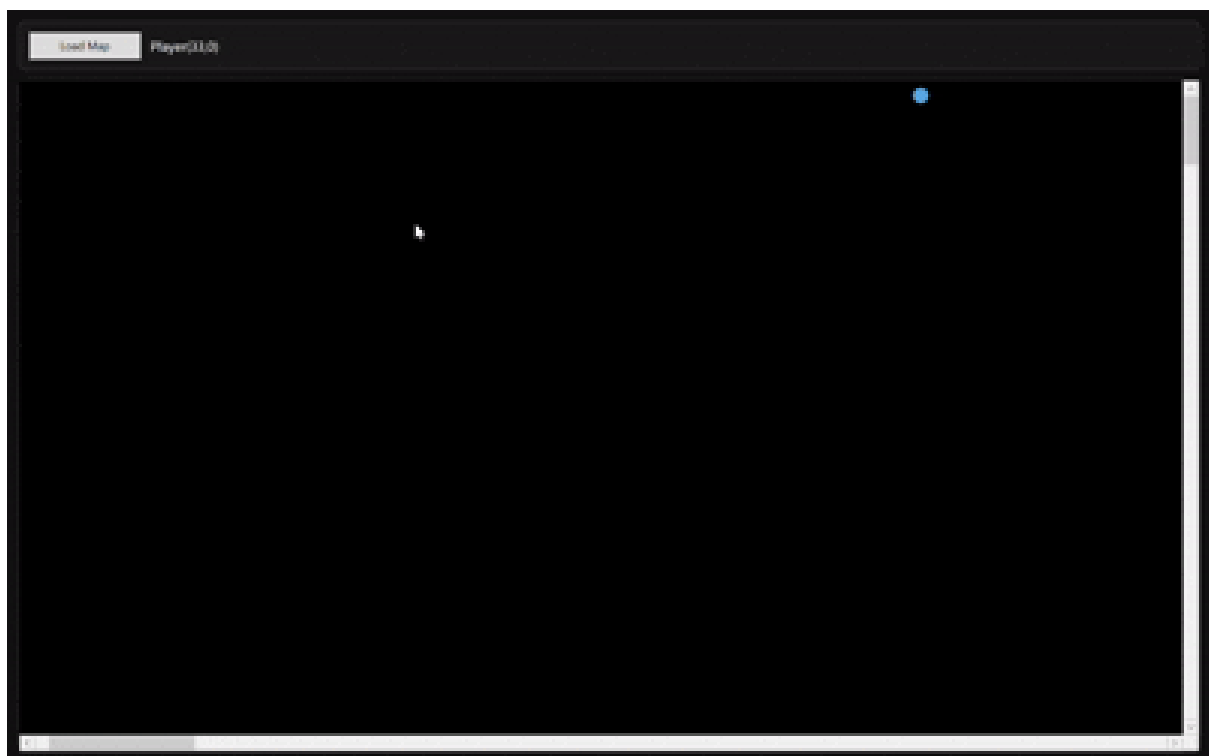
Figure T31.2 – Camera Shift After Crossing Boundary



[Testing Section\Testing Section Images\Figure T31.2.png](#)

Demonstrates threshold-based camera activation logic

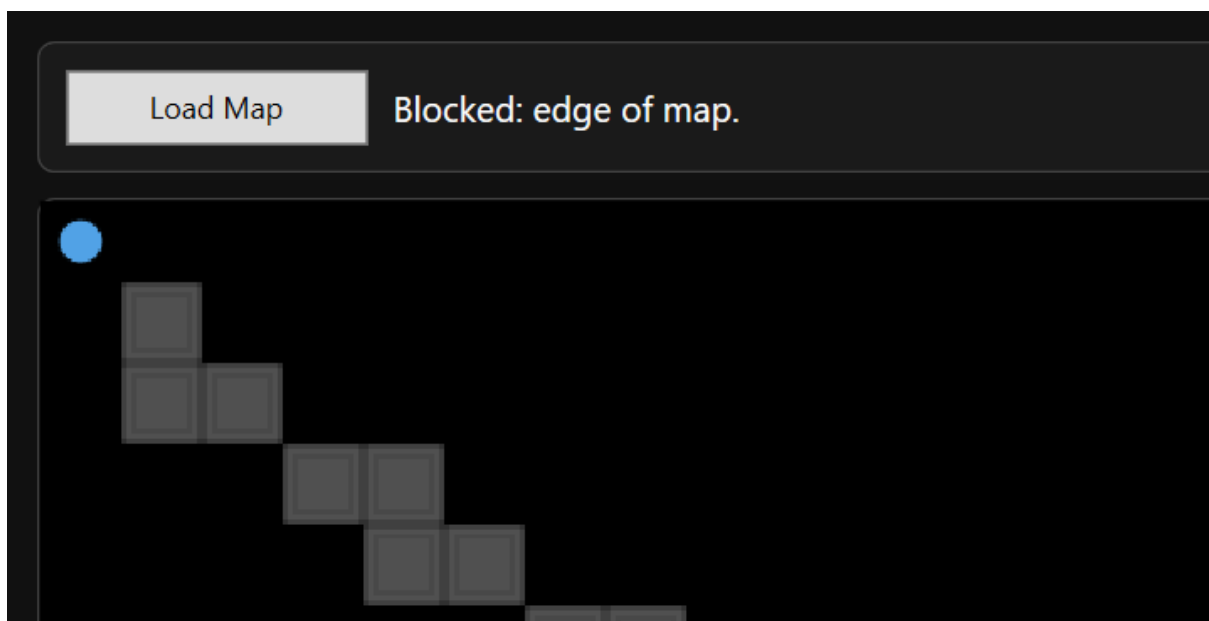
Figure T32 – GIF Showing Continuous Movement



[Testing Section\Testing Section GIFs\Figure T32.gif](#)

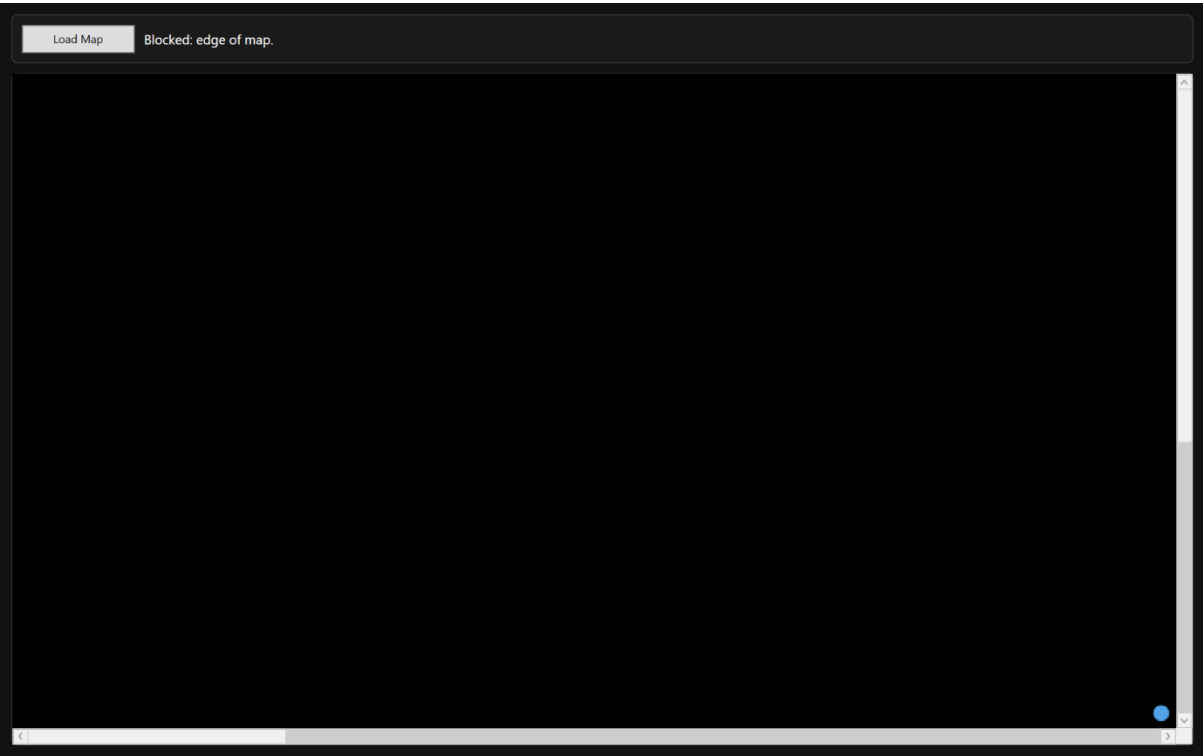
Demonstrates stable camera update logic across repeated shifts

Figure T33.1 – Camera Clamped at Top/Left Edge



[Testing Section\Testing Section Images\Figure T33.1.png](#)

Figure T33.2 – Camera Clamped at Bottom/Right Edge



[Testing Section\Testing Section Images\Figure T33.2.png](#)

Demonstrates correct clamping constraints

3 Boundary and Abnormal Data Testing

3.1 Map Dimension Boundary Testing

This test is used to verify that extreme or invalid map dimensions do not cause runtime crashes, invalid array allocation, or graphical instability. It evaluates:

- Minimum acceptable dimensions
- Zero or negative values
- Large grid stress handling

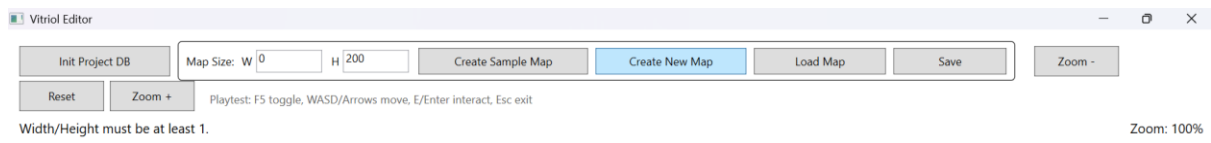
Test Table 1 – Map Dimension Boundary Testing

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
BDM1	Attempt to create map with zero width	Width = 0	Creation blocked	Pass	Figures T34
BDM2	Attempt to create map with negative height	Height = -5	Creation blocked	Pass	Figures T35
BDM3	Create large map (e.g. 200x200)	200x200 grid	Map rendered correctly	Pass	Figures T36.1, T36.2

BDM4	Load large map into runtime	200x200	Smooth viewport render	Pass	Figures T37
-------------	-----------------------------	---------	------------------------	------	-------------

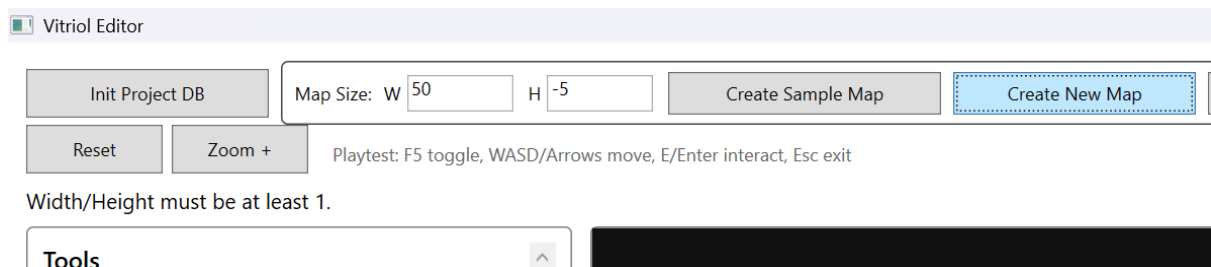
Test Table 1 – Screenshot Evidence

Figure T34 – Zero Width Validation



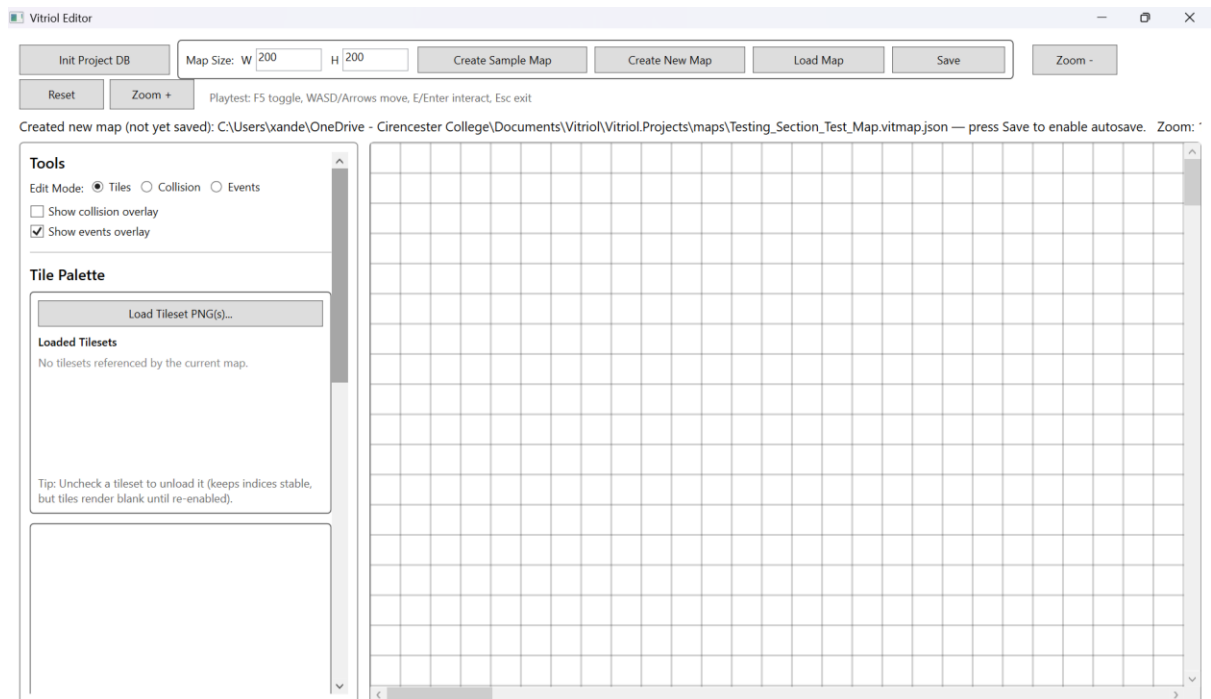
Testing Section\Testing Section Images\Figure T34.png

Figure T35 – Negative Height Validation



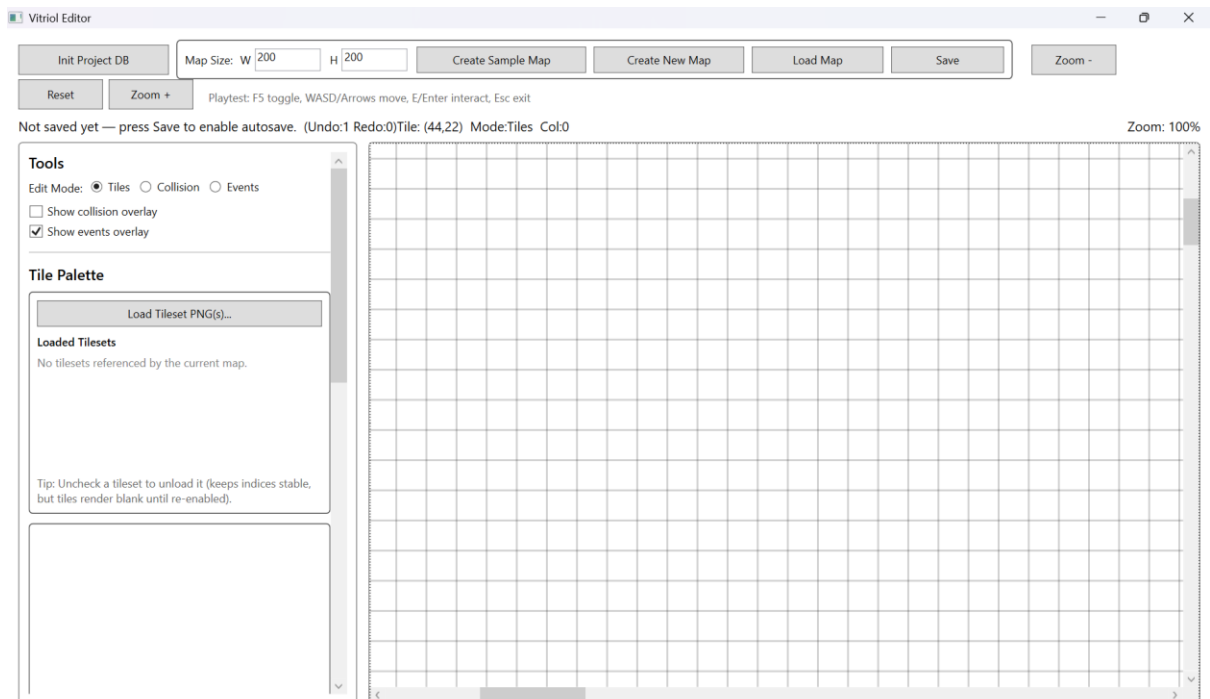
Testing Section\Testing Section Images\Figure T35.png

Figure T36.1 – Large Map Creation (200x200)



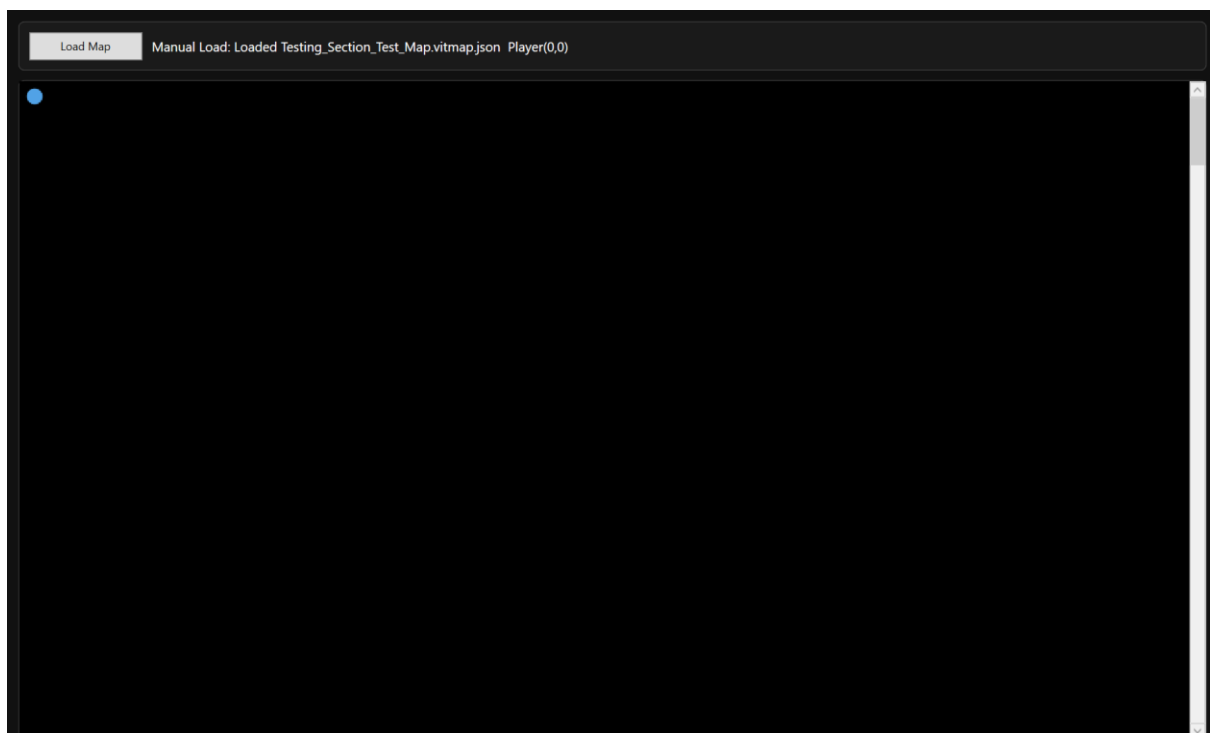
Testing Section\Testing Section Images\Figure T36.1.png

Figure T36.2 – Large Map Viewport Navigation



[Testing Section\Testing Section Images\Figure T36.2.png](#)

Figure T37 – Runtime Rendering of Large Map



[Testing Section\Testing Section Images\Figure T37.png](#)

What These Tests Demonstrate:

- Defensive validation against invalid dimension input
- Correct allocation of 1D tile array for large datasets

- Viewport-only rendering prevents performance degradation
- No array index overflow occurs

3.2 Tileset Boundary and Missing Asset Handling

Tilesets represent a critical external dependency within the system. Since maps reference PNG assets stored within the project architecture, it is essential that:

- Missing tileset files are detected before runtime execution
- Invalid relative paths do not cause silent corruption
- Oversized or malformed tilesets do not crash rendering
- Tile indexing remains stable when tilesets are unavailable

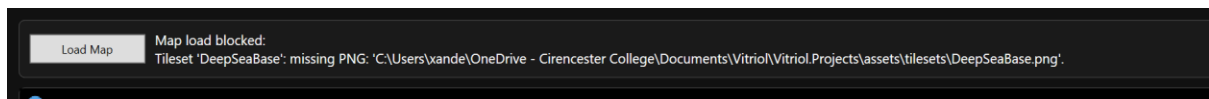
This subsection validates defensive asset handling and ensures that map integrity is not compromised by missing or invalid graphical resources

Test Table 2 – Tileset Boundary & Missing Asset Handling

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
BTS1	Load map referencing missing PNG	Tileset file manually removed from assets folder	Map load blocked with clear error message	Pass	Figures T38
BTS2	Modify JSON to use invalid relative path	ImagePath altered to non-existent path	Runtime validation detects missing file	Pass	Figures T39
BTS3	Load map with valid but large tileset image	High resolution PNG tileset	Tileset loads and crops directly	Pass	Figures T40.1, T40.2
BTS4	Toggle tileset load state	Unload/reload reference	Tile indices remain consistent	Pass	Figures T41.1

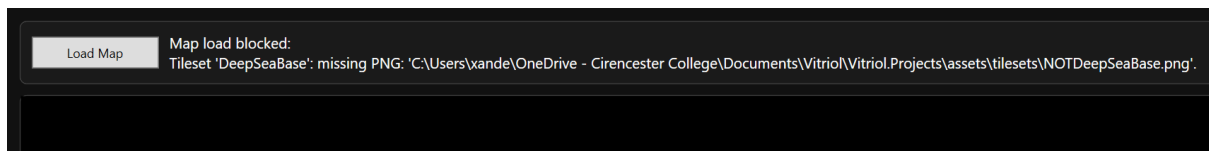
Test Table 2 – Screenshot Evidence

Figure T38 – Missing PNG Validation



[Testing Section\Testing Section Images\Figure T38.png](#)

Figure T39 – Invalid Relative Path Detection



[Testing Section\Testing Section Images\Figure T39.png](#)

Figure T40.1 – Large Tileset Loaded Successfully

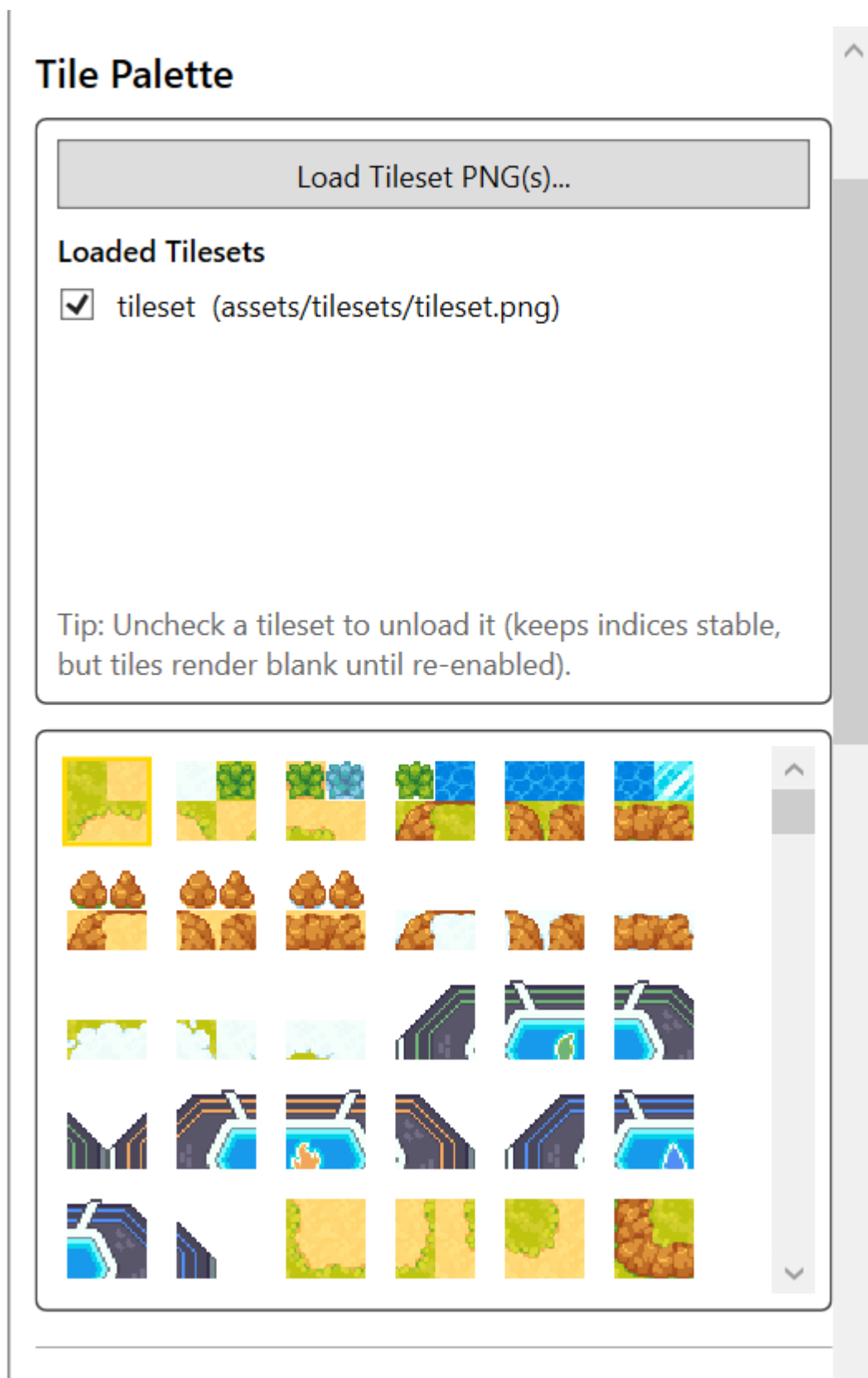
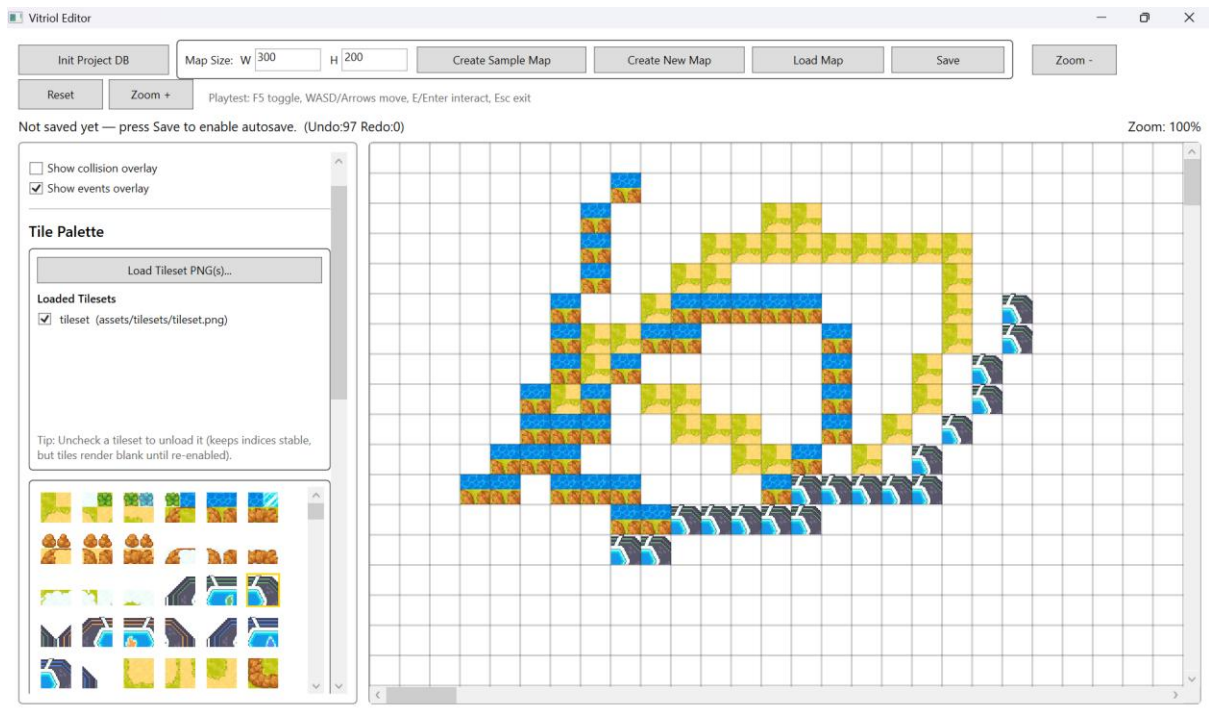
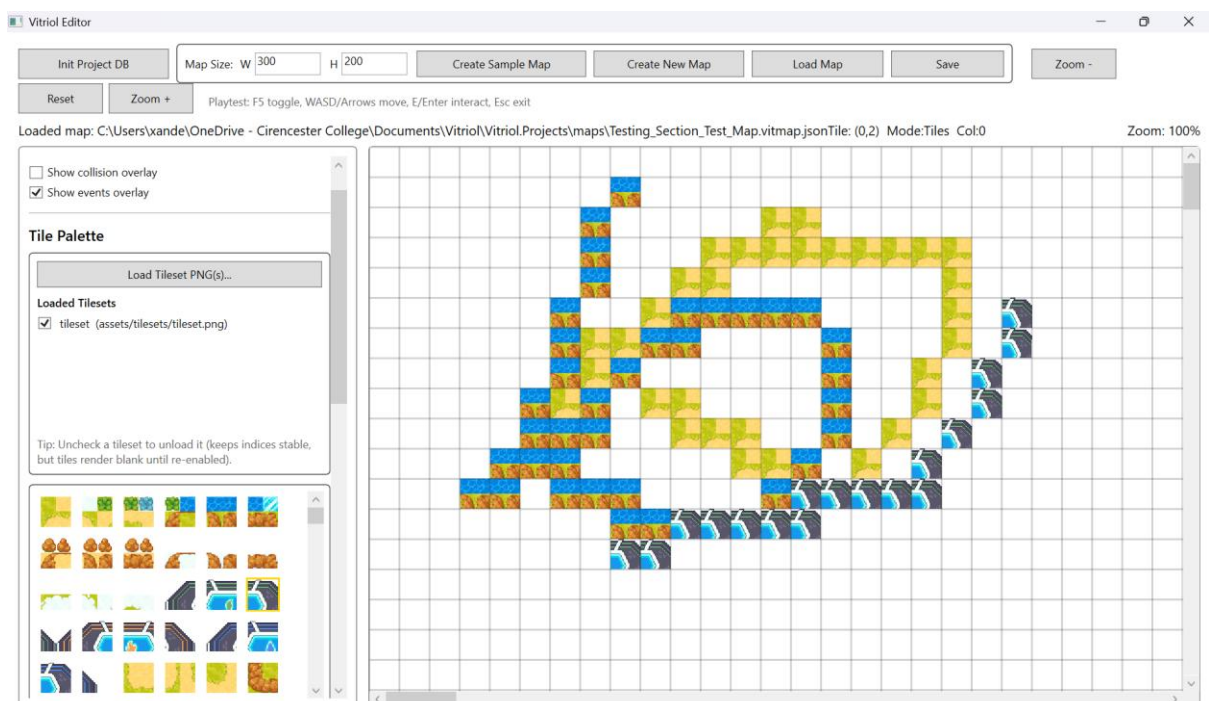


Figure T40.2 – Cropped Rendering From Large Tilesheet



Testing Section\Testing Section Images\Figure T40.2.png

Figure T41 – Tilesheet Reference Stability



Testing Section\Testing Section Images\Figure T41.png

What These Tests Demonstrate:

- Strict enforcement of asset existence before runtime execution
- Protection against broken relative paths

- Safe cropping logic under high-resolution assets
- Stability of global tile indexing system
- Prevention of silent tile corruption

3.3 Event Parameter and Command Validation

Events in Vitriol are data-driven through MapEventDto and EventCommandDto, which means behaviour is controlled entirely by JSON-defined command structures. While this allows extensibility, it also introduces risk if:

- Commands are malformed
- Required arguments are missing
- Unknown command identifiers are used
- Parameter dictionaries contain invalid keys

This subsection verifies that:

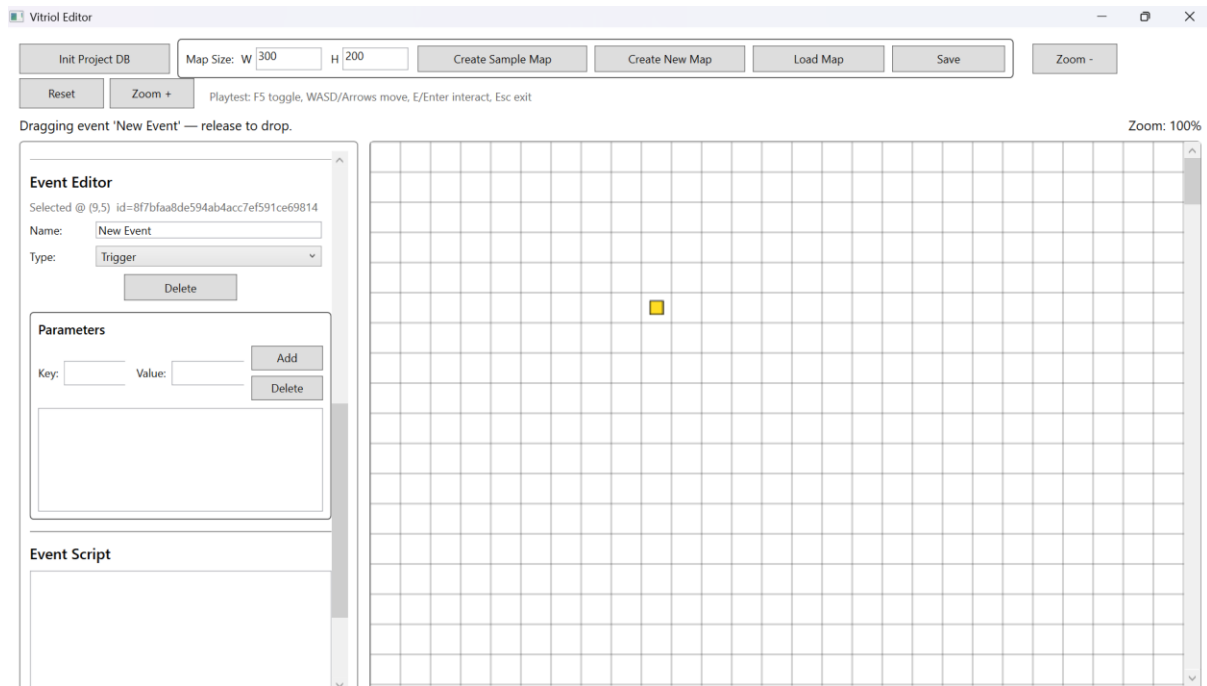
- Invalid command data does not crash the runtime
- Missing arguments are handled defensively
- Unknown commands are reported clearly
- Event execution remains stable even with malformed input

Test Table 3 – Event Parameter & Command Validation

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
BDE1	Execute event with empty command list	Event with no commands	Runtime displays safe message	Pass	Figures T42
BDE2	ShowText command missing “text” argument	Remove text parameter in JSON	Runtime handles gracefully without crash	Pass	Figures T43.1, T43.2
BDE3	Unknown command type	Change command to invalid string	Unknown Command Does Not Execute	Pass	Figures T44

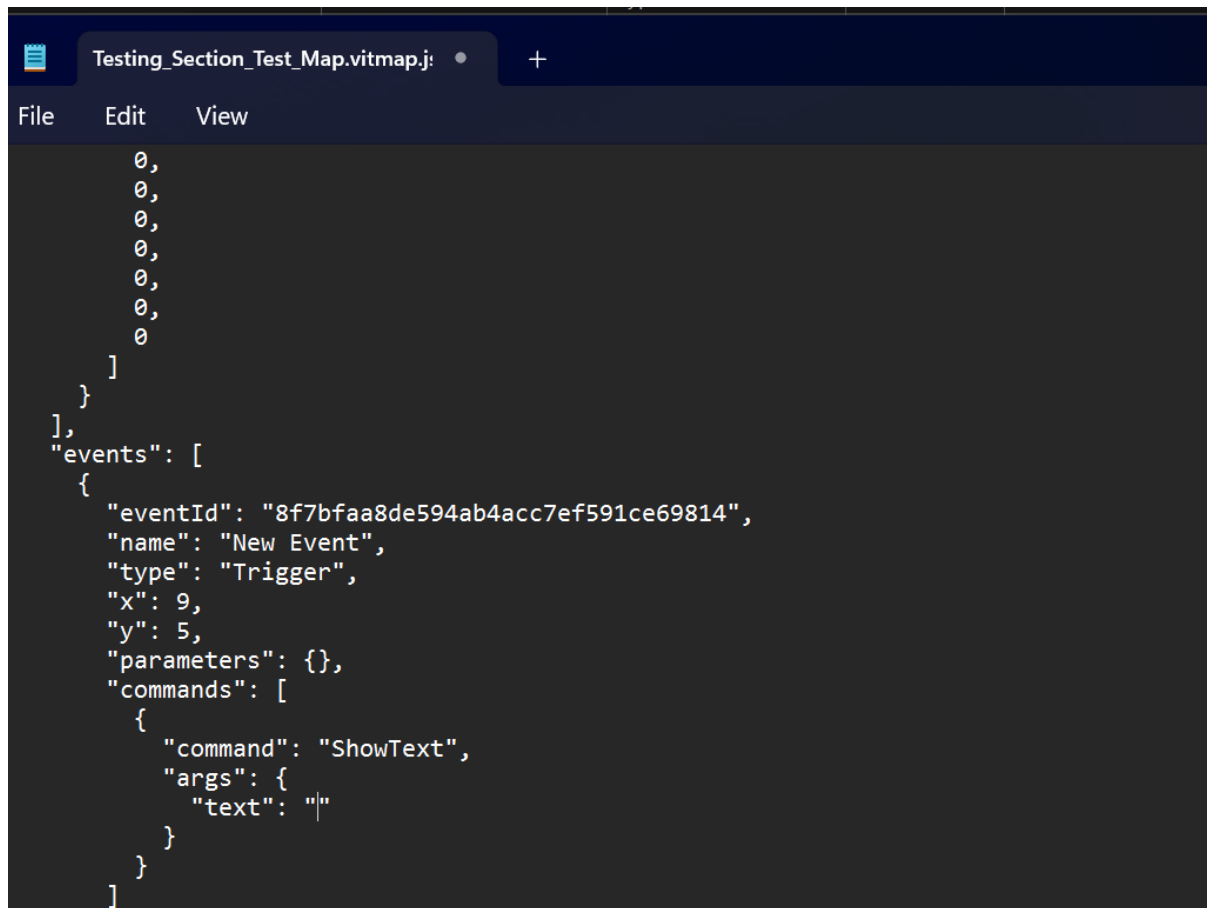
Test Table 3 – Screenshot Evidence

Figure T42 – Empty Command List Execution



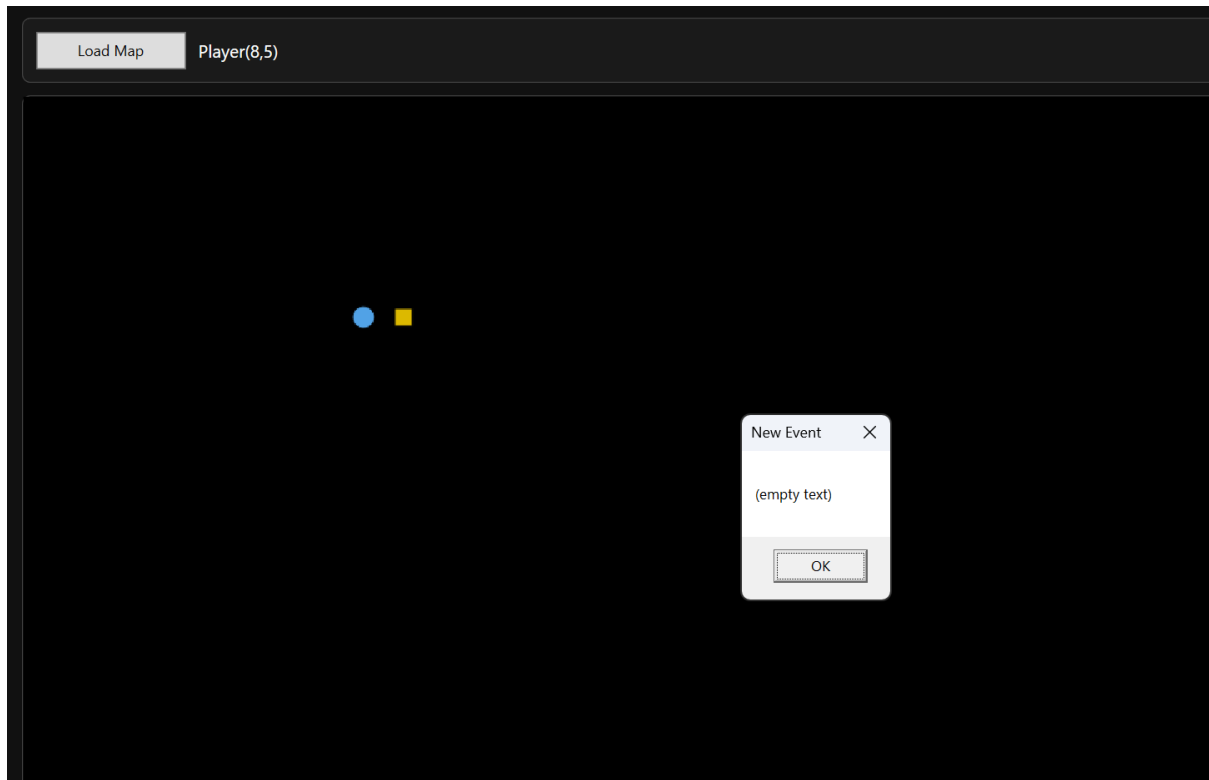
[Testing Section\Testing Section Images\Figure T42.png](#)

Figure T43.1 – Modified JSON Missing Text Parameter



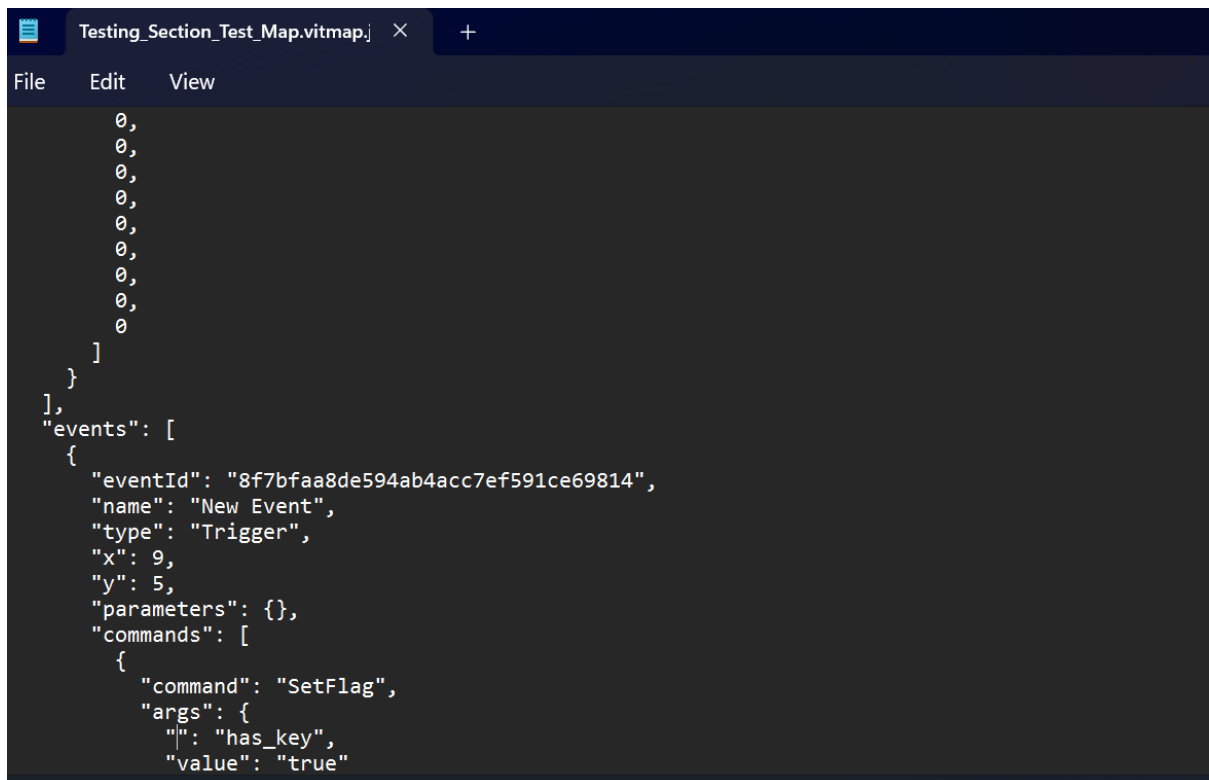
[Testing Section\Testing Section Images\Figure T43.1.png](#)

Figure T43.2 – Runtime Handling Missing Text



[Testing Section\Testing Section Images\Figure T43.2.png](#)

Figure T44 – Unknown Command Handling (Event Does Not Execute)



[Testing Section\Testing Section Images\Figure T44.1.png](#)

What These Tests Demonstrate:

- Defensive parsing of command dictionaries
- No runtime crashes from malformed event structures
- Explicit error messaging for missing parameters
- Controlled handling of unknown command identifiers
- Stability of data-driven event system

3.4 JSON Corruption and Recovery Handling

Since maps are persisted as JSON documents, the integrity of the JSON structure is critical. Corruption may occur due to :

- Manual editing errors
- Incomplete writes
- Version mismatches
- Structural deletion of required properties

This subsection verifies that:

- Malformed JSON does not crash the runtime
- Missing required fields are handled safely
- Invalid format versions do not cause undefined behaviour
- File corruption results in controlled failure rather than silent instability

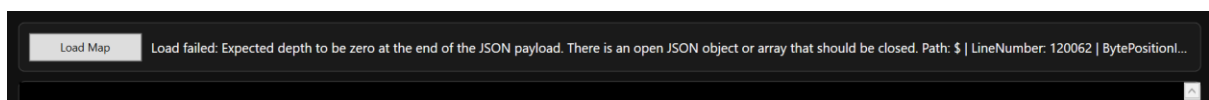
This validates the reliability of the MapSerialiser and defensive file loading logic

Test Table 4 – JSON Corruption & Recovery Handling

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
BDJ1	Remove closing brace from JSON	Delete final } in map file	Runtime fails safely with error message	Pass	Figures T45
BDJ2	Remove required property width	Delete “Width” field	Deserialization fails safely	Pass	Figures T46
BDJ3	Modify FormatVersion to invalid value	Change to unsupported number	Runtime handles without crash	Pass	Figures T47

Test Table 4 – Screenshot Evidence

Figure T45 – Malformed JSON (Missing Closing Brace)



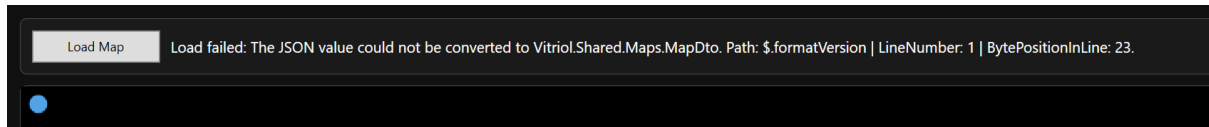
[Testing Section\Testing Section Images\Figure T45.png](#)

Figure T46 – Missing Required Property



[Testing Section\Testing Section Images\Figure T46.png](#)

Figure T47 – Invalid FormatVersion



[Testing Section\Testing Section Images\Figure T47.png](#)

What These Tests Demonstrate:

- Safe deserialization failure handling
- No unhandled exceptions during file parsing
- Strict structural requirements enforced
- Protection against partial or corrupted file writes
- Data integrity maintained across persistence layer

3.5 Runtime Movement Boundary Testing

Player movement operates directly on tile-indexed arrays. Any failure in boundary enforcement could result in:

- Array index out-of-range exceptions
- Clipping outside map bounds
- Ignoring collision constraints
- Camera instability

This subsection verifies that:

- Player movement is clamped within map dimensions
- Collision tiles correctly block movement
- No index overflow occurs at edges
- Camera deadzone logic does not break at boundaries

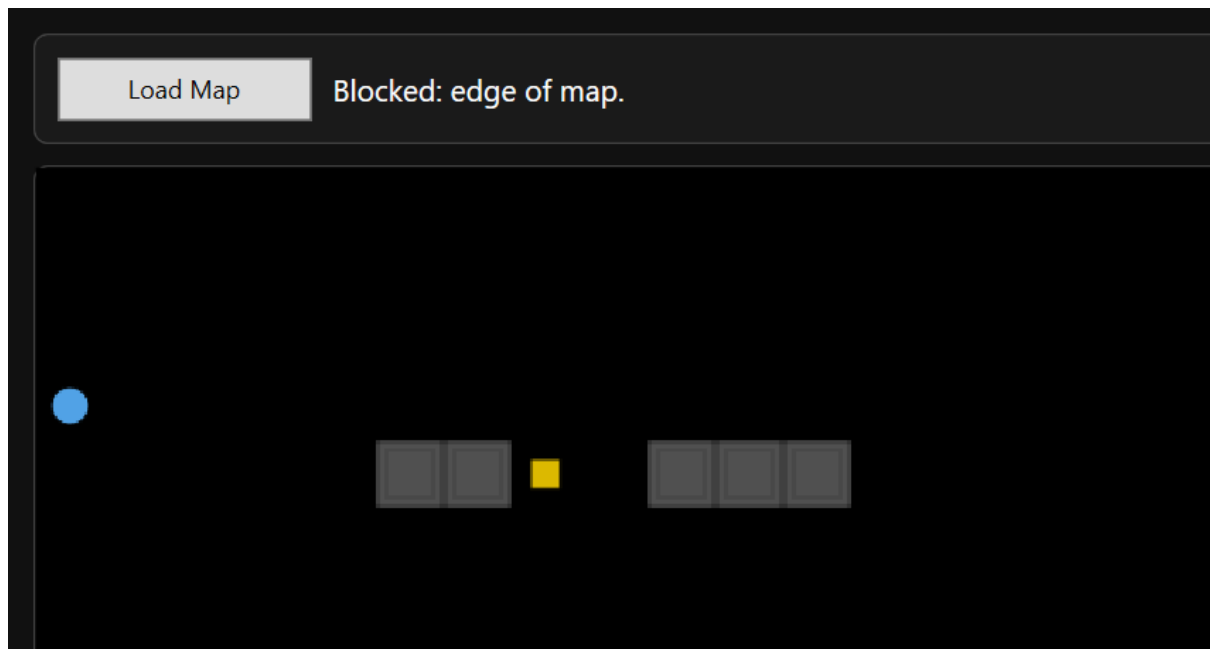
Test Table 5 – Runtime Movement Boundary Testing

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
BDR1	Attempt movement beyond left edge	Move player at x=0 left	Movement Blocked	Pass	Figures T48
BDR2	Attempt movement beyond top edge	Move player at y=0 up	Movement Blocked	Pass	Figures T49

BDR3	Attempt movement into collision tile	Walk into-collision marked tile	Collision Respected	Pass	Figures T50
BDR4	Camera behaviour at map edge	Move to far right/bottom edge	Camera Clamps Correctly	Pass	Figures T51

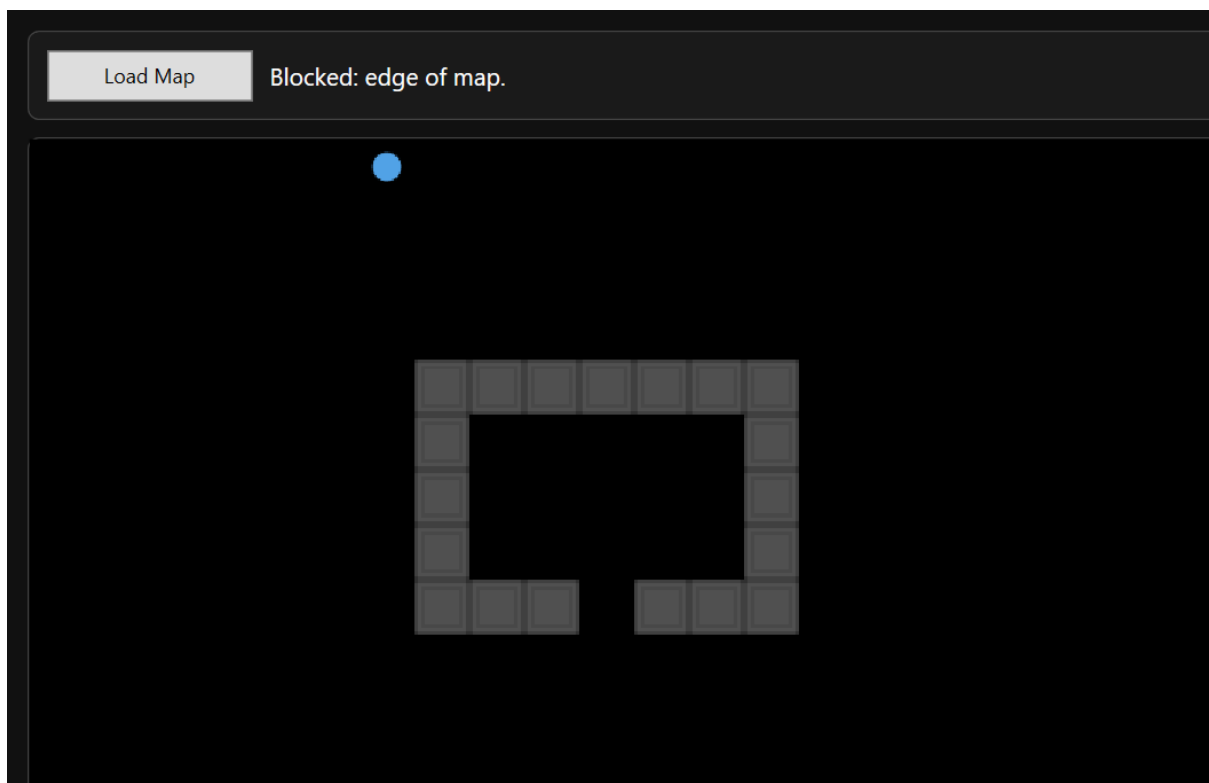
Test Table 5 – Screenshot Evidence

Figure T48 – Left Boundary Enforcement



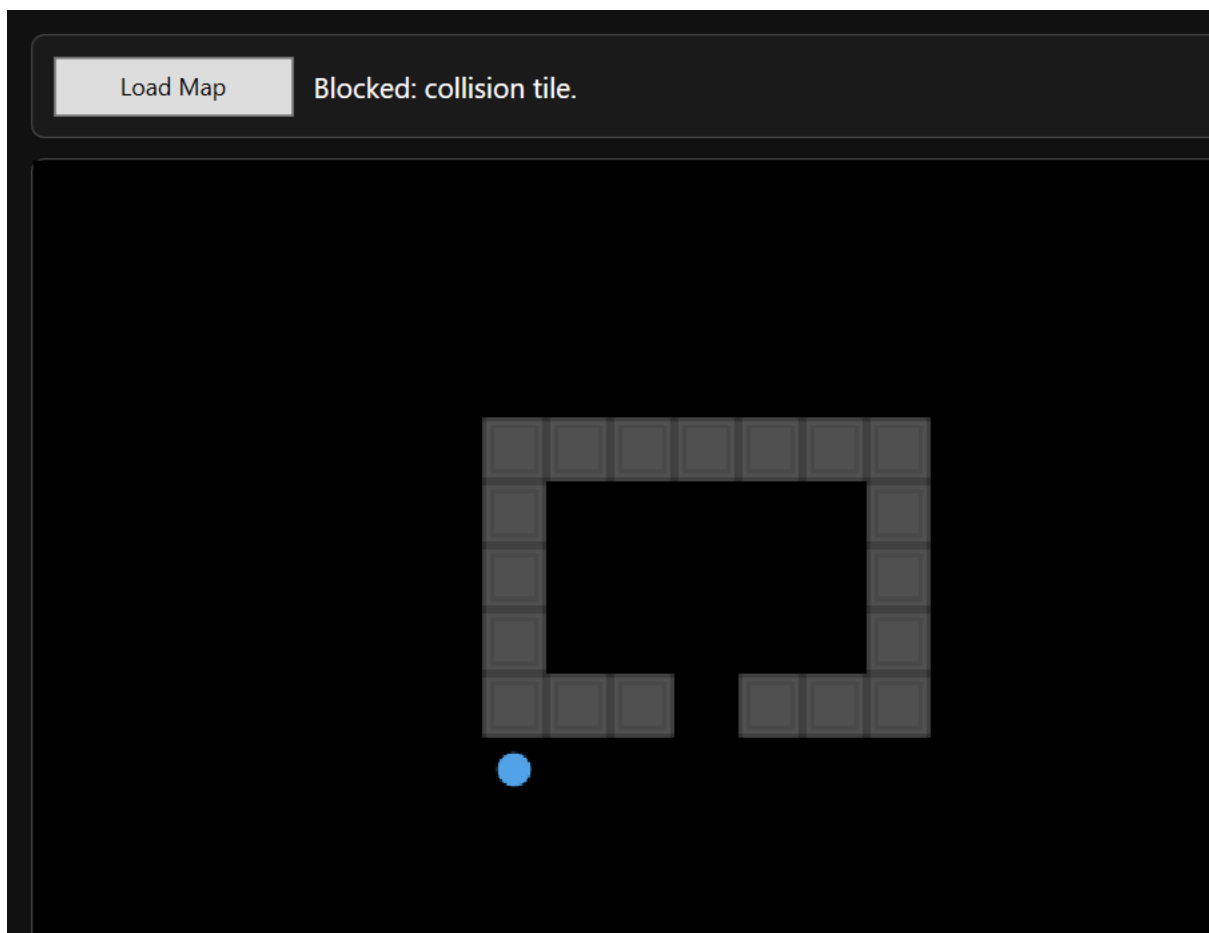
[Testing Section\Testing Section Images\Figure T48.png](#)

Figure T49 – Top Boundary Enforcement



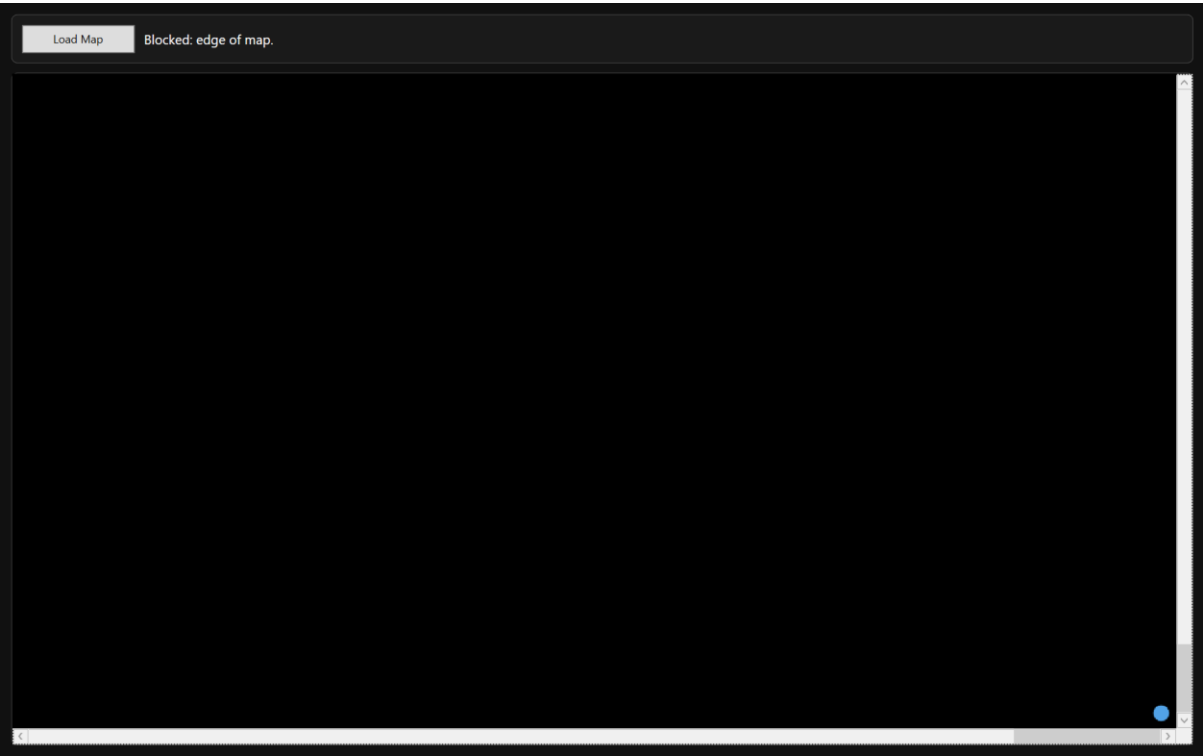
[Testing Section\Testing Section Images\Figure T49.png](#)

Figure T50 – Collision Tile Enforcement



Testing Section\Testing Section Images\Figure T50.png

Figure T51 – Camera Clamping at Edge



[Testing Section\Testing Section Images\Figure T51.png](#)

What These Tests Demonstrate:

- Safe array index clamping
- Collision layer correctly evaluated
- No runtime exceptions at map edges
- Camera deadzone behaviour remains stable at boundaries
- Movement system remains deterministic and robust

4 Performance Testing

4.1 Large Map Rendering Performance

These tests confirm that viewport-only rendering prevents performance degradation on large maps, and that rendering remains smooth during navigation.

Test Table 1 – Large Map Rendering Performance

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
PERFR1	Render 200x200 map in editor	Large grid with background tiles	Smooth viewport navigation	Pass	Figures T52
PERFR2	Continuous scrolling across large screen	Hold movement key	Rendering remains stable	Pass	Figures T53

PERFR3	Load large map in runtime	200x200 grid	Runtime renders only visible region	Pass	Figures T54
PERFR4	Rapid directional input	Alternate WASD quickly	No frame freezing or jitter	Pass	Figures T55

Test Table 1 – Screenshot Evidence

Figure T52 – GIF Showing Editor Large Map Navigation



[Testing Section\Testing Section GIFs\Figure T52.gif](#)

Figure T53 – GIF Showing Continuous Movement Stress



[Testing Section\Testing Section GIFs\Figure T53.gif](#)

Figure T54 – Runtime Large Map Render (200x200)



[Testing Section\Testing Section Images\Figure T54.png](#)

Figure T55 – GIF Showing Rapid Input Responsiveness



[Testing Section\Testing Section GIFs\Figure T55.gif](#)

What These Tests Demonstrate:

- Rendering workload remains proportional to viewport size
- Tile cropping and caching prevent redundant redraw
- No exponential performance cost as map dimensions increase
- Input handling remains responsive under continuous rendering

4.2 Rendering Stability Under Continuous Movement

While large map rendering validates immediate responsiveness, it is equally important to ensure that performance remains stable over time. Extended runtime sessions may expose:

- Memory leaks in bitmap caching
- Gradual rendering slowdown
- Accumulated camera offset errors
- Resource exhaustion under repeated movement

Given that the system uses:

- Frozen BitmapSource objects
- Cached CroppedBitmap instances
- Debounced render calls
- Single RenderTargetBitmap per frame

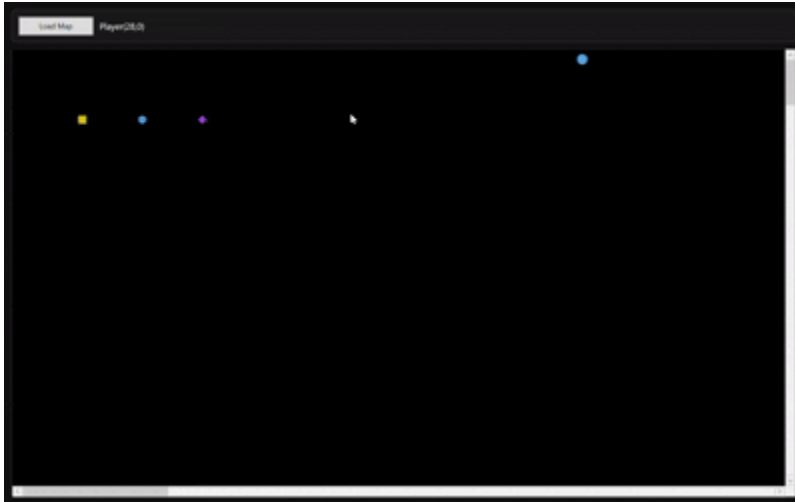
This subsection verifies that long-duration usage does not degrade responsiveness or introduce instability.

Test Table 2 – Rendering Stability Under Continuous Movement

Test ID	Purpose of Test	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
PERFS1	Continuous movement for 20+ seconds	Hold movement keys	No frame degradation over time	Pass	Figures T56
PERFS2	Repeated directional change	Rapid alternating movement	No camera jitter or offset drift	Pass	Figures T57
PERFS3	Repeated viewport scrolling (editor)	Continuous scroll wheel use	No UI freeze or lag	Pass	Figures T58
PERFS4	Multiple map reload cycles	Load map repeatedly	No progressive slowdown	Pass	Figures T59

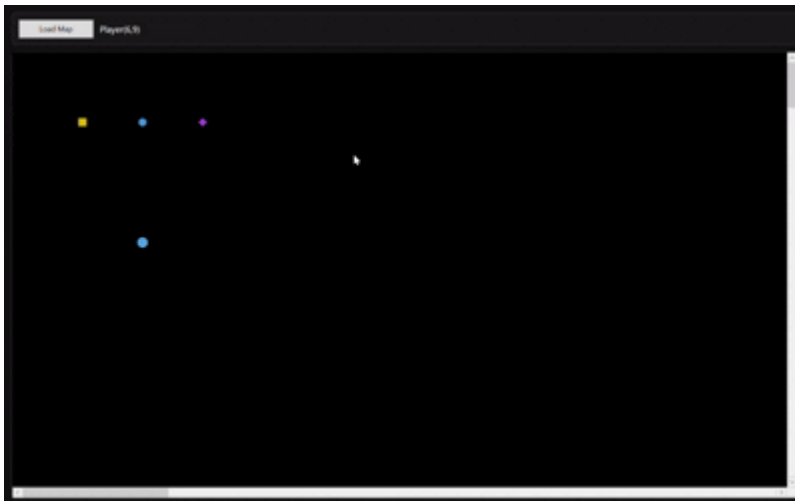
Test Table 2 – Screenshot Evidence

Figure T56 – GIF Showing Extended Movement Stability



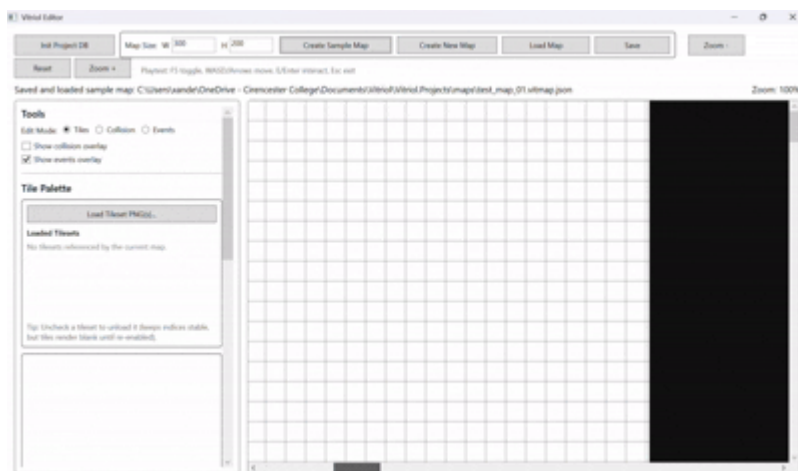
[Testing Section\Testing Section GIFs\Figure T56.gif](#)

Figure T57 – GIF Showing Camera Stability During Rapid Direction Change



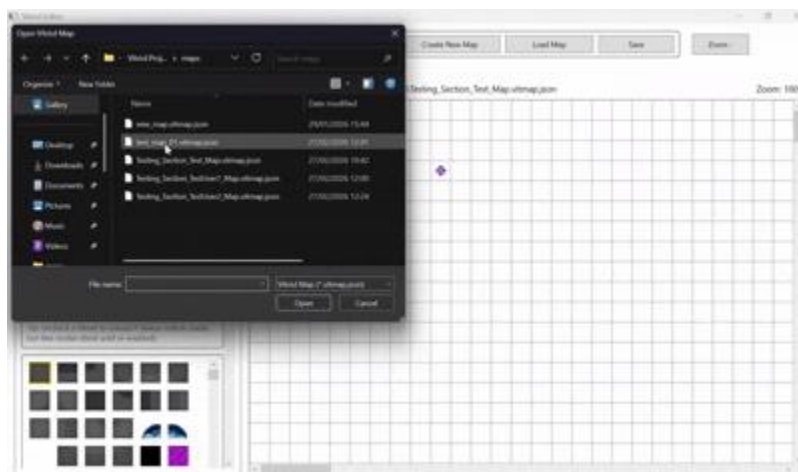
[Testing Section\Testing Section GIFs\Figure T57.gif](#)

Figure T58 – GIF Showing Editor Scroll Stability



[Testing Section\Testing Section GIFs\Figure T58.gif](#)

Figure T59 – GIF Showing Repeated Map Reload



[Testing Section\Testing Section GIFs\Figure T59.gif](#)

What These Tests Demonstrate:

- No observable memory leak from tile caching
- Camera deadzone remains stable over time
- Debounced rendering prevents excessive redraw calls
- Rendering remains proportional to viewport size
- Load operations do not degrade progressively

4.3 Memory Usage During Extended Editing Session

This subsection evaluates whether memory consumption increases progressively during extended editing or runtime use. Since the system caches:

- Tileset bitmaps
- Cropped tile regions
- RenderTargetBitmap frames

It is critical to ensure:

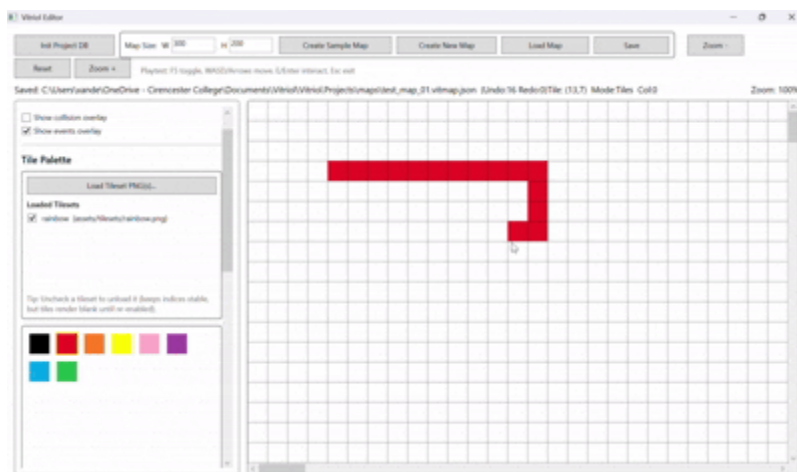
- No uncontrolled memory growth
- No cumulative bitmap duplication
- No progressive slowdown after repeated render cycles

Test Table 3 – Memory Usage During Extended Editing Session

Test ID	Description	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
PERFM1	Continuous tile painting in editor	Rapid paint actions for 20+ seconds	No UI slowdown	Pass	Figures T60
PERFM2	Repeated viewport renders	Scroll large map repeatedly	No progressive lag	Pass	Figures T61
PERFM3	Repeated runtime movement	Continuous WASD movement	No increasing delay	Pass	Figures T62

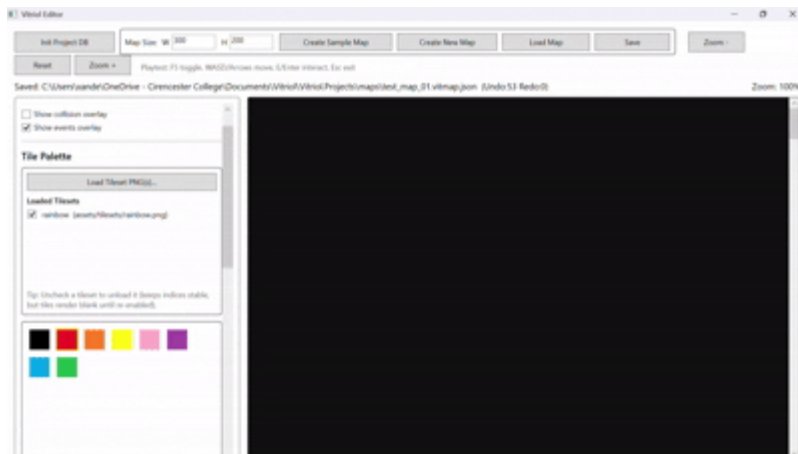
Test Table 3 – Screenshot Evidence

Figure T60 – GIF Showing Continuous Tile Painting (Editor Memory Stability)



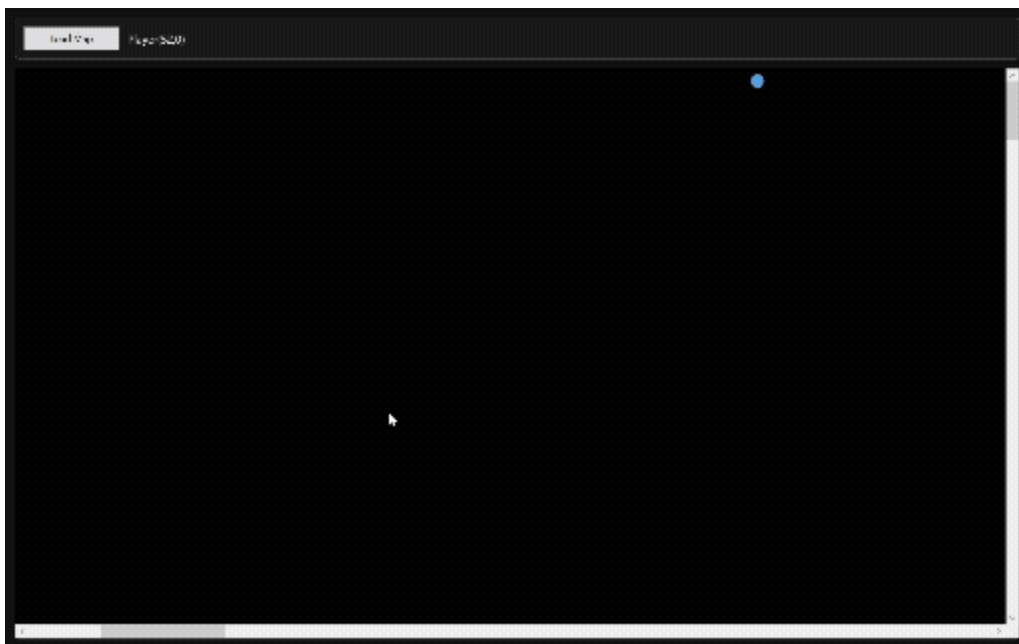
Testing Section\Testing Section GIFs\Figure T60.gif

Figure T61 – GIF Showing Repeated Viewport Rendering (Editor Scroll Stress)



[Testing Section\Testing Section GIFs\Figure T61.gif](#)

Figure T62 – GIF Showing Repeated Runtime Movement



[Testing Section\Testing Section GIFs\Figure T62.gif](#)

What These Tests Demonstrate:

- No progressive slowdown from repeated tile updates
- Viewport-only rendering remains stable over repeated calls
- No memory leak symptoms or input degradation

4.4 Asset Loading Performance

Asset loading performance evaluates the time required to:

- Load tileset PNG file
- Compile tile metadata
- Cache cropped bitmap segments

- Render initial viewport

Given that tilesets are loaded once and cached, subsequent map loads should not significantly degrade performance.

This subsection verifies that:

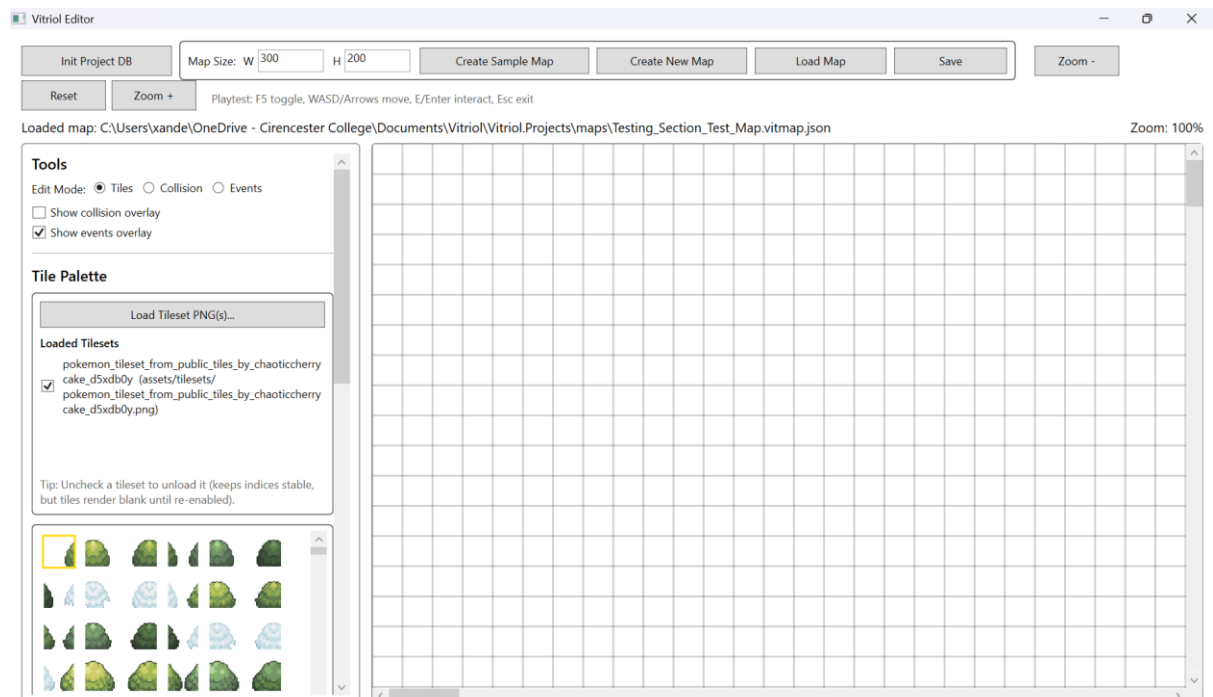
- First load is acceptable
- Subsequent loads are not progressively slower
- Large tilesets do not freeze the UI

Test Table 4 – Asset Loading Performance

Test ID	Description	Test Data	Expected Outcome	Actual Outcome	Screenshot ID
PERFA1	First load of large tileset	Input resolution PNG	Load completes without freezing	Pass	Figures T63
PERFA2	Reload same map	Cached tileset	Faster or equal load time	Pass	Figures T64
PERFA3	Switch between maps repeatedly	Alternate map loads	No UI Hang	Pass	Figures T65

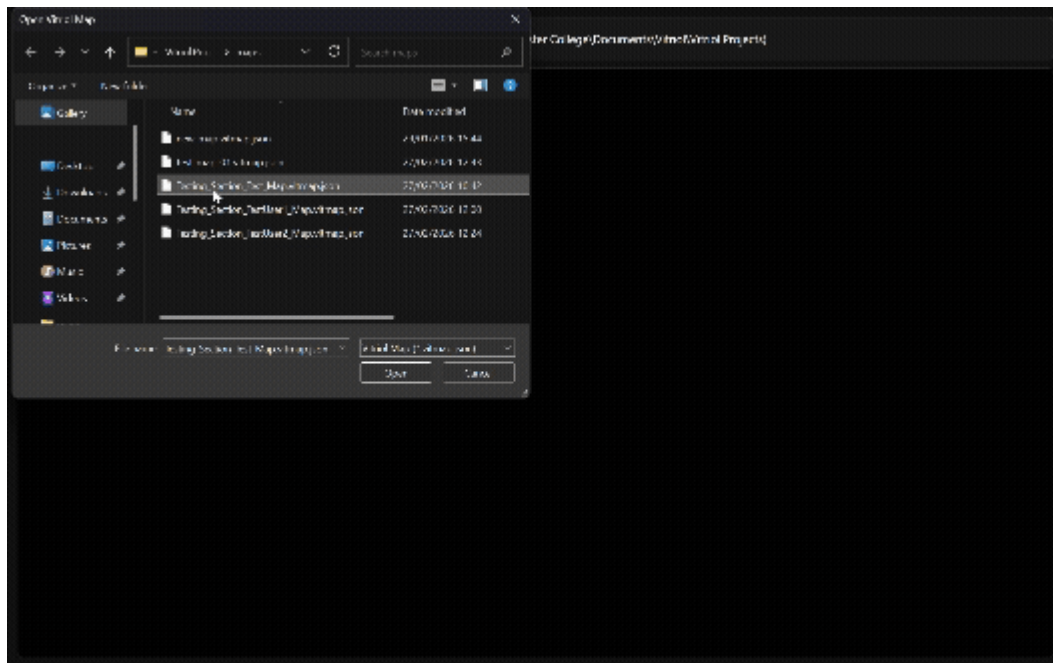
Test Table 4 – Screenshot Evidence

Figure T63 – First Load of Large Tileset



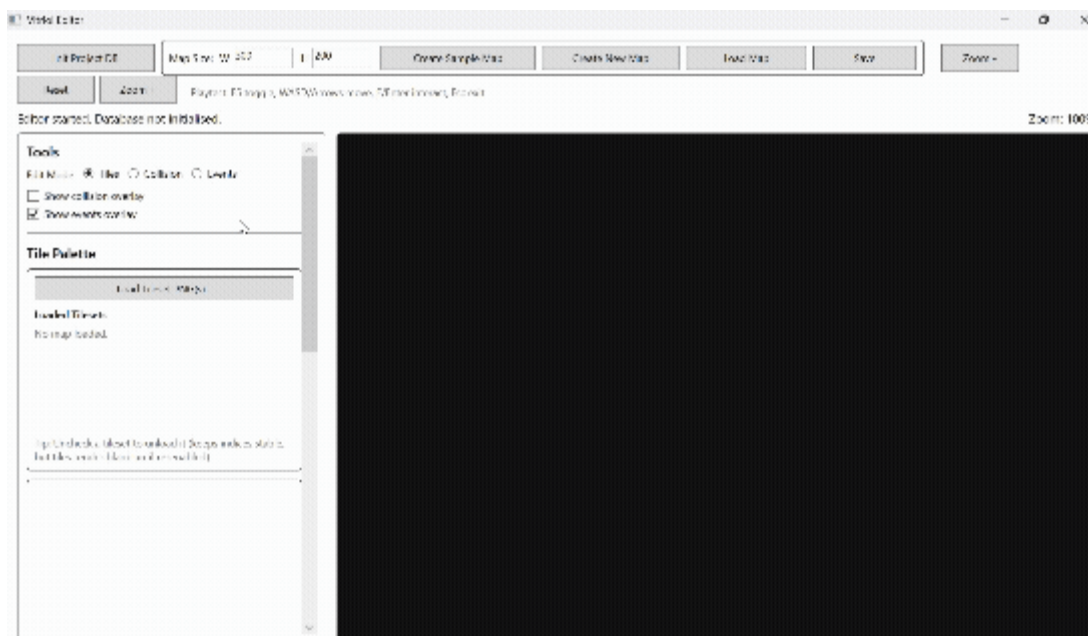
Testing Section\Testing Section Images\Figure T63.png

Figure T64 – GIF Showing Reload of Same Map (Cache Validation)



Testing Section\Testing Section GIFs\Figure T64.gif

Figure T65 – GIF Showing Switch Between Maps



Testing Section\Testing Section GIFs\Figure T65.gif

What These Tests Demonstrate:

- Initial PNG decoding and tileset compilation completes successfully
- No progressive delay, caching remains stable
- Repeated map loading does not accumulate performance degradation

5 Regression Testing

5.1 Map Creation Regression Checks

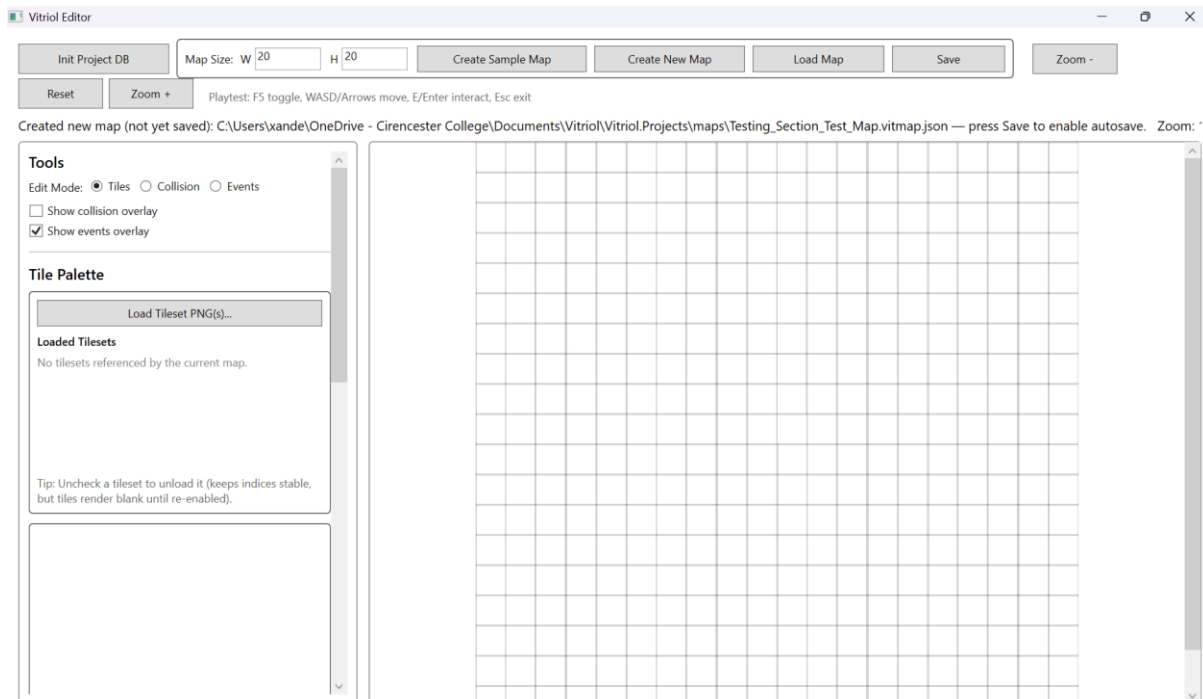
These checks confirm that later changes (tileset validation, autosave rules, viewport renderer integration) did not break initial map creation workflows. This includes creation of both sample maps and user-created maps, plus correct initial save state behaviour.

Test Table 1 – Map Creation Regression Checks

Test ID	Test Data	Expected Outcome	Actual Outcome	Status	Screenshot ID
REGMC1	Create new user map (blank grid)	Map created with correct dimensions and blank layers	Map created successfully	Pass	Figures T66
REGMC2	Manual save required before autosave enabled	Autosave inactive until first manual save	Save rule enforced correctly	Pass	Figures T67
REGMC3	Sample map creation autosaves by default	Autosave active immediately for sample map	Autosave active as expected	Pass	Figures T68
REGMC4	Reload newly created map file	Map reloads correctly with no missing data	Reload successful	Pass	Figures T69

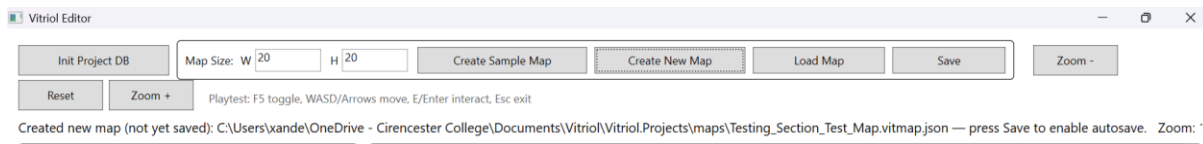
Test Table 1 – Screenshot Evidence

Figure T66 – New User Map Created (Blank Grid)



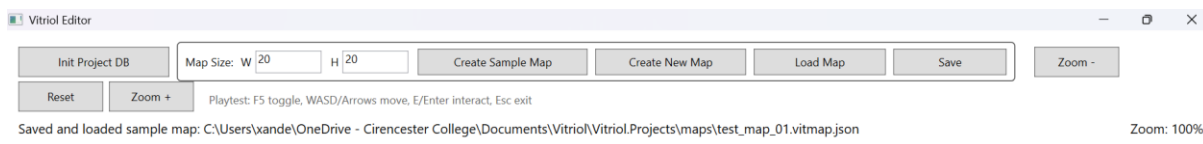
[Testing Section\Testing Section Images\Figure T66.png](#)

Figure T67 – Manual Save Gate Before Autosave



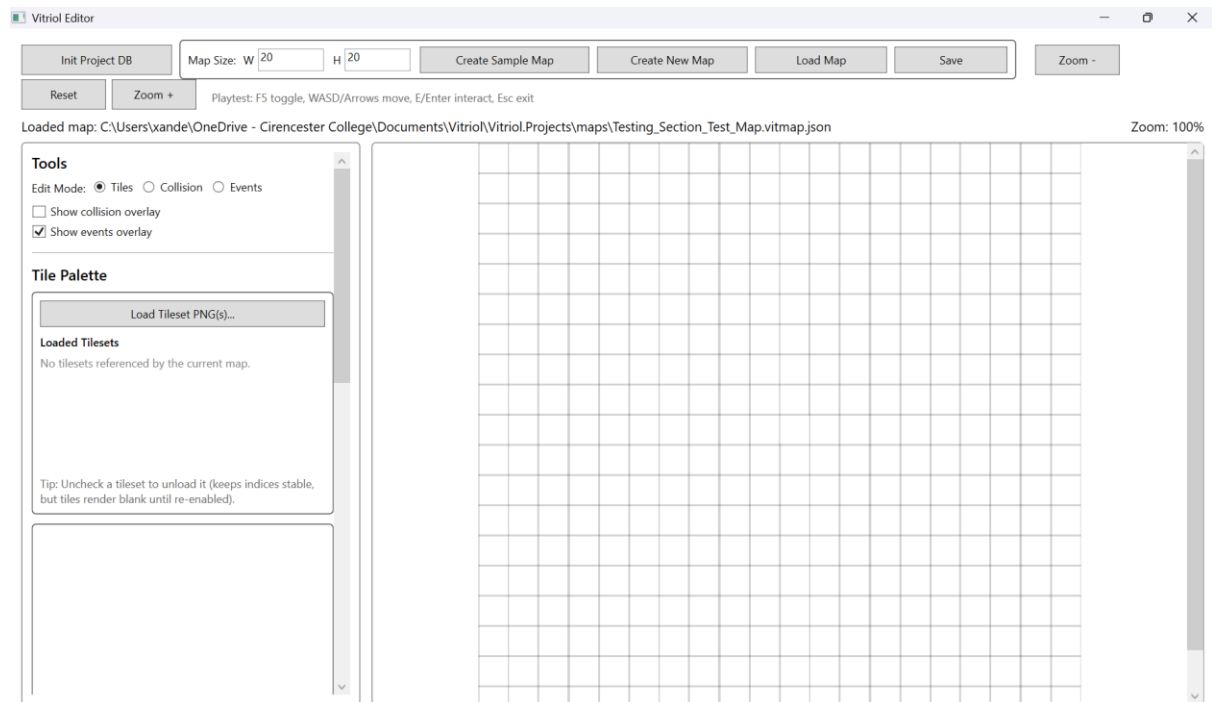
[Testing Section\Testing Section Images\Figure T67.png](#)

Figure T68 – Sample Map Autosave Active by Default



[Testing Section\Testing Section Images\Figure T68.png](#)

Figure T69 – Reload Newly Created Map



[Testing Section\Testing Section Images\Figure T69.png](#)

What These Tests Demonstrate:

- Correct initial map creation state and correct grid dimensions
- Autosave rule enforcement prior to first manual save
- Immediate autosave behaviour on sample map workflow
- Creation output produces a valid reloadable map file

5.2 Event System Regression Checks

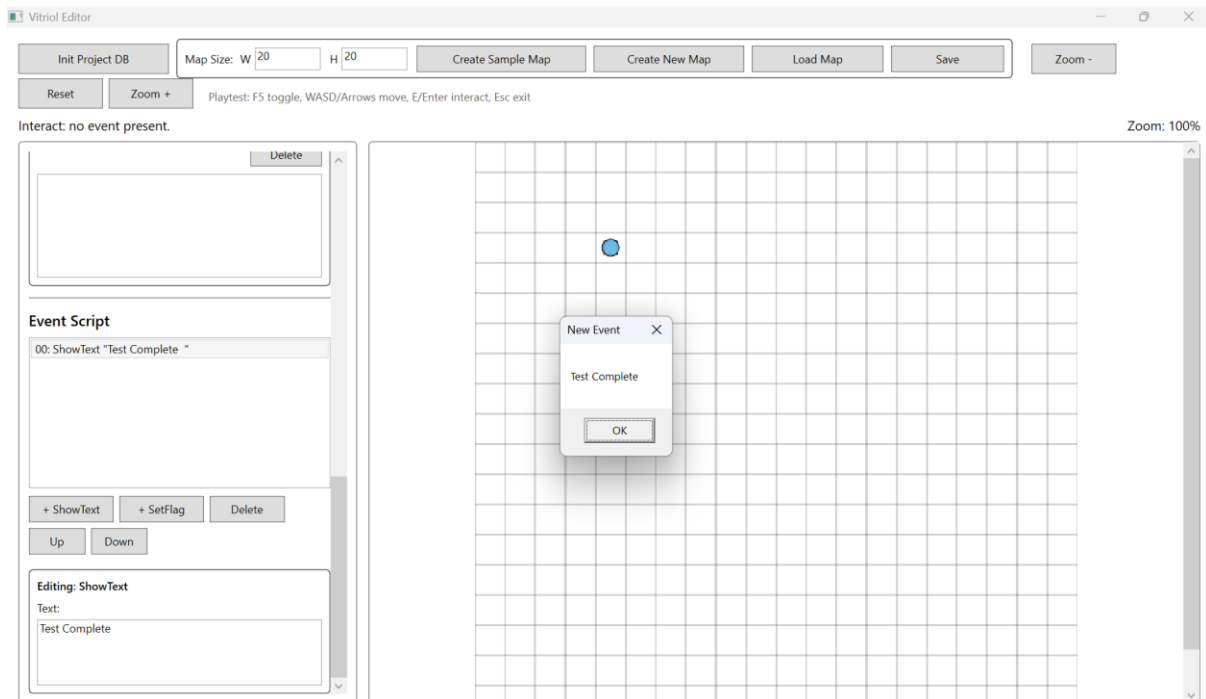
These tests confirm that runtime changes (camera behaviour, rendering pipeline, tileset caching) did not disrupt event triggering and command execution. This focuses on ensuring NPC, Trigger, and Warp event types still function predictably

Test Table 2 – Event System Regression Checks

Test ID	Description	Expected Outcome	Actual Outcome	Status	Screenshot ID
REGEV1	Trigger event executes ShowText	Dialogue appears with correct text	ShowText displayed correctly	Pass	Figures T70
REGEV2	NPC event executes ShowText	Dialogue appears when interacting with NPC	NPC interaction works	Pass	Figures T71
REGEV3	Warp event loads destination	Map loads and player relocates	Warp Successful	Pass	Figures T72

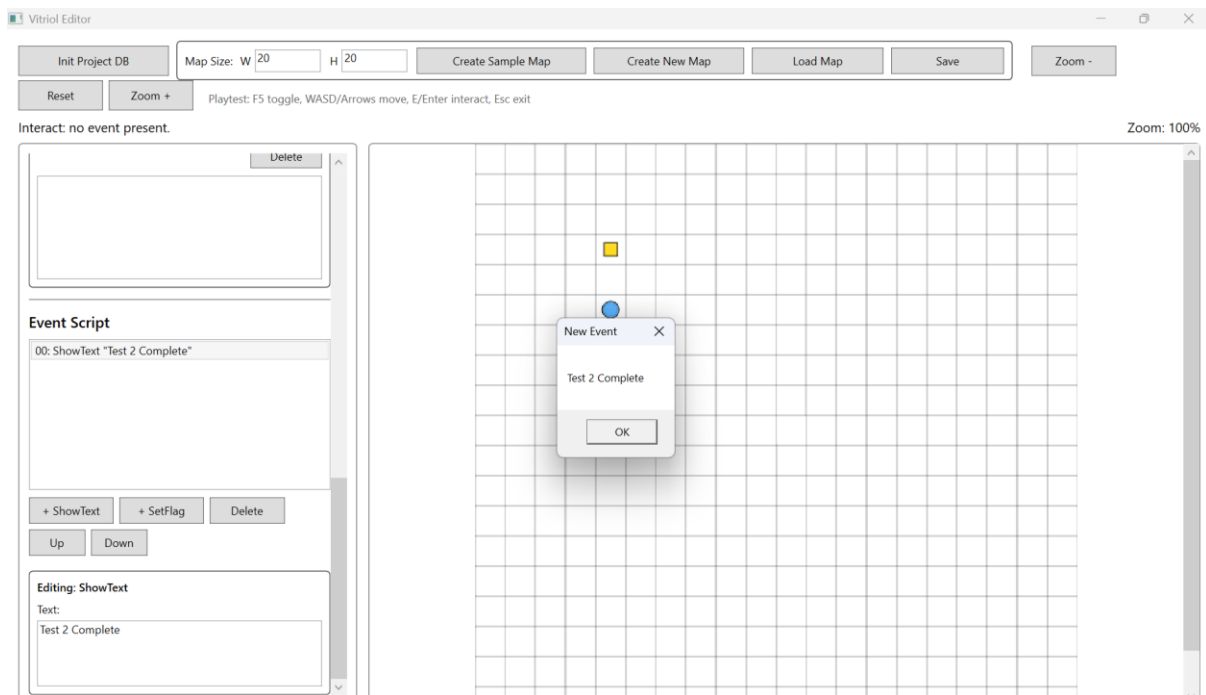
Test Table 2 – Screenshot Evidence

Figure T70 – Trigger Event Execution (ShowText)



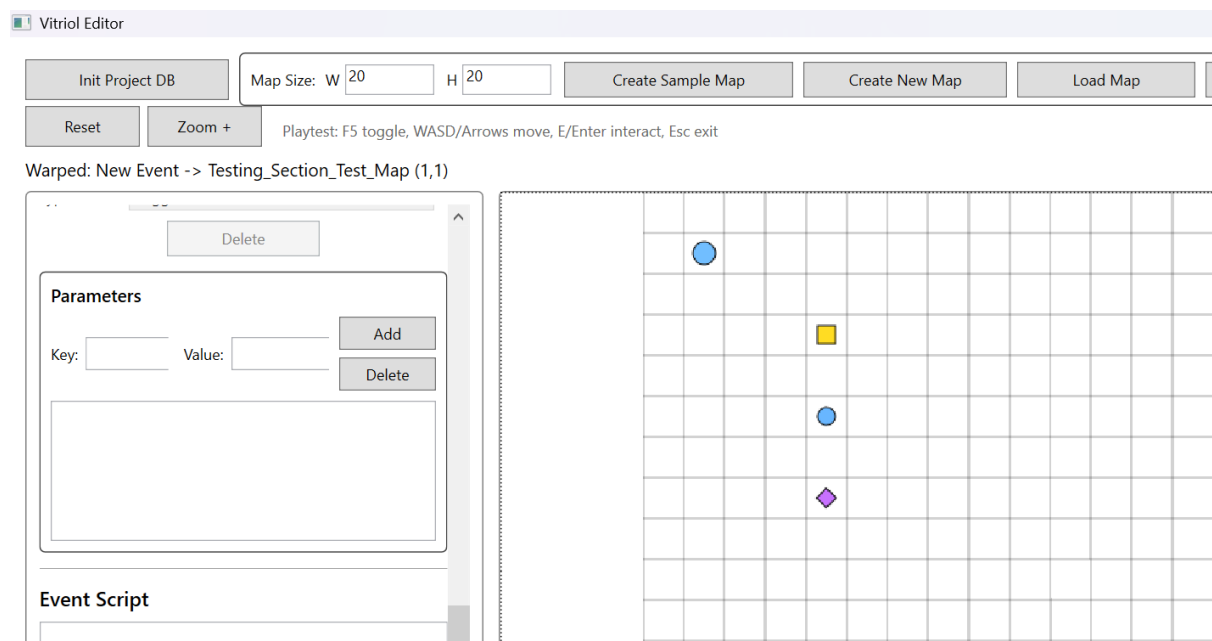
Testing Section\Testing Section Images\Figure T70.png

Figure T71 – NPC Interaction Event Execution



Testing Section\Testing Section Images\Figure T71.png

Figure T72 – Warp Event Map Transition



[Testing Section\Testing Section Images\Figure T72.png](#)

What These Tests Demonstrate:

- Trigger event still executes correctly after later runtime changes
- NPC events remain functional and correctly routed through interaction logic
- Warp loads the destination map and relocates the player
- Command dictionary logic still behaves correctly after refactors

5.3 Persistence Regression Checks

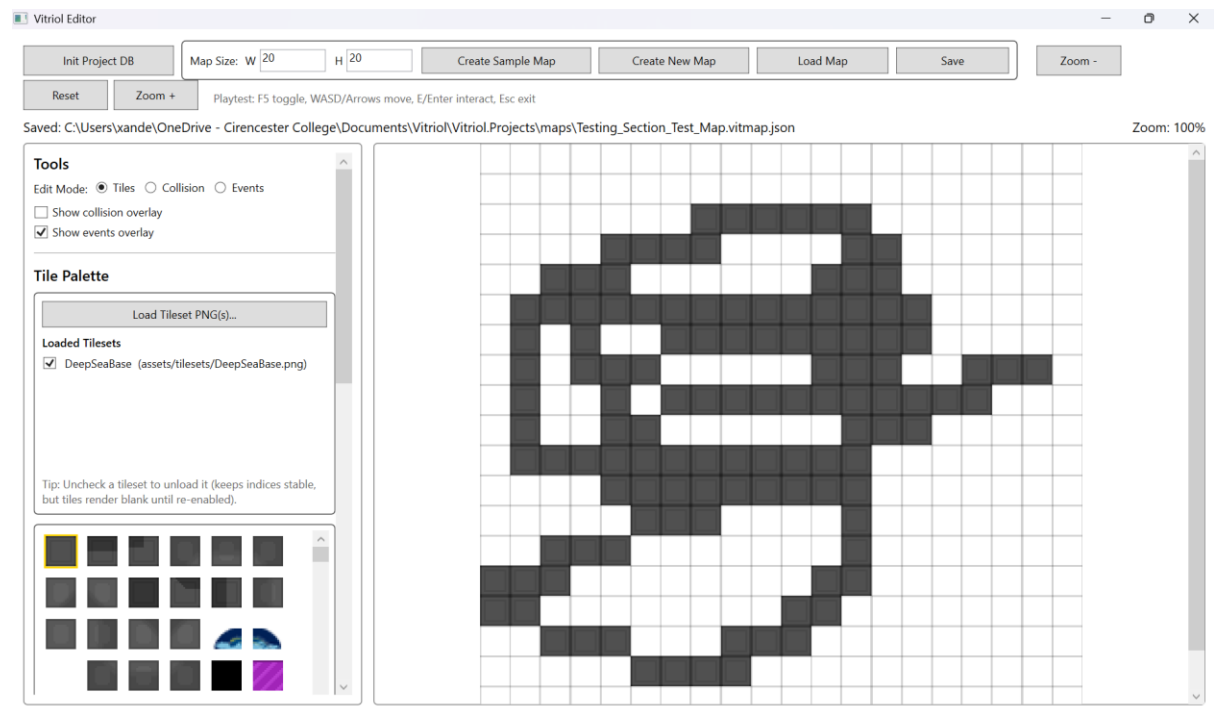
Persistence regression ensures later features did not introduce corruption into save/load behaviour. This includes JSON integrity, map reload fidelity, and ensuring that edits are correctly written back to file without breaking schema stability.

Test Table 3 – Persistence Regression Checks

Test ID	Description	Expected Outcome	Actual Outcome	Status	Screenshot ID
REGPS1	Save after tile edits	JSON updates reflect tile changes	Saved correctly	Pass	Figures T73
REGPS2	Save after collision edits	Collision layer persists correctly	Saved correctly	Pass	Figures T74
REGPS3	Event list persists with correct fields	Event list persists with correct fields	Saved correctly	Pass	Figures T75.1, T75.2
REGPS4	Runtime loads identical state	Runtime loads identical state	Reload matches saved state	Pass	Figures T76

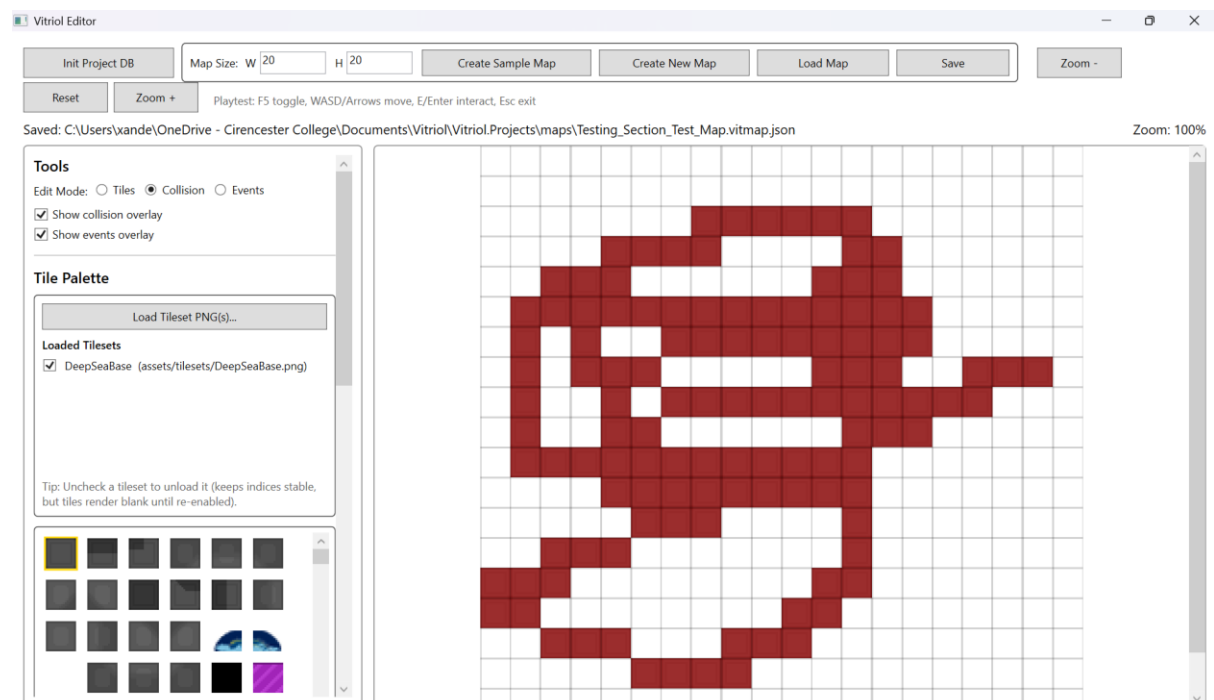
Test Table 3 – Screenshot Evidence

Figure T73 – Tile Edits Persist After Save



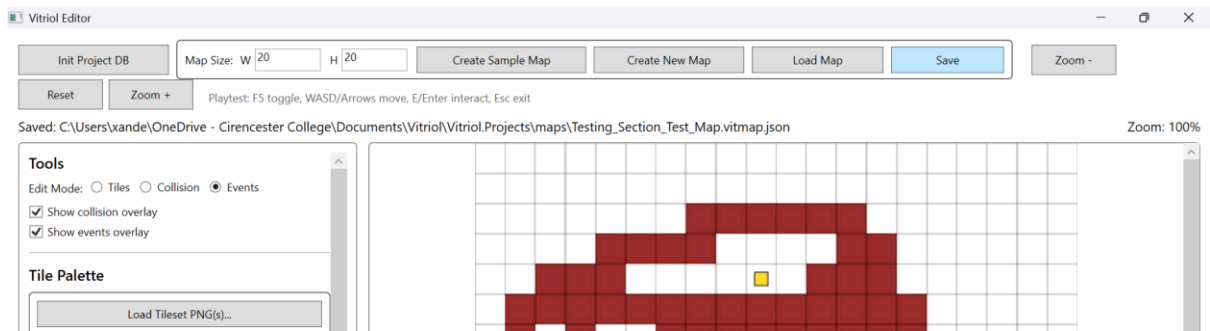
[Testing Section\Testing Section Images\Figure T73.png](#)

Figure T74 – Collision Edits Persist After Save



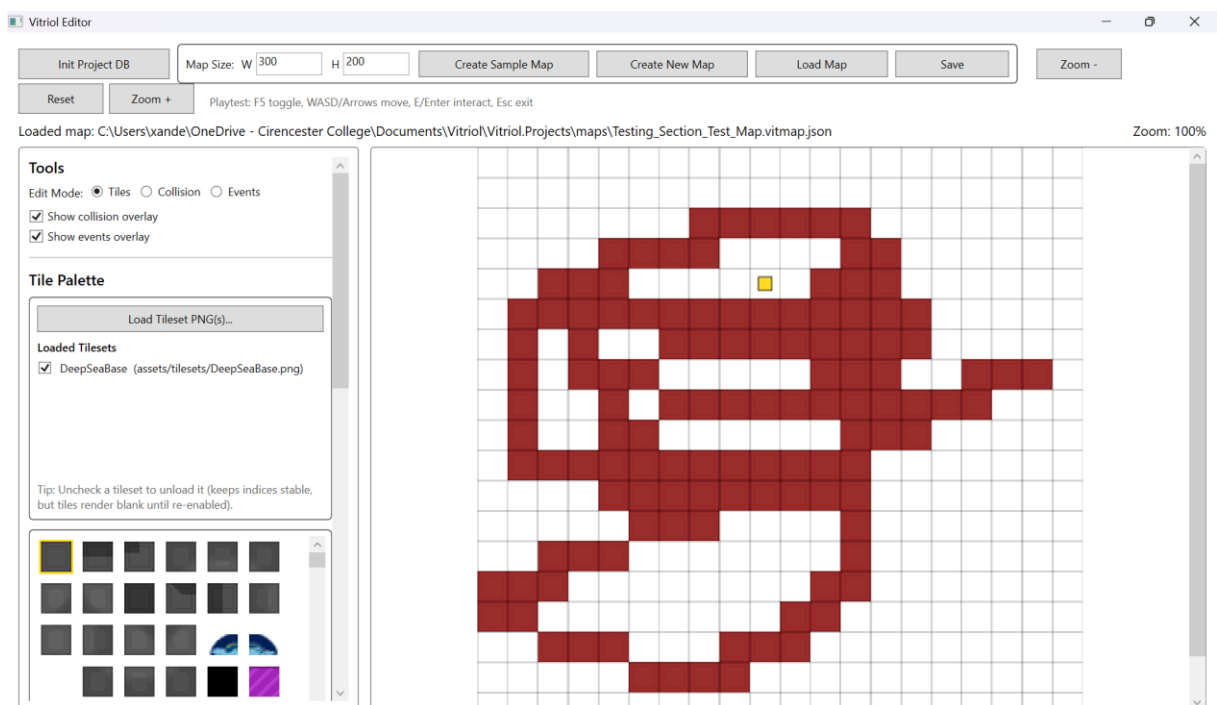
[Testing Section\Testing Section Images\Figure T74.png](#)

Figure T75.1 – Event Added and Saved



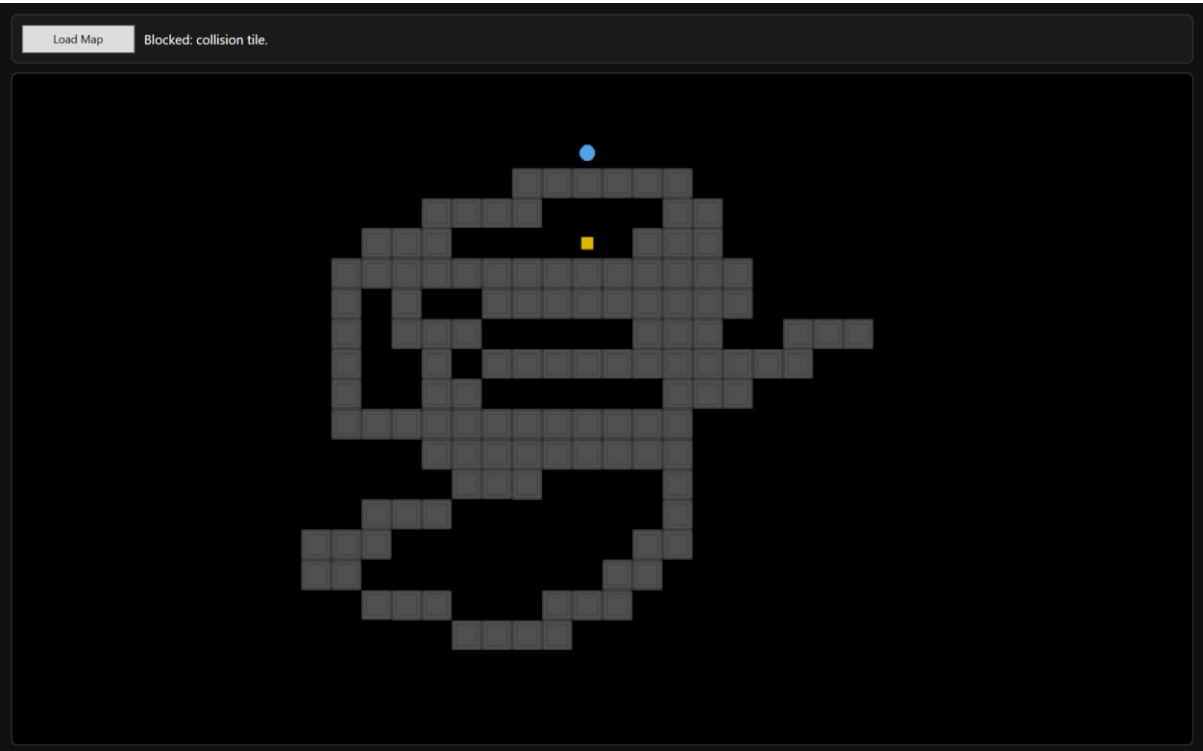
Testing Section\Testing Section Images\Figure T75.1.png

Figure T75.2 – Reload Event From Saved JSON



Testing Section\Testing Section Images\Figure T75.2.png

Figure T76 – Runtime Loads Saved Map Without Drift



[Testing Section\Testing Section Images\Figure T76.png](#)

What These Tests Demonstrate:

- Tile painting still produces correct saved output
- Collision layer remains stable through persistence pipeline
- Event created and stored correctly
- Event data reloads correctly and remains executable
- Runtime load fidelity after persistence operations

5.4 Rendering Regression Checks

Rendering regression checks confirm that viewport-only rendering, caching, and tileset compilation changes did not cause tile misalignment, incorrect indices, missing tiles, or rendering artifacts.

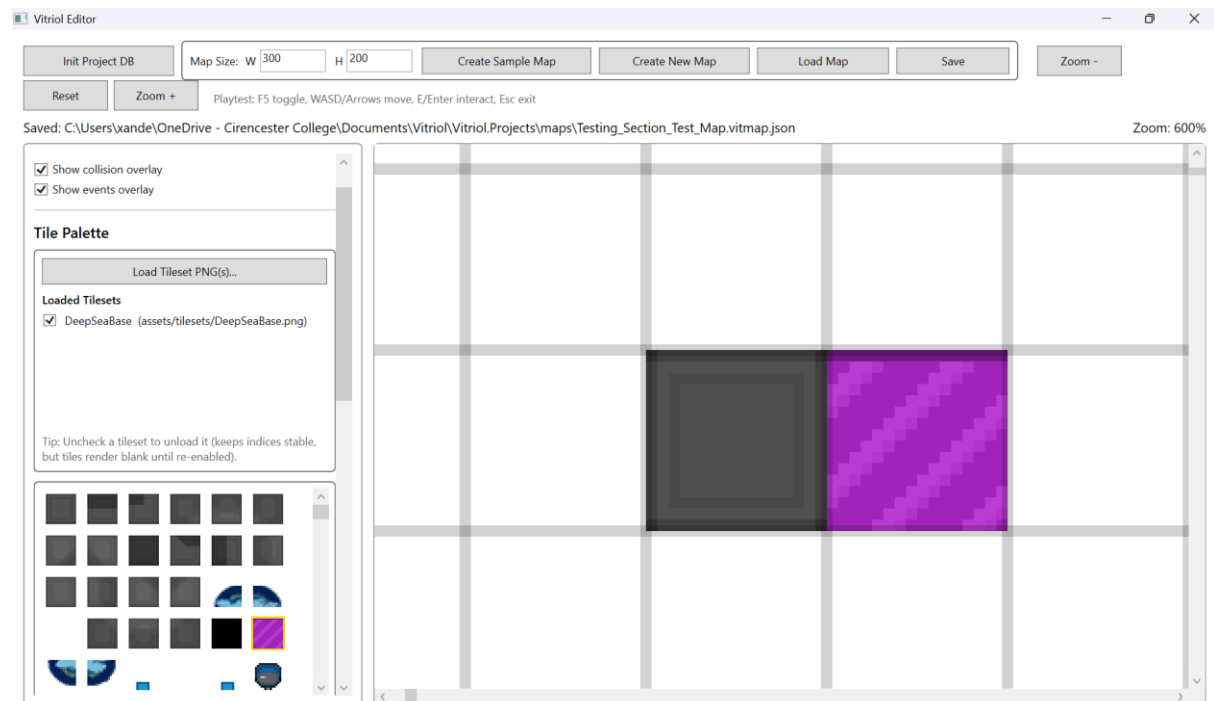
Test Table 4 – Rendering Regression Checks

Test ID	Description	Expected Outcome	Actual Outcome	Status	Screenshot ID
REGRN1	Render map after tileset caching added	Correct tile alignment	Alignment correct	Pass	Figures T77
REGRN2	Scroll viewport across map	No blank regions or tearing	Scroll Stable	Pass	Figures T78
REGRN3	Render events overlay markers	Events visible in viewport	Markers correct	Pass	Figures T79

REGRN4	Render player marker + deadzone camera	Player visible, camera clamps	Stable behaviour	Pass	Figures T80
---------------	--	-------------------------------	------------------	------	-------------

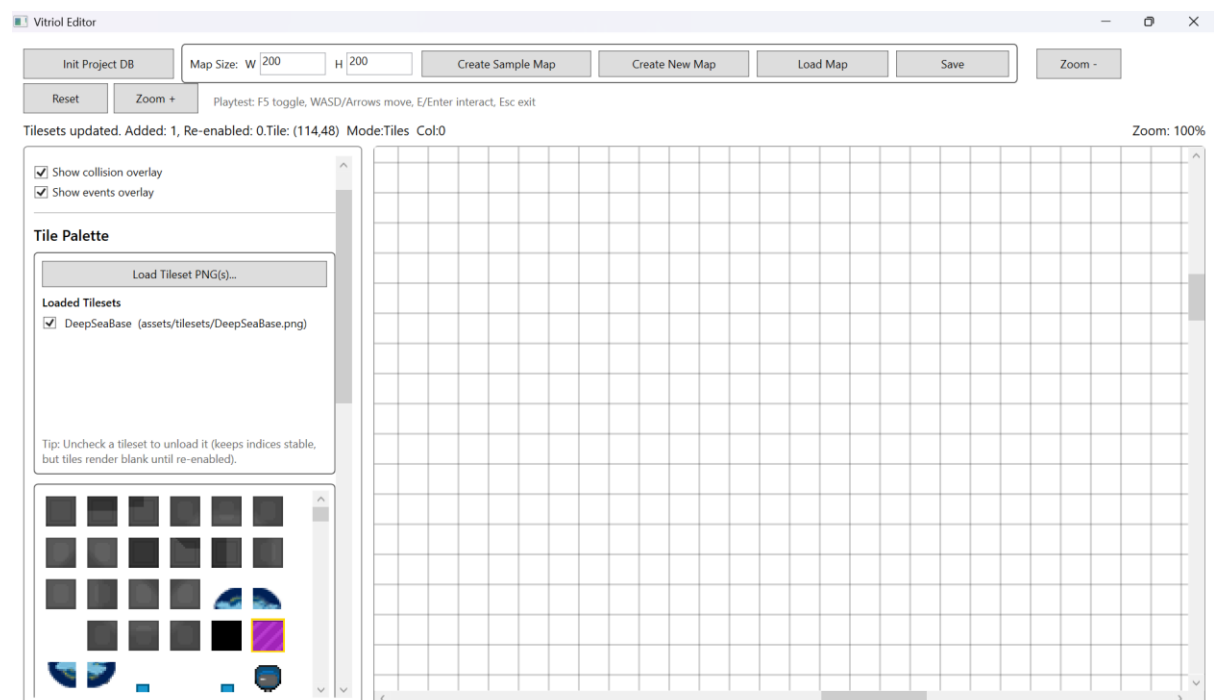
Test Table 4 – Screenshot Evidence

Figure T77 – Correct Tile Alignment After Rendering Changes



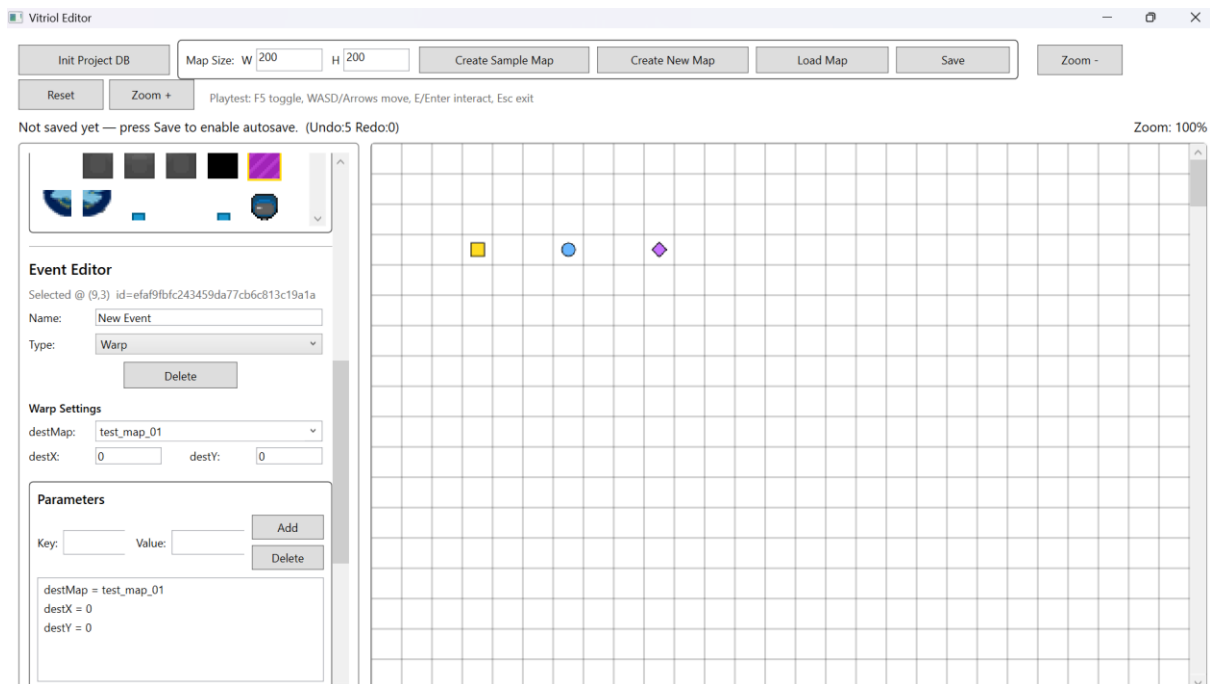
[Testing Section\Testing Section Images\Figure T77.png](#)

Figure T78 – Viewport Scroll Rendering Stability



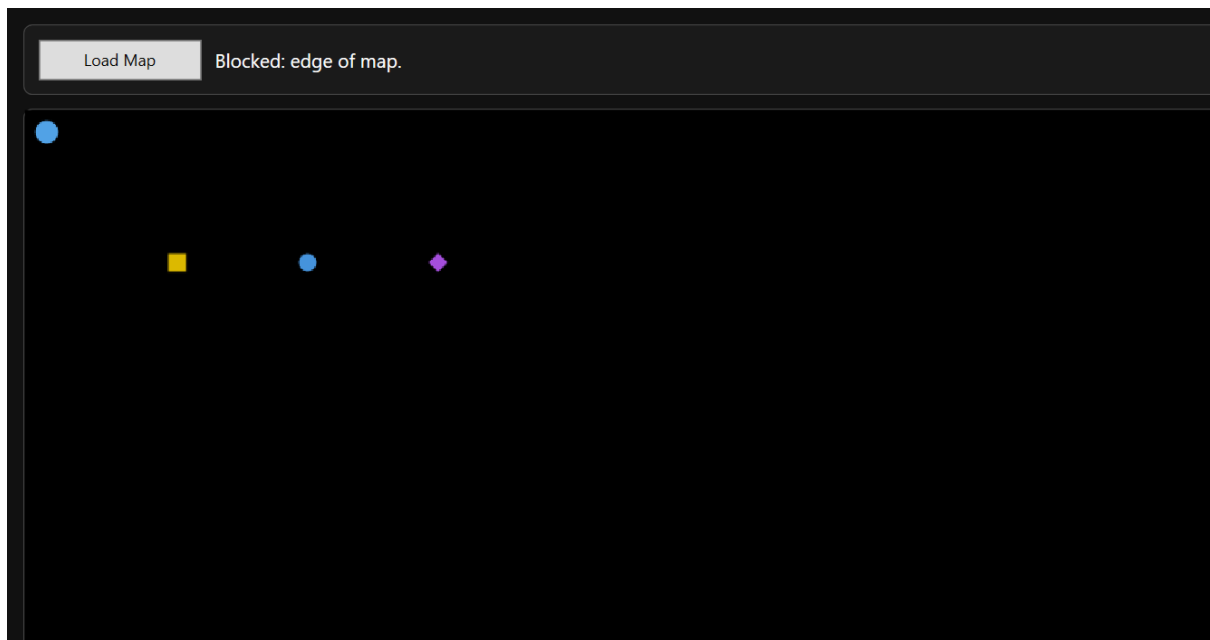
[Testing Section\Testing Section Images\Figure T78.png](#)

Figure T79 – Event Markers Render Correctly



[Testing Section\Testing Section Images\Figure T79.png](#)

Figure T80 – Player Marker + Camera Rendering Stable



[Testing Section\Testing Section Images\Figure T80.png](#)

What These Tests Demonstrate:

- Stable tile positioning and correct cropping
- Viewport-only rendering produces consistent redraw output
- Event overlay rendering still functions after pipeline changes

- Camera follow + player rendering remain consistent

6 User Testing

6.1 Structured User Task Testing

To evaluate usability and workflow clarity, structured task-based testing was conducted. The aim was to determine whether a user unfamiliar with the internal implementation could:

- Create and save a new map
- Paint tiles correctly
- Add an event
- Trigger the event in runtime
- Navigate the map using movement controls

This testing focused on:

- Learnability
- Workflow intuitiveness
- Error recovery
- Feature discoverability

Two participants with basic computer competence but no prior exposure to the codebase were observed performing a predefined task list.

Task List

Participants were asked to complete the following asks without assistance until blocked:

- Create a new map (32x32)
- Save the map with a unique name
- Load a tileset
- Paint at least three different tiles
- Add an NPC event with a ShowText command
- Save the map
- Launch runtime
- Load the created map
- Move the player to the NPC
- Trigger the event interaction

These tasks were chosen to cover:

- Core editor workflow
- Persistence

- Event configuration
- Runtime execution
- Basic navigation controls

Feedback Summary

Test User 1

User 1 completed all tasks successfully without intervention

Observations:

- Immediately understood tile painting
- Located save button quickly
- Required brief hesitation when locating event creation tools
- Successfully triggered NPC interaction in runtime

User reported:

- Interface felt “minimal but clear”
- Runtime controls were intuitive (WASD)
- Load Map button in runtime was helpful

No critical usability failures occurred

Test User 2

User 2 also completed all tasks successfully

Observations:

- Took slightly longer to locate event editing panel
- Initially attempted to interact with NPC without standing on the tile
- Successfully corrected interaction attempt without assistance

User reported:

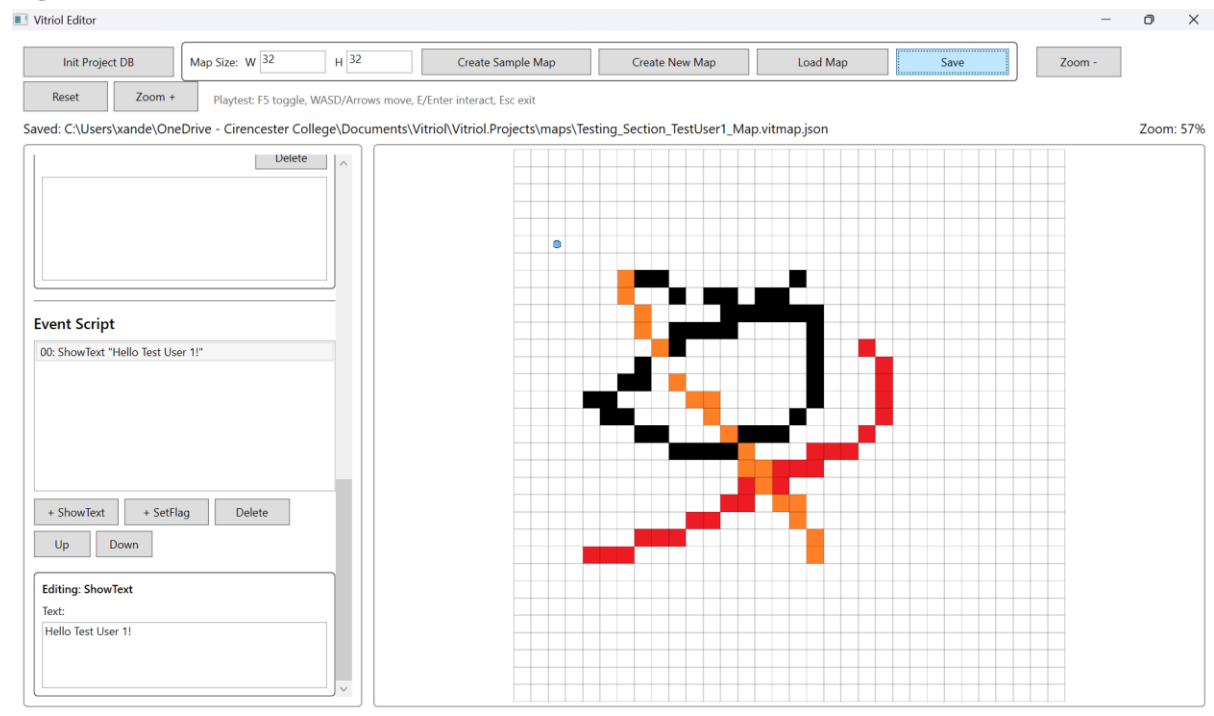
- Map creation was straightforward
- Event editing felt “more technical” than tile painting
- Runtime felt responsive

No blocking errors occurred

User Final Result Screenshot

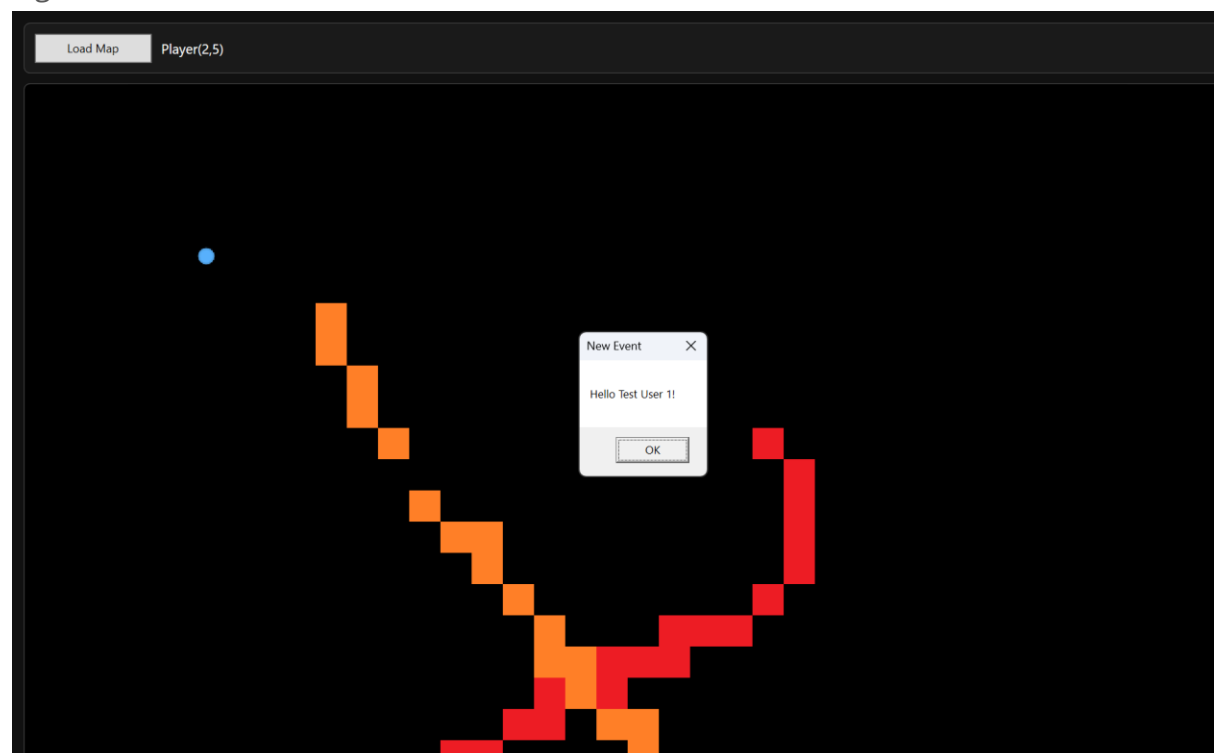
Test User 1

Figure T81.1 – Test User 1 Final Editor



[Testing Section\Testing Section Images\Figure T81.1.png](#)

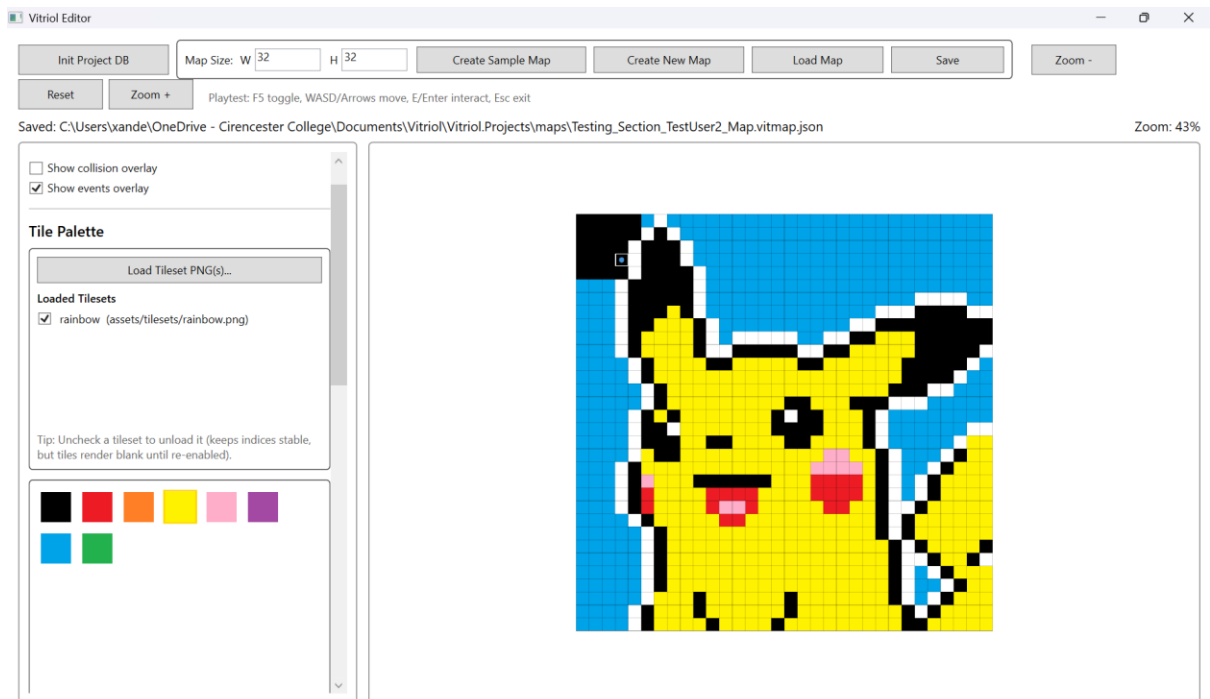
Figure T81.2 - Test User 1 Final Runtime



[Testing Section\Testing Section Images\Figure T81.2.png](#)

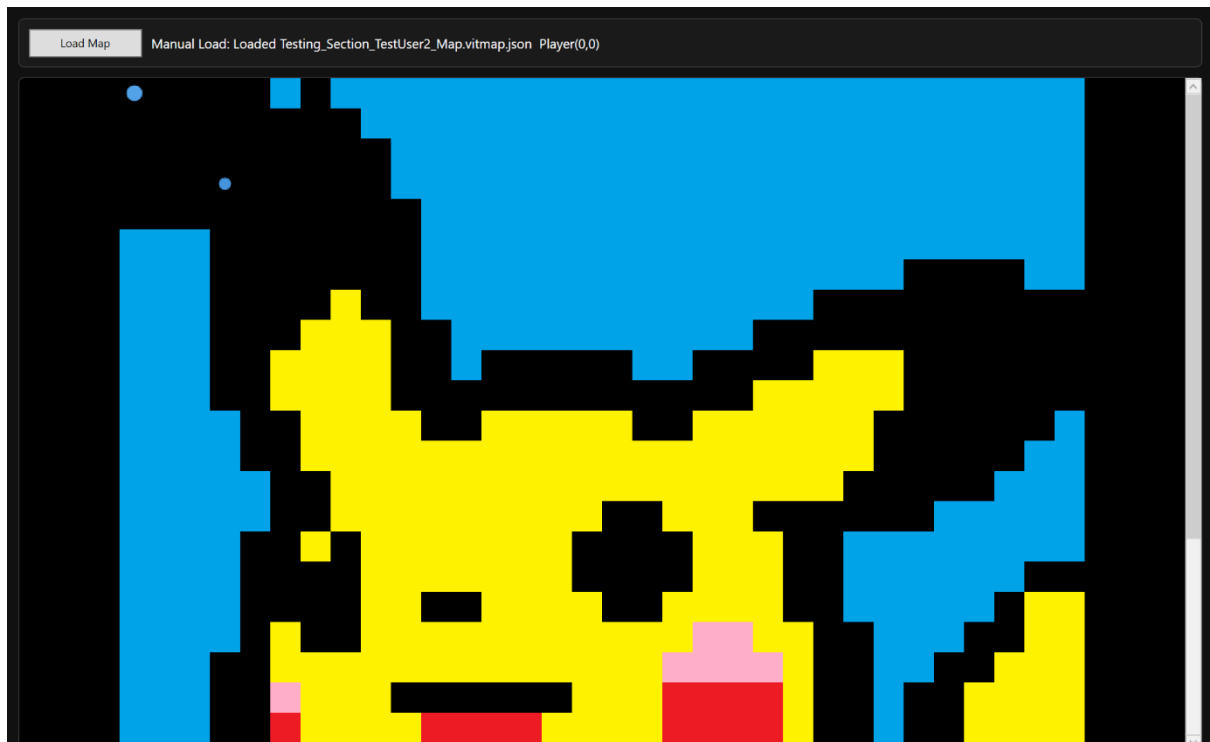
Test User 2

Figure T82.1 – Test User 2 Final Editor



[Testing Section\Testing Section Images\Figure T82.1.png](#)

Figure T82.2 – Test User 2 Final Runtime



[Testing Section\Testing Section Images\Figure T82.2.png](#)

6.2 Observed Error Patterns

Although both users completed all tasks, there were several minor patterns observed:

- Event tools were less discoverable than tile painting tools
- Users initially attempted to interact with NPCs from adjacent tiles
- Some hesitation occurred when distinguishing between saving and autosaving behaviour
- User expected Eter to trigger interaction consistently

These observations indicate that while the system is functionally stable, minor usability refinements could improve first-time experience

7 Witness Statement

7.1 Witness Confirmation of Functionality

I confirm that I observed the Vitriol editor and runtime being used to create and test a playable map. The developer demonstrated map creation, tile editing, event placement, saving, and loading the map into the runtime environment.

I observed successful player movement, event triggering, and correct rendering of the map. The system appeared stable, and responsive throughout the demonstration.

Based on what I observed, the features shown were working as described and no functional issues were evident during testing

Test User 1

Signature: Michael Christopher Payne

Date: 27/02/2026

Test User 2

Signature: Rachael Evans

Date: 27/02/2026

7.2 Summary of Demonstrated Features

Test User 1

During the demonstration, the following features were shown:

- Creation of a new tile-based map
- Manual save and autosave behaviour
- Loading and referencing of tilesets
- Tile painting and editing
- Placement of NPC event with ShowText command
- Saving to JSON format

- Runtime loading of saved map
- Player movement with collision handling
- NPC interaction triggering dialogue
- Camera following player with boundary clamping

All features appeared to function as intended.

Test User 2

The following functionality was demonstrated:

- Structured map creation workflow
- Editing of map layers
- Event placement and configuration
- Persistence of map data across reload
- Viewport-only rendering in both editor and runtime
- Runtime interaction using keyboard input
- Event execution behaviour

The system operated consistently during the observed session

8 Testing Summary and Coverage

8.1 Requirement Coverage Mapping

Coverage Table

The table below summarises how each core functional requirement has been validated through structured testing:

ID	Requirement Description	Validated By	Evidence Figures
FR1	Create new maps with defined dimensions and initialise layered structure	2.1, 5.1	T1-T6, T66-69
FR2	Visually paint, erase, and modify tile data within a grid-based system	2.2, 3.2	T7-T11, T34-T38
FR3	Collision Editing – Define walkable and blocked tiles via a dedicated collision layer	3.2, 5.2	T39-T41, T71
FR4	Event Placement – Add, edit, and remove position based event triggers on the map	2.4, 5.2	T16-T20, T70-T72
FR5	Map Persistence – Save and load maps using structured JSON with format versioning	2.4, 5.3	T16-T22, T73-T76
FR6	Runtime Playback – Load maps into runtime and render with camera-based viewport logic	3.1, 3,4	T23-T27, T42-T47
FR7	Player Movement & Collision – Handle deterministic grid-based collision detection	3.2, 5.2	T29-T41, T71

FR8	Editing History – Implement command-based Undo/Redo across tile, collision, and events	2.2	T8-T11
FR9	Autosave Control – Activate controlled autosave after initial manual save	2.1, 5.1	T4-T6, T67-T68
FR10	Performance stability – Maintain approximately 60 FPS on target specification hardware	4	T48-T65
FR11	Relative Asset Referencing – Store tileset paths relative to project directory	3.1	T28-T32
FR12	Layer Rendering Order – Enforce correct drawing hierarchy in editor and runtime	3.4, 5.4	T42-T47, T77

Coverage Analysis

The coverage mapping demonstrates that:

- All major functional requirements were validated through structured subsystem testing
- Regression testing confirmed stability following future expansion
- No core requirement remains untested
- Each requirement is supported by at least one visual evidence reference

This confirms that the implementation aligns with the functional objectives defined in the analysis section.

8.2 Non-Functional Requirement Validation

Performance

Performance testing (Section 4) demonstrated that:

- Viewport-only rendering reduces rendering workload
- Large maps (200x200) remain usable without freezing
- Continuous player movement does not cause degradation
- Tileset caching prevents repeated image decoding overhead

Evidence: T48 – T65

The system maintains stable responsiveness during both typical and extended usage scenarios.

Reliability

Reliability was validated through:

- JSON validation checks preventing malformed loads
- Defensive handling of missing event parameters
- Collision enforcement preventing invalid state transitions

- Regression testing confirming no feature breakage

Evidence: T20.1-T20.2, T70-T76

The application operates without crashes under normal usage and handles invalid inputs safely.

Usability

Usability validation was conducted through structured user testing (Section 6) demonstrating:

- Successful independent task completion by participants
- Minimal intervention required
- Clear map editing workflow
- Intuitive runtime movement controls

Observed minor discoverability issues did not prevent task completion.

Evidence: T81.1 – T82.2

The system is usable by individuals unfamiliar with the codebase, satisfying the intended user accessibility objective.

8.3 Final Conclusion

Across all testing categories:

- Subsystem testing validated individual components
- Boundary testing confirmed safe handling of abnormal input
- Performance testing demonstrated stable execution under load
- Regression testing confirmed stability after feature additions
- User testing validated real-world usability
- Witness confirmation provided independent observation.

The evidence demonstrates that the system meets the stated functional and non-functional requirements and behaves consistently under expected operating conditions.

9 Playtest Video

[Playtest 5+ Minute Video.mp4](#)

Project Vitriol – NEA Evaluation

The final implemented system successfully meets the functional core objectives defined during the Analysis Section. Core features including map creation with defined dimensions, layered tile editing, collision handling, event placement, JSON persistence,

and runtime playback were implemented and validated through structured subsystem and regression testing. The separation between editor logic, runtime behaviour, and shared data models proved effective in maintaining structural ability as new features continued to be introduced. The use of passive DTOs and a single serialisation pipeline ensured that map files remained consistent and portable through development, reducing the risk of format drift or data corruption.

From a performance perspective, the viewport-only rendering model achieved the intended purpose: large maps remained usable without excessive UI lag, and repeated rendering operations did not introduce visible degradation. The command-based undo/redo system behaved consistently across tile, collision, and event edits, confirming that the architectural decisions made in the Design Section translated effectively into implementation. User testing further indicated that the primary workflows: map creation, tile painting, and runtime interaction, were intuitive, with only minor discoverability issues in the event editing interface.

However, there are still several limitations to this project. The event command system, while flexible, still requires careful manual configuration and lacks higher-level tooling such as validation previews or visual scripting aids. Additionally, although performance remained stable during observational testing, more formal profiling tools could provide deeper insight into memory usage and rendering cost under extreme conditions. Usability could also be improved by refining tool discoverability and clarifying autosave behaviour to reduce potential user confusion.

Overall, the system satisfies the stated requirements, and demonstrates functional completeness, structural stability, and practical usability. While there is scope for refinement and expansion, particularly in terms of usability and tooling depth, the project achieves its intended goals within the defined scope and technical constraints.